

Boring-NixOS

K19G

2026-09-09

Table of contents

Boring NixOS	35
Boring NixOS	36
Who this is for	36
What “boring” means here	36
How to read	37
How examples work	37
Map of the book	37
What this book is not	39
Introduction	40
The Reproducibility Crisis	41
The Reproducibility Crisis	42
Mental model	42
Worked examples	42
Case 1: Detecting Ambient Host Contamination	42
Case 2: Purity and Isolated Evaluation	43
Case 3: Hermetic Sandboxing in Action	44
The trap	44
The boring rule	45
Try this	45
Installing Nix	46
Installing Nix	47
Mental model	47
Worked examples	48
Case 1: Multi-user install on Linux	48
Case 2: Enable flakes in nix.conf	48
Case 3: First substitute from cache.nixos.org	49
Case 4: A pinned flake instead of a floating channel	49
Case 5: macOS (same daemon, different volume)	50
The trap	50
The boring rule	51
Try this	51
Installing NixOS	52

Installing NixOS	53
Mental model	53
Worked examples	54
Case 1: Disposable VM, not a daily driver	54
Case 2: Partition, mount, generate config	54
Case 3: A tiny first <code>configuration.nix</code>	55
Case 4: The first rebuild on the installed system	56
Case 5: Rollback without a rescue disk	57
The trap	57
The boring rule	57
Try this	58
 Nix Fundamentals	 59
 Store Model	 60
The <code>/nix/store</code> Anatomy	61
The <code>/nix/store</code> Anatomy	62
Mental model	62
Worked examples	63
Case 1: Follow a binary into the store	63
Case 2: Read-only and epoch timestamps	63
Case 3: Two versions, no collision	64
Case 4: What is <i>not</i> in the store	64
Case 5: Name a path without guessing the hash	64
The trap	64
The boring rule	65
Try this	65
 Content-Addressable Paths	 66
Content-Addressable Paths	67
Mental model	67
Worked examples	68
Case 1: Input hash changes when the builder changes	68
Case 2: Store paths are immutable, not “updated in place”	68
Case 3: Fixed-output identity is the hash	69
Case 4: Why CA helps a cache	69
Case 5: Inspect which kind a path is	69
The trap	70
The boring rule	70
Try this	70

Hashes and Dependencies	71
Cryptographic Hashes and Inputs	72
Cryptographic Hashes and Inputs	73
Mental model	73
Worked examples	73
Case 1: Instantiate before you build	73
Case 2: One flag, new path	74
Case 3: The compiler is an input	75
Case 4: Purity — the sandbox cannot see <code>/usr</code>	75
Case 5: Pin the input set with a lockfile	76
The trap	76
The boring rule	76
Try this	76
Deterministic Dynamic Linking and RPATH	77
Deterministic Dynamic Linking and RPATH	78
Mental model	78
Worked examples	78
Case 1: Read RUNPATH off a real binary	78
Case 2: Which libraries would actually load	79
Case 3: The interpreter is a store path too	79
Case 4: Runtime closure matches RUNPATH	79
Case 5: LD_LIBRARY_PATH punches a hole	79
The trap	80
The boring rule	80
Try this	80
Dependency Graphs	81
Dependency Closures and Graphs	82
Dependency Closures and Graphs	83
Mental model	83
Worked examples	83
Case 1: Runtime closure of ripgrep	83
Case 2: Sizes, not just names	84
Case 3: Direct references versus the whole tree	84
Case 4: Why gcc is missing from runtime	84
Case 5: Copy what production needs	85
The trap	85
The boring rule	85
Try this	85
The Nix Database (db.sqlite)	86

The Nix Database (db.sqlite)	87
Mental model	87
Worked examples	87
Case 1: Referrers — who keeps this path alive?	87
Case 2: Valid path check	88
Case 3: Deriver link	88
Case 4: Registration is why <code>nix copy</code> exists	88
Case 5: Where the file lives (look, don't poke)	89
The trap	89
The boring rule	89
Try this	89
 Garbage Collection	 90
 GC Roots and Space Reclamation	 91
 GC Roots and Space Reclamation	 92
Mental model	92
Worked examples	93
Case 1: See the roots	93
Case 2: A <code>result</code> symlink is a root	93
Case 3: Safe collect versus drop generations	93
Case 4: Why CI disks fill up	94
Case 5: Do not GC the system you are running	94
The trap	94
The boring rule	95
Try this	95
 Store Optimization and Deduplication	 96
 Store Optimization and Deduplication	 97
Mental model	97
Worked examples	97
Case 1: Run it once and read the report	97
Case 2: Automatic on NixOS	98
Case 3: Prove two paths share an inode	98
Case 4: Logical size versus disk usage	98
Case 5: Safe with GC	99
The trap	99
The boring rule	99
Try this	99
 First Expressions	 100
 The Derivation Primitive	 101

The Derivation Primitive	102
Mental model	102
Worked examples	103
Case 1: A one-file banner	103
Case 2: Instantiate versus build	103
Case 3: Environment attributes	103
Case 4: The <code>runCommand</code> spelling you will actually write	104
Case 5: Failure when <code>\$out</code> is missing	104
The trap	105
The boring rule	105
Try this	105
Building Multi-File Artifacts	107
Building Multi-File Artifacts	108
Mental model	108
Worked examples	108
Case 1: A directory with a script and a data file	108
Case 2: Patch the shebang so the sandbox shell is not required at runtime	109
Case 3: <code>writeShellApplication</code> — the boring wrapper	109
Case 4: Multiple outputs are a later tool	110
Case 5: Install into a profile to get <code>PATH</code>	110
The trap	111
The boring rule	111
Try this	111
Inspecting the Store	112
Inspecting Closures: <code>why-depends</code>, <code>path-info</code>, <code>nix-tree</code>	113
Inspecting Closures: <code>why-depends</code>, <code>path-info</code>, <code>nix-tree</code>	114
Mental model	114
Worked examples	114
Case 1: Closure size in one line	114
Case 2: <code>why-depends</code>	115
Case 3: ASCII tree without extra tools	115
Case 4: <code>nix-tree</code> for interactive blame	116
Case 5: Watch a build with <code>nom</code>	116
The trap	116
The boring rule	117
Try this	117
Nix Language	118
Nix Expressions and Evaluation	119

Nix Expressions and Evaluation	120
Mental model	120
Worked examples	120
Case 1: Arithmetic is an expression	120
Case 2: A file is one <code>let ... in</code>	120
Case 3: Laziness skips <code>throw</code>	121
Case 4: <code>--strict</code> / JSON vs <code>thunks</code>	122
Case 5: No side effects at eval time	122
The trap	122
The boring rule	123
Try this	123
Values and Immutability	124
Values and Immutability	125
Mental model	125
Worked examples	125
Case 1: Merge does not rewrite	125
Case 2: Lists concatenate to a new list	126
Case 3: Same name twice is an error	126
Case 4: Inner <code>let</code> shadows, outer unchanged	127
Case 5: Deep merge is not <code>//</code>	127
The trap	128
The boring rule	128
Try this	128
Data Types and String Interpolation	129
Data Types and String Interpolation	130
Mental model	130
Worked examples	130
Case 1: <code>builtins.typeOf</code>	130
Case 2: Interpolation and indented strings	131
Case 3: Path vs string	132
Case 4: Integer interpolation fails	132
Case 5: <code>null</code> is not <code>"</code>	132
The trap	133
The boring rule	133
Try this	133
Let-In Blocks and Local Bindings	134
Let-In Blocks and Local Bindings	135
Mental model	135
Worked examples	135
Case 1: Desk URL from parts	135
Case 2: Order does not matter	136
Case 3: Shadowing	136
Case 4: Unused <code>throw</code> is fine	137

Case 5: <code>inherit</code> from a set	137
The trap	138
The boring rule	138
Try this	138
Functions and Argument Defaults	139
Functions and Argument Defaults	140
Mental model	140
Worked examples	140
Case 1: Curry vs <code>attrset</code>	140
Case 2: <code>@</code> captures extras	141
Case 3: Unexpected argument without	141
Case 4: <code>secure ? false</code>	142
Case 5: Apply with <code>--apply</code>	142
The trap	143
The boring rule	143
Try this	143
Attribute Sets and Recursive Definitions	144
Attribute Sets and Recursive Definitions	145
Mental model	145
Worked examples	145
Case 1: Merge and <code>or</code>	145
Case 2: <code>rec</code> for sibling URLs	146
Case 3: <code>//</code> does not update <code>rec</code> dependents	146
Case 4: Missing attr without <code>or</code>	147
Case 5: <code>//</code> is shallow (again)	147
The trap	148
The boring rule	148
Try this	148
Conditionals, Lists, and Built-in Operations	149
Conditionals, Lists, and Built-in Operations	150
Mental model	150
Worked examples	150
Case 1: Flags	150
Case 2: <code>map / filter</code>	151
Case 3: <code>else</code> is required	151
Case 4: Concatenate packages-style lists	152
Case 5: Lazy unused branch	152
The trap	152
The boring rule	153
Try this	153
Importing Files and Building Abstractions	154

Importing Files and Building Abstractions	155
Mental model	155
Worked examples	155
Case 1: Function file + caller	155
Case 2: Staging port	156
Case 3: <code>lib.optional</code> without a full <code>nixpkgs</code> (local copy)	156
Case 4: Relative paths are relative to the file	157
Case 5: <code><nixpkgs></code> is ambient	157
The trap	157
The boring rule	157
Try this	157
The Nix REPL and Debugging Expressions	158
The Nix REPL and Debugging Expressions	159
Mental model	159
Worked examples	160
Case 1: Basic REPL session	160
Case 2: Loading <code>nixpkgs</code>	161
Case 3: Loading a flake	163
Case 4: Debugging an expression with a wrong attribute path	165
Case 5: <code>builtins.trace</code> for printf-style debugging	167
The trap	168
The boring rule	169
Try this	170
Dev Environments	171
Nix Shells and the devShell Concept	172
Nix Shells and the devShell Concept	173
Mental model	173
Worked examples	173
Case 1: Simple <code>shell.nix</code>	173
Case 2: Setting Environment Variables in a Shell	174
The trap	175
The boring rule	175
Try this	175
Multi-Language Development Environments	176
Multi-Language Development Environments	177
Mental model	177
Worked examples	177
Case 1: Unified Polyglot Stack	177
Case 2: C Libraries and <code>pkg-config</code>	178
The trap	178
The boring rule	179

Try this	179
Flakes for Reproducible Dev Environments	180
Flakes for Reproducible Dev Environments	181
Mental model	181
Worked examples	181
Case 1: Minimal Flake Development Shell	181
Case 2: Updating Dependencies	182
The trap	183
The boring rule	183
Try this	183
Environment Management with Home Manager	184
Environment Management with Home Manager	185
Mental model	185
Worked examples	185
Case 1: Personal tools only	185
Case 2: Project compiler stays in the flake	186
Case 3: direnv composes, it does not replace HM	187
Case 4: Same nixpkgs as the OS when you can	187
Case 5: Rollback is per layer	187
The trap	187
The boring rule	187
Try this	187
Migrating from Docker Desktop to Nix DevShells	188
Migrating from Docker Desktop to Nix DevShells	189
Mental model	189
Worked examples	189
Case 1: The Docker Compose Pattern vs The Nix devShell Pattern	189
Case 2: Hybrid Workflow — Native Tooling with Containerized Backends	190
The trap	191
The boring rule	191
Try this	191
Automatic Dev Shells with direnv	192
Automatic Dev Shells with direnv	193
Mental model	193
Lifecycle	193
Worked examples	194
Case 1: Installing direnv and nix-direnv via Home Manager	194
Case 2: A Go desk project	195
Case 3: A polyglot project with named devShells	196
Case 4: Caching and GC safety	198

The trap	200
Using plain <code>use nix</code> instead of <code>use flake</code>	200
The boring rule	201
Try this	201
nix-darwin: Declarative macOS Configuration	202
nix-darwin: Declarative macOS Configuration	203
Mental model	203
Worked examples	204
Case 1: Minimal darwinConfiguration	204
Case 2: One flake for Linux and macOS	205
Case 3: Declarative Homebrew casks via nix-homebrew	207
Case 4: system.defaults macOS tweaks	210
The trap	212
The boring rule	213
Try this	213
Nix on a Foreign Linux	214
Nix on a Foreign Linux	215
Mental model	215
Worked examples	215
Case 1: Daemon is already installed — use it	215
Case 2: Standalone Home Manager on Fedora/Ubuntu	216
Case 3: Project shells without touching <code>/usr</code>	216
Case 4: SELinux and <code>/nix</code>	217
Case 5: Uninstall without wrecking the distro	217
The trap	217
The boring rule	217
Try this	217
Vendor Binaries with nix-ld and FHS Envs	219
Vendor Binaries with nix-ld and FHS Envs	220
Mental model	220
Worked examples	221
Case 1: Enable nix-ld on NixOS	221
Case 2: See what is still missing	221
Case 3: <code>buildFHSEnv</code> for an installer that shells out	222
Case 4: Package the vendor blob only when you must	222
Case 5: Foreign Linux	222
The trap	223
The boring rule	223
Try this	223

Packages and Overlays	224
The Nix Package Manager in Practice	225
The Nix Package Manager in Practice	226
Mental model	226
Worked examples	226
Case 1: One-shot	226
Case 2: <code>nix run</code>	227
Case 3: Profile install + list	227
Case 4: Rollback	227
Case 5: Search	228
The trap	228
The boring rule	228
Try this	228
Understanding the Flakes Input System	229
Understanding the Flakes Input System	230
Mental model	230
Worked examples	230
Case 1: <code>follows</code> in the tree	230
Case 2: One-input update	231
Case 3: Audit the lock	231
Case 4: <code>path:</code> for a sibling repo	232
Case 5: Check after a bump	232
The trap	232
The boring rule	232
Try this	232
Creating and Composing Overlays	233
Creating and Composing Overlays	234
Mental model	234
Worked examples	234
Case 1: Add a script to pkgs	234
Case 2: Compose two overlays	235
Case 3: <code>overrideAttrs</code> from <code>prev</code>	235
Case 4: Infinite recursion (the foot-gun)	236
Case 5: Flake overlay export	236
The trap	236
The boring rule	236
Try this	237
Custom Package Definitions	238
Custom Package Definitions	239
Mental model	239

Worked examples	239
Case 1: <code>writeShellApplication</code> recipe	239
Case 2: <code>stdenv</code> + <code>wrapProgram</code>	240
Case 3: Override an input	240
Case 4: Flake package	241
Case 5: <code>meta</code>	241
The trap	241
The boring rule	242
Try this	242
Managing Dependencies Across Projects	243
Managing Dependencies Across Projects	244
Mental model	244
Worked examples	244
Case 1: Tooling flake	244
Case 2: Consumer with <code>follows</code>	245
Case 3: Prefer packages over overlay	246
Case 4: Update the pin	246
Case 5: CI uses git, not <code>path:</code>	246
The trap	246
The boring rule	246
Try this	247
Fetchers and Fixed-Output Derivations	248
Fetchers and Fixed-Output Derivations	249
Mental model	249
Worked examples	249
Case 1: <code>fetchurl</code> with a probe hash	249
Case 2: <code>fetchFromGitHub</code>	250
Case 3: <code>nix-prefetch</code> so you do not fake-hash in a loop	251
Case 4: A package <code>src</code>	251
Case 5: <code>lib.fakeHash</code> is a development aid	251
The trap	251
The boring rule	252
Try this	252
NIX_PATH, Channels, and Pinning Inputs	253
NIX_PATH, Channels, and Pinning Inputs	254
Mental model	254
Worked examples	254
Case 1: See what <code><nixpkgs></code> actually is	254
Case 2: Flake pin (the default)	255
Case 3: <code>npins</code> without flakes	255
Case 4: Override <code>NIX_PATH</code> for one command (lab only)	255
Case 5: Fetch a tarball by hash (no flake, no <code>npins</code>)	256
The trap	256

The boring rule	256
Try this	256
Building From Source	257
The Derivation Build Process	258
The Derivation Build Process	259
Mental model	259
Worked examples	259
Case 1: One C file, no autotools	259
Case 2: <code>doCheck</code>	260
Case 3: <code>\$src</code> as a real tree	261
Case 4: Inspect the wrapped compiler	261
Case 5: <code>outputs</code> (preview)	261
The trap	262
The boring rule	262
Try this	262
Building C and C++ Projects with CMake	263
Building C and C++ Projects with CMake	264
Mental model	264
Worked examples	264
Case 1: Tiny CMake project	264
Case 2: <code>cmakeFlags</code>	265
Case 3: Ninja	266
Case 4: Link zlib	266
Case 5: Tests	266
The trap	266
The boring rule	267
Try this	267
Rust Packages with Cargo	268
Rust Packages with Cargo	269
Mental model	269
Worked examples	269
Case 1: <code>buildRustPackage + cargoHash</code>	269
Case 2: <code>cargoLock.lockFile</code>	270
Case 3: Native libs (openssl, pkg-config)	270
Case 4: Tests	271
Case 5: Crane (optional)	271
The trap	271
The boring rule	271
Try this	272
Python Packages with Pip and Setuptools	273

Python Packages with Pip and Setuptools	274
Mental model	274
Worked examples	274
Case 1: Application	274
Case 2: Library vs application	275
Case 3: Tests	276
Case 4: Dev shell	276
Case 5: Missing from nixpkgs	276
The trap	276
The boring rule	276
Try this	277
Go Modules with buildGoModule	278
Go Modules with buildGoModule	279
Mental model	279
Worked examples	279
Case 1: Stdlib-only binary	279
Case 2: Dependencies — probe vendorHash	280
Case 3: Pin the toolchain	281
Case 4: CGO_ENABLED=0	281
Case 5: Tests in the sandbox	281
The trap	282
The boring rule	282
Try this	282
Cross-Compilation for Multiple Platforms	283
Cross-Compilation for Multiple Platforms	284
Mental model	284
Worked examples	284
Case 1: Cross hello	284
Case 2: Your C program	284
Case 3: nativeBuildInputs vs buildInputs	285
Case 4: Flake overlay	285
Case 5: Prefer a builder for NixOS	286
The trap	286
The boring rule	286
Try this	286
Trivial Builders	287
Trivial Builders	288
Mental model	288
Worked examples	288
Case 1: writeText	288
Case 2: writeShellApplication	289
Case 3: runCommand	289
Case 4: symlinkJoin	290

Case 5: <code>writeTextDir</code> for a default config	290
The trap	290
The boring rule	291
Try this	291
Source Filtering	292
Source Filtering	293
Mental model	293
Worked examples	293
Case 1: <code>cleanSource</code>	293
Case 2: <code>fileset</code> union	294
Case 3: Exclude by name	294
Case 4: <code>gitignore</code>	294
Case 5: Prove it with <code>--dry-run</code>	295
The trap	295
The boring rule	295
Try this	295
Patching Packages	296
Patching Packages	297
Mental model	297
Worked examples	297
Case 1: In-tree patch on your package	297
Case 2: Overlay on <code>nixpkgs</code> hello	298
Case 3: <code>fetchpatch2</code> from a PR	298
Case 4: <code>sed</code> in <code>postPatch</code> (last resort)	298
Case 5: Drop an upstream patch	299
The trap	299
The boring rule	299
Try this	299
Node.js Packages	300
Node.js Packages	301
Mental model	301
Worked examples	301
Case 1: <code>buildNpmPackage</code>	301
Case 2: A CLI with a bin	302
Case 3: <code>pnpm</code>	302
Case 4: Node in a <code>devShell</code> , not in the output	303
Case 5: Native addons	303
The trap	303
The boring rule	303
Try this	303
Go Toolchains and <code>gomod2nix</code>	304

Go Toolchains and gomod2nix	305
Mental model	305
Worked examples	305
Case 1: Stay on <code>buildGoModule</code>	305
Case 2: Pin Go like fenix pins rustc	305
Case 3: gomod2nix generate	306
Case 4: <code>vendor/</code> in git (air-gap)	307
Case 5: Private modules	307
The trap	307
The boring rule	307
Try this	308
Go Development Environments	309
Go Development Environments	310
Mental model	310
Worked examples	310
Case 1: Shell that matches the package	310
Case 2: <code>direnv</code>	311
Case 3: Tests as you type	311
Case 4: CGO shell when you mean it	312
Case 5: Private <code>GOPRIVATE</code>	312
The trap	312
The boring rule	312
Try this	312
Cross-Compiling Go	313
Cross-Compiling Go	314
Mental model	314
Worked examples	314
Case 1: linux/arm64 from x86_64, no CGO	314
Case 2: Flake packages per arch	315
Case 3: CGO — use <code>pkgsCross</code> or a builder	315
Case 4: Windows / Darwin as GOOS	316
Case 5: file and go version <code>-m</code>	316
The trap	316
The boring rule	316
Try this	316
Packaging Anti-Patterns	317
Packaging Anti-Patterns	318
Mental model	318
Worked examples	318
Case 1: <code><nixpkgs></code> in production	318
Case 2: Fat <code>src</code>	319
Case 3: IFD	319
Case 4: Compiler on the profile	319

Case 5: Overlay recursion and secrets	319
The trap	320
The boring rule	320
Try this	320
NixOS System	321
The NixOS Module System Architecture	322
The NixOS Module System Architecture	323
Mental model	323
Worked examples	324
Case 1: A small desk module	324
Case 2: Consume it from a host	325
Case 3: Priorities — <code>mkDefault</code> versus <code>mkForce</code>	325
Case 4: Types catch mistakes at eval	326
Case 5: Inspect options without guessing	326
The trap	327
The boring rule	327
Try this	327
Configuring Your First NixOS Machine	328
Configuring Your First NixOS Machine	329
Mental model	329
Worked examples	330
Case 1: A readable host module	330
Case 2: Flake output for the same host	330
Case 3: <code>test</code> before <code>switch</code>	331
Case 4: Dry-activate and generations	332
Case 5: Split files without a framework	332
The trap	333
The boring rule	333
Try this	333
Service Management with systemd	334
Service Management with systemd	335
Mental model	335
Worked examples	336
Case 1: A desk worker unit	336
Case 2: Prefer a <code>nixpkgs</code> module when it exists	337
Case 3: Timers instead of <code>cron</code>	337
Case 4: Secrets as files, not unit text	338
Case 5: See what switch will restart	338
The trap	338
The boring rule	339
Try this	339

Networking, Firewall, and Security	340
Networking, Firewall, and Security	341
Mental model	341
Worked examples	341
Case 1: Hostname, DNS, firewall allow-list	341
Case 2: systemd-networkd static address (server)	342
Case 3: NetworkManager on the workstation	343
Case 4: WireGuard with a key file	343
Case 5: Trust a VPN interface without opening the world	344
The trap	344
The boring rule	344
Try this	345
Users, Groups, and SSH Configuration	346
Users, Groups, and SSH Configuration	347
Mental model	347
Worked examples	347
Case 1: Immutable users and hardened sshd	347
Case 2: Hash a console password	348
Case 3: Root is not a daily login	349
Case 4: Groups for devices, not for folklore	349
Case 5: Prove password SSH is dead	350
The trap	350
The boring rule	351
Try this	351
Hardware Configuration and Drivers	352
Hardware Configuration and Drivers	353
Mental model	353
Worked examples	353
Case 1: Read the generated file; do not invent it	353
Case 2: Firmware and microcode	354
Case 3: nixos-hardware for a known laptop	355
Case 4: Graphics — Mesa first, vendor blobs only if needed	355
Case 5: KVM on a workstation that runs VMs	356
The trap	356
The boring rule	356
Try this	357
Declarative Disk Partitioning with Disko	358
Declarative Disk Partitioning with Disko	359
Mental model	359
Worked examples	359
Case 1: Lab VM — one disk, no LUKS	359
Case 2: Metal — by-id + LUKS + Btrfs subvolumes	361

Case 3: Flake input	362
Case 4: nixos-anywhere	363
Case 5: Parameterise the device per host	363
The trap	363
The boring rule	364
Try this	364
Ephemeral Roots and Impermanence	365
Ephemeral Roots and Impermanence	366
Mental model	366
Worked examples	366
Case 1: tmpfs root + persist list	366
Case 2: SSH host keys — the one everyone forgets	367
Case 3: Desk service state	368
Case 4: Flake input	368
Case 5: Prove it on a VM	369
The trap	370
The boring rule	370
Try this	370
Image Generation with nixos-generators	371
Image Generation with nixos-generators	372
Mental model	372
Worked examples	373
Case 1: Bootable rescue ISO for the infrastructure desk	373
Case 2: QEMU qcow2 image for local VM testing	375
Case 3: Raspberry Pi SD card image (aarch64 cross-compile from x86_64)	377
Case 4: Sharing a NixOS module between a live machine and an image build	379
The trap	382
Building sd-aarch64 on x86_64 without cross-compilation	382
The boring rule	383
Try this	383
Backups and Generation Hygiene	385
Backups and Generation Hygiene	386
Mental model	386
Worked examples	386
Case 1: What to include	386
Case 2: Restic on NixOS	387
Case 3: Restore drill (the part people skip)	388
Case 4: Generation window on NixOS	388
Case 5: /boot full is a different incident	388
The trap	389
The boring rule	389
Try this	389

Home Manager	390
Home Manager and the Module System	391
Home Manager and the Module System	392
Mental model	392
Worked examples	393
Case 1: A standalone <code>home.nix</code>	393
Case 2: A flake with <code>homeConfigurations</code>	393
Case 3: User packages are not system services	394
Case 4: Generation rollback for <code>\$HOME</code>	395
Case 5: <code>stateVersion</code> is a birth certificate	395
The trap	395
The boring rule	395
Try this	396
Managing Dotfiles Declaratively	397
Managing Dotfiles Declaratively	398
Mental model	398
Worked examples	398
Case 1: Inline JSON from Nix	398
Case 2: Source a file that lives in git	399
Case 3: On-change hook	400
Case 4: Mutable file when a program insists on writing	400
Case 5: Inspect what this generation linked	400
The trap	400
The boring rule	401
Try this	401
Shell and Editor Configuration	402
Shell and Editor Configuration	403
Mental model	403
Worked examples	403
Case 1: Git as a module, not a ritual	403
Case 2: Starship prompt	404
Case 3: Bash aliases and history	405
Case 4: Neovim with plugins from <code>nixpkgs</code>	405
Case 5: Login shell versus module	406
The trap	406
The boring rule	406
Try this	406
Application Settings and Preferences	407
Application Settings and Preferences	408
Mental model	408

Worked examples	408
Case 1: Alacritty	408
Case 2: SSH client config	409
Case 3: tmux	410
Case 4: VS Code extensions from nixpkgs	410
Case 5: Fonts the terminal named	411
The trap	411
The boring rule	411
Try this	411
Combining NixOS and Home Manager	412
Combining NixOS and Home Manager	413
Mental model	413
Worked examples	413
Case 1: Minimal unified module	413
Case 2: Flake wiring	414
Case 3: Split files per user	415
Case 4: Backup files on clobber	415
Case 5: Rollback includes \$HOME	416
The trap	416
The boring rule	416
Try this	416
CI/CD with Nix	417
Setting Up Binary Caches	418
Setting Up Binary Caches	419
Mental model	419
Worked examples	419
Case 1: Consume a team cache on NixOS	419
Case 2: Push from a workstation (once)	420
Case 3: Verify a hit versus a miss	420
Case 4: Serve a tiny cache with <code>nix copy</code>	420
Case 5: Generate a signing key for a self-hosted cache	421
The trap	421
The boring rule	421
Try this	421
GitHub Actions with Nix	422
GitHub Actions with Nix	423
Mental model	423
Worked examples	423
Case 1: Check and build, push to Cachix	423
Case 2: <code>nix flake check</code> is the gate	424
Case 3: Magic Nix Cache when you do not have Cachix yet	424

Case 4: Pin the installer, pin nixpkgs	425
Case 5: Free disk on the runner	425
The trap	425
The boring rule	425
Try this	426
GitLab CI Integration	427
GitLab CI Integration	428
Mental model	428
Worked examples	428
Case 1: Official Nix image, flakes on	428
Case 2: Cachix from GitLab	429
Case 3: Cache /nix on a self-hosted runner	429
Case 4: rules so forks do not push	430
Case 5: Show what you shipped	430
The trap	431
The boring rule	431
Try this	431
Build Caching Strategies	432
Build Caching Strategies	433
Mental model	433
Worked examples	433
Case 1: Team cache is the real cache	433
Case 2: magic-nix-cache as a spare tyre	434
Case 3: lib.cleanSource / gitignore	434
Case 4: Vendor once, compile often	435
Case 5: See what would rebuild	435
The trap	436
The boring rule	436
Try this	436
Testing in CI Pipelines	437
Testing in CI Pipelines	438
Mental model	438
Worked examples	438
Case 1: Hermetic Go tests	438
Case 2: Flake checks	439
Case 3: CI workflow	439
Case 4: VM test job (KVM)	440
Case 5: Fail the gate	440
The trap	440
The boring rule	440
Try this	441
Secrets Management in CI	442

Secrets Management in CI	443
Mental model	443
Worked examples	443
Case 1: Private flake inputs without putting the PAT in Nix	443
Case 2: Eval sops without decrypting values into the store	444
Case 3: The leak, on purpose, once	444
Case 4: Cachix write only on main	445
Case 5: <code>nix flake check</code> without credentials	445
The trap	446
The boring rule	446
Try this	446
Remote Builders	447
Remote Builders	448
Mental model	448
Worked examples	448
Case 1: NixOS client configuration	448
Case 2: One-shot without a module	449
Case 3: Builder host — NixOS	449
Case 4: Darwin → Linux	450
Case 5: Features — why tests ran locally	451
The trap	451
The boring rule	451
Try this	451
Containers and Kubernetes	452
Creating Container Images with Nix (dockerTools)	453
Creating Container Images with Nix (dockerTools)	454
Mental model	454
Worked examples	454
Case 1: Layered image	454
Case 2: <code>vendorHash</code> for a real Go module	455
Case 3: Inspect layers	456
Case 4: Flake package	456
Case 5: Nobody is home in the image	456
The trap	456
The boring rule	457
Try this	457
Kubernetes Manifests as Nix Modules	458
Kubernetes Manifests as Nix Modules	459
Mental model	459
Worked examples	459
Case 1: A Deployment as JSON	459

Case 2: Same flake as the image	460
Case 3: Apply	460
Case 4: ConfigMap from Nix, not from kubectl create	461
Case 5: When the cluster cannot run Nix	461
The trap	461
The boring rule	461
Try this	461
Helm Charts Generated from Nix	463
Helm Charts Generated from Nix	464
Mental model	464
Worked examples	464
Case 1: Vendor a chart tarball	464
Case 2: Render values from Nix	465
Case 3: Wrapper script in a flake	465
Case 4: <code>helm upgrade</code> with a pinned package	465
Case 5: Diff in CI	466
The trap	466
The boring rule	466
Try this	466
GitOps with Argo CD and Flux	467
GitOps with Argo CD and Flux	468
Mental model	468
Worked examples	468
Case 1: Generated tree layout	468
Case 2: Flux GitRepository (illustrative)	469
Case 3: Kustomization pointing at generated YAML	469
Case 4: CI gate	469
Case 5: Argo Application (if the org standardised on Argo)	470
The trap	470
The boring rule	470
Try this	471
Service Mesh and Observability	472
Service Mesh and Observability	473
Mental model	473
Worked examples	473
Case 1: App metrics without a mesh	473
Case 2: Probe the endpoint	474
Case 3: NetworkPolicy before sidecar	474
Case 4: If you must mesh	474
Case 5: Logs on NixOS already exist	475
The trap	475
The boring rule	475
Try this	475

Infrastructure as Code	476
Managing Terraform Providers with Nix	477
Managing Terraform Providers with Nix	478
Mental model	478
Worked examples	478
Case 1: Shell with pinned providers	478
Case 2: Flake devShell	479
Case 3: Offline init	479
Case 4: Tiny plan	479
Case 5: CI	480
The trap	480
The boring rule	480
Try this	480
OpenTofu Integration	481
OpenTofu Integration	482
Mental model	482
Worked examples	482
Case 1: devShell	482
Case 2: Same <code>main.tf</code> as Terraform	483
Case 3: Backend still not in Nix	483
Case 4: CI	483
Case 5: Do not mix CLIs	483
The trap	483
The boring rule	484
Try this	484
Ansible Provisioning with Nix	485
Ansible Provisioning with Nix	486
Mental model	486
Worked examples	486
Case 1: Pinned ansible CLI	486
Case 2: Inventory in git	487
Case 3: Collections pinned, not <code>ansible-galaxy</code> at runtime	487
Case 4: Playbook that only talks to legacy	487
Case 5: Refuse NixOS in the inventory	487
The trap	488
The boring rule	488
Try this	488
Cloud Provider Configuration	489
Cloud Provider Configuration	490
Mental model	490

Worked examples	490
Case 1: Official AMI lookup (OpenTofu)	490
Case 2: Image you built	491
Case 3: user-data is not configuration.nix	491
Case 4: Security group matches the firewall chapter	491
Case 5: Instance tags	491
The trap	491
The boring rule	492
Try this	492
Multi-Cloud Deployment Patterns	493
Multi-Cloud Deployment Patterns	494
Mental model	494
Worked examples	494
Case 1: Shared module, two hosts	494
Case 2: stateVersion	494
Case 3: Images	495
Case 4: Separate tofu directories	495
Case 5: WireGuard between clouds	495
The trap	495
The boring rule	495
Try this	495
Secrets and Security	496
Why Secrets Are Hard with Declarative Config	497
Why Secrets Are Hard with Declarative Config	498
Mental model	498
Worked examples	498
Case 1: Prove the leak	498
Case 2: builtins.readFile is the same leak	499
Case 3: Reference a runtime path	499
Case 4: Hashes versus plaintext	500
Case 5: Hunt the closure	500
The trap	500
The boring rule	500
Try this	500
Using SOPS for Encrypted Configuration (sops-nix)	501
Using SOPS for Encrypted Configuration (sops-nix)	502
Mental model	502
Worked examples	502
Case 1: Recipients and a secrets file	502
Case 2: NixOS module	503
Case 3: Age key from SSH host key	504

Case 4: Flake input	504
Case 5: Missing owner	504
The trap	504
The boring rule	505
Try this	505
HashiCorp Vault Integration	506
HashiCorp Vault Integration	507
Mental model	507
Worked examples	507
Case 1: Agent module	507
Case 2: Template without secret values in Nix	508
Case 3: Reload the app when the lease rotates	509
Case 4: AppRole files	509
Case 5: When not to use Vault	509
The trap	510
The boring rule	510
Try this	510
Age Encryption for Simple Secrets	511
Age Encryption for Simple Secrets	512
Mental model	512
Worked examples	512
Case 1: Encrypt a file to yourself	512
Case 2: SSH public key as recipient	513
Case 3: agenix rules file	513
Case 4: Consume on NixOS	513
Case 5: Rekey when the fleet grows	514
The trap	514
The boring rule	514
Try this	514
Security Hardening Patterns	516
Security Hardening Patterns	517
Mental model	517
Worked examples	517
Case 1: Sandbox a desk unit	517
Case 2: Kernel sysctl baseline	518
Case 3: nixpkgs service modules already sandbox	519
Case 4: JIT exception	519
Case 5: Analyze before and after	519
The trap	519
The boring rule	520
Try this	520

Platform Engineering	521
The Internal Developer Platform Pattern	522
The Internal Developer Platform Pattern	523
Mental model	523
Worked examples	523
Case 1: Platform flake	523
Case 2: Init from template	524
Case 3: Consumer lock	524
Case 4: Baseline NixOS module	524
Case 5: Version the platform	525
The trap	525
The boring rule	525
Try this	525
Monorepo Strategies with Nix	526
Monorepo Strategies with Nix	527
Mental model	527
Worked examples	527
Case 1: fileset src	527
Case 2: flake outputs	528
Case 3: PR path filter (CI)	528
Case 4: linkFarm for a smoke set	528
Case 5: Do not use recursive flakes	528
The trap	529
The boring rule	529
Try this	529
Multi-Platform Development and Distribution	530
Multi-Platform Development and Distribution	531
Mental model	531
Worked examples	531
Case 1: eachDefaultSystem	531
Case 2: List systems explicitly	532
Case 3: Remote builder instead of qemu	532
Case 4: Check matrix in CI	532
Case 5: Release artifacts	532
The trap	532
The boring rule	533
Try this	533
Build Optimization and Performance Tuning	534
Build Optimization and Performance Tuning	535
Mental model	535

Worked examples	535
Case 1: Daemon knobs (NixOS 26.05)	535
Case 2: IFD is slow	536
Case 3: Profiler	536
Case 4: <code>--dry-run</code> after a comment-only change	536
Case 5: Remote vs local	536
The trap	536
The boring rule	536
Try this	537
Debugging and Troubleshooting Production Issues	538
Debugging and Troubleshooting Production Issues	539
Mental model	539
Worked examples	539
Case 1: Build log	539
Case 2: Eval trace	539
Case 3: Unit	540
Case 4: Why is python in the closure?	540
Case 5: Rollback	540
The trap	540
The boring rule	540
Try this	540
Testing Strategies for Nix Configurations	541
Testing Strategies for Nix Configurations	542
Mental model	542
Worked examples	542
Case 1: flake checks	542
Case 2: NixOS test	542
Case 3: CI	543
Case 4: Hermetic tests	543
Case 5: Do not test only eval	543
The trap	543
The boring rule	543
Try this	544
Maintenance, Upgrades, and Migration	545
Maintenance, Upgrades, and Migration	546
Mental model	546
Worked examples	546
Case 1: Lock bump inside 26.05	546
Case 2: Release upgrade	546
Case 3: <code>stateVersion</code> stays	547
Case 4: Home Manager follows	547
Case 5: Record	547
The trap	547

The boring rule	547
Try this	547
Enterprise Operational Checklists	548
Enterprise Operational Checklists	549
Mental model	549
Worked examples	549
Case 1: Daily (human)	549
Case 2: Weekly	549
Case 3: Before a fleet apply	549
Case 4: After a security advisory	550
Case 5: New host	550
The trap	550
The boring rule	550
Try this	550
Fleet Deployments with Colmena	551
Fleet Deployments with Colmena	552
Mental model	552
Worked examples	552
Case 1: Hive from flake	552
Case 2: Canary	553
Case 3: Dry-run	553
Case 4: Build on the builder	553
Case 5: Output	553
The trap	553
The boring rule	553
Try this	554
Lightweight MicroVMs with microvm.nix	555
Lightweight MicroVMs with microvm.nix	556
Mental model	556
Worked examples	556
Case 1: Guest in the flake	556
Case 2: Run	557
Case 3: vs nixosTest	557
Case 4: Persist nothing unless listed	557
Case 5: Host virtualisation	557
The trap	557
The boring rule	558
Try this	558
Content-Addressed Nix Internals	559
Content-Addressed Nix Internals	560
Mental model	560

Worked examples	560
Case 1: FOD is enough	560
Case 2: Input-addressed comment rebuild	560
Case 3: <code>nix show-derivation</code>	561
Case 4: When CA helps a farm	561
Case 5: 2.35 lazy flake copy is not CA	561
The trap	561
The boring rule	561
Try this	561
 Production Capstone Lab	 562
 Production Architecture and Threat Model	 563
 Production Architecture and Threat Model	 564
Mental model	564
Worked examples	565
Case 1: Inventory in the flake	565
Case 2: Trust boundary	565
Case 3: Secret inventory	565
Case 4: Failure drill list	566
Case 5: What this capstone is not	566
The trap	566
The boring rule	566
Try this	566
 Disko Storage and Ephemeral Base System	 567
 Disko Storage and Ephemeral Base System	 568
Mental model	568
Worked examples	568
Case 1: The Production Disko Module	568
Case 2: The Impermanence Persistent State Contract	570
The trap	571
The boring rule	571
Try this	571
 Hardened Networking and WireGuard Mesh	 572
 Hardened Networking and WireGuard Mesh	 573
Mental model	573
Worked examples	573
Case 1: WireGuard Mesh Module	573
Case 2: Strict nftables Firewall Configuration for Database Node	574
The trap	575
The boring rule	575
Try this	575

Distributed Application Stack and Secrets	576
Distributed Application Stack and Secrets	577
Mental model	577
Worked examples	577
Case 1: Caddy Reverse Proxy Configuration (<code>gw-01</code>)	577
Case 2: Sandboxed Application Service (<code>app-01</code>)	578
The trap	579
The boring rule	579
Try this	579
Automated Fleet Rollout and Disaster Recovery	580
Automated Fleet Rollout and Disaster Recovery	581
Mental model	581
Worked examples	581
Case 1: The Fleet Flake (<code>flake.nix</code>)	581
Case 2: Executing Fleet Rollout with Canary Verification	582
Case 3: Zero-Touch Disaster Recovery Playbook	583
The trap	583
The boring rule	583
Try this	584
Appendices	585
Glossary of Terms	586
Glossary of Terms	587
Derivation (<code>.drv</code>)	587
Nix Store (<code>/nix/store</code>)	587
Closure	587
Reference scanning	587
Flake	587
DevShell	587
Binary cache / substituter	587
NAR	588
GC root	588
Home Manager	588
Overlay	588
Module system	588
Fixed-output derivation (FOD)	588
RUNPATH	588
nix-ld	588
FHS env (<code>buildFHSEnv</code>)	588
Remote builder	589
stateVersion	589
Generation	589
Impermanence	589

Command Cheat Sheet	590
Command Cheat Sheet	591
Flakes and development	591
Build, run, eval	591
Inspect closures	591
Fetchers	592
Copy and caches	592
Remote builders	592
NixOS	592
Home Manager	593
GC and disk	593
Backup (restic)	593
Nix and NixOS Version Reference	594
Nix and NixOS Version Reference	595
Nix CLI Releases	595
Nix 2.18	595
Nix 2.19	596
Nix 2.20	597
Nix 2.21	597
Nix 2.22	598
Nix 2.23	599
Nix 2.24	600
NixOS Releases	601
NixOS 23.05 — Stoa	601
NixOS 23.11 — Tapir	602
NixOS 24.05 — Uakari	603
NixOS 24.11 — Vicuna	605
Nix 2.28 – 2.34 (bridge)	606
Nix 2.35	606
NixOS 25.05 / 25.11	607
NixOS 26.05 — Yarara	607
Quick-Reference Matrix	608

Boring NixOS

Boring NixOS

A direct, practical guide to configuring infrastructure that builds identically every time: **deterministic, immutable, and refreshingly boring**.

Ambiguity is the enemy of reliability. When machines mutate silently through ad-hoc commands, builds drift, onboarding breaks, and outages follow. The boring default is to define the full closure in code — from developer compilers to multi-region cloud clusters — so every environment evaluates identically six months later.

Baseline: Nix **2.35+** · NixOS **26.05+** · Flakes enabled. Standalone and self-contained. You do not need any other title in this library.

Who this is for

- Engineers, platform developers, and SREs who want systems that build identically on a laptop, CI runner, and bare-metal server.
- Developers looking for instant, clean project environments without container lag or host library pollution.
- Teams moving away from fragile setup scripts and mutable system administration toward pure declarative configuration.

You do not need prior experience with Nix or functional programming. A terminal, a standard text editor, and basic Linux familiarity are all you need.

What “boring” means here

Nix is a purely functional package manager and domain-specific language. Applied with discipline, it eliminates entire categories of operational bugs: conflicting shared libraries, broken symlinks, broken OS upgrades, and undocumented host drift.

When over-complicated, it can become an impenetrable maze of nested overlays and tangled macros.

Boring NixOS teaches clear, sustainable defaults: - Plain, readable expressions over clever meta-programming. - Explicit inputs pinned by cryptographic lockfiles. - Modular, auditable NixOS system definitions that scale without surprise.

How to read

1. Read the chapter concept and mental model.
2. Type the expression into its designated file (`default.nix`, `shell.nix`, `flake.nix`, or `configuration.nix`).
3. Run the exact build or evaluation command.
4. Experiment with the **Try this** exercises to verify how the store and evaluator behave.

The chapters are sequenced progressively. Install Nix (and optionally NixOS) in part 0, then the store and the language. You can jump to devShells, NixOS services, or CI caching once those basics exist.

How examples work

Every code listing is a complete, runnable expression or file. Nothing is an incomplete fragment left for you to guess.

```
# default.nix
let
  message = "reproducible desk online";
in
message
```

```
nix-instantiate --eval default.nix
```

"reproducible desk online"

A recurring scenario — a small team infrastructure desk (workstations, devShells, microservices, deployment pipelines) — connects the exercises so each technique solves a practical engineering challenge.

Map of the book

Part	Folder	What you leave with
0	00-introduction	Why imperative setups drift; install Nix; install NixOS on a disposable VM
1	01-nix-fundamentals	<code>/nix/store</code> , hashes, <code>RUNPATH</code> , closures, <code>db.sqlite</code> , GC, raw derivations, <code>why-depends</code> / <code>path-info</code> / <code>nix-tree</code>

Part	Folder	What you leave with
2	02-nix-language	Types, <code>let ... in</code> , functions, attrsets, lists, imports, the REPL
3	03-dev-environments	<code>mkShell</code> , polyglot toolchains, flakes, <code>direnv</code> , <code>nix-darwin</code> , Nix on a foreign Linux, <code>nix-ld</code> / FHS
4	04-packages-and-overlays	<code>nixpkgs</code> , flake inputs, overlays, custom packages, fetchers, <code>NIX_PATH</code> / <code>npins</code>
5	05-building-from-source	<code>stdenv</code> , C/CMake, Rust, Python, Go modules / <code>gomod2nix</code> / Go shells / Go cross, Node, trivial builders, source filters, patches, anti-patterns
6	06-nixos-system	Modules, first machine, <code>systemd</code> , firewall, users, hardware, Disko, impermanence, generators, backups and GC windows
7	07-home-manager	User modules, dotfiles, shell/editor, app settings, one-flake NixOS+HM
8	08-cicd	Binary caches, GitHub Actions, GitLab CI, caching strategy, tests, secrets, remote builders
9	09-containers-k8s	<code>dockerTools</code> images, k8s as modules, Helm, GitOps, mesh (optional late path)
10	10-iac	Terraform/OpenTofu providers, Ansible, cloud patterns
11	11-secrets-security	Why the store is public; <code>sops-nix</code> ; Age; Vault; hardening
12	12-platform-engineering	IDP, monorepos, fleet, tuning, debug, tests, upgrades, Colmena, microVMs, CA internals
13	13-production-capstone	Threat model, Disko + ephemeral root, WireGuard, app stack, fleet rollback
99	99-appendices	Glossary, command cheat sheet, Nix/NixOS version notes

What this book is not

- Not an exhaustive reference manual of all 100,000+ packages in `nixpkgs`.
- Not an exercise in esoteric functional programming puzzles.
- Not a guide to maintaining legacy channels and unpinned channels.

It is a concrete guide to writing reproducible systems that remain easy to understand, maintain, and operate.

Introduction

The Reproducibility Crisis

The Reproducibility Crisis

Software engineering has spent decades tolerating ambient state: hidden dependencies, global machine mutations, and “works on my machine” excuses. The boring default of this book is: **every development environment, dependency, and production system must be a pure, declarative, reproducible expression from zero.**

Mental model

Traditional operations rely on imperative mutation. You provision a virtual machine or container, run a sequence of `apt-get`, `curl | bash`, and `pip install` commands, and hope that: 1. Upstream package repositories haven’t updated or removed files. 2. Network connections remain uninterrupted mid-install. 3. System state matches the exact sequence of commands previously tested.

When any of these fail, you get **Heisenbugs**: intermittent build failures, subtle ABI mismatches, and production outages that disappear when you attempt to isolate them.

Traditional Imperative Model:

```
Base OS -> [Step 1: apt-get] -> [Step 2: pip install] -> [Step 3: git clone] -> ??? (Unknown)
```

Nix Declarative Model:

```
Inputs (Source + Hashes) -> Pure Derivation -> Immutable /nix/store Path (Deterministic)
```

Nix replaces stateful mutation with pure mathematical evaluation: given the exact same source code and inputs, it always yields the exact same byte-for-byte system closure.

Worked examples

Case 1: Detecting Ambient Host Contamination

Save as `test_environment.sh`. Inspect how standard shell scripts unknowingly depend on host binaries and paths.

```
# test_environment.sh
#!/usr/bin/env bash
set -euo pipefail

echo "Inspecting system path dependencies:"
which python3 || echo "python3 missing"
```

```
which node || echo "node missing"
which gcc || echo "gcc missing"
```

Run:

```
bash test_environment.sh
```

Output (on a typical developer workstation):

```
Inspecting system path dependencies:
/usr/bin/python3
/home/user/.nvm/versions/node/v20.0.0/bin/node
/usr/bin/gcc
```

Notice how three completely different tools live in three different unpinned locations (`/usr/bin`, user home directories, global OS paths). If a colleague runs the exact same script on a fresh laptop, it fails immediately.

Case 2: Purity and Isolated Evaluation

Save as `pure_check.nix`. A minimal Nix expression evaluated in isolation.

```
# pure_check.nix
let
  system = builtins.currentSystem;
  message = "No ambient host contamination allowed";
in
{
  inherit system message;
}
```

Run:

```
nix-instantiate --eval pure_check.nix
```

Output:

```
{ message = "No ambient host contamination allowed"; system = "x86_64-linux"; }
```

Evaluating this expression does not query the internet, look at `/usr/local`, or depend on environment variables.

Case 3: Hermetic Sandboxing in Action

Nix builds run in sandboxed namespaces where network access and non-declared filesystem paths are inaccessible.

Save as `sandbox_check.nix`:

```
# sandbox_check.nix
derivation {
  name = "sandbox-proof";
  system = builtins.currentSystem;
  builder = "/bin/sh";
  args = [
    "-c"
    "echo 'Built inside isolated namespace' > $out"
  ];
}
```

Run:

```
nix-build sandbox_check.nix
```

Output:

```
these 1 derivations will be built:
  /nix/store/10h9v...-sandbox-proof.drv
building '/nix/store/10h9v...-sandbox-proof.drv'...
/nix/store/v73ka...-sandbox-proof
```

Inspect output:

```
cat result
```

Output:

```
Built inside isolated namespace
```

The output was generated purely from the declared inputs.

The trap

Relying on “Golden Images” (pre-baked AMIs, Docker images without lockfiles, or manual server setup steps documented in internal wikis).

Wiki page: "Run these 14 commands to set up the desk service..."

By week three, one PPA repository goes offline, a patch release bumps a minor dependency, and onboarding is broken for every new hire.

The fix: Check in code that defines the environment completely down to cryptographic hashes. If it builds once, it builds forever.

The boring rule

- Never mutate system state in-place with ad-hoc shell commands.
- Pin all toolchain and library dependencies via cryptographic hashes.
- Eliminate dependency on the host's ambient `/usr/bin` and `$PATH`.
- Every environment must be reproducible from a fresh Git checkout.

Try this

1. Run `env` in your current shell and note how many variables point to user home directories or mutable local paths.
2. In `pure_check.nix`, add a field `timestamp = builtins.currentTime`; and observe how pure evaluation restricts non-deterministic built-ins.

Installing Nix

Installing Nix

The rest of this book assumes a working Nix installation with **flakes** and the **nix-command** CLI enabled. The boring default is: **a multi-user daemon install, flakes on, and nix run nixpkgs#hello succeeding before you write a single expression.**

You do not need NixOS yet. Nix the package manager runs on ordinary Linux and on macOS. NixOS (the operating system) comes in the next chapter.

Mental model

Nix is three things that share a name:

Piece	What it is
Nix the tool	The nix binary and the nix-daemon that builds and substitutes store paths
The store	/nix/store , a read-only tree of hashed paths
NixOS	A Linux distribution whose entire system is a Nix derivation

This chapter installs the first two. The daemon (multi-user) install is the one that matches production: unprivileged users talk to **nix-daemon** over a socket; builds run as dedicated **nixbld** users; the store is owned by root.

Single-user installs (**~/ .nix-profile** only, no daemon) exist. Skip them on a shared workstation. They fight SELinux, they fight **/nix** permissions, and they do not match the NixOS mental model you will use later.

Flakes are still marked experimental in some docs. On this book's baseline (Nix 2.35+) they are the boring interface: **flake.nix** plus **flake.lock**, no ambient **<nixpkgs>** channel required.

```
curl installer
```

```
/nix created + nix-daemon enabled + your user in the trusted group
```

```
nix.conf: experimental-features = nix-command flakes
```

```
nix run nixpkgs#hello → "Hello, world!"
```

Worked examples

Case 1: Multi-user install on Linux

On a throwaway VM or a Linux workstation you control, run the official multi-user installer. Read the script. Then execute it:

```
curl -L https://nixos.org/nix/install -o nix-install.sh
less nix-install.sh
sh nix-install.sh --daemon
```

Output (abbreviated):

```
installing in multi-user mode...
alright! we're going to call sudo
copying Nix to /nix/store...
providing a `nix` command in your PATH...
setting up the nix-daemon systemd service...
Nix: just installed
```

Open a **new** login shell so PATH and NIX_PROFILES are set. Confirm:

```
nix --version
systemctl is-active nix-daemon
```

Output:

```
nix (Nix) 2.35.2
active
```

The daemon must be active. If it is not, `systemctl enable --now nix-daemon` and check `journalctl -u nix-daemon`.

Case 2: Enable flakes in nix.conf

The installer may already have turned flakes on. Check:

```
nix show-config | grep experimental-features
```

If the line does not include both `nix-command` and `flakes`, write them yourself.

On a multi-user install the file is `/etc/nix/nix.conf` (root). On a single-user leftover it is `~/.config/nix/nix.conf`. Prefer the system file:

```
# /etc/nix/nix.conf
experimental-features = nix-command flakes
```


Restart the daemon so the change is picked up:

```
sudo systemctl restart nix-daemon
```

`nix flake --help` must print help, not an “experimental feature disabled” error.

Case 3: First substitute from cache.nixos.org

You have not compiled anything. `nix run` should **download** a closure:

```
nix run nixpkgs#hello
```

Output:

```
copying path '/nix/store/...-hello-2.12.1' from 'https://cache.nixos.org'...
Hello, world!
```

If this compiles from source for minutes, your substituters are missing. Confirm:

```
nix show-config | grep substituters
```

<https://cache.nixos.org> must be listed. The matching public key is shipped with Nix; you do not paste it by hand.

Case 4: A pinned flake instead of a floating channel

`nix run nixpkgs#hello` follows whatever `nixpkgs` your registry currently points at. That is fine for a smoke test. For desk work, pin.

Save as `flake.nix` in an empty directory:

```
# flake.nix
{
  description = "Desk smoke test";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in
    {
      packages.${system}.default = pkgs.hello;
    };
}
```

If you are on aarch64, change `system` to "aarch64-linux" (or "aarch64-darwin" on Apple silicon).

```
nix flake lock
nix run
```

Output:

Hello, world!

`flake.lock` now pins the exact nixpkgs revision. Commit both files. The next machine that clones this directory gets the same `hello`.

Case 5: macOS (same daemon, different volume)

On macOS the installer creates an APFS volume for `/nix` (the root filesystem is signed and cannot hold `/nix` directly). Run the same official script with `--daemon`. After install:

```
nix --version
nix run nixpkgs#hello
```

Do not install Nix with Homebrew. Homebrew's `nix` package fights the daemon, the volume, and the store layout. One installer, one `/nix`.

The trap

The trap is `curl | sh` without reading, then a second installer on top. Mixing the official installer, the Determinate installer, and a distro package (`dnf install nix`, `apt install nix-bin`) produces two daemons, two `nix.conf` files, and a store the next `nix-collect-garbage` is afraid to touch.

Pick **one** installer and keep it:

Situation	Installer
Generic Linux VM, this book	Official multi-user (<code>--daemon</code>)
Team that already standardised on Determinate	Determinate, and only that
Fedora with a packaged <code>nix</code> you are required to use	The distro package, then enable flakes by hand

If you already have a broken mix:

```
# last resort: uninstall, then one clean install
sudo systemctl stop nix-daemon
# follow the uninstall steps for whichever installer you actually used
```

Do not `rm -rf /nix` while a daemon is running.

A second trap is enabling flakes in `~/.config/nix/nix.conf` while the daemon reads `/etc/nix/nix.conf`. Your shell says flakes are on; `nix-build` via the daemon says they are not. Put `experimental-features` in the file the **daemon** reads.

The boring rule

- Multi-user daemon install. One `/nix`. One `nix-daemon`.
- `experimental-features = nix-command flakes` in the daemon's `nix.conf`.
- Prove the install with `nix run nixpkgs#hello` substituting from `cache.nixos.org`, not compiling.
- Pin `nixpkgs` in a `flake.lock` as soon as you keep the directory.
- Do not stack installers. Do not install Nix via Homebrew.

Try this

1. Run `nix run nixpkgs#cowsay -- "desk online"` and confirm the NAR came from `cache.nixos.org` (the “copying path ... from” line).
2. In Case 4's directory, run `nix flake metadata` and write down the locked `nixpkgs` revision. Change nothing, run it on a second machine (or a container with Nix), and confirm the revision matches.
3. Comment out `experimental-features` in `nix.conf`, restart the daemon, and run `nix flake show`. Restore the line after you have seen the error.
4. Run `ls -ld /nix /nix/store` and note the owners. The store should not be writable by your ordinary user.

Installing NixOS

Installing NixOS

Nix the package manager is not NixOS. NixOS is a Linux distribution whose kernel, initrd, systemd units, users, and `/etc` are one derivation. The boring default is: **install once on a disposable VM, generate hardware config, write a tiny `configuration.nix`, and treat `nixos-rebuild switch` as the only way the machine changes.**

Do not practise on the only disk that holds your photographs.

Mental model

The installer ISO is a live NixOS. You boot it, partition a disk, generate a hardware snapshot, write a system expression, and run `nixos-install`. That produces generation 1. From then on you edit code and rebuild. You do not `apt install`. You do not `systemctl enable` by hand.

File	Role
<code>hardware-configuration.nix</code>	Disk UUIDs, file systems, kernel modules. Generated. Rarely edited.
<code>configuration.nix</code>	Hostname, users, packages, services, bootloader. You write this.
<code>/nix/var/nix/profiles/system</code>	Pointer at the current generation
Bootloader menu	Every previous generation you can roll back to

installer ISO

partition + mount → `/mnt` and `/mnt/boot`

`nixos-generate-config --root /mnt`

edit `/mnt/etc/nixos/configuration.nix`

`nixos-install` → reboot → generation 1

This book targets **NixOS 26.05+**. After install, `nixos-version` should agree.

Worked examples

Case 1: Disposable VM, not a daily driver

Create a UEFI VM:

Resource	Starting point
Firmware	UEFI
RAM	4 GB (8 GB is more comfortable)
Disk	40 GB
CPU	2 cores

Download the graphical or minimal NixOS 26.05 ISO from the NixOS download page. Attach it. Boot. You are in a live environment with a root shell (minimal ISO) or a desktop (graphical ISO).

Check firmware:

```
ls /sys/firmware/efi
```

If that directory exists, you are in UEFI mode. Match the VM firmware to the bootloader you will enable (`systemd-boot` for UEFI).

Case 2: Partition, mount, generate config

On a single empty virtual disk `/dev/vda` (names vary: `vda`, `sda`, `nvme0n1`):

```
lsblk
```

A boring GPT layout for UEFI:

Partition	Size	Type	Mount
<code>/dev/vda1</code>	1 GB	EFI System	<code>/mnt/boot</code>
<code>/dev/vda2</code>	rest	Linux filesystem	<code>/mnt</code>

Using `parted` and `mkfs` by hand is acceptable for this first machine. `Disko` (later in the book) replaces this for fleets. For one VM:

```
parted /dev/vda -- mklabel gpt
parted /dev/vda -- mkpart ESP fat32 1MiB 1GiB
parted /dev/vda -- set 1 esp on
parted /dev/vda -- mkpart primary ext4 1GiB 100%

mkfs.fat -F32 /dev/vda1
mkfs.ext4 /dev/vda2
```

```
mount /dev/vda2 /mnt
mkdir -p /mnt/boot
mount /dev/vda1 /mnt/boot
```

Generate the hardware snapshot:

```
nixos-generate-config --root /mnt
ls /mnt/etc/nixos/
```

Output:

```
configuration.nix
hardware-configuration.nix
```

Open `hardware-configuration.nix`. You should see the file-system UUIDs for `/` and `/boot`. Do not invent UUIDs. Leave this file as generated.

Case 3: A tiny first `configuration.nix`

Replace the generated `configuration.nix` with something you can read in one screen. Save as `/mnt/etc/nixos/configuration.nix`:

```
# configuration.nix
{ config, pkgs, ... }:

{
  imports = [ ./hardware-configuration.nix ];

  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;

  networking.hostName = "desk-vm";
  time.timeZone = "UTC";
  i18n.defaultLocale = "en_US.UTF-8";

  users.users.deskadmin = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    initialPassword = "changeme";
  };

  environment.systemPackages = with pkgs; [
    git
    vim
    curl
  ];
}
```

```
services.openssh.enable = true;

system.stateVersion = "26.05";
}
```

`stateVersion` is **not** “the release I want to be on.” It is “the release whose defaults this machine was born with.” Leave it at the release you installed. Changing it later without reading the release notes is how databases and state dirs break.

Set a real password after first login (`passwd`). `initialPassword` is a lab convenience; do not ship it.

Install and reboot:

```
nixos-install
reboot
```

Log in as `deskadmin`. Confirm:

```
nixos-version
hostname
```

Output:

```
26.05.9440.6aefcda (Yarara)
desk-vm
```

Case 4: The first rebuild on the installed system

You are now on the machine. Edit `/etc/nixos/configuration.nix` (or, once you move to a flake, the flake in git). Add `ripgrep` to `environment.systemPackages` and activate:

```
sudo nixos-rebuild switch
```

Output:

```
building the system configuration...
updating GRUB/systemd-boot...
activating the configuration...
setting up /etc...
reloading user units...
```

```
which rg
rg --version
```

`rg` is on `PATH`. It came from the new generation, not from a manual download.

List generations:


```
sudo nix-env --list-generations --profile /nix/var/nix/profiles/system
```

You should see generation 1 (install) and generation 2 (ripgrep). The bootloader menu lists both.

Case 5: Rollback without a rescue disk

Break the machine on purpose in the VM. In `configuration.nix`, set a nonsense timezone or remove `boot.loader.systemd-boot.enable` — something that still **builds** but is wrong — or, safer, add a package, switch, then:

```
sudo nixos-rebuild switch --rollback
```

Output:

```
switching to generation 1
activating the configuration...
```

`rg` is gone. The bootloader default is generation 1 again. If a rebuild leaves the system unbootable, pick the previous generation in `systemd-boot` (hold Space at firmware, then select the older NixOS entry).

The trap

The trap is **mutating the live system after install**. `passwd` for the first login is fine. `useradd`, `systemctl enable nginx`, `curl | sh` into `/usr` are not. Those changes vanish on the next rebuild, or worse, they survive in `/etc` as unmanaged files that fight the next activation.

A second trap is editing `hardware-configuration.nix` to “clean it up.” The file is ugly because hardware is ugly. If you replace UUIDs with `/dev/vda2`, the next disk reorder will fail to boot.

A third trap is treating `stateVersion` as an upgrade knob. Upgrade by changing the flake input (or channel) to a new release and reading the release notes. Keep `stateVersion` at the original value unless a release note tells you to bump a specific option.

The boring rule

- Practise on a VM. UEFI. One ESP, one root.
- `nixos-generate-config` owns `hardware-configuration.nix`. You own `configuration.nix`.
- `system.stateVersion` is the birth release. Leave it.
- After install, the only mutation path is `nixos-rebuild switch` (or `test`, or `boot`).
- Know `--rollback` and the bootloader menu before you need them.

Try this

1. After Case 4, run `sudo nixos-rebuild test` with a hostname change. Confirm `hostname` changed, reboot, and confirm it reverted (test does not update the bootloader default).
2. Run `sudo nixos-rebuild dry-activate` and read which units would restart. No change should be applied.
3. Add `cowsay` to `environment.systemPackages`, switch, run `cowsay desk`, then rollback and confirm `cowsay` is missing.
4. Open `hardware-configuration.nix` and write down the UUID of `/`. Compare with `lsblk -f`. They must match.

Nix Fundamentals

Store Model

The `/nix/store` Anatomy

The `/nix/store` Anatomy

Everything Nix builds, downloads, or installs lands in `/nix/store`. The boring default is: **treat the store as a read-only database of paths named by hash, and never move, rename, or chmod anything inside it.**

Mental model

Every store path looks like this:

```
/nix/store/<hash>--<name>--<version>
```

Example:

```
/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0
```

Piece	Meaning
Hash	32 characters of Nix base32. Identity of this path.
Name + version	Human label. Not identity. Two <code>ripgrep-14.1.0</code> builds can coexist.
The directory	A complete artifact: <code>bin/</code> , <code>lib/</code> , <code>share/</code> , or a single file.

The hash is computed from the **inputs** of the derivation (the usual case) or from the **contents** of the output (fixed-output and content-addressed paths). Either way, a different hash is a different path. Nothing is overwritten.

Permissions are read-only. Timestamps are the Unix epoch (Jan 1 1970) so two machines producing the same bytes do not disagree about mtime.

```
/nix
  store/      immutable artifacts
  var/nix/
    db/       SQLite index of those artifacts
    profiles/ generations (GC roots)
    gcroots/  extra roots (result, direnv, ...)
```

Your shell's `rg` is not “the ripgrep in `/usr/bin`”. It is a symlink into a specific store path.

Worked examples

Case 1: Follow a binary into the store

```
which rg
ls -l $(which rg)
ls -ld $(dirname $(which rg))/..
```

Output (hashes will differ on your machine):

```
/run/current-system/sw/bin/rg
lrwxrwxrwx 1 root root 68 Jan  1  1970 /run/current-system/sw/bin/rg -> /nix/store/z1k947f6y788
dr-xr-xr-x 1 root root 4096 Jan  1  1970 /nix/store/z1k947f6y788sfaivs931h26hsyb3506-ripgrep-14
```

On a Nix-on-Linux user profile the first hop may be `~/.nix-profile/bin/rg` instead of `/run/current-system`. The last hop is always `/nix/store/...`

Case 2: Read-only and epoch timestamps

```
ls -ld $(readlink -f $(which nix))
```

Output:

```
-r-xr-xr-x 1 root root 15728640 Jan  1  1970 /nix/store/7vqw9388aaaaaaaaaaaaaaaaaaaaaaa-nix-2.
```

- Mode `r-xr-xr-x` (or `r-xr-xr-x` on the directory `r-xr-xr-x`): no write bit.
- Date `Jan 1 1970`: not “this was built at the epoch”; the timestamp is **normalised** so it is not an input to the next hash.

Try to touch it:

```
touch $(readlink -f $(which nix))
```

Output:

```
touch: cannot touch '/nix/store/...-nix-2.35.2/bin/nix': Permission denied
```

Case 3: Two versions, no collision

```
nix build nixpkgs#hello --out-link hello-a
ls -ld hello-a
```

Output:

```
lrwxrwxrwx 1 deskadmin deskadmin 55 Jan  1 12:00 hello-a -> /nix/store/...-hello-2.12.1
```

`hello-a` is a **GC root** in the current directory (the **result**-style symlink). The store path is the real artifact. Installing a different `nixpkgs` revision of `hello` produces a second directory next to the first. Both stay until nothing roots them.

Case 4: What is *not* in the store

```
ls /usr/bin/rg 2>&1 || true
echo $PATH | tr ':' '\n' | head
```

On NixOS, `/usr/bin` is nearly empty (`env`, maybe `sh`). Tools live under `/run/current-system/sw/bin` and `/nix/store`. On Nix-on-Linux, `/usr/bin` still has the host distro; Nix tools are extra entries on `PATH`. Mixing them is how “it works in my shell” happens. Prefer the store path.

Case 5: Name a path without guessing the hash

```
nix path-info nixpkgs#hello
```

Output:

```
/nix/store/...-hello-2.12.1
```

`nix path-info -Sh nixpkgs#hello` adds closure size. You do not construct store paths by concatenating strings in a shell script.

The trap

The trap is `sudo chmod -R u+w /nix/store/...` so you can **patch a file**. The path’s hash promised those bytes. After you write to it, every later build that trusted that path is lying. The daemon may also refuse to register the path on `--verify`.

If a binary is wrong, rebuild it with different inputs. You get a new hash. The old path stays until GC.

A related trap: copying a store path to another machine with `scp` and dropping it under `/nix/store` by hand. Use `nix copy` so the database is updated. A directory that is not registered in `db.sqlite` is invisible to Nix and eligible for nothing useful.

The boring rule

- All Nix artifacts live under `/nix/store/<hash>-<name>`.
- Follow `which` until you see `/nix/store`. That is the real path.
- Never `rm`, `mv`, `chmod`, or `touch` inside the store.
- Use `nix path-info / nix-store -q` to name paths. Do not paste hashes from memory.
- Treat `result` and profile symlinks as pointers, not as the software.

Try this

1. Run `readlink -f $(which bash)` (or `$(which sh)`) and count how many symlink hops precede `/nix/store`.
2. `ls /nix/store | wc -l` — that is how many artifacts this machine currently holds. Run it again after `nix run nixpkgs#cowsay -- hi` and see the count move.
3. `stat $(readlink -f $(which nix))` and confirm `Access: (0555)` (or similar, no write) and a 1970 timestamp.
4. Try `echo x >> $(readlink -f $(which nix))` and save the exact error. That error is the product.

Content-Addressable Paths

Content-Addressable Paths

Most Nix store paths you meet are **input-addressed**: the hash is a digest of the derivation (builder, env, input paths), not of the bytes that came out. **Content-addressed** (CA) paths hash the output itself. The boring default is: **understand the difference, keep producing ordinary input-addressed builds, and let fixed-output derivations (fetchers) be the CA paths you actually use every day.**

Mental model

Kind	Hash comes from	Same output, different builder?
Input-addressed	The <code>.drv</code> (inputs, builder, env)	Two different store paths
Content-addressed / FOD	The output bytes (and hash mode)	Same store path

Input-addressed is why a comment in a build script rebuilds everything downstream: the `.drv` changed, so the output path changed, even if `gcc` emitted identical bytes.

Fixed-output derivations (the `fetchurl` / `fetchFromGitHub` family) are the everyday CA case. You declare `hash = "sha256-..."`; Nix may use the network; it then **checks** the bytes against that hash. The store path is a function of the hash, not of the URL. Change the URL to a mirror, keep the hash, keep the path.

Experimental CA derivations for *ordinary* builds exist (`contentAddressed` in Nix). They are not the boring default for a desk workstation. The platform-engineering internals chapter covers them later.

Input-addressed:

```
.drv    hash    /nix/store/AAAA-desk-api
          (even if bytes == yesterday's BBBB-desk-api)
```

Fixed-output:

```
declared hash    /nix/store/HHHH-source.tar.gz
(URL is just how we get the bytes; the hash is identity)
```

Worked examples

Case 1: Input hash changes when the builder changes

Save as `banner-a.nix`:

```
# banner-a.nix
let
  pkgs = import <nixpkgs> {};
in
derivation {
  name = "desk-banner";
  system = builtins.currentSystem;
  builder = "${pkgs.bash}/bin/bash";
  args = [ "-c" "echo 'Desk System Online' > $out" ];
}
```

Save as `banner-b.nix` — same bytes out, extra comment in the script:

```
# banner-b.nix
let
  pkgs = import <nixpkgs> {};
in
derivation {
  name = "desk-banner";
  system = builtins.currentSystem;
  builder = "${pkgs.bash}/bin/bash";
  args = [ "-c" "echo 'Desk System Online' > $out # lab note" ];
}
```

```
nix-instantiate banner-a.nix
nix-instantiate banner-b.nix
```

Output:

```
/nix/store/aaaaaaaaaaaaaaaaaaaaaaaaaaaaa-desk-banner.drv
/nix/store/bbbbbbbbbbbbbbbbbbbbbbbbbbb-desk-banner.drv
```

Two `.drv` files. Two future output paths. The echo is the same; the input is not.

Case 2: Store paths are immutable, not “updated in place”

```
touch $(readlink -f $(which nix)) 2>&1 || true
```

Output:

```
touch: cannot touch '/nix/store/...-nix-2.35.2/bin/nix': Permission denied
```

Nix never overwrites `AAAA-desk-banner` with new bytes. It builds `CCCC-desk-banner` and swings a profile symlink. That is input-addressing plus immutability working together.

Case 3: Fixed-output identity is the hash

Save as `fod.nix`:

```
# fod.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.fetchurl {
  url = "https://cache.nixos.org/nix-cache-info";
  hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
}

nix-build fod.nix
```

The build **fails** (unless you lucked into that hash). The error is the point:

```
error: hash mismatch in fixed-output derivation '/nix/store/...-nix-cache-info':
  specified: sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=
    got:     sha256-<the real hash>
```

Paste the `got` value into `hash`, rebuild, and the path is now determined by those bytes. Point `url` at a different mirror of the **same** file and the store path stays put.

Case 4: Why CA helps a cache

If two CI jobs fetch the same tarball with the same hash, they share one store path. If two CI jobs compile the same C file with two different comment-only builder changes, input-addressed Nix stores two outputs even when `cmp` would say the binaries match.

That waste is real. It is also cheaper than debugging “why did the binary change when the source didn’t” because a timestamp leaked into an input. Boring teams live with input-addressed builds and pin fetch hashes.

Case 5: Inspect which kind a path is

```
nix path-info --json $(nix-build '<nixpkgs>' -A hello --no-out-link) | head
```

Look for `ca` / `narHash` fields when present. Ordinary `hello` from `nixpkgs` is input-addressed. A `fetchurl` output is fixed-output. You do not need to memorise JSON; you need to know which family you are looking at when a cache misses.

The trap

The trap is **turning off the hash** (`hash = ""`; or `sha256 = lib.fakeSha256` left in production) so the fetch “just works.” The next time GitHub retags, or a mirror is replaced, you silently take new bytes at the same URL. The whole point of a FOD is the mismatch error.

The other trap is enabling experimental CA derivations on a laptop because a blog post promised automatic sharing. Keep CA-for-builds in the lab until the whole team agrees. Fetchers already gave you the CA that matters.

The boring rule

- Everyday builds are input-addressed. A `.drv` change is a new path, even if the bytes look the same.
- Fetchers are fixed-output. Identity is the **hash you wrote**.
- Never leave a dummy hash in a committed file.
- Do not `chmod` the store to “fix” a path. Build a new one.
- Treat experimental content-addressed builds as optional, not as the desk default.

Try this

1. Change only the `name` in `banner-a.nix` from `desk-banner` to `desk-banner-v2`, instantiate, and confirm the `.drv` hash moved.
2. Run Case 3, copy the `got` hash, rebuild, then run `nix path-info -Sh result` and note the size.
3. Flip one character of the working hash and read the mismatch error. That error is success: Nix refused the bytes.
4. Explain in one sentence, to a colleague, why `hello` from `nixpkgs` and a `fetchurl` tarball are hashed differently.

Hashes and Dependencies

Cryptographic Hashes and Inputs

Cryptographic Hashes and Inputs

Two machines produce the same store path only if they evaluate the same derivation. The boring default is: **every input — source, compiler, flags, patches, build script — is part of the hash, and a one-bit change is a new path, not an in-place upgrade.**

Mental model

Traditional package managers treat `ripgrep-14.1.0` as identity. It is a label. A `rg` built against `glibc 2.38` is not the same program as a `rg` built against `glibc 2.39`.

Nix hashes the `.drv` file: a Nix-language serialisation of the builder, environment, and input store paths. The output path is `/nix/store/<hash-of-that-drv>-<name>`.

`sources + toolchain + env + builder`

`.drv file` `SHA` `output path`

`sandbox build` `bytes at that path`

If the build is allowed to see the network, the time of day, or `/usr/bin/cc`, the hash is a lie. The sandbox exists so the hash means something.

Worked examples

Case 1: Instantiate before you build

Save as `sample.nix`:

```
# sample.nix
let
  pkgs = import <nixpkgs> {};
in
derivation {
  name = "desk-sample";
  system = builtins.currentSystem;
  builder = "${pkgs.bash}/bin/bash";
```

```
args = [ "-c" "echo ok > $out" ];
}
```

```
nix-instantiate sample.nix
```

Output:

```
/nix/store/7vqw...-desk-sample.drv
```

No compiler ran. The hash is already decided. `cat` that `.drv` (it is text) and you will see the builder path, name, and system.

Case 2: One flag, new path

Save as `opt.nix`:

```
# opt.nix
{ cflags ? "-O2" }:
let
  pkgs = import <nixpkgs> {};
in
derivation {
  name = "desk-opt";
  system = builtins.currentSystem;
  builder = "${pkgs.bash}/bin/bash";
  inherit cflags;
  args = [ "-c" "echo $cflags > $out" ];
}
```

```
nix-instantiate opt.nix
nix-instantiate --argstr cflags -O3 opt.nix
```

Output:

```
/nix/store/aaaa...-desk-opt.drv
/nix/store/bbbb...-desk-opt.drv
```

`-O2` versus `-O3` is two derivations. Both can live on disk. Nothing was upgraded in place.

Case 3: The compiler is an input

```
nix-instantiate '<nixpkgs>' -A hello
nix-store -q --references $(nix-instantiate '<nixpkgs>' -A hello)
```

Output (abbreviated):

```
/nix/store/...-hello-2.12.1.drv
/nix/store/...-stdenv-linux.drv
/nix/store/...-source
...
```

hello does not hash “a C compiler.” It hashes **this** stdenv, which hashes **this** gcc, which hashes **this** source tarball. That is why a nixpkgs bump rebuilds the world: gcc moved, so everything that listed gcc as an input moved.

Case 4: Purity — the sandbox cannot see /usr

Save as `impure.nix`:

```
# impure.nix
let
  pkgs = import <nixpkgs> {};
in
derivation {
  name = "desk-impure";
  system = builtins.currentSystem;
  builder = "${pkgs.bash}/bin/bash";
  args = [ "-c" "/usr/bin/id > $out" ];
}

nix-build impure.nix
```

Output:

```
building '/nix/store/...-desk-impure.drv'...
/bin/bash: line 1: /usr/bin/id: No such file or directory
error: builder for '/nix/store/...-desk-impure.drv' failed
```

The sandbox does not contain `/usr`. That failure is the hash doing its job: if `/usr/bin/id` had been used, two machines with different `id` binaries would still have produced the “same” store path.

Case 5: Pin the input set with a lockfile

```
nix flake new ./desk-pin
cd desk-pin
nix flake lock
nix hash path flake.lock
```

`flake.lock` records the git revision and NAR hash of `nixpkgs`. Two laptops with the same lock file instantiate the same `.drv` hashes. Without the lock, `nixpkgs` is a moving label.

The trap

The trap is `nix-build --impure` (or `builtins.currentTime`, or `filterSource` that includes `.git`) so a “quick test” leaks host state into the hash. The path then differs between CI and a laptop for reasons that do not show up in `git diff`.

A second trap is trusting version numbers in `pname` / `version` as identity. They are labels inside the path. The hash is identity. Rename freely; never assume 1.0.0 means the same bytes as last week’s 1.0.0.

The boring rule

- The `.drv` is hashed **before** the build. Instantiate to see identity.
- Toolchains, flags, and sources are inputs. Changing any of them is a new path.
- Keep the sandbox on. A build that needs `/usr` is not a Nix build.
- Pin `nixpkgs` (flake lock or a pinned fetch). Do not build against a floating channel on a desk that ships artifacts.
- Version strings are labels. Hashes are names.

Try this

1. Add an environment variable `desk = "window-a";` to `sample.nix`, instantiate, and confirm the `.drv` hash changed even though `echo ok` did not.
2. Run `nix-store --query --binding builder $(nix-instantiate sample.nix)` and confirm it points at `bash` in `/nix/store`, not `/bin/bash`.
3. Build `opt.nix` twice (`-02` and `-03`), then `diff -u $(nix-build opt.nix) $(nix-build --argstr cflags -03 opt.nix)`.
4. Search a `.drv` with `grep gcc $(nix-instantiate '<nixpkgs>' -A hello)` and note that `gcc` appears as a store path, not as the word “gcc”.

Deterministic Dynamic Linking and RPATH

Deterministic Dynamic Linking and RPATH

On a conventional Linux system, `rg` asks the dynamic linker for `libc.so.6` and takes whatever `/lib` currently holds. A glibc upgrade then changes every running binary at once. The boring default is: **bake the exact store path of every needed library into the ELF RUNPATH, and leave `LD_LIBRARY_PATH` alone.**

Mental model

When Nix links a binary it sets `DT_RUNPATH` (or older `DT_RPATH`) to store directories:

ELF header

```
RUNPATH = /nix/store/7r44p...-glibc-2.39/lib:/nix/store/...-gcc-.../lib
```

`ld.so` looks **there**, not in `/lib`, not in `/usr/lib`. Two `rg` binaries can sit on the same machine, each with its own glibc, and neither notices the other.

Reference scanning then records those store paths as **runtime references**. Garbage collection will not delete glibc while a rooted `rg` still points at it.

```
rg --RUNPATH--> glibc-2.39/lib
    --references (db.sqlite) --> same glibc path
```

Worked examples

Case 1: Read RUNPATH off a real binary

```
readelf -d $(readlink -f $(which rg)) | grep -E 'RPATH|RUNPATH'
```

Output:

```
0x000000000000001d (RUNPATH)
```

```
Library runpath: [/nix/store/7r44p12l4b72j9m9l09k96ygh
```

If `rg` is a wrapper script, follow it with `readlink -f` or inspect `$(nix path-info nixpkgs#ripgrep)/bin/rg`.

Case 2: Which libraries would actually load

```
ldd $(readlink -f $(which rg))
```

Output (abbreviated):

```
linux-vdso.so.1 (0x00007ffd...)
libgcc_s.so.1 => /nix/store/...-xgcc-...-libgcc/lib/libgcc_s.so.1
libc.so.6 => /nix/store/...-glibc-2.39/lib/libc.so.6
/nix/store/...-glibc-2.39/lib/ld-linux-x86-64.so.2
```

Every => target is under /nix/store. Nothing resolves to /lib/x86_64-linux-gnu.

Case 3: The interpreter is a store path too

```
readelf -l $(readlink -f $(which rg)) | grep interpreter
```

Output:

```
[Requesting program interpreter: /nix/store/...-glibc-2.39/lib/ld-linux-x86-64.so.2]
```

The kernel loads **that** ld-linux, not /lib64/ld-linux-x86-64.so.2. This is why a Nix binary copied to a machine without /nix dies with No such file or directory even though the file is there: the interpreter path is missing.

Case 4: Runtime closure matches RUNPATH

```
nix-store -q --references $(readlink -f $(which rg)) | xargs dirname | xargs dirname
```

or, cleaner:

```
nix path-info -r $(nix path-info nixpkgs#ripgrep) | head
```

You should see glibc (and a few others), not gcc, not the source tarball. Compilers are build-time; RUNPATH is runtime.

Case 5: LD_LIBRARY_PATH punches a hole

```
LD_LIBRARY_PATH=/usr/lib ldd $(readlink -f $(which rg)) | head
```

Depending on the host, ldd may now show /usr/lib for some libraries. That is a **different program** than the one Nix hashed. It may work today and segfault after a distro upgrade.

Nix wrappers sometimes set LD_LIBRARY_PATH for a specific package that was not patched. That wrapper is part of the derivation. Your shell profile setting a global LD_LIBRARY_PATH is not.

The trap

The trap is **exporting `LD_LIBRARY_PATH` in `.bashrc` to make a vendor binary start**. Every Nix binary in that shell now prefers the vendor's `libssl.so`. The next OpenSSL update on the host becomes a Nix outage.

The fix for vendor binaries is `nix-ld` or `buildFHSEnv` (later chapter), not a global search path.

A second trap is `patchelf --set-rpath /usr/lib` on a store path. The path is read-only, and even if you copy it out, you have left the closure Nix knows about.

The boring rule

- Trust `RUNPATH`. Inspect it with `readelf -d` and `ldd`.
- Runtime references in the Nix database should match those libraries.
- Do not set global `LD_LIBRARY_PATH`, `LD_PRELOAD`, or `DYLD_*` to “fix” Nix.
- A binary copied off the store needs its interpreter and `RUNPATH` closure (`nix copy`, not `scp bin/rg`).
- Vendor ELF files get `nix-ld` or an FHS env, not a profile hack.

Try this

1. Compare `readelf -d $(which curl)` on a Nix `curl` versus, if you have one, `/usr/bin/curl`. Note where `libc` lives in each.
2. Run `nix path-info -Shr nixpkgs#ripgrep` and find `glibc` in the closure. Confirm the same hash appears in `RUNPATH`.
3. `file $(readlink -f $(which rg))` — confirm it is dynamically linked. (A few Nix packages are static; they have no `RUNPATH`. That is also fine.)
4. In a subshell, `export LD_LIBRARY_PATH=/does/not/exist` and run `rg --version`. Then unset it. If something fails only with the export, you have felt the hole.

Dependency Graphs

Dependency Closures and Graphs

Dependency Closures and Graphs

A store path is never alone. The boring default is: **know the runtime closure (what you must copy to production) versus the build-time closure (what the sandbox needed), and measure both with `nix-store` / `nix path-info` before you ship.**

Mental model

Nix records **direct references** by scanning output files for the 32-character hash of other store paths. If `rg` contains the string `/nix/store/7r44p...-glibc-2.39`, `glibc` is a runtime reference. `gcc` was used to compile `rg` but does not appear in the binary, so it is **not** in the runtime closure.

Term	Command	What you get
Direct references	<code>nix-store -q --references P</code>	Immediate edges out of P
Referrers	<code>nix-store -q --referrers P</code>	Paths that point at P
Runtime closure	<code>nix-store -q --requisites P</code>	P plus everything it needs to run
Deriver	<code>nix-store -q --deriver P</code>	The <code>.drv</code> that built P

build-time: `gcc, headers, cmake, fetchers`
(sandbox only)

output: `rg references glibc, libgcc, ...`

runtime closure = `rg` + those references, transitively

Deploy the runtime closure. Do not deploy `gcc` because you compiled on this machine.

Worked examples

Case 1: Runtime closure of `ripgrep`

```
nix-store -q --requisites $(nix path-info nixpkgs#ripgrep)
```

Output (typical shape):

```
/nix/store/7r44p12l4b72j9m9l09k96yghz02f8n7-glibc-2.39
/nix/store/...-libgcc-...
/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0
```

A handful of paths. Not a compiler toolchain.

Case 2: Sizes, not just names

```
nix path-info -Shr nixpkgs#ripgrep
```

Flags:

Flag	Meaning
-S	Size of this path
-h	Human units
-r	Closure (requisites)

The last line is the closure you would copy with `nix copy`. If that number is hundreds of megabytes for a “small” CLI, a wrapper script has pulled in Python or glib.

Case 3: Direct references versus the whole tree

```
echo 'references:'
nix-store -q --references $(nix path-info nixpkgs#ripgrep)
echo 'requisites:'
nix-store -q --requisites $(nix path-info nixpkgs#ripgrep)
```

References are one hop. Requisites are the transitive closure. For `rg` they are often almost the same. For a desktop environment they are not.

Case 4: Why gcc is missing from runtime

```
nix-store -q --deriver $(nix path-info nixpkgs#hello)
nix-store -q --references $(nix-store -q --deriver $(nix path-info nixpkgs#hello))
```

The `.drv` references `stdenv` (`gcc`, `binutils`, ...). The **output** of `hello` does not. Reference scanning dropped the toolchain. That is the entire point of separate build-time and runtime graphs.

Case 5: Copy what production needs

```
nix copy --to file://$PWD/desk-nar nixpkgs#ripgrep
ls desk-nar
```

The NAR cache contains the runtime closure, not gcc. A colleague can `nix copy --from file://$PWD/desk-nar` and run `rg` without compiling.

On a live cache this is `nix copy --to ssh://desk-cache` or a Cachix push. Same graph.

The trap

The trap is **shipping result by scp** — one directory — and discovering the binary dies on the server because `glibc` is not there. **result** is a symlink to **one** path. The closure is that path plus requisites.

The other trap is reading `--requisites` on a `.drv` and panicking at the size. A `.drv`'s requisites include build-time inputs. Query the **output** path (`nix path-info nixpkgs#ripgrep`), not the `deriver`, when you mean “what runs.”

The boring rule

- Runtime closure = `--requisites` of the **output** path.
- Measure with `nix path-info -Shr` before you call a package “small.”
- Deploy with `nix copy` (or a binary cache), never with a single `scp` of `bin/`.
- Build-time tools belong in the `.drv`, not in production.
- If a runtime closure contains a compiler, a wrapper or `propagatedBuildInputs` leaked. Fix the package.

Try this

1. Compare `nix path-info -Sh nixpkgs#hello` (one path) with `nix path-info -Shr nixpkgs#hello` (closure). Write down both numbers.
2. Run `nix-store -q --graph $(nix path-info nixpkgs#hello)` and pipe to `head`. You should see `hello` and `glibc` nodes.
3. `nix-store -q --referrers $(nix path-info nixpkgs#glibc | head -1)` — many packages point at `glibc`. That is why it survives GC.
4. Intentionally `scp` only the `hello` store directory to a `tmp` dir and try to run it with `./bin/hello`. Then explain the interpreter error using the `RPATH` chapter.

The Nix Database (db.sqlite)

The Nix Database (db.sqlite)

`/nix/store` is a directory tree. Nix does not discover it by walking the tree on every command. The boring default is: **the daemon owns `/nix/var/nix/db/db.sqlite`, and you query it with `nix-store / nix path-info`, never with `sqlite3`.**

Mental model

The database answers three questions:

1. Is this path **valid** (registered, complete, not half-copied)?
2. What does it **reference**, and what **refers to** it?
3. Which **.drv** **derived** it?

<code>/nix/store/</code>	bytes
<code>/nix/var/nix/db/</code>	index of those bytes
<code>db.sqlite</code>	
<code>schema</code>	

If you copy files into `/nix/store` with `cp`, the database does not know them. `nix-store --query` will not list them. GC will not preserve them. The next `--verify` may delete them as garbage.

The daemon holds a lock on the database. That is why two `nix-build` processes can run at once without corrupting registration.

Worked examples

Case 1: Referrers — who keeps this path alive?

```
nix-store -q --referrers $(nix path-info nixpkgs#ripgrep)
```

Output (shape):

```
/nix/store/z1k947f6y788sfaivs931h26hsvb3506-ripgrep-14.1.0
/nix/store/...-system-path
```

A path with **no** referrers and **no** GC root is collectable. `ripgrep` itself appearing in the list is normal (a path is sometimes listed relative to how you queried). Profiles and `result` symlinks show up as referrers once they point here.

Case 2: Valid path check

```
nix-store --query --hash $(nix path-info nixpkgs#hello)
nix-store --verify --check-contents 2>&1 | tail
```

`--verify --check-contents` rehashes every registered path. It is slow; it is the tool you run after a disk scare, not every morning.

Output on a healthy store:

```
reading the Nix database...
checking path existence...
checking hashes...
```

No “path is corrupted” lines.

Case 3: Deriver link

```
nix-store -q --deriver $(nix path-info nixpkgs#hello)
```

Output:

```
/nix/store/...-hello-2.12.1.drv
```

Substituted paths (downloaded from a cache, never built here) may report `unknown-deriver`. That is not corruption. The bytes arrived without the `.drv`.

Case 4: Registration is why `nix copy` exists

```
mkdir -p /tmp/not-a-store
cp -a $(nix path-info nixpkgs#hello) /tmp/not-a-store/
nix-store -q --requisites /tmp/not-a-store/hello-* 2>&1 || true
```

Output:

```
error: path '/tmp/not-a-store/...' is not valid
```

The bytes exist. Nix does not care. `nix copy --to file:///tmp/desk-cache nixpkgs#hello` registers them in that cache’s database.

Case 5: Where the file lives (look, don't poke)

```
sudo ls -l /nix/var/nix/db/
```

Output:

```
-rw-r--r-- 1 root root ... db.sqlite  
-rw-r--r-- 1 root root ... schema
```

Owned by root. On a multi-user install your account talks to the daemon; the daemon talks to SQLite. You do not.

The trap

The trap is **opening db.sqlite in a GUI and deleting a row** to “free space.” The store directory and the database then disagree. `--verify` will complain; GC may delete the wrong thing; the daemon may refuse to start.

The other trap is `sudo rm -rf /nix/store/some-hash-...` because `du` scared you. The database still lists the path as valid. The next build that expected those bytes fails in a way that looks like a nixpkgs bug.

Use `nix-collect-garbage`. Use `nix-store --delete` only on a path with zero referrers, and only when you mean it.

The boring rule

- Query with `nix-store -q` and `nix path-info`. Never write SQL against `db.sqlite`.
- Paths that are not registered are not Nix paths, even if they sit under `/nix/store`.
- After filesystem damage, `nix-store --verify --check-contents`. Then restore from a cache, do not patch the DB.
- Substituted artifacts may lack a deriver. That is normal.
- Delete via GC, not `rm`.

Try this

1. `nix-store -q --referrers-closure $(nix path-info nixpkgs#hello)` and estimate how much of your profile depends on `hello` (probably little unless you installed it).
2. Run `nix-store --verify` without `--check-contents` (fast existence check). Then decide whether you need the slow rehash.
3. `nix path-info --json nixpkgs#hello` and identify `narHash`, `references`, and `deriver` in the JSON.
4. Attempt `nix-store --delete $(nix path-info nixpkgs#glibc)` and read the “cannot delete path because it is still alive” error. That error is the database doing its job.

Garbage Collection

GC Roots and Space Reclamation

GC Roots and Space Reclamation

Nix never overwrites a path, so `/nix/store` grows until something says “this is still needed.” The boring default is: **roots keep closures alive; `nix-collect-garbage` deletes everything that is not reachable from a root; `-d` is how you drop old generations on purpose.**

Mental model

A **GC root** is a pointer *outside* the store (or a special symlink inside `gcroots/`) that names a store path. The collector starts from roots and walks `--referrers` in reverse: anything unreachable is garbage.

Common roots:

Root	Typical path	Keeps alive
Current NixOS system	<code>/run/current-system</code> and <code>/nix/var/nix/profiles/system</code>	The running OS plus old generations
User profile	<code>~/.nix-profile</code>	<code>nix profile</code> / Home Manager user env
Home Manager generations	<code>~/.local/state/nix/profiles/home-manager</code>	<code>home-manager switch</code> results
<code>result</code> symlink	<code>./result</code> → registered in <code>gcroots/auto</code>	The last <code>nix-build</code> in that directory
<code>direnv</code>	<code>.direnv/flake-profile</code>	The cached devShell
Channels / flake registry	<code>~/.nix-defexpr</code> , registries	Pin data, not usually huge

`roots` → `profiles` / `result` / `current-system`

walk references

keep those paths; delete the rest

`nix-collect-garbage` without `-d` deletes **orphans** (paths with no root) but **keeps generations**. `-d` deletes old generations first, then orphans. That is the difference between “I freed 80 MB” and “I freed 12 GB.”

Worked examples

Case 1: See the roots

```
ls /nix/var/nix/gcroots
ls -l /nix/var/nix/gcroots/auto 2>/dev/null | head
nix-store --gc --print-roots 2>/dev/null | head
```

You should recognise `result`, `profiles`, and `/run/current-system` (on NixOS).

Case 2: A result symlink is a root

```
nix build nixpkgs#hello --out-link /tmp/hello-result
nix-store --gc --print-roots | grep hello-result
```

Output:

```
/tmp/hello-result -> /nix/store/...-hello-2.12.1
```

Delete the symlink, then collect:

```
rm /tmp/hello-result
nix-collect-garbage
```

Output:

```
finding garbage collector roots...
deleting garbage...
... store paths deleted, ... MiB freed
```

If `hello` is still in your profile, it will **not** be deleted. Only the extra root is gone.

Case 3: Safe collect versus drop generations

```
nix-collect-garbage
```

Keeps every NixOS / Home Manager generation. Safe on a Monday morning.

```
nix-collect-garbage --delete-older-than 14d
```

Drops generations older than fourteen days, then deletes newly orphaned paths. Prefer this over `-d` on a machine you might still want to roll back.

```
sudo nix-collect-garbage -d
```

Drops **all** old generations of all profiles. The running system and the current user profile stay. Rollback targets vanish. Use it when disk is the incident.

On NixOS, `nix.gc.automatic = true;` plus `nix.gc.options = "--delete-older-than 14d";` is the boring timer.

Case 4: Why CI disks fill up

CI runners that `nix build` without deleting **result**, without a root that expires, and without GC between jobs accumulate every closure they ever substituted. The boring CI pattern:

```
nix build .#desk-api
nix-collect-garbage --delete-older-than 7d
```

or a persistent runner with `auto-optimise-store` and a nightly GC. Ephemeral GitHub runners are destroyed anyway; self-hosted runners are not.

Case 5: Do not GC the system you are running

```
readlink /run/current-system
sudo nix-collect-garbage
ls /run/current-system
```

`/run/current-system` is a root. The collector will not delete the running closure. It **will** delete old generations if you passed `-d`. After `-d`, `nixos-rebuild switch --rollback` has nowhere to go except whatever generations you still have.

The trap

The trap is `sudo rm -rf /nix/store/*` because `df` is 100%. That races the daemon, corrupts `db.sqlite`, and can unboot NixOS. Always GC.

The second trap is running `-d` on a laptop the day before a risky upgrade, then needing the previous generation at 2 a.m. Keep two weeks of generations on machines that boot humans.

The third trap is leaving a **result** in `$HOME` forever. It pins an old clang forever. `find ~ -name result -type l` once a quarter.

The boring rule

- Roots keep closures. No root, GC is allowed to delete.
- `nix-collect-garbage` is the only deletion tool.
- `--delete-older-than 14d` on workstations; `-d` when disk is the incident.
- Delete leftover `result` symlinks you do not need.
- On NixOS, enable automatic GC with a retention window — not `rm`.

Try this

1. `nix build nixpkgs#cowsay --out-link /tmp/cow && nix-store --gc --print-roots | grep /tmp/cow`. Remove the link and GC. Confirm the cowsay path is gone **unless** something else roots it.
2. Count generations: `sudo nix-env --list-generations --profile /nix/var/nix/profiles/system` (NixOS) or `nix-env --list-generations (user)`.
3. Dry-run: `nix-collect-garbage --dry-run` and read what would go. Then run without `--dry-run` only if you agree.
4. On a NixOS lab VM, add `nix.gc.automatic = true;` and `nix.gc.dates = "weekly";`, switch, and `systemctl status nix-gc.timer`.

Store Optimization and Deduplication

Store Optimization and Deduplication

Two packages often contain identical files: licenses, headers, man pages, `.pyc` with the same hash. Without help, each copy occupies its own disk blocks. The boring default is: **nix-store --optimise** (or `nix.optimise.automatic`) **hard-links identical files to one inode**, and you never do this with `cp` or a dedup FUSE.

Mental model

Optimisation walks `/nix/store`, hashes file contents, and when two files match:

- They keep two directory entries (two paths).
- They share one inode (one set of blocks).
- They remain read-only. A later rebuild never writes through a hard link; it creates a new path.

```
/nix/store/pkgA/share/COPYING
                                inode 1001  (one copy on disk)
/nix/store/pkgB/share/COPYING
```

This is **not** content-addressed store paths. Paths stay input-addressed and distinct. Only file *contents* are shared.

It is also not compression. `du` after `--optimise` drops; `nix path-info -S` (logical size) may look similar because it counts each path separately.

Worked examples

Case 1: Run it once and read the report

```
sudo nix-store --optimise
```

Output:

```
hashing files in /nix/store...
31420 files linked, 850.40 MiB freed
```

On a fresh VM the number may be small. On a desk that has compiled three gcc versions it can be gigabytes.

Case 2: Automatic on NixOS

Save as `optimise.nix` (import from `configuration.nix`):

```
# optimise.nix
{
  nix.optimise.automatic = true;
  nix.optimise.dates = [ "03:45" ];
  nix.settings.auto-optimize-store = true;
}
```

Option	When it runs
<code>nix.settings.auto-optimize-store</code>	After each build, newly added files are linked
<code>nix.optimise.automatic</code>	A systemd timer walks the whole store

Turn both on for a workstation. `auto-optimize-store` catches new builds; the timer catches paths that arrived via substitution.

Apply:

```
sudo nixos-rebuild switch
systemctl status nix-optimise.timer
```

Case 3: Prove two paths share an inode

Pick a file that is likely duplicated (a license):

```
find /nix/store -name COPYING -type f 2>/dev/null | head -5
```

If you have at least two, compare inodes:

```
stat -c '%i %n' /nix/store/<hash-a>-* /COPYING /nix/store/<hash-b>-* /COPYING
```

After `--optimize`, the `%i` values match. `ls -l` shows a link count `n` greater than 1.

Case 4: Logical size versus disk usage

```
nix path-info -Sh nixpkgs#hello
df -h /nix
```

`path-info -S` reports the size of that path's files as if they were unique. `df` reports actual blocks after hard links. Both numbers are useful; they are not the same question. When finance asks “how much does hello weigh,” use `nix path-info -Shr` (closure, logical). When the disk is full, use `df` and GC.

Case 5: Safe with GC

Optimise, then GC, in either order. Hard links do not keep a path alive. GC deletes directory trees; the inode goes away when the last link is removed.

```
sudo nix-store --optimise
sudo nix-collect-garbage --delete-older-than 14d
```

That pair is the boring monthly (or the automatic timers).

The trap

The trap is a **third-party deduplication tool** (duperemove, a Btrfs dedup cron, zfs dedup=on “because Nix”) on top of Nix’s own hard links. Filesystem-level dedup plus Nix hard links is extra CPU for little gain, and some tools copy-on-write in ways that fight the read-only store.

Btrfs and ZFS compression (compress=zstd) is fine. That is not dedup. Leave Nix to hard-link identical files.

The other trap is running `--optimise` while blaming it for a “missing” file. Optimise never deletes a path. If a file vanished, GC or a failed disk did it.

The boring rule

- `nix-store --optimise` or the NixOS timer. Not a random dedup cron.
- Enable `auto-optimise-store` so new builds are linked as they land.
- Measure disk with `df`; measure closures with `nix path-info -Shr`.
- Compression (Btrfs/ZFS) is complementary. Filesystem dedup is not needed.
- Optimise does not replace GC. Run both.

Try this

1. Note `df -h /nix` (or `/`), run `sudo nix-store --optimise`, note `df` again. Record the delta.
2. On NixOS, add the snippet from Case 2, switch, and `systemctl list-timers | grep nix`.
3. `stat` two identical COPYING files and confirm the inode number. If you only have one, `nix build nixpkgs#hello nixpkgs#cowsay` first and look again.
4. Run `nix-store --optimise` a second time immediately. The “files linked” count should be near zero. That is how you know it is idempotent.

First Expressions

The Derivation Primitive

The Derivation Primitive

Every package, every system, every `mkShell` eventually becomes a `derivation { ... }`. The boring default is: **know the three required fields, write one by hand once, then use `pkgs.runCommand` / `stdenv.mkDerivation` for the rest of your career.**

Mental model

`derivation` is a builtin. It does not compile C. It produces a `.drv` file describing a build:

Field	Required	Meaning
<code>name</code>	yes	Label in the store path
<code>system</code>	yes	"x86_64-linux", "aarch64-linux", "aarch64-darwin", ...
<code>builder</code>	yes	Store path of the executable that will run
<code>args</code>	no	Arguments to the builder
<code>other attrs</code>	no	Become environment variables inside the sandbox

`$out` is set by Nix to the output path **before** the builder runs. The builder's job is to create that file or directory. If `$out` is missing when the builder exits 0, the build fails.

`builder = "/bin/sh"` is a lie on a sandboxed Linux: `/bin/sh` is not in the sandbox. Point `builder` at `${pkgs.bash}/bin/bash`.

```
derivation { ... }
```

```
.drv (instantiate)
```

```
sandbox runs builder
```

```
$out registered in the store
```

Worked examples

Case 1: A one-file banner

Save as `minimal.nix`:

```
# minimal.nix
let
  pkgs = import <nixpkgs> {};
in
derivation {
  name = "desk-banner";
  system = builtins.currentSystem;
  builder = "${pkgs.bash}/bin/bash";
  args = [
    "-c"
    "echo 'Desk System Online' > $out"
  ];
}
```

```
nix-build minimal.nix
cat result
```

Output:

```
these 1 derivations will be built:
  /nix/store/...-desk-banner.drv
building '/nix/store/...-desk-banner.drv'...
/nix/store/...-desk-banner
Desk System Online
```

result is a GC root pointing at `$out`.

Case 2: Instantiate versus build

```
nix-instantiate minimal.nix
cat $(nix-instantiate minimal.nix) | head
```

The `.drv` is text. You will see `builder`, `system`, and `name`. `nix-build` is `instantiate` + `realise`.

Case 3: Environment attributes

Save as `env.nix`:

```
# env.nix
let
  pkgs = import <nixpkgs> {};
in
derivation {
  name = "desk-env";
  system = builtins.currentSystem;
  builder = "${pkgs.bash}/bin/bash";
  deskWindow = "A";
  args = [ "-c" "echo window=$deskWindow > $out" ];
}
```

```
nix-build env.nix && cat result
```

Output:

```
window=A
```

Every extra attribute is an environment variable. Nested attribute sets are **not** allowed here (the builtin is strict about types). That is one reason `stdenv.mkDerivation` exists: it accepts a richer `attrset` and flattens it.

Case 4: The `runCommand` spelling you will actually write

Save as `run.nix`:

```
# run.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.runCommand "desk-banner" { } ''
  echo 'Desk System Online' > $out
''
```

```
nix-build run.nix && cat result
```

Same `$out` idea. `runCommand` fills in `builder`, `system`, and `stdenv` so you do not hard-code `bash`. Prefer this in real files. Use raw `derivation` when you are explaining Nix, not when you are packaging the desk API.

Case 5: Failure when `$out` is missing

Save as `missing.nix`:

```
# missing.nix
let
```



```

    pkgs = import <nixpkgs> {};
in
derivation {
  name = "desk-missing";
  system = builtins.currentSystem;
  builder = "${pkgs.bash}/bin/bash";
  args = [ "-c" "echo forgot" ];
}

nix-build missing.nix

```

Output:

```

building '/nix/store/...-desk-missing.drv'...
forgot
error: builder for '/nix/store/...-desk-missing.drv' failed to produce output path '/nix/store/...'

```

The builder exited 0. Nix still failed. `$out` is a contract.

The trap

The trap is `builder = "/bin/sh"` copied from an ancient gist. It works only with the sandbox off, or on some macOS setups with a magic `/bin/sh`. On NixOS it fails with `No such file or directory`. Use `${pkgs.bash}/bin/bash` or `runCommand`.

The other trap is writing to `$out` and then also expecting a directory layout without `mkdir`. If `$out` should be a directory, `mkdir -p $out` first; do not echo into `$out` and then `mkdir $out/bin`.

The boring rule

- Raw `derivation` once, so the `.drv` is not magic. Then `runCommand` / `mkDerivation`.
- `builder` is a **store path**, not `/bin/sh`.
- Create `$out`. That is the whole job.
- Extra attributes become env vars. Keep them strings, paths, or lists of those.
- `result` is a GC root. Delete it when you are done poking.

Try this

1. Change the `echo` text in `minimal.nix`, rebuild, and confirm the store path hash changed.
2. `nix-instantiate minimal.nix` and search the `.drv` for `bash`. It must be a `/nix/store/...-bash-...` path.
3. Rewrite `minimal.nix` as `runCommand` (Case 4) and `diff` the two `cat result` outputs — they should match.

4. In `env.nix`, add `deskShift = "morning";` and print both variables. Confirm a rebuild produced a new path.

Building Multi-File Artifacts

Building Multi-File Artifacts

`$out` can be a file or a directory. Real packages are directories: `bin/`, `lib/`, `share/`. The boring default is: `mkdir -p $out/bin` first, write files, `chmod +x` the executables, and smoke-test with `./result/bin/...`

Mental model

Nix does not care about FHS except as a convention other tools expect. If you put a binary at `$out/bin/desk-status`, `nix-env`, Home Manager, and `environment.systemPackages` will put `$out/bin` on `PATH`. If you put it at `$out/desk-status`, they will not.

Path	Usual contents
<code>\$out/bin</code>	Executables on <code>PATH</code>
<code>\$out/lib</code>	Shared libraries (<code>RUNPATH</code> targets)
<code>\$out/share</code>	Data, man pages, completions
<code>\$out/etc</code>	Default config (not live <code>/etc</code>)

`pkgs.runCommand` is enough for scripts and generated trees. Compilers come later (`stdenv.mkDerivation`).

Worked examples

Case 1: A directory with a script and a data file

Save as `multifile.nix`:

```
# multifile.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.runCommand "desk-assets" { } ''
  mkdir -p $out/bin $out/share
  cat > $out/bin/desk-status << 'EOF'
  #!/bin/sh
  echo status: ready
  EOF
  chmod +x $out/bin/desk-status
  echo 'Desk version 1.0' > $out/share/version.txt
''
```

```
nix-build multifile.nix
./result/bin/desk-status
cat result/share/version.txt
```

Output:

```
/nix/store/...-desk-assets
status: ready
Desk version 1.0
```

Case 2: Patch the shebang so the sandbox shell is not required at runtime

`#!/bin/sh` works on NixOS (there is a `/bin/sh`). It is still sloppy. Point the shebang at store bash:

Save as `shebang.nix`:

```
# shebang.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.runCommand "desk-status" { inherit (pkgs) bash; } ''
  mkdir -p $out/bin
  cat > $out/bin/desk-status << EOF
  #!${pkgs.bash}/bin/bash
  echo status: ready
  EOF
  chmod +x $out/bin/desk-status
''

nix-build shebang.nix
head -1 result/bin/desk-status
```

Output:

```
#!/nix/store/...-bash-5.2/bin/bash
```

`nix-store -q --references result` now includes bash. That is correct: the script **runs** bash.

Case 3: `writeShellApplication` — the boring wrapper

Save as `write.nix`:

```
# write.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.writeShellApplication {
```

```

name = "desk-status";
runtimeInputs = [ pkgs.coreutils ];
text = ''
    echo "status: ready"
    date -u +%Y-%m-%d
'';
}

```

```

nix-build write.nix
./result/bin/desk-status

```

Output:

```

status: ready
2026-09-09

```

`writeShellApplication` sets the shebang, `set -o errexit / nounset / pipefail`, and puts `runtimeInputs` on `PATH` via a wrapper. Prefer it over hand-rolled `echo > $out/bin`.

Case 4: Multiple outputs are a later tool

Some packages split `$out` and `$dev` (headers) so runtime closures stay small. You will meet `outputs = [ "out" "dev" ]`; in the packaging part. For desk scripts, one `$out` is enough. Do not invent extra outputs until a closure-size measurement says headers are the problem.

Case 5: Install into a profile to get PATH

```

nix-build write.nix
nix profile install ./result
desk-status

```

Output:

```

status: ready
2026-09-09

```

On NixOS, the same package in `environment.systemPackages` is the system-wide equivalent. `./result/bin/...` is the lab check; a profile is how humans run it tomorrow.

The trap

The trap is **forgetting `chmod +x`**. The file is in `$out/bin`, activation puts the directory on `PATH`, and the kernel says `Permission denied`. The build succeeded.

The other trap is writing a shebang of `#!/usr/bin/env bash` and then not putting `bash` on `PATH` at runtime. `env` looks up `bash` in the **user's** `PATH`, which may be missing or may be the host's `bash`. Store-path shebangs (`writeShellApplication`) do not guess.

The boring rule

- `$out` is a directory for anything with more than one file. `mkdir -p` first.
- Executables live in `$out/bin` and are executable.
- Prefer `writeShellApplication` for scripts.
- Shebangs should be store paths, not `/usr/bin/env`.
- Smoke-test `./result/bin/...` before you add the package to a profile.

Try this

1. Add `$out/share/man/man1/desk-status.1` with a one-line man page. Rebuild and `ls result/share/man/man1`.
2. Drop `chmod +x` from Case 1, rebuild, and run `./result/bin/desk-status`. Restore `chmod` after you see `Permission denied`.
3. `nix-store -q --references $(nix-build shebang.nix --no-out-link)` and find `bash`. Then do the same for Case 1 (`#!/bin/sh`) and compare.
4. Convert Case 1 to `writeShellApplication` and add `pkgs.hello` to `runtimeInputs`. Call `hello` from the script.

Inspecting the Store

**Inspecting Closures: why-depends, path-info,
nix-tree**

Inspecting Closures: `why-depends`, `path-info`, `nix-tree`

Shipping a “small” service that secretly contains `gcc` is a weekly desk accident. The boring default is: **measure the closure before you copy it, then ask `why-depends` when a path you did not expect shows up.**

Mental model

Three questions, three tools:

Question	Tool
How big is this, including everything it needs?	<code>nix path-info -Shr</code>
Why is this path inside that closure?	<code>nix why-depends A B</code>
Let me walk the tree with a keyboard	<code>nix-tree</code>

`nix-store -q --tree` is the classic ASCII dump. `nix-tree` is the same graph with navigation. `nom` (`nix-output-monitor`) wraps `nix build` so you watch fetch/build in a table instead of a scrollback.

You already used `--requisites` in the closures chapter. This chapter is the daily loop: size, blame, browse.

Worked examples

Case 1: Closure size in one line

```
nix path-info -Shr nixpkgs#hello
```

Output (shape):

```
/nix/store/...-glibc-2.39      31.0M  31.0M
/nix/store/...-hello-2.12.1   226K   31.2M
```

The last column on the last line is the number you put in a review: **hello’s runtime closure is ~31 MB**, almost all `glibc`.

Compare a fat wrapper:

```
nix path-info -Sh nixpkgs#ripgrep
nix path-info -Shr nixpkgs#ripgrep | tail -1
```

If the closure is dozens of times the single-path size, something was propagated that should not have been.

Case 2: why-depends

“Why does the desk API closure contain python3?”

```
nix why-depends nixpkgs#ripgrep nixpkgs#glibc
```

Output (shape):

```
/nix/store/...-ripgrep-14.1.0
/nix/store/...-glibc-2.39
```

A longer chain (hello → wrapper → python) prints as a path of edges. That chain is the fix list: cut the edge you did not intend (a `propagatedBuildInputs`, a `wrapProgram` that pulled in a huge interpreter, a `share/` file that embeds a store path).

If there is **no** dependence:

```
nix why-depends nixpkgs#hello nixpkgs#ripgrep
```

Output:

```
'hello' does not depend on 'ripgrep'
```

Case 3: ASCII tree without extra tools

```
nix-store -q --tree $(nix path-info nixpkgs#hello)
```

Output (shape):

```
/nix/store/...-hello-2.12.1
/nix/store/...-glibc-2.39
...
```

Useful on a server where you will not install `nix-tree`. Pipe to `less`.

Case 4: nix-tree for interactive blame

```
nix run nixpkgs#nix-tree -- $(nix path-info nixpkgs#hello)
```

or, if `nix-tree` is in your system packages:

```
nix-tree $(nix path-info nixpkgs#hello)
```

Keys (typical):

Key	Action
j / k	Move
enter	Open a path's references
s	Sort by size
q	Quit

Sort by size first. The top of the list is where the megabytes are.

Case 5: Watch a build with nom

```
nix run nixpkgs#nix-output-monitor -- build nixpkgs#hello
```

`nom` is a frontend. It does not change hashes. Use it when a long `nixos-rebuild` is a wall of identical “copying path” lines and you want to know whether you are substituting or compiling.

On NixOS you can alias:

```
# configuration.nix fragment
environment.systemPackages = [ pkgs.nix-output-monitor ];
```

Then `nom build .#desk-api`.

The trap

The trap is **reading `du -sh result`**. `result` is a symlink to **one** path. `du` does not follow the `glibc` reference. You report 200 KB, production downloads 31 MB, and someone “optimises” the wrong thing.

Always `nix path-info -Shr`.

The second trap is **why-depends** in the wrong direction. `nix why-depends A B` means “why does A depend on B” — A is the parent closure, B is the unexpected guest.

The boring rule

- Size with `nix path-info -Shr` on the **output** path.
- Blame with `nix why-depends parent child`.
- Browse with `nix-tree` (or `--tree` on a server).
- `du` on `result` is not a closure size.
- `nom` is optional UX. It is not part of the hash.

Try this

1. Record `nix path-info -Shr nixpkgs#hello` and `nix path-info -Shr nixpkgs#git`. Git is larger. `why-depends` git against perl or python and see which interpreter, if any, is in the runtime closure.
2. `nix-store -q --tree $(nix path-info nixpkgs#hello) | wc -l` — that is the node count of a tiny closure.
3. Install `nix-tree` in a devShell, open hello, sort by size, and name the largest path.
4. Run a `nix build` twice, once plain and once under `nom`. Confirm the resulting store path is identical (`nix path-info`).

Nix Language

Nix Expressions and Evaluation

Nix Expressions and Evaluation

Nix does not run a list of statements. It evaluates **one** pure expression. The boring default is: **a file is a single value, the same inputs always yield the same AST, and laziness means unused branches (including throw) never run.**

This part uses the Nix **2.35** CLI (`nix eval`). Flakes come in part 3–4; here the language is enough.

Mental model

Bash mutates the world from top to bottom. Nix does not:

1. **One expression per file.** `2 + 2`, `{ a = 1; }`, and a 400-line package are the same kind of thing.
2. **Deterministic.** Same tree, same result. No clock, no `$HOME`, no “it depends who ran it.”
3. **Lazy.** Defining fifty attributes and asking for `.port` does not compute the other forty-nine.

Bash: step → mutate → step → mutate → leftover state

Nix: root expression → demand-driven subexpressions → a value

`nix eval --expr` is a scratchpad. `nix eval --file` evaluates a file. `--json` is for machines.

Worked examples

Case 1: Arithmetic is an expression

```
nix eval --expr '1 + 2 * 3'
```

Output:

7

Precedence is ordinary. There is no `print`. The value *is* the result.

Case 2: A file is one `let ... in`

Save as `root_expression.nix`:


```
# root_expression.nix
let
  deskId = "terminal-01";
  status = "online";
in
{
  id = deskId;
  state = status;
  ready = true;
}

nix eval --file root_expression.nix --json
```

Output:

```
{"id":"terminal-01","ready":true,"state":"online"}
```

--json sorts keys. The file did not “run” three assignments; it produced one attrset.

Case 3: Laziness skips throw

Save as laziness_proof.nix:

```
# laziness_proof.nix
let
  validField = "desk-service";
  brokenField = throw "This error is never triggered!";
in
{
  service = validField;
  broken = brokenField;
}

nix eval --file laziness_proof.nix --apply 'x: x.service'
```

Output:

```
"desk-service"
```

broken exists on the set. It is not forced. Force it:

```
nix eval --file laziness_proof.nix --apply 'x: x.broken'
```

Output (shape):

error: This error is never triggered!

That error is the proof: evaluation is demand-driven.

Case 4: `--strict` / JSON vs thunks

```
nix eval --expr '{ a = 1 + 1; b = throw "no"; }' --apply 'x: x.a'
```

Output:

2

`nix-instantiate --eval` without `--strict` can print thunks as `<CODE>`. Prefer `nix eval` (2.35) plus `--json` or `--apply` so you see values, not internals.

Case 5: No side effects at eval time

Save as `no_echo.nix`:

```
# no_echo.nix
let
  # This does not print. It is a string.
  msg = "desk online";
in
msg
```

```
nix eval --file no_echo.nix
```

Output:

"desk online"

There is no `echo` during eval. Writes happen later, in a **derivation** sandbox (fundamentals / packaging). If you wanted a file on disk, you have not evaluated yet — you have not built.

The trap

The trap is **expecting `echo` or `curl` inside a `.nix` file to run when you `nix eval`**. Eval is memory. Network and files belong in FOD fetchers and builders.

A second trap: two expressions in one file with no `let`. That is a parse error. One root.

The boring rule

- One expression per file.
- `nix eval --expr / --file` on Nix 2.35. `--json` when you want stable output.
- Laziness: unused `throw` is silent; accessed `throw` is an eval error.
- No side effects at eval. Builds write `$out`.
- `--apply` to pick a field without evaluating the whole set.

Try this

1. Case 3: eval `.service` then `.broken`. Predict before you run.
2. `nix eval --expr 'let x = 1 / 0; in 4'` — does it error? (Integer division by zero is forced only if `x` is used.)
3. Put two attrsets in one file without `let`. Read the parse error.
4. `nix eval --expr '{ a = 1; }' --json` and compare to `--impure` — you should not need `--impure` here.

Values and Immutability

Values and Immutability

Once a Nix value exists, nothing mutates it. The boring default is: **//** and **++** allocate new values; names in a **let** are bound once; the original attrset is still what you passed in.

Mental model

Python dicts surprise you:

```
config = {"port": 8080}
mutate(config) # original changed
```

Nix:

1. Attrsets, lists, and strings do not have setters.
2. **a // b** is a **new** set. **a** is unchanged.
3. Memoisation is safe because nothing behind a pointer can change.

```
{ a = 1; }
                                { a = 1; b = 2; } (new)
{ b = 2; }
first set still { a = 1; }
```

Worked examples

Case 1: Merge does not rewrite

Save as `immutable_merge.nix`:

```
# immutable_merge.nix
let
  baseDesk = {
    location = "building-a";
    tier = "standard";
  };
  upgradedDesk = baseDesk // {
    tier = "executive";
    desksCount = 12;
  }
```

```

    };
  in
  {
    original = baseDesk;
    upgraded = upgradedDesk;
  }

```

```
nix eval --file immutable_merge.nix --json
```

Output:

```
{"original":{"location":"building-a","tier":"standard"},"upgraded":{"desksCount":12,"location":
```

baseDesk.tier is still "standard". Fifty functions can receive baseDesk.

Case 2: Lists concatenate to a new list

Save as immutable_list.nix:

```

# immutable_list.nix
let
  activeTables = [ 1 2 3 ];
  extendedTables = activeTables ++ [ 4 5 ];
in
{
  initial = activeTables;
  result = extendedTables;
}

```

```
nix eval --file immutable_list.nix --json
```

Output:

```
{"initial":[1,2,3],"result":[1,2,3,4,5]}
```

There is no .push.

Case 3: Same name twice is an error

Save as double_bind.nix:

```

# double_bind.nix
let
  x = 1;

```

```

    x = 2;
  in
  x

nix eval --file double_bind.nix

```

Output (shape):

error: attribute 'x' already defined at double_bind.nix:3

Use a second name (x0, xNext) or an inner let that **shadows**.

Case 4: Inner let shadows, outer unchanged

Save as shadow.nix:

```

# shadow.nix
let
  x = 1;
in
{
  outer = x;
  inner = let x = 2; in x;
}

nix eval --file shadow.nix --json

```

Output:

```
{"inner":2,"outer":1}
```

Case 5: Deep merge is not //

```

# shallow.nix
let
  a = { desk = { port = 80; host = "a"; }; };
  b = { desk = { port = 443; }; };
in
a // b

nix eval --file shallow.nix --json

```

Output:

```
{"desk":{"port":443}}
```

`host` is **gone**. `//` replaces the `desk` attr wholesale. Nested merge is `lib.recursiveUpdate` (imports chapter) or a function that rebuilds the nest. Do not expect `//` to patch deep keys.

The trap

The trap is `let x = 1; x = 2;`. The trap after that is assuming `//` is deep. Both look like assignment in other languages. They are not.

The boring rule

- Values do not change. New names, new values.
- `//` shallow. `++` new list.
- One binding per name per `let`.
- Shadow in a nested `let`, do not reassign.
- For nested config, write a function or `lib.recursiveUpdate`, not a pile of `//`.

Try this

1. `nix eval --expr 'let a = { x = 1; }; b = a // { x = 2; }; in a.x'` — expect 1.
2. Case 5: add `lib` later; for now rebuild `{ desk = a.desk // b.desk; }` by hand and keep `host`.
3. Duplicate a `let` name; save the error text.
4. `nix eval --expr '[ 1 ] ++ [ 1 ]'` — duplicates are allowed; lists are not sets.

Data Types and String Interpolation

Data Types and String Interpolation

Nix checks types when a value is **used**, not at parse time. The boring default is: **know the six primitives, interpolate with `${...}`, `toString` numbers, and treat unquoted paths as store copies.**

Mental model

Type	Examples
int	42, -1
float	3.14
bool	true, false
string	"desk", ''multi''
path	./config.nix, /etc/os-release
null	null

Also later: lists, attrsets, functions, derivations.

`"${port}"` is an error if `port` is an int. `"${toString port}"` is the boring spelling.

`'' ... ''` is a multi-line string. Leading indentation of the closing `''` is stripped.

Unquoted `./file` interpolated into a string **copies** file into `/nix/store` and pastes that path.
Quoted `"./file"` is just characters.

Worked examples

Case 1: `builtins.typeOf`

Save as `types_check.nix`:

```
# types_check.nix
{
  intType = builtins.typeOf 100;
  floatType = builtins.typeOf 3.5;
  boolType = builtins.typeOf true;
  stringType = builtins.typeOf "desk";
  pathType = builtins.typeOf ./types_check.nix;
  nullType = builtins.typeOf null;
```

```
}
```

```
nix eval --file types_check.nix --json
```

Output:

```
{"boolType": "bool", "floatType": "float", "intType": "int", "nullType": "null", "pathType": "path", "str
```

Case 2: Interpolation and indented strings

Save as interpolation.nix:

```
# interpolation.nix
let
  appName = "desk-agent";
  port = 9090;
  logDir = "/var/log/desk";
  endpoint = "http://127.0.0.1:${toString port}";
  systemdService = ''
    [Unit]
    Description=${appName} Service

    [Service]
    ExecStart=/bin/false
    Environment=ENDPOINT=${endpoint}
    StandardOutput=append:${logDir}/output.log
  '';
in
{
  inherit endpoint systemdService;
}
```

```
nix eval --file interpolation.nix --apply 'x: x.endpoint'
```

Output:

```
"http://127.0.0.1:9090"
```

```
nix eval --file interpolation.nix --apply 'x: x.systemdService'
```

The unit text has no leftover leading spaces from the Nix indent. That is why '' exists for systemd snippets.

Case 3: Path vs string

```
nix eval --expr 'let f = ./types_check.nix; in "${f}"'
```

Output (shape):

```
"/nix/store/...-types_check.nix"
```

```
nix eval --expr '"./types_check.nix"'
```

Output:

```
"./types_check.nix"
```

The first copied bytes into the store (eval of a path). The second is a string you could pass to a builder as a **name**, not as content.

Case 4: Integer interpolation fails

Save as bad_interp.nix:

```
# bad_interp.nix
let port = 8080; in "listen ${port}"
```

```
nix eval --file bad_interp.nix
```

Output (shape):

```
error: cannot coerce an integer to a string
```

Fix: "listen \${toString port}".

Case 5: null is not ""

```
nix eval --expr 'builtins.typeOf null'
nix eval --expr 'null == ""'
```

Output:

```
"null"
false
```

or on attrsets (`set.x` or `3`) is not the same as `null`. Missing attr `null` attr.

The trap

The trap is **quoting a path** (`src = "./src"`) and wondering why the builder cannot see your files. Unquoted `./src` is the FOD/source. A string is letters.

The other trap is interpolating a derivation into a string at **eval** (`"${pkgs.hello}"`) — that **forces a build** (or at least a path). Fine in a module; surprising in a “just eval” scratch file.

The boring rule

- `toString` before interpolating ints (and most non-strings).
- `'...'` for scripts and units.
- Unquoted paths when Nix should hash the file.
- Quoted paths when you mean a literal string.
- `builtins.typeOf` when a merge “looks like” the wrong kind.

Try this

1. `nix eval --expr 'builtins.typeOf (toString 42)' → "string".`
2. Case 4: add `toString`, re-eval.
3. `nix eval --expr 'let p = 1; in "${p}"'` and save the coerce error.
4. Compare `"${./types_check.nix}"` with `toString ./types_check.nix` — both store paths, slightly different spelling rules; pick one style and keep it.

Let-In Blocks and Local Bindings

Let-In Blocks and Local Bindings

There is no `x = 1` that you later change. The boring default is: **let** names, **in** uses them, bindings may mention each other in any order, and unused names (even throw) stay cheap.

Mental model

```
let
  b = a + 10;
  a = 5;
in
b    # 15
```

Order in the `let` is not execution order. Nix builds a graph. `b` needs `a`; `a` is 5.

`inherit x`; in an attrset is `x = x`; — copy the name from scope.

Nested `let` shadows. The outer name is still there for the outer `in`.

Worked examples

Case 1: Desk URL from parts

Save as `desk_service.nix`:

```
# desk_service.nix
let
  domain = "internal.desk";
  port = 443;
  protocol = "https";
  serviceUrl = "${protocol}://${domain}:${toString port}";
in
{
  inherit domain port;
  url = serviceUrl;
}
```

```
nix eval --file desk_service.nix --json
```

Output:

```
{"domain":"internal.desk","port":443,"url":"https://internal.desk:443"}
```

Case 2: Order does not matter

Save as `order.nix`:

```
# order.nix
let
  c = b + 1;
  b = a + 1;
  a = 1;
in
c

nix eval --file order.nix
```

Output:

3

Write names in dependency order anyway. Humans read top-down even when Nix does not.

Case 3: Shadowing

Save as `scope.nix`:

```
# scope.nix
let
  tier = "standard";
in
{
  outerTier = tier;
  overridden =
    let
      tier = "premium";
    in
      tier;
}

nix eval --file scope.nix --json
```

Output:

```
{"outerTier":"standard","overridden":"premium"}
```


Case 4: Unused throw is fine

Save as `unused.nix`:

```
# unused.nix
let
  domain = "internal.desk";
  unused = throw "never evaluated";
in
domain
```

```
nix eval --file unused.nix
```

Output:

```
"internal.desk"
```

A 200-line `let` of unused helpers is still a readability problem. Laziness does not make a mess cheap for the next reader.

Case 5: inherit from a set

Save as `inherit_from.nix`:

```
# inherit_from.nix
let
  cfg = { host = "desk"; port = 8080; extra = true; };
in
{
  inherit (cfg) host port;
}
```

```
nix eval --file inherit_from.nix --json
```

Output:

```
{"host":"desk","port":8080}
```

`extra` stayed in `cfg`. `inherit (cfg) host port` is the boring way to pick keys.

The trap

The trap is a **200-line let of unrelated helpers**. Split files (`import`, next chapters) when a binding cluster has a name (`db`, `tls`, `desk-api`).

The other trap is circular `let`: `a = b; b = a;`. Eval loops. `rec` attrsets have the same foot-gun (next chapter).

The boring rule

- `let` for names. `in` for the value you mean.
- Bindings can be in any order; write them in human order.
- `inherit` / `inherit (set)` instead of `host = host`.
- Unused bindings can `throw`; still delete them if they confuse.
- Split large `lets` into files.

Try this

1. Add `unused = throw "x";` to `desk_service.nix`; eval still works.
2. Reverse three bindings as in Case 2; eval.
3. `inherit (cfg) missing;` and read the error.
4. Nested `let` that shadows `port`; print both outer and inner.

Functions and Argument Defaults

Functions and Argument Defaults

A Nix function takes **one** argument. The boring default for desk config is: `{ name, port ? 8080, ... }:` — **named attrs, defaults with ?, ... only when extras are allowed.**

Mental model

```
x: x + 1
```

Two “parameters” are currying: `x: y: x + y`, called as `add 1 2`. Configuration code should not look like that.

Named args:

```
{ host, port ? 80 }: "${host}:${toString port}"
```

Piece	Meaning
<code>{ host, port }</code>	Both required
<code>port ? 80</code>	Optional
<code>...</code>	Ignore extra attrs
<code>args@{ host, ... }</code>	Bind the whole set as args

Without `...`, an unexpected key is an eval error. That is often what you want.

Worked examples

Case 1: Curry vs attrset

Save as `functions_demo.nix`:

```
# functions_demo.nix
let
  add = x: y: x + y;
  formatServer = { host, port ? 80 }: "${host}:${toString port}";
in
{
  sum = add 10 25;
  defaultPort = formatServer { host = "api.desk"; };
}
```

```
    customPort = formatServer { host = "api.desk"; port = 443; };
}
```

```
nix eval --file functions_demo.nix --json
```

Output:

```
{"customPort":"api.desk:443","defaultPort":"api.desk:80","sum":35}
```

Call `formatServer` with a missing `host` — error. That is better than `"null:80"`.

Case 2: @ captures extras

Save as `capture.nix`:

```
# capture.nix
let
  makeProfile = args@{ user, role ? "member", ... }: {
    username = user;
    userRole = role;
    rawInput = args;
  };
in
makeProfile { user = "alice"; team = "desk-ops"; }

nix eval --file capture.nix --json
```

Output:

```
{"rawInput":{"team":"desk-ops","user":"alice"},"userRole":"member","username":"alice"}
```

team survived in `rawInput` because of

Case 3: Unexpected argument without ...

Save as `strict_fn.nix`:

```
# strict_fn.nix
let
  f = { host }: host;
in
f { host = "desk"; extra = 1; }
```

```
nix eval --file strict_fn.nix
```

Output (shape):

```
error: function 'anonymous lambda' called with unexpected argument 'extra'
```

Package functions (`{ stdenv, fetchurl }`) stay strict so typos fail. Module-shaped `{ config, pkgs, ... }`: uses `...` because the module system passes extra keys.

Case 4: `secure ? false`

Save as `url.nix`:

```
# url.nix
let
  formatServer = { host, port ? 80, secure ? false }:
    let
      scheme = if secure then "https" else "http";
      p = if secure && port == 80 then 443 else port;
    in
      "${scheme}://${host}:${toString p}";
in
{
  a = formatServer { host = "api.desk"; };
  b = formatServer { host = "api.desk"; secure = true; };
}
```

```
nix eval --file url.nix --json
```

Output:

```
{"a":"http://api.desk:80","b":"https://api.desk:443"}
```

Case 5: Apply with `--apply`

```
nix eval --file functions_demo.nix --apply 'x: x.customPort'
```

Output:

```
"api.desk:443"
```

Functions are values. You can pass them around; you cannot mutate a default after the fact — you call again with a different set.

The trap

The trap is ... **on every function** so “it never errors.” Typos (`prt = 443`) silently drop. Use ... at module boundaries. Keep package headers strict.

The other trap is currying five levels (`a: b: c: d: e:`) for config. Named attrsets.

The boring rule

- Config functions take `{ ... }`.
- `?` for defaults. Required keys stay required.
- ... only when extras are part of the contract.
- `@args` when you must forward the whole set.
- Unexpected-argument errors are a feature.

Try this

1. Case 4: add `path ? ""` and append it when non-empty.
2. Drop ... from Case 2 and pass `team`; read the error.
3. `nix eval --expr '({ x }: x) { x = 1; y = 2; }'`.
4. Write `id = x: x`; and `nix eval --expr '(import ./id.nix) 41'` after saving `id.nix` as `x: x`.

Attribute Sets and Recursive Definitions

Attribute Sets and Recursive Definitions

Attrsets are how Nix holds config. The boring default is: **plain { }**, merge with **//**, read with **.** and **or**, and use **rec** only when siblings need each other and you will not **//** over them later.

Mental model

```
{ host = "127.0.0.1"; port = 8080; }
```

Op	Role
s.port	Access; missing → error
s.timeout or 30	Default if missing
a // b	Shallow merge; right wins
rec { x = 1; y = x; }	Siblings see each other

desk.db.port = 5432; in a let bind is nested sugar in some positions (nested in rec / let via x.y =). Prefer nested { desk.db = { port = 5432; }; } when learning.

Worked examples

Case 1: Merge and or

Save as attr_operations.nix:

```
# attr_operations.nix
let
  defaultConfig = {
    host = "127.0.0.1";
    port = 8080;
    workers = 4;
  };
  productionOverrides = {
    port = 443;
    workers = 16;
  };
  finalConfig = defaultConfig // productionOverrides;
```

```

in
{
  inherit finalConfig;
  customTimeout = finalConfig.timeout or 30;
}

```

```
nix eval --file attr_operations.nix --json
```

Output:

```
{"customTimeout":30,"finalConfig":{"host":"127.0.0.1","port":443,"workers":16}}
```

Case 2: rec for sibling URLs

Save as `rec_demo.nix`:

```

# rec_demo.nix
rec {
  domain = "desk.corp";
  authService = "https://auth.${domain}";
  billingService = "https://billing.${domain}";
  endpoints = [ authService billingService ];
}

nix eval --file rec_demo.nix --apply 'x: x.authService'

```

Output:

```
"https://auth.desk.corp"
```

A `let` around the same names is often clearer than `rec`. Use `rec` when the set *is* the value you export.

Case 3: `//` does not update rec dependents

Save as `rec_override.nix`:

```

# rec_override.nix
let
  base = rec { x = 1; y = x + 1; };
  updated = base // { x = 10; };
in
{
  inherit (updated) x y;
}

```

```
nix eval --file rec_override.nix --json
```

Output:

```
{"x":10,"y":2}
```

y is still 2. It closed over the original x. If overrides must recompute, use a function:

```
make = { x }: rec { inherit x; y = x + 1; };  
make { x = 10; } # y = 11
```

Case 4: Missing attr without or

```
nix eval --file attr_operations.nix --apply 'x: x.finalConfig.timeout'
```

Output (shape):

```
error: attribute 'timeout' missing
```

Production config: or at the edges, required keys in the module types (part 6).

Case 5: // is shallow (again)

```
# nest.nix  
let  
  a = { tls = { enable = true; port = 443; }; };  
  b = { tls = { enable = false; }; };  
in  
a // b
```

```
nix eval --file nest.nix --json
```

Output:

```
{"tls":{"enable":false}}
```

port vanished. Rebuild `tls = a.tls // b.tls` or `lib.recursiveUpdate`.

The trap

The trap is **rec plus // and expecting dependents to move**. Case 3. Prefer **let** + function.

The other trap is **s.foo.bar** when **foo** might be missing — error on **foo**, not on **bar**. **s.foo.bar or 1** does not save you; **or** binds the whole **s.foo.bar** only if the syntax is **(s.foo.bar) or 1** vs **s.foo.bar or 1** — in Nix, **or** is tight on the selection. Write **(s.foo or {}).bar or 1** when nested.

The boring rule

- **.** and **or** for reads. **//** for shallow overlays of config.
- **rec** for small self-contained sets you will not patch.
- Overrides that must recompute → function, not **rec //**.
- Nested data: merge one level at a time or **recursiveUpdate**.
- Missing attributes should error in internal code; **or** at user-facing edges.

Try this

1. Access **finalConfig.invalid** without **or**.
2. Rewrite **rec_demo.nix** as **let domain = ...; in { ... }** without **rec**.
3. Case 3: replace **//** with **make { x = 10; }** and confirm **y == 11**.
4. **(s.tls or {}).port or 443** on a set without **tls**.

Conditionals, Lists, and Built-in Operations

Conditionals, Lists, and Built-in Operations

`if` in Nix is a **value**, not a statement. The boring default is: `if c then a else b` (else required), lists with `++`, and `builtins.map / filter / length / elem`.

Mental model

```
if isProd then 5 else 1
```

No `if` without `else`. No `if` that “does” something. Both branches are expressions; the unused one stays lazy.

Lists:

```
[ 1 "two" ./three (4 + 5) ]
```

Elements are space-separated. `[ 1 2 ] ++ [ 3 ]` is `[ 1 2 3 ]`. Indexing is `builtins.elemAt xs 0` (0-based). There is no `xs[0]` in the language.

`lib.optional cond x` is `if cond then [ x ] else [ ]` — the boring way to maybe-append one item.

Worked examples

Case 1: Flags

Save as `conditional_config.nix`:

```
# conditional_config.nix
let
  isProduction = true;
in
{
  replicas = if isProduction then 5 else 1;
  logging = if isProduction then "warn" else "debug";
}

nix eval --file conditional_config.nix --json
```

Output:

```
{"logging":"warn","replicas":5}
```

Flip `isProduction` to `false` and eval again.

Case 2: map / filter

Save as `list_operations.nix`:

```
# list_operations.nix
let
  tableNumbers = [ 1 2 3 4 5 6 7 8 9 10 ];
  isPriority = n: n <= 3;
  priorityTables = builtins.filter isPriority tableNumbers;
  deskWorkers = builtins.map (n: { table = n; status = "active"; }) priorityTables;
in
{
  inherit deskWorkers;
  n = builtins.length priorityTables;
  hasTen = builtins.elem 10 tableNumbers;
}

nix eval --file list_operations.nix --json
```

Output:

```
{"deskWorkers":[{"status":"active","table":1},{"status":"active","table":2},{"status":"active",
```

Case 3: else is required

Save as `bad_if.nix`:

```
# bad_if.nix
if true then 1

nix eval --file bad_if.nix
```

Output (shape):

```
error: syntax error, unexpected end of file, expecting 'else'
```

To omit an attr, do not skip `else`. Use:

```
{ } // (if enable then { workers = 4; } else { })
```

or `lib.optionalAttrs enable { workers = 4; }`.

Case 4: Concatenate packages-style lists

Save as `packages.nix`:

```
# packages.nix
let
  common = [ "git" "jq" ];
  extra = [ "ripgrep" ];
in
common ++ extra ++ (if true then [ "htop" ] else [ ])
```

```
nix eval --file packages.nix --json
```

Output:

```
["git","jq","ripgrep","htop"]
```

This is how `environment.systemPackages` concatenates across modules (the module system merges lists). Here you do it by hand.

Case 5: Lazy unused branch

Save as `lazy_if.nix`:

```
# lazy_if.nix
if true then "ok" else throw "no"
```

```
nix eval --file lazy_if.nix
```

Output:

```
"ok"
```

The `throw` is not forced. Invert the condition to see it.

The trap

The trap is `if cond then x`; with no `else`, copied from Bash. Parse error.

The other trap is `[ 1, 2 ]` with commas. Nix lists are **spaces**. Commas are `attrset` / `function-arg` style. `[ 1, 2 ]` is often a parse error or a surprising function call.

The boring rule

- `if` always has `else`. Both sides are values.
- Spaces in lists. `++` to join.
- `map` / `filter` / `length` / `elem` from `builtins`.
- `optional` / `optionalAttrs` (`lib`) for maybe-one-item.
- Unused `else throw` stays quiet — good for flags, bad if you meant to test both.

Try this

1. `builtins.length` already in Case 2; add `builtins.elem 3 priorityTables`.
2. Write `[ 1, 2 ]` and read the error (or the odd parse).
3. Case 5 with `false`.
4. `lib.optional` — after you have `pkgs.lib` in the imports chapter, replace the `if true then [ "htop" ] else [ ]` pattern.

Importing Files and Building Abstractions

Importing Files and Building Abstractions

One file does not hold a desk. The boring default is: **small files that return functions**, `import ./x.nix { ... }` with explicit args, and no `<nixpkgs>` in anything you ship.

Mental model

`import` evaluates a path to a value:

```
import ./constants.nix
import ./service.nix { port = 8080; }
```

The imported file cannot see your `let` names unless you pass them. There is no implicit global `pkgs` unless you created one.

`<nixpkgs>` looks up `NIX_PATH`. Two laptops, two channels, two worlds. Flakes (next parts) pin `github:NixOS/nixpkgs/nixos-26.05`. This chapter still uses relative `./` files.

`pkgs.lib` (once you have `pkgs`) is optional, `mkIf`, `recursiveUpdate`, `toUpper`, ... — prefer it over reinventing `if cond then [ x ] else [ ]`.

Worked examples

Case 1: Function file + caller

Save as `database.nix`:

```
# database.nix
{ environment }:

let
  isProduction = environment == "production";
in
{
  dbHost = if isProduction then "db.desk.corp" else "127.0.0.1";
  dbPort = if environment == "staging" then 5433 else 5432;
  poolSize = if isProduction then 50 else 5;
}
```

Save as `app.nix`:

```
# app.nix
let
  dbConfig = import ./database.nix { environment = "production"; };
in
{
  appName = "desk-billing";
  database = dbConfig;
}

nix eval --file app.nix --json
```

Output:

```
{"appName":"desk-billing","database":{"dbHost":"db.desk.corp","dbPort":5432,"poolSize":50}}
```

Case 2: Staging port

```
nix eval --expr '(import ./database.nix { environment = "staging"; }).dbPort'
```

Output:

```
5433
```

Case 3: lib.optional without a full nixpkgs (local copy)

Save as lib_demo.nix:

```
# lib_demo.nix
let
  optional = cond: x: if cond then [ x ] else [ ];
  features = [ "core-api" ]
    ++ optional true "metrics-exporter"
    ++ optional false "debug-profiler";
in
features

nix eval --file lib_demo.nix --json
```

Output:

```
["core-api","metrics-exporter"]
```

With nixpkgs 26.05: `lib.optional` is the same idea. Do not copy half of `lib` into every repo; `pkgs.lib` once you import `nixpkgs`.

Case 4: Relative paths are relative to the file

`import ./database.nix` in `app.nix` looks next to `app.nix`, not next to your shell `cwd`. `nix eval --file ./app.nix` from another directory still finds `./database.nix` beside `app.nix`.

Case 5: `<nixpkgs>` is ambient

```
nix eval --impure --expr 'builtins.nixPath'
```

Whatever `nixpkgs=` points at is **not** in `flake.lock`. Fine for a 10-second repl. Not fine for the desk flake. Next parts: `inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";`.

The trap

The trap is `import <nixpkgs> {}` in production modules. CI, a coworker, and a 6-month-old laptop disagree. Pin 26.05 in a lockfile.

The other trap is circular `import`: `a.nix` imports `b.nix` imports `a.nix`. Eval loop. Pass data downward; do not make a cycle.

The boring rule

- Files return functions. Callers pass args.
- `import ./relative.nix` — path relative to the file.
- No `<nixpkgs>` in shipped config. Flake pin `nixos-26.05`.
- Use `pkgs.lib` instead of a private util soup — once `pkgs` exists.
- No import cycles.

Try this

1. `import ./database.nix { environment = "staging"; }` and check `dbPort == 5433`.
2. Move `app.nix` eval to another `cwd`; confirm it still works.
3. `nix eval --impure --expr '<nixpkgs>'` and note it is a path, not a lock.
4. Split `desk_service.nix` from the `let` chapter into `url.nix` + `import`.

The Nix REPL and Debugging Expressions

The Nix REPL and Debugging Expressions

The boring default before editing any Nix file is **opening `nix repl` `.#` first**. It gives you an interactive evaluator for the current flake, lets you inspect types and attribute paths without touching the file system, and surfaces errors instantly — the same role Python’s REPL or `ghci` plays for Haskell.

Mental model

`nix repl` is an incremental expression evaluator. Each line you type is parsed and evaluated; the result is printed. The session accumulates bindings: assign with `x = 42` and `x` stays in scope for the rest of the session.

Key REPL meta-commands:

Command	What it does
<code>:t expr</code>	Print the inferred type of <code>expr</code>
<code>:e expr</code>	Open the definition of <code>expr</code> in <code>\$EDITOR</code>
<code>:b drv</code>	Realise a derivation (<code>nix-store --realise</code>)
<code>:p expr</code>	Pretty-print a value, expanding lazy thunks
<code>:l path</code>	Load a <code>.nix</code> file or flake into scope
<code>:r</code>	Reload — re-evaluate all <code>:l</code> expressions
<code>:?</code>	List every meta-command

There are two common launch modes:

```
nix repl           # empty scope
nix repl 'nixpkgs' # loads nixpkgs; pkgs = import <nixpkgs> {}
nix repl .#        # loads the flake in the current directory
```

Nix evaluation is **lazy**. The REPL only evaluates what you ask for. An attribute set with a broken nested path will not error until you actually access that path.

Worked examples

Case 1: Basic REPL session

This walkthrough covers arithmetic, strings, attribute sets, and function application — the primitives you will inspect constantly at the infrastructure desk.

Start the REPL with no arguments:

```
nix repl
```

Output:

```
Nix 2.35.2
Type :? for help.
```

```
nix-repl>
```

Arithmetic:

```
nix-repl> 2 + 2
4
```

```
nix-repl> 1024 * 1024
1048576
```

```
nix-repl> 7 / 2
3
```

Division is integer division for two integers. Use floats explicitly if you need fractional results.

Strings and interpolation:

```
nix-repl> "desk-" + "api"
"desk-api"
```

```
nix-repl> host = "db.desk.corp"
```

```
nix-repl> "connecting to ${host}:5432"
"connecting to db.desk.corp:5432"
```

Attribute sets:


```
nix-repl> svc = { name = "billing"; port = 8080; }
```

```
nix-repl> svc.name  
"billing"
```

```
nix-repl> svc.port  
8080
```

Function application:

```
nix-repl> greet = name: "Hello, ${name}!"
```

```
nix-repl> greet "ops-team"  
"Hello, ops-team!"
```

Type introspection with `:t`:

```
nix-repl> :t 42  
an integer
```

```
nix-repl> :t "desk-api"  
a string
```

```
nix-repl> :t svc  
a set
```

```
nix-repl> :t greet  
a function
```

```
nix-repl> :t [ 1 2 3 ]  
a list
```

```
nix-repl> :t true  
a Boolean
```

`:t` is the fastest way to catch a “I expected a string but got a set” mismatch before it becomes a build error.

Case 2: Loading `nixpkgs`

`nix repl 'nixpkgs'` imports `nixpkgs` and places its attribute set into scope. This is the standard way to inspect package metadata, library functions, and attribute paths at the infrastructure desk.

```
nix repl 'nixpkgs'
```

Output:

```
Nix 2.35.2
```

```
Type :? for help.
```

```
Loading installable 'nixpkgs'...
```

```
Added 22756 variables.
```

```
nix-repl>
```

Query a package version:

```
nix-repl> go.version
```

```
"1.22.3"
```

```
nix-repl> postgresql.version
```

```
"16.3"
```

```
nix-repl> prometheus.version
```

```
"2.51.2"
```

Inspect package metadata:

```
nix-repl> go.meta.description
```

```
"The Go Programming Language"
```

```
nix-repl> go.meta.homepage
```

```
"https://go.dev/"
```

Browse `lib` functions. Every function under `lib` is an attribute — tab-complete works:

```
nix-repl> lib.strings.toUpperCase "hello"
```

```
"HELLO"
```

```
nix-repl> lib.strings.concatStringsSep ", " [ "api" "worker" "scheduler" ]
```

```
"api, worker, scheduler"
```

```
nix-repl> lib.lists.flatten [ [ 1 2 ] [ 3 [ 4 5 ] ] ]
```

```
[ 1 2 3 4 5 ]
```

```
nix-repl> lib.attrsets.mapAttrs (k: v: v + 1) { a = 1; b = 2; }
```

```
{ a = 2; b = 3; }
```

Jump to a function's source with `:e`:

```
nix-repl> :e lib.strings.concatStringsSep
```

This opens `lib/strings.nix` at the definition of `concatStringsSep` in your `$EDITOR`. It is the fastest way to understand what a library function actually does without searching the `nixpkgs` repository manually.

Check whether an attribute exists before using it:

```
nix-repl> lib.attrsets.hasAttr "version" go
true
```

```
nix-repl> lib.attrsets.hasAttr "doesNotExist" go
false
```

Case 3: Loading a flake

`nix repl .#` loads the flake in the current directory. Every top-level output — `packages`, `devShells`, `nixosConfigurations`, `apps` — becomes directly accessible in scope.

The infrastructure desk flake used throughout this section:

```
# flake.nix
{
  description = "Desk infrastructure flake";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in
    {
      packages.${system}.default = pkgs.writeShellApplication {
        name = "desk-hello";
        text = ''
          echo "Infrastructure desk tooling - online."
        '';
      };

      devShells.${system}.default = pkgs.mkShell {
        packages = [ pkgs.go pkgs.gopls pkgs.golangci-lint ];
        shellHook = ''
```

```

        echo "desk devShell ready"
    '';
};

nixosConfigurations.workstation = nixpkgs.lib.nixosSystem {
    inherit system;
    modules = [ ./hosts/workstation/configuration.nix ];
};
}

```

Load the flake:

```
nix repl .#
```

Output:

Nix 2.35.2

Type :? for help.

Loading installable '.#'...

Added 7 variables.

nix-repl>

Explore outputs:

```
nix-repl> :t packages
```

a set

```
nix-repl> packages.x86_64-linux.default.name
```

"desk-hello"

```
nix-repl> :t devShells.x86_64-linux.default
```

a derivation

```
nix-repl> devShells.x86_64-linux.default.buildInputs
```

[«derivation /nix/store/...-go-1.22.3.drv» ...]

Explore NixOS configuration options:

```
nix-repl> nixosConfigurations.workstation.config.networking.hostName
```

"workstation"

```
nix-repl> :t nixosConfigurations.workstation.config.environment.systemPackages
```

a list

Build a package directly from the REPL with `:b`:

```
nix-repl> :b packages.x86_64-linux.default
```

Output:

This derivation produced the following outputs:

```
out -> /nix/store/r7kzxq3j4b5c6d7e8f9g-desk-hello
```

After `:b` succeeds you can inspect the output path in the shell without leaving the session. `:b` is the fastest build-test loop available: no terminal switching, no retyping the flake attribute path.

After editing `flake.nix`, reload without restarting:

```
nix-repl> :r
```

Output:

```
Loading installable '.#'...  
Added 7 variables.
```

Case 4: Debugging an expression with a wrong attribute path

The infrastructure desk's `shell.nix` has a typo — `pkgs.golint` instead of the correct `pkgs.golangci-lint`. Reproduce the error and then isolate it in the REPL.

Save the broken file:

```
# shell.nix  
{ pkgs ? import <nixpkgs> {} }:  
  
pkgs.mkShell {  
  packages = [  
    pkgs.go  
    pkgs.golint          # wrong: this attribute was removed from nixpkgs years ago  
    pkgs.gopls  
  ];  
}
```

Evaluate with `nix-instantiate --eval`:

```
nix-instantiate --eval shell.nix
```

Output:

```
error: attribute 'golint' missing
```

```
at /nix/store/...-source/pkgs/top-level/all-packages.nix:9423:5:
```

```
9422|   golang-migrate = callPackage ../tools/misc/golang-migrate { };
9423|   golint = throw "golint was removed from nixpkgs. Use golangci-lint instead."
9424|
```

```
... while evaluating the attribute 'golint'
at /home/ops/desk/shell.nix:6:5:
```

```
5|   pkgs.go
6|   pkgs.golint
7|   pkgs.gopls
```

The error is an **evaluation-time** error: Nix never started building anything. Now reproduce the same inspection in the REPL to narrow it down interactively:

```
nix repl 'nixpkgs'
```

```
nix-repl> pkgs = import <nixpkgs> {}
```

```
nix-repl> :t pkgs.go
a derivation
```

```
nix-repl> :t pkgs.golangci-lint
a derivation
```

```
nix-repl> :t pkgs.golint
error: attribute 'golint' missing
... while evaluating the attribute 'golint'
```

The REPL pinpoints exactly which path is broken. Fix the file:

```
# shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  packages = [
    pkgs.go
    pkgs.golangci-lint # correct attribute name
    pkgs.gopls
  ];
}
```

Re-evaluate:

```
nix-instantiate --eval shell.nix
```

Output:

```
«derivation /nix/store/...-nix-shell.drv»
```

Case 5: `builtins.trace` for `printf`-style debugging

When a conditional chooses the wrong branch or a version check silently produces unexpected output, `builtins.trace` prints a diagnostic line during evaluation and then returns its second argument unchanged. It is the Nix equivalent of a `print` statement.

Save as `default.nix`:

```
# default.nix
let
  pkgs = import <nixpkgs> {};

  # Confirm which version of a package is selected at eval time.
  tracedPkg = pkg:
    builtins.trace
      "checking version: ${pkg.pname}-${pkg.version}"
      pkg;

  selectedGo = tracedPkg pkgs.go;
in
{
  inherit selectedGo;
  name = selectedGo.pname;
}
```

Evaluate:

```
nix eval --file default.nix name
```

Output:

```
trace: checking version: go-1.22.3
"go"
```

The `trace:` prefix appears on `stderr`; the final value ("`go`") appears on `stdout`. Scripts that parse `nix eval` output are unaffected by trace lines.

Chain multiple traces to track a pipeline of transformations:

```
# default.nix
let
  pkgs = import <nixpkgs> {};

  step1 = builtins.trace "step1: selecting go"    pkgs.go;
  step2 = builtins.trace "step2: selecting gopls"  pkgs.gopls;
  step3 = builtins.trace "step3: building list"    [ step1 step2 ];
in
  step3
```

Evaluate:

```
nix eval --file default.nix
```

Output:

```
trace: step1: selecting go
trace: step2: selecting gopls
trace: step3: building list
[ «derivation /nix/store/...-go-1.22.3.drv» «derivation /nix/store/...-gopls-0.15.3.drv» ]
```

Because Nix is lazy, traces only fire when the binding is actually forced. If `step2` were never referenced elsewhere, its trace would never appear. This makes `builtins.trace` a precise tool: it confirms a value was evaluated, not just defined.

Remove all `builtins.trace` calls before committing. They print to `stderr` on every evaluation, including CI and deployment pipelines, producing confusing noise in logs.

The trap

Confusing evaluation-time errors with build-time errors.

An **evaluation-time error** (error: attribute 'foo' missing, error: value is a set while a string was expected) is detected by the Nix evaluator before any builder process starts. It appears immediately in `nix eval`, `nix repl`, and `nix build` — with no build log.

A **build-time error** (a compiler error, a failing test, a missing source file) is detected by the builder process after the evaluator has successfully constructed the derivation. It appears in the build log.

Evaluation-time error — REPL:

```
nix-repl> :b packages.x86_64-linux.typo
error: flake output attribute 'packages.x86_64-linux.typo' is not a derivation or path
```


No `.drv` was written. The evaluator rejected the expression before attempting to build.

Evaluation-time error — CLI:

```
nix build .#packages.x86_64-linux.typo
```

```
error: flake output attribute 'packages.x86_64-linux.typo' is not a derivation or path
```

Build-time error — REPL:

```
nix-repl> :b packages.x86_64-linux.desk-service
building '/nix/store/abc123-desk-service.drv'...
error: builder for '/nix/store/abc123-desk-service.drv' failed with exit code 1;
      last 10 log lines:
      > ./cmd/main.go:12:2: undefined: config.LoadEnv
```

Build-time error — CLI:

```
nix build .#packages.x86_64-linux.desk-service
```

```
error: builder for '/nix/store/abc123-desk-service.drv' failed with exit code 1;
      last 10 log lines:
      > ./cmd/main.go:12:2: undefined: config.LoadEnv
```

The evaluator succeeded (a `.drv` was written). The Go compiler failed inside the sandbox. The fix belongs in the source code, not in the Nix expression.

Diagnostic rule: if you see a `/nix/store/...drv` path in the error message, the evaluator succeeded and the problem is in the build. If there is no `.drv` path, the problem is in the Nix expression itself.

The boring rule

- Open `nix repl .#` before editing any file in a flake. Keep it open alongside your editor.
 - Use `:t expr` to sanity-check that a binding holds the type you expect before wiring it into a larger expression.
 - Use `builtins.trace` to confirm which branch a conditional takes and which version of a package is selected. Remove all traces before committing.
 - Use `:b drv` inside the REPL to test builds interactively instead of switching terminals and retyping `nix build .#...`
 - After editing `flake.nix`, type `:r` to reload the flake without restarting the session.
 - A `.drv` path in the error means eval succeeded; the bug is in source code. No `.drv` means eval failed; the bug is in the Nix expression.
-

Try this

1. Start `nix repl 'nixpkgs'`. Find the `version` and `meta.license.shortName` of `pkgs.ripgrep`. Then use `:e pkgs.ripgrep` to open its package definition and read what `buildInputs` it declares.
2. In a flake-based project on your workstation, run `nix repl .#`. Type `:t devShells.x86_64-linux.default` to confirm it is a derivation, then `:b devShells.x86_64-linux.default` to build its environment closure. Inspect the output path with `nix path-info --closure-size /nix/store/<result>` — how large is the closure in megabytes?
3. Write a `probe.nix` that accesses a missing attribute (`pkgs.nonexistentTool`). Evaluate it with `nix-instantiate --eval probe.nix`. Then load the same file in `nix repl` with `:l probe.nix`. Does the error fire on load, or only when you type the binding name and press Enter? What does this tell you about Nix's lazy evaluation?
4. Add `builtins.trace` calls to an existing `shell.nix` or `flake.nix` to print the version of each package in a list using `builtins.map` and `builtins.concatStringsSep`. Run `nix eval --file shell.nix` and confirm the traces appear. Then remove the traces, re-evaluate, and verify `stderr` is clean.

Dev Environments

Nix Shells and the devShell Concept

Nix Shells and the devShell Concept

Development environments rot when developers install tools directly onto their host machines. The boring default is: **define a `shell.nix` or `devShell` declaring the exact tools required, and activate it with `nix-shell` or `nix develop`.**

Mental model

A Nix devShell is an ephemeral execution environment: 1. It downloads or builds only the packages listed in `packages` or `buildInputs`. 2. It sets up `$PATH`, environment variables, and compiler flags in a child shell. 3. It does not alter your workstation's global system directories. 4. When you exit the shell (`exit` or `Ctrl+D`), your environment reverts instantly to normal.

Host OS (`$PATH = /usr/bin`)

```
nix-shell starts
  Injects /nix/store/abc...-go-1.27/bin into $PATH
  Runs shellHook
  Developer compiles and tests
```

```
exit drops back to pristine Host OS ($PATH = /usr/bin)
```

Worked examples

Case 1: Simple `shell.nix`

Save as `shell.nix`.

```
# shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  packages = [
    pkgs.ripgrep
    pkgs.jq
  ];

  shellHook = ''
    echo "=== Desk Shell Ready ==="
```

```
    '';  
}
```

Run:

```
nix-shell --run "echo Shell active"
```

Output:

```
=== Desk Shell Ready ===  
Shell active
```

The tools are available only within that session.

Case 2: Setting Environment Variables in a Shell

Save as `env_shell.nix`.

```
# env_shell.nix  
{ pkgs ? import <nixpkgs> {} }:  
  
pkgs.mkShell {  
  packages = [ pkgs.curl ];  
  
  env = {  
    DESK_API_URL = "https://api.desk.internal";  
    DESK_TIMEOUT = "30";  
  };  
  
  shellHook = ''  
    echo "Targeting API: $DESK_API_URL"  
    '';  
}
```

Run:

```
nix-shell env_shell.nix --run "echo Timeout: \${DESK_TIMEOUT}"
```

Output:

```
Targeting API: https://api.desk.internal  
Timeout: 30
```

Environment variables are isolated and declared in code.

The trap

Installing development compilers like Go, Rust, or Node globally with system package managers (`apt`, `brew`). Different projects inevitably require different versions, leading to version conflicts.

The fix: Use a `shell.nix` in each repository.

The boring rule

- Every project repository must have a root `shell.nix` or `flake.nix`.
- Use `pkgs.mkShell` with `packages` for modern shells.
- Use `shellHook` for local environment configuration and diagnostics.

Try this

1. Add `pkgs.tree` to `shell.nix` and run `nix-shell --run "tree -L 1"`.

Multi-Language Development Environments

Multi-Language Development Environments

Modern systems frequently combine multiple languages: a Go backend, a TypeScript frontend, Python data scripts, and C shared libraries. The boring default is: **declare all runtimes, compilers, and library headers inside a single unified `mkShell` definition.**

Mental model

In traditional systems, managing polyglot stacks requires coordinating multiple independent version managers (`asdf`, `nvm`, `pyenv`, `rustup`) and installing system development packages (`libssl-dev`, `libpq-dev`).

Nix unifies this: - Compilers (`go`, `rustc`, `nodejs`) are just packages. - C libraries (`openssl`, `postgresql`) provide headers and shared objects automatically. - `pkg-config` is wired up automatically when included in `nativeBuildInputs`.

Worked examples

Case 1: Unified Polyglot Stack

Save as `polyglot.nix`.

```
# polyglot.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  packages = [
    pkgs.go
    pkgs.nodejs_22
    (pkgs.python3.withPackages (ps: [ ps.requests ps.pydantic ]))
    pkgs.sqlite
  ];

  shellHook = ''
    echo "Polyglot environment loaded: Go, Node, Python, SQLite"
  '';
}
```

Run:

```
nix-shell polyglot.nix --run "go version && node --version && python3 --version"
```

Output:

```
Polyglot environment loaded: Go, Node, Python, SQLite
go version go1.27.1 linux/amd64
v22.11.0
Python 3.12.7
```

All runtimes coexist without path or library collision.

Case 2: C Libraries and pkg-config

Save as `native_c.nix`. A shell that configures C headers and libraries for applications that compile native extensions.

```
# native_c.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  nativeBuildInputs = [ pkgs.pkg-config ];
  buildInputs = [ pkgs.openssl pkgs.zlib ];

  shellHook = ''
    echo "OpenSSL cflags: $(pkg-config --cflags openssl)"
  '';
}
```

Run:

```
nix-shell native_c.nix --run "echo C libraries configured"
```

Output:

```
OpenSSL cflags: -I/nix/store/...-openssl-3.3.2-dev/include
C libraries configured
```

`pkg-config` points directly to the isolated store header directories.

The trap

Relying on host system header files like `/usr/include/openssl`. If a teammate is on a different Linux distribution or macOS, paths differ and builds fail.

The fix: Always include header and library dependencies explicitly in `buildInputs`.

The boring rule

- Group build-time tools (like `pkg-config`, `cmake`) in `nativeBuildInputs`.
- Group runtime libraries and headers (like `openssl`, `zlib`) in `buildInputs`.
- Use language-specific package wrappers (like `python3.withPackages`).

Try this

1. In `polyglot.nix`, add `pkgs.cargo` and test running `cargo --version`.

Flakes for Reproducible Dev Environments

Flakes for Reproducible Dev Environments

Classic `shell.nix` still relies on `<nixpkgs>`, which can vary across developer machines depending on their channel state. The boring default is: **use modern Nix Flakes (`flake.nix` + `flake.lock`) to pin the exact commit of nixpkgs across the entire team.**

Mental model

Nix Flakes introduce three critical improvements: 1. **Pinned Lockfiles (`flake.lock`)**: Automatically records the exact Git commit hash of every dependency. 2. **Standard Output Schema**: Predictable structure (`packages.${system}`, `devShells.${system}.default`). 3. **Pure Evaluation**: Flakes disallow reading undeclared files, ambient environment variables, or mutable network channels.

`flake.nix` (declares inputs & outputs)

`flake.lock` (records exact Git commit of nixpkgs)

`nix develop` (creates identical shell on all machines)

Worked examples

Case 1: Minimal Flake Development Shell

Save as `flake.nix`.

```
# flake.nix
{
  description = "Desk management service development shell";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  };

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
```

```

    pkgs = import nixpkgs { inherit system; };
in
{
    devShells.${system}.default = pkgs.mkShell {
        packages = [
            pkgs.go
            pkgs.gopls
            pkgs.golangci-lint
        ];

        shellHook = ''
            echo "=== Pinned Go 1.27 Dev Environment ==="
        '';
    };
};
}

```

Run:

```
nix develop --command go version
```

Output:

```

=== Pinned Go 1.27 Dev Environment ===
go version go1.27.1 linux/amd64

```

The exact commit of nixpkgs is frozen into flake.lock.

Case 2: Updating Dependencies

When you are ready to update packages across the team, update the lockfile explicitly:

Run:

```
nix flake update
```

Output:

```

updating lock file '/home/user/project/flake.lock':
• Updated input 'nixpkgs':
  'github:NixOS/nixpkgs/...' (2026-09-01)
→ 'github:NixOS/nixpkgs/...' (2026-09-08)

```

Updates are an intentional Git commit, never an accidental surprise.

The trap

Forgetting to check `flake.lock` into Git. Without `flake.lock`, teammates generate their own lockfiles on different dates, losing strict bit-for-bit reproducibility.

The fix: Always commit `flake.lock` alongside `flake.nix`.

The boring rule

- Always use Flakes for new development environments.
- Always commit `flake.lock` to source control.
- Use `nix develop` to enter Flake-based development shells.

Try this

1. Inspect `flake.lock` with `cat flake.lock` or `jq . flake.lock` and locate the `rev` (revision) field.

Environment Management with Home Manager

Environment Management with Home Manager

`nix develop` is the **project**. Home Manager is the **person**. The boring default is: **personal git/prompt/editor in Home Manager; compilers only in the project's devShell** — never **Go 1.x in `home.packages`** “for every repo”.

Part 7 teaches Home Manager as a NixOS user module. This chapter is the boundary with part 3's shells, including on a foreign Linux laptop (Nix 2.35, nixpkgs 26.05).

Mental model

```
~/ .nix-profile      Home Manager: jq, bat, git config, nvim
direnv / nix develop
    PATH prepends  go, gopls, postgresql client  for this repo only
```

When you `cd` out, the compiler vanishes. Your `git` aliases stay. That split is how two repos can pin two Go versions without fighting.

`home.stateVersion = "26.05"`; on a **new** user born on 26.05. If the user was created on 24.11, leave `"24.11"`.

Worked examples

Case 1: Personal tools only

Save as `home.nix`:

```
# home.nix
{ pkgs, ... }:

{
  home.username = "deskuser";
  home.homeDirectory = "/home/deskuser";
  home.stateVersion = "26.05";

  home.packages = with pkgs; [
    htop
    bat
    fzf
    jq
```

```

    ripgrep
  ];

  programs.git = {
    enable = true;
    userName = "Desk Engineer";
    userEmail = "engineer@desk.corp";
  };
}

home-manager switch --flake .#deskuser
which jq

```

jq is a store path. It is not gcc.

Case 2: Project compiler stays in the flake

Save as `flake.nix` in the service repo:

```

# flake.nix
{
  description = "Desk API";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in
    {
      devShells.${system}.default = pkgs.mkShell {
        packages = [ pkgs.go pkgs.gopls pkgs.postgresql ];
      };
    };
}

# outside the shell
which go || echo 'no go'
nix develop --command which go

```

Home Manager must **not** also install `pkgs.go`. If it does, `which go` outside the project still works — and CI (which only has the flake) does not match your laptop.

Case 3: direnv composes, it does not replace HM

`.envrc` with `use flake` prepends the `devShell`. Starship from Home Manager still runs. You do not put `programs.git` in the project flake.

Case 4: Same nixpkgs as the OS when you can

On NixOS 26.05, Home Manager follows the system `nixpkgs` (part 7). On Ubuntu + Nix 2.35, pin `nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";` in the `user` flake too, so `jq` is the same closure as CI.

Case 5: Rollback is per layer

```
home-manager generations
home-manager switch --rollback
```

This does not roll back the project's `go. nix develop` is not a Home Manager generation. Two rollbacks, two tools, two meanings.

The trap

The trap is `home.packages = [ pkgs.go pkgs.nodejs pkgs.rustc ]`; as a “developer workstation.” The next repo needs another version. You overlay. CI diverges. Put toolchains in the repo flake.

The boring rule

- Home Manager: identity, editor, prompt, generic CLIs.
- `devShell`: language toolchains and DB clients the app needs.
- `stateVersion` is birth, not “current channel.”
- Same `nixpkgs` pin as the desk OS when the machine is NixOS 26.05.
- Do not install compilers globally “so `PATH` always has `go`.”

Try this

1. Add `pkgs.hello` to `home.packages`, `switch`, `hello`, remove it, `switch`.
2. `which go` in and out of `nix develop` on a repo that lists `pkgs.go`. Out must fail (or be distro `go` you should not use).
3. `echo $PATH` inside `direnv` and label HM vs `devShell` entries.
4. Set `home.stateVersion` wrongly to a future value on a lab user, read the warning, restore the birth value.

Migrating from Docker Desktop to Nix DevShells

Migrating from Docker Desktop to Nix DevShells

Many teams default to running basic development tools inside Docker containers to avoid dependency drift. The boring default is: **migrate local development toolchains to native Nix devShells for near-zero startup latency, direct editor integration, and fractional resource consumption.**

Mental model

Dimension	Docker Desktop Container	Nix devShell
Architecture	Linux VM + Container Daemon	Native Host Process
Startup Latency	5 – 20 seconds	< 50 milliseconds
File I/O Performance	Slow across VM bind-mounts	Native host filesystem speed
IDE Integration	Requires Remote-Containers plugins	Direct local LSP & debugger integration
Resource Usage	4 – 8 GB RAM continuously	Exact binary memory only when executed

Containers remain the right choice for isolated services (e.g. running an actual PostgreSQL database or Redis instance). They are the wrong choice for running simple compilers (go, rustc, npm).

Worked examples

Case 1: The Docker Compose Pattern vs The Nix devShell Pattern

Compare a typical `docker-compose.yml` with a clean `flake.nix`.

Docker setup requires bind mounts, ports, and a persistent background VM.

The Nix equivalent:

Save as `flake.nix`:

```
# flake.nix
{
  description = "Fast native desk development environment";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
```

```

outputs = { self, nixpkgs }:
  let
    pkgs = import nixpkgs { system = "x86_64-linux"; };
  in
  {
    devShells.x86_64-linux.default = pkgs.mkShell {
      packages = [
        pkgs.go
        pkgs.golangci-lint
        pkgs.delve
      ];
    };
  };
}

```

Run:

```
nix develop --command go test ./...
```

Output:

```
ok      desk/service    0.015s
```

Compilation and tests run at native host speed without VM translation layers or volume-mount caching slowdowns.

Case 2: Hybrid Workflow — Native Tooling with Containerized Backends

The recommended production pattern: - **Language toolchains & linters:** Run natively via Nix devShell. - **Stateful databases & caches:** Run as lightweight containers via Podman or Docker.

Save as `start_services.sh`:

```

# start_services.sh
#!/usr/bin/env bash
set -euo pipefail

echo "Launching isolated database container..."
podman run -d --name desk-db -p 5432:5432 -e POSTGRES_PASSWORD=secret postgres:16-alpine ||

echo "Running migrations with native Nix-provided toolchain:"
psql -h 127.0.0.1 -U postgres -c "SELECT 'Desk database online' AS status;"

```

Run:

```
nix-shell -p postgresql --run "bash start_services.sh"
```

Output:

```
Launching isolated database container...
Running migrations with native Nix-provided toolchain:
      status
-----
Desk database online
(1 row)
```

The CLI tools are provided instantly by Nix; the database runs isolated in a container.

The trap

Running full GUI editors or IDEs inside containers or remote VMs when simple local toolchain isolation is all that was needed.

The fix: Use Nix to provide the compilers and tools on your host OS. Your editor connects directly to native binaries on `$PATH`.

The boring rule

- Use Nix devShells for compilers, language servers, CLI tools, and linters.
- Reserve containers for long-running stateful services (databases, message queues).
- Enjoy native host file I/O speed and seamless editor integration.

Try this

1. Compare the time taken to run `go version` natively in a Nix devShell vs inside a `docker run --rm golang go version` container.

Automatic Dev Shells with direnv

Automatic Dev Shells with direnv

Running `nix develop` by hand is ceremony you will forget. `direnv` eliminates it: the moment you `cd` into a project directory, your shell gains every tool that project declared. `nix-direnv` makes that transition instant and GC-safe. Together they are the boring default for any infrastructure desk running multiple projects side by side.

Mental model

`direnv` is a shell hook. After you add it to your shell's init file, it intercepts every directory change. When the new directory contains a file named `.envrc`, `direnv` evaluates that file and exports the resulting environment variables into your running shell. When you leave the directory, those variables vanish.

`nix-direnv` provides a single function — `use flake` — that you call from `.envrc`. It replaces `direnv`'s slower built-in `use nix` with a version that:

1. Builds the devShell once and caches the result in `/nix/store`.
2. Creates a GC-root symlink at `.direnv/flake-profile` so that `nix-collect-garbage -d` cannot evict the store paths the shell depends on.
3. Re-evaluates only when `flake.nix`, `flake.lock`, or `.envrc` changes — not on every `cd`.

Lifecycle

```
cd project/
  direnv detects .envrc
    nix-direnv: cache hit? → restore env (ms)
      cache miss? → nix build → cache → restore env (s/min, once)
        PATH, PKG_CONFIG_PATH, ... exported into current shell
cd ..
  direnv unloads the environment
```

The only files a project needs beyond `flake.nix` are:

File	Purpose	Commit?
<code>.envrc</code>	tells <code>direnv</code> to <code>use flake</code>	yes
<code>.direnv/</code>	<code>nix-direnv</code> cache and GC root	no (gitignore)

Worked examples

Case 1: Installing direnv and nix-direnv via Home Manager

Home Manager is the standard way to install direnv on a NixOS workstation. The module handles shell hook injection automatically — you do not touch `.bashrc` or `.zshrc` by hand.

Save as `home.nix`:

```
# home.nix
{ pkgs, ... }:

{
  programs.direnv = {
    enable = true;          # installs direnv and adds the shell hook
    nix-direnv.enable = true; # replaces built-in use_nix with nix-direnv
  };

  # Optional: silence direnv on every cd; it still prints on changes.
  home.sessionVariables = {
    DIRENV_LOG_FORMAT = "";
  };
}
```

Apply the configuration:

```
home-manager switch
```

Output:

```
Starting Home Manager activation
Activating checkFilesChanged
Activating installPackages
replacing /home/ops/.nix-profile → /nix/store/...-home-manager-path
Activating linkGeneration
Activating onFilesChange
Activating reloadSystemd
```

Open a new shell (or `exec $SHELL`) so the hook takes effect. Now trust the first project directory:

```
direnv allow ~/projects/desk
```

Output:

```
direnv: loading ~/projects/desk/.envrc
direnv: using flake
direnv: nix-direnv: renewed cache
direnv: export +AR +AS +CC +CONFIG_SHELL ... +buildInputs
```

`direnv allow` is a one-time command per directory. It records a hash of `.envrc` in `~/.local/share/direnv/allow/`. If `.envrc` changes later you must run `direnv allow` again — a deliberate safety gate.

Case 2: A Go desk project

The infrastructure desk maintains a small Go service that talks to the internal metrics API. The flake declares a devShell with `go`, `gopls`, and `gotools`. The `.envrc` is a single line.

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk metrics service";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    flake-utils.url = "github:numtide/flake-utils";
  };

  outputs = { self, nixpkgs, flake-utils }:
    flake-utils.lib.eachDefaultSystem (system:
      let
        pkgs = nixpkgs.legacyPackages.${system};
      in
      {
        devShells.default = pkgs.mkShell {
          name = "desk-metrics";
          packages = [
            pkgs.go_1_23
            pkgs.gopls
            pkgs.gotools # goimports, etc.
          ];

          shellHook = ''
            echo "desk-metrics shell: $(go version)"
          '';
        };
      });
}
```

Save as `.envrc`:

```
# .envrc
use flake
```

Allow and enter:

```
cd ~/projects/desk-metrics
direnv allow
```

Output:

```
direnv: loading ~/projects/desk-metrics/.envrc
direnv: using flake
direnv: nix-direnv: building flake output 'devShell.x86_64-linux.default'
[... nix build output ...]
direnv: nix-direnv: renewed cache
desk-metrics shell: go version go1.23.4 linux/amd64
direnv: export +AR +AS +CC +CGO_ENABLED +GOPATH ... +buildInputs
```

Subsequent entries are instant:

```
cd ..
cd ~/projects/desk-metrics
```

Output:

```
direnv: loading ~/projects/desk-metrics/.envrc
direnv: using flake
direnv: nix-direnv: using cached dev shell
desk-metrics shell: go version go1.23.4 linux/amd64
direnv: export +AR +AS +CC +CGO_ENABLED +GOPATH ... +buildInputs
```

Verify the toolchain:

```
which go gopls goimports
```

Output:

```
/nix/store/...-go-1.23.4/bin/go
/nix/store/...-gopls-0.16.2/bin/gopls
/nix/store/...-gotools-2024-01-01/bin/goimports
```

Case 3: A polyglot project with named devShells

The desk's internal portal has a Go backend and a Node/TypeScript frontend. They live in the same repository but need separate toolchains. A single flake with two named devShells keeps everything in one place.

Save as `flake.nix`:

```

# flake.nix
{
  description = "Desk portal - backend + frontend";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    flake-utils.url = "github:numtide/flake-utils";
  };

  outputs = { self, nixpkgs, flake-utils }:
    flake-utils.lib.eachDefaultSystem (system:
      let
        pkgs = nixpkgs.legacyPackages.${system};
      in
      {
        devShells.backend = pkgs.mkShell {
          name = "portal-backend";
          packages = [
            pkgs.go_1_23
            pkgs.gopls
            pkgs.gotools
            pkgs.sqlite
          ];
          shellHook = ''
            echo "[backend] $(go version)"
          '';
        };

        devShells.frontend = pkgs.mkShell {
          name = "portal-frontend";
          packages = [
            pkgs.nodejs_22
            pkgs.nodePackages.typescript
            pkgs.nodePackages.typescript-language-server
          ];
          shellHook = ''
            echo "[frontend] node $(node --version)"
          '';
        };
      });
}

```

Each subdirectory gets its own `.envrc` pointing at the correct output.

Save as `backend/.envrc`:

```
# backend/.envrc
use flake ..#backend
```

Save as frontend/.envrc:

```
# frontend/.envrc
use flake ..#frontend
```

Allow both, then switch between them:

```
direnv allow backend
direnv allow frontend
cd backend
```

Output:

```
direnv: loading ~/projects/portal/backend/.envrc
direnv: using flake ..#backend
direnv: nix-direnv: using cached dev shell
[backend] go version go1.23.4 linux/amd64
direnv: export +AR +AS +CC ... +buildInputs
```

```
cd ../frontend
```

Output:

```
direnv: unloading
direnv: loading ~/projects/portal/frontend/.envrc
direnv: using flake ..#frontend
direnv: nix-direnv: using cached dev shell
[frontend] node v22.9.0
direnv: export +NODE_PATH ... +buildInputs
```

The Go tools are absent from the frontend shell and vice versa. No manual activation or deactivation is needed.

Case 4: Caching and GC safety

Every time nix-direnv builds a devShell it writes a symlink under `.direnv/` that acts as a Nix GC root. This prevents `nix-collect-garbage -d` from deleting the store paths your shell depends on.

Inspect the structure after the first `use flake`:

```
ls -la ~/projects/desk-metrics/.direnv/
```

Output:

```
total 16
drwxr-xr-x 3 ops ops 4096 Sep  9 10:12 .
drwxr-xr-x 8 ops ops 4096 Sep  9 10:12 ..
drwxr-xr-x 2 ops ops 4096 Sep  9 10:12 flake-inputs
lrwxrwxrwx 1 ops ops   80 Sep  9 10:12 flake-profile -> /nix/store/...-desk-metrics-profile
lrwxrwxrwx 1 ops ops   80 Sep  9 10:12 flake-profile-1-link -> /nix/store/...-desk-metrics-profile
```

```
ls -la ~/projects/desk-metrics/.direnv/flake-inputs/
```

Output:

```
total 8
drwxr-xr-x 2 ops ops 4096 Sep  9 10:12 .
drwxr-xr-x 3 ops ops 4096 Sep  9 10:12 ..
lrwxrwxrwx 1 ops ops   83 Sep  9 10:12 nixpkgs -> /nix/store/...-source
```

Now run garbage collection:

```
nix-collect-garbage -d
```

Output:

```
finding garbage collector roots...
removing stale link from '/nix/var/nix/gcroots/per-user/ops/...' to '...'
deleting garbage...
deleting '/nix/store/...-some-old-package'
...
2.31 GiB freed
```

Your devShell store paths survive because `.direnv/flake-profile-1-link` is registered as a GC root. Re-enter the directory:

```
cd ~/projects/desk-metrics
```

Output:

```
direnv: loading ~/projects/desk-metrics/.envrc
direnv: using flake
direnv: nix-direnv: using cached dev shell
desk-metrics shell: go version go1.23.4 linux/amd64
direnv: export +AR ...
```

The shell is still instant — nothing was collected.

Add `.direnv/` to the project's `.gitignore`:

```
# .gitignore
.direnv/
```

The symlinks are machine-local. Committing them would break other developers' paths.

The trap

Using plain `use nix` instead of `use flake`

`direnv` ships with a built-in `use_nix` helper for legacy `shell.nix` files. A common mistake is writing `.envrc` like this for a flake project:

```
# .envrc ← WRONG for a flake project
use nix
```

`use nix` has no cache. It invokes `nix-shell` on every directory entry. For a devShell with many dependencies, that means several seconds of evaluation on every `cd`.

Symptom:

```
cd ~/projects/desk-metrics
```

Output:

```
direnv: loading ~/projects/desk-metrics/.envrc
direnv: using nix
[4.2s pause]
direnv: export +AR +AS +CC ...
```

Fix: use `use flake` (provided by `nix-direnv`):

```
# .envrc ← correct
use flake
```

Result:

```
cd ~/projects/desk-metrics
```

Output:

```
direnv: loading ~/projects/desk-metrics/.envrc
direnv: using flake
direnv: nix-direnv: using cached dev shell
direnv: export +AR +AS +CC ...
```


The difference is a cache layer. `use flake` stores the built environment as a profile and reloads it by reading symlinks — no Nix evaluation occurs on a cache hit.

A related mistake is forgetting to re-run `direnv allow` after editing `.envrc`. `direnv` blocks evaluation of changed files until you explicitly approve them:

```
direnv: error ~/projects/desk-metrics/.envrc is blocked. Run `direnv allow` to approve its content
```

This is a security feature, not a bug. Run `direnv allow` after every intentional `.envrc` change.

The boring rule

- **Commit `.envrc`** with a single line: `use flake` (or `use flake .#<name>` for named shells). It is part of the project contract.
- **Gitignore `.direnv/`**. Its contents are machine-local GC roots and cached profiles.
- **Run `direnv allow` explicitly** every time `.envrc` changes. Treat it as a checkpoint: you reviewed the file and approved it.
- **Never use `use nix` in a flake project**. The built-in has no cache and no GC safety. Always use the `nix-direnv use flake`.
- **Install via Home Manager**, not ad-hoc. The `programs.direnv` module wires the shell hook correctly for `bash`, `zsh`, and `fish` without manual init-file edits.
- **One `.envrc` per shell boundary**. Subdirectory `.envrc` files can reference the parent flake with `use flake ../#<name>`. Do not duplicate flake outputs.

Try this

1. **Baseline timing**. In an existing flake project, write an `.envrc` with `use nix` and measure the cold entry time with `time (cd project && cd ..)`. Then switch to `use flake`, run `direnv allow`, and measure again. Record both numbers and note the difference.
2. **Named-shell workspace**. Create a `~/projects/portal` repository with the two-shell flake from Case 3. Add `backend/.envrc` and `frontend/.envrc`. Verify that `which go` returns a path only inside `backend/` and `which node` returns a path only inside `frontend/`.
3. **GC resilience**. Enter a `direnv`-managed project and note a store path from `echo $buildInputs | tr ' ' '\n' | head -1`. Run `nix-collect-garbage -d`. Confirm the store path still exists with `ls /nix/store/<path>`. Then delete `.direnv/` manually, re-enter the directory, and observe `nix-direnv` rebuilding from scratch.
4. **Audit your trust**. Run `cat ~/.local/share/direnv/allow/*` to list all allowed `.envrc` hashes on your workstation. For each corresponding directory, open `.envrc` and confirm the content matches what you expect. Revoke any stale entries with `direnv deny <path>`.

nix-darwin: Declarative macOS Configuration

nix-darwin: Declarative macOS Configuration

Half the infrastructure desk runs macOS laptops; the other half runs NixOS servers. Without `nix-darwin`, the macOS machines drift — Homebrew packages installed by hand, `defaults write` commands nobody remembers, and a setup doc that is always three months out of date. The boring default is: **declare macOS system configuration in the same flake that manages Linux, and apply it with `darwin-rebuild switch`.**

Mental model

NixOS uses `nixosConfigurations` to describe Linux machines. `nix-darwin` adds `darwinConfigurations` for macOS machines. Both live in the same `flake.nix`.

```
flake.nix
  nixosConfigurations.desk-server  ← nixos-rebuild switch
  darwinConfigurations.desk-mac    ← darwin-rebuild switch
```

`nix-darwin` manages:

Layer	Managed by nix-darwin
System packages	<code>environment.systemPackages</code>
Homebrew casks	<code>homebrew.casks</code> (via <code>nix-homebrew</code>)
launchd daemons & agents	<code>launchd.daemons</code> , <code>launchd.agents</code>
macOS defaults	<code>system.defaults.*</code>
Environment variables	<code>environment.variables</code>
Shell initialisation	<code>programs.zsh</code> , <code>programs.fish</code>
Per-user config	Home Manager as a module

`nix-darwin` **cannot** manage SIP-protected system paths, kernel extensions, or anything that macOS's System Integrity Protection locks away. Attempting to do so produces a clear error at build time — not a silent failure at runtime.

The `apply` command is the macOS twin of `nixos-rebuild switch`:

```
nixos-rebuild switch --flake .#desk-server  ← Linux
darwin-rebuild switch --flake .#desk-mac    ← macOS
```

Worked examples

Case 1: Minimal darwinConfiguration

This is the smallest working `nix-darwin` flake. It sets two `system.defaults` tweaks that every developer on the team wants: auto-hiding Dock and a fast key-repeat rate.

`nix-darwin` is not part of `nixpkgs`. It is its own flake input, pinned like any other dependency.

Save as `flake.nix`:

```
# flake.nix
{
  description = "desk-mac - minimal nix-darwin configuration";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

    nix-darwin = {
      url = "github:LnL7/nix-darwin/master";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  outputs = { self, nixpkgs, nix-darwin }:
    let
      system = "aarch64-darwin";          # M-series Mac; use x86_64-darwin for Intel
      pkgs    = nixpkgs.legacyPackages.${system};
    in
    {
      darwinConfigurations."desk-mac" = nix-darwin.lib.darwinSystem {
        inherit system;
        modules = [
          ({ pkgs, ... }: {
            # System packages available to all users
            environment.systemPackages = [
              pkgs.ripgrep
              pkgs.jq
              pkgs.git
            ];

            # macOS system defaults
            system.defaults = {
              dock.autohide = true;
              dock.autohide-delay = 0.0;
              dock.autohide-time-modifier = 0.2;

              NSGlobalDomain.InitialKeyRepeat = 15;    # delay before repeat (lower = faster)
            };
          });
        ];
      };
    }
  };
}
```

```

        NSGlobalDomain.KeyRepeat = 2;           # repeat interval (lower = faster)
    };

    # nix-darwin requires this to be set
    system.stateVersion = 5;

    # Required: tell nix-darwin which Nix package to use
    nix.package = pkgs.nix;

    # Allow unfree packages if needed
    nixpkgs.config.allowUnfree = true;
  })
];
};
};
}

```

Install `nix-darwin` once on a fresh Mac, then apply the configuration:

```
darwin-rebuild switch --flake .#desk-mac
```

Output:

```

building the system configuration...
setting up /etc/zshrc...
setting up /etc/bashrc...
setting up /etc/nix/nix.conf...
system defaults applied: dock.autohide = true, KeyRepeat = 2
reloading Dock...
setting up launchd services...
/run/current-system -> /nix/store/3zqr9m4s...-darwin-system

```

System generation: desk-mac-1 (2024-11-20)

Every subsequent `darwin-rebuild switch` is idempotent. Running it twice produces no changes.

Case 2: One flake for Linux and macOS

The desk team maintains one repository. `desk-server` is a NixOS machine. `desk-mac` is a developer's MacBook. Both receive the same core tools from a shared `commonPackages` list.

Save as `flake.nix`:

```

# flake.nix
{
  description = "Desk infrastructure - Linux + macOS from one flake";

```

```

inputs = {
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  nix-darwin = {
    url = "github:LnL7/nix-darwin/master";
    inputs.nixpkgs.follows = "nixpkgs";
  };
};

outputs = { self, nixpkgs, nix-darwin }:
  let
    # Packages every desk engineer needs regardless of OS
    commonPackages = pkgs: [
      pkgs.ripgrep
      pkgs.jq
      pkgs.git
      pkgs.go
    ];
  in
  {
    # Linux server
    nixosConfigurations."desk-server" = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        ({ pkgs, ... }: {
          environment.systemPackages = commonPackages pkgs;

          # Server-only: enable the SSH daemon
          services.openssh.enable = true;

          system.stateVersion = "26.05";
        })
      ];
    };

    # macOS workstation
    darwinConfigurations."desk-mac" = nix-darwin.lib.darwinSystem {
      system = "aarch64-darwin";
      modules = [
        ({ pkgs, ... }: {
          environment.systemPackages = commonPackages pkgs ++ [
            pkgs.mas # Mac App Store CLI - macOS only
          ];

          system.defaults.dock.autohide = true;
        })
      ];
    };
  }

```

```

        system.stateVersion = 5;
        nix.package = pkgs.nix;
        nixpkgs.config.allowUnfree = true;
    })
];
};
};
}

```

Apply to the server (run on the Linux machine):

```
nixos-rebuild switch --flake .#desk-server
```

Output:

```

building the system configuration...
activating the configuration...
setting up /etc/ssh/sshd_config...
starting sshd.service

```

Apply to the Mac (run on the macOS machine):

```
darwin-rebuild switch --flake .#desk-mac
```

Output:

```

building the system configuration...
system defaults applied
setting up /etc/zshrc...
/run/current-system -> /nix/store/7f2kp1rs...-darwin-system

```

ripgrep, jq, git, and go are now in `/run/current-system/sw/bin` on both machines. Both were updated from a single source of truth.

Case 3: Declarative Homebrew casks via nix-homebrew

`nix-darwin` can drive Homebrew declaratively through the `nix-homebrew` module. GUI macOS apps — which are not in `nixpkgs` — are listed in code and installed automatically during `darwin-rebuild switch`. No manual `brew install --cask` commands.

`nix-homebrew` is a separate flake input that bridges `nix-darwin` and Homebrew. It installs Homebrew itself under `/opt/homebrew` if absent and keeps the tap and cask list in sync.

Save as `flake.nix`:

```

# flake.nix
{

```

```

description = "desk-mac with declarative Homebrew casks";

inputs = {
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  nix-darwin = {
    url = "github:LnL7/nix-darwin/master";
    inputs.nixpkgs.follows = "nixpkgs";
  };

  nix-homebrew = {
    url = "github:zhaofengli/nix-homebrew";
    inputs.nixpkgs.follows = "nixpkgs";
  };

  homebrew-core = {
    url = "github:homebrew/homebrew-core";
    flake = false;
  };

  homebrew-cask = {
    url = "github:homebrew/homebrew-cask";
    flake = false;
  };
};

outputs = { self, nixpkgs, nix-darwin, nix-homebrew,
             homebrew-core, homebrew-cask }:
{
  darwinConfigurations."desk-mac" = nix-darwin.lib.darwinSystem {
    system = "aarch64-darwin";
    modules = [
      # Wire in nix-homebrew
      nix-homebrew.darwinModules.nix-homebrew
    ]
    {
      nix-homebrew = {
        enable = true;
        enableRosetta = true;    # required on Apple Silicon for x86_64 casks
        user = "alice";          # the user that owns the Homebrew prefix

        taps = {
          "homebrew/homebrew-core" = homebrew-core;
          "homebrew/homebrew-cask" = homebrew-cask;
        };

        # Prevent Homebrew from auto-updating outside nix-darwin
        autoMigrate = true;
      };
    }
  };
}

```



```

    };
  }

  ({ pkgs, ... }: {
    environment.systemPackages = [
      pkgs.ripgrep
      pkgs.jq
      pkgs.git
    ];

    # Casks are GUI apps not available in nixpkgs
    homebrew = {
      enable = true;

      casks = [
        "ghostty"    # GPU-accelerated terminal
        "1password"  # password manager
        "arc"         # browser
      ];

      # Remove casks that are no longer in the list on next switch
      onActivation.cleanup = "zap";
    };

    system.stateVersion = 5;
    nix.package = pkgs.nix;
    nixpkgs.config.allowUnfree = true;
  })
];
};
};
}

```

Apply the configuration:

```
darwin-rebuild switch --flake .#desk-mac
```

Output:

```

building the system configuration...
==> Homebrew taps are up to date.
==> Installing cask ghostty
==> Installing cask 1password
==> Installing cask arc
system defaults applied
/run/current-system -> /nix/store/9xbp3v7c...-darwin-system

```

Adding a cask is a one-line diff in `homebrew.casks`. Removing it from the list and running `darwin-rebuild switch` uninstalls it automatically because `onActivation.cleanup = "zap"`.

Case 4: `system.defaults` macOS tweaks

`system.defaults` is `nix-darwin`'s typed interface to `defaults write`. Every key maps to a known macOS preference domain. Values are type-checked at build time — a typo is a build error, not a silent no-op.

This module captures the settings the desk team applies to every macOS workstation. Save it as a standalone module imported by the main `flake.nix`.

Save as `macos-defaults.nix`:

```
# macos-defaults.nix
{ pkgs, lib, ... }:

{
  system.defaults = {

    # Finder
    finder = {
      AppleShowAllExtensions = true;    # always show .go, .nix, .yaml suffixes
      AppleShowAllFiles      = true;    # show hidden dot-files
      ShowPathbar             = true;    # breadcrumb at bottom of every window
      ShowStatusBar           = true;    # file count and disk space in status bar
      FXPreferredViewStyle    = "Nlsv"; # default to list view
      FXDefaultSearchScope    = "SCcf"; # search current folder by default
      _FXSortFoldersFirst     = true;    # directories appear before files
    };

    # Keyboard
    NSGlobalDomain = {
      ApplePressAndHoldEnabled = false; # disable character picker; enables key repeat
      InitialKeyRepeat         = 15;    # 225 ms before repeat starts (default: 25)
      KeyRepeat                 = 2;     # 30 ms between repeats      (default: 6)
    };

    # Dock
    dock = {
      autohide           = true;
      autohide-delay     = 0.0;
      autohide-time-modifier = 0.2;
      show-recents       = false;    # no "recent apps" section
      tileSize           = 48;
      minimize-to-application = true; # minimise into app icon, not Dock shelf
      mru-spaces         = false;    # keep Space order fixed
    };
  };
}
```

```

};

# Screenshots
screenshot = {
  location      = "/Users/alice/Screenshots";
  type          = "png";
  disable-shadow = true;
};

# Trackpad
trackpad = {
  Clicking          = true;    # tap-to-click
  TrackpadThreeFingerDrag = true;
};
};

# Suppress .DS_Store on network and USB volumes via activation script
system.activationScripts.extraActivation.text = ''
  defaults write com.apple.desktopservices DSDontWriteNetworkStores -bool true
  defaults write com.apple.desktopservices DSDontWriteUSBStores      -bool true
'';
}

```

Import it from your flake:

```

# flake.nix (modules list excerpt)
modules = [
  ./macos-defaults.nix
  ({ pkgs, ... }: {
    environment.systemPackages = [ pkgs.ripgrep pkgs.jq pkgs.git ];
    system.stateVersion = 5;
    nix.package = pkgs.nix;
  })
];

```

Apply:

```
darwin-rebuild switch --flake .#desk-mac
```

Output:

```

building the system configuration...
system defaults applied:
finder.AppleShowAllExtensions = true
finder.ShowPathbar = true
dock.autohide = true
NSGlobalDomain.KeyRepeat = 2
NSGlobalDomain.InitialKeyRepeat = 15

```

```
reloading Dock...
reloading Finder...
/run/current-system -> /nix/store/2njw8x9q...-darwin-system
```

Preferences take effect immediately. `InitialKeyRepeat` and `KeyRepeat` take effect after the next login.

The trap

`nix-darwin` does not replace the macOS kernel. It manages user space. Developers who have used NixOS sometimes reach for `boot.kernelModules`, `boot.kernelParams`, or `hardware.*` options in a `darwinConfiguration`. These options do not exist on Darwin.

Attempting it:

```
# flake.nix ← WRONG: kernel options are Linux-only
darwinConfigurations."desk-mac" = nix-darwin.lib.darwinSystem {
  system = "aarch64-darwin";
  modules = [
    ({ ... }: {
      boot.kernelModules = [ "wireguard" ]; # does not exist on Darwin
    })
  ];
};
```

Run:

```
darwin-rebuild switch --flake .#desk-mac
```

Output:

```
error: The option `boot.kernelModules' does not exist. Definition values:
- In anonymous module at `'/nix/store/.../flake.nix':
  [ "wireguard" ]
(use '--show-trace' to show detailed location information)
```

The error fires at **build time**, before any change touches the live system.

The fix: On macOS, kernel extensions must be installed through Apple-signed packages or system recovery. WireGuard on macOS runs as a userspace process (`wireguard-go`), not a kernel module. Declare the `wireguard-go` binary in `environment.systemPackages` instead.

Similarly, SIP (System Integrity Protection) prevents `nix-darwin` from writing to `/System`, `/usr` (except `/usr/local`), or `/sbin`. Do not attempt to place files in those paths. Everything belongs in `/nix/store` and `/run/current-system`, which SIP does not protect.

The boring rule

- One `flake.nix` holds both `nixosConfigurations` and `darwinConfigurations`.
- Share tooling via a `commonPackages` function; append OS-specific packages in each configuration.
- Use `system.defaults` for every preference that would otherwise be a `defaults write` command.
- Run Home Manager as a `nix-darwin` module (`home-manager.darwinModules.home-manager`) for per-user dot-file management.
- Use `nix-homebrew` to manage Homebrew casks declaratively; set `onActivation.cleanup = "zap"` so removed casks are uninstalled automatically.
- Run `darwin-rebuild switch --flake .#<hostname>` after every change. No change is complete until the switch succeeds.
- Do not fight SIP. If an operation requires disabling SIP, it belongs in a one-time provisioning script during machine enrollment — not in `nix-darwin`.

Try this

1. **Add a cask.** Add `"rectangle"` (a window manager) to `homebrew.casks` in Case 3's flake and run `darwin-rebuild switch`. Confirm `Rectangle.app` appears in `/Applications`. Then remove it from the list, switch again, and verify it is gone.
2. **Shared package drift.** Fork Case 2's flake. Add `pkgs.kubectl` to `commonPackages`. Apply to both `desk-server` (via `nixos-rebuild switch`) and `desk-mac` (via `darwin-rebuild switch`). Confirm `kubectl version --client` returns the same version string on both machines.
3. **Defaults archaeology.** On a macOS machine, run `defaults read com.apple.dock | grep autohide` and note the current value. Add `system.defaults.dock.autohide = true` to Case 1's flake, switch, and run the same `defaults read` command again to confirm the value changed.
4. **Trigger the trap deliberately.** In a `darwinConfiguration`, add `boot.kernelModules = [ "wireguard" ]` and run `darwin-rebuild switch`. Read the error message. Then replace it with `environment.systemPackages = [ pkgs.wireguard-go ]`, switch again, and confirm `wg-quick` is now on `$PATH`.

Nix on a Foreign Linux

Nix on a Foreign Linux

Most of the desk will not reinstall the OS this quarter. The boring default is: **install the Nix daemon on the existing Ubuntu/Fedora/Arch machine, use flakes + Home Manager + direnv, and leave apt/dnf for the kernel, firmware, and the desktop the vendor already glued together.**

This is not NixOS. `/etc`, `systemd` units, and the kernel stay the host's. You steal reproducible **user** environments and **project** shells. When you are ready to own the machine, the NixOS install chapter is the jump.

Mental model

Layer	Owner on a foreign distro
Kernel, glibc of <code>/usr</code> , display manager	Distro (<code>dnf</code> , <code>apt</code>)
<code>/nix/store</code> , <code>nix-daemon</code>	Nix
User CLIs, <code>git</code> , <code>prompt</code> , <code>nvim</code>	Home Manager (standalone)
Per-repo compilers	<code>nix develop</code> / <code>direnv</code>

`PATH` will contain **both** `/usr/bin` and `~/.nix-profile/bin`. Put the Nix profile **first** so `jq` is the one you pinned.

Do not replace `/usr/bin/python3` with a Nix python. The distro's tools (`PackageKit`, `GNOME` extensions, vendor installers) expect the distro python.

<code>/usr/bin/...</code>	distro, leave it
<code>~/.nix-profile/bin</code>	Home Manager
<code>\$PRJ/.direnv</code>	project shell (<code>direnv</code>)

Worked examples

Case 1: Daemon is already installed — use it

After the Installing Nix chapter:

```
systemctl is-active nix-daemon
nix --version
echo $PATH | tr ':' '\n' | head
```

If `~/nix-profile/bin` is missing from `PATH`, your login shell did not source `/nix/var/nix/profiles/default/etc/profile.d/nix-daemon.sh`.
For bash:

```
# ~/.bashrc fragment (until Home Manager owns bash)
. /nix/var/nix/profiles/default/etc/profile.d/nix-daemon.sh
```

Then switch bash to Home Manager (Home Manager part) so this fragment goes away.

Case 2: Standalone Home Manager on Fedora/Ubuntu

```
nix run home-manager/release-26.05 -- init --switch
```

or, with the flake from the Home Manager part:

```
home-manager switch --flake .#deskadmin
```

`home.username` and `home.homeDirectory` must match the Unix account (`id -un`, `echo $HOME`).
Home Manager will not create `/home/deskadmin` on a foreign distro if you invented a name.

Case 3: Project shells without touching /usr

Save as `flake.nix` in a Go service repo:

```
# flake.nix
{
  description = "Desk metrics on a foreign Linux";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in
    {
      devShells.${system}.default = pkgs.mkShell {
        packages = [ pkgs.go pkgs.gopls ];
      };
    }
}
```

```
nix develop --command go version
```

The host may also have `apt install golang`. which `go` inside `nix develop` must be a store path.
Outside, it may be `/usr/bin/go`. That split is expected. `direnv` makes the project one automatic.

Case 4: SELinux and /nix

On Fedora, SELinux can deny the daemon. If `nix-build` fails with `Permission denied` and `dmesg` mentions `nix-daemon`:

```
# lab only - prefer the distro's documented SELinux policy for Nix
sudo ausearch -m avc -ts recent | head
```

Follow the distro package docs when you installed Nix via `dnf`. If you used the official installer, a `systemd` service plus `/nix` labeled `etc_t` or the installer-provided policy is required. Do not `setenforce 0` as a lifestyle.

Case 5: Uninstall without wrecking the distro

```
sudo systemctl stop nix-daemon
# official installer ships an uninstall script; use it
sudo rm -rf /nix
```

Then remove the `PATH` snippet from `~/.bashrc`. Distro packages (`vim`, `firefox`) stay. That is the whole point of not replacing `/usr`.

The trap

The trap is `environment.systemPackages` thinking on Ubuntu: writing a `configuration.nix` and wondering why `nixos-rebuild` is not installed. There is no NixOS module system on this host. Home Manager + flakes. If you want `services.openssh`, install NixOS.

The other trap is mixing distro Node and Nix Node on the same `PATH` without `direnv`. CI uses Nix; your laptop uses `/usr/bin/node`; lockfiles disagree. Always enter the project shell.

The boring rule

- Nix daemon + flakes + Home Manager + `direnv` on a foreign distro. Distro owns the OS.
- Nix `PATH` first for **your** CLIs. Distro `python/node` stay for **vendor** tools.
- No `configuration.nix` until the machine is NixOS.
- SELinux: use the distro's Nix policy, not `setenforce 0`.
- Project compilers only from `nix develop`.

Try this

1. `which jq` before and after `home-manager` switch with `jq` in `home.packages`. The second should be `/nix/store/...`

2. In a throwaway repo, Case 3, `nix develop --command which go` versus `which go` in a fresh terminal.
3. `echo $PATH` and label each entry as distro / Nix profile / direnv / other.
4. Confirm `systemctl is-enabled nix-daemon` is enabled so a reboot still builds.

Vendor Binaries with nix-ld and FHS Envs

Vendor Binaries with nix-ld and FHS Envs

Nix binaries look for `ld-linux` and `libssl.so` under `/nix/store`. A vendor AppImage, a JetBrains extractor, or a `curl | tar` toolchain looks for `/lib64/ld-linux-x86-64.so.2`. The boring default is: `programs.nix-ld` for random ELF files, `buildFHSEnv` for programs that require an FHS tree, and never a global `LD_LIBRARY_PATH`.

Mental model

Three kinds of program live on a desk:

Kind	Loader	Fix
Nix package	Store <code>ld-linux</code> + <code>RUNPATH</code>	None. Already correct.
Vendor ELF	Hard-coded <code>/lib64/ld-linux...</code>	<code>nix-ld</code> intercepts that path
“Must see <code>/usr/lib</code> ” installer	FHS layout	<code>buildFHSEnv</code> / <code>steam-run</code>

`nix-ld` installs a **compatibility loader** at the path vendor binaries request. That loader then finds libraries from a Nix-provided list (`programs.nix-ld.libraries`). The vendor file stays where you downloaded it; it does not enter `/nix/store` unless you package it.

`buildFHSEnv` builds a **bubblewrapped** `/usr` with the packages you listed. The process sees `libc.so` in the usual places. Use it when `nix-ld` is not enough (scripts that `dlopen("/usr/lib/libfoo.so")`).

vendor binary

requested interpreter: `/lib64/ld-linux-x86-64.so.2`

`nix-ld` stub at that path

real libraries from `programs.nix-ld.libraries` (store paths)

Worked examples

Case 1: Enable nix-ld on NixOS

Save as `nix-ld.nix`:

```
# nix-ld.nix
{ pkgs, ... }:

{
  programs.nix-ld.enable = true;
  programs.nix-ld.libraries = with pkgs; [
    stdenv.cc.cc
    zlib
    openssl
    curl
    icu
    nss
    nspr
    libxml2
    libGL
    xorg.libX11
  ];
}
```

```
sudo nixos-rebuild switch
ls -l /lib64/ld-linux-x86-64.so.2
```

Output (shape):

```
lrwxrwxrwx 1 root root ... /lib64/ld-linux-x86-64.so.2 -> /nix/store/...-nix-ld-.../lib/ld-linux-x86-
```

A vendor `./desk-toolbox` that died with `No such file or directory` (missing interpreter) should now start — or fail on a **named** missing library, which you add to `libraries`.

Case 2: See what is still missing

```
ldd ./desk-toolbox | grep 'not found'
```

Output (shape):

```
libfoo.so.1 => not found
```

Find it in `nixpkgs`, add `pkgs.foo` to `programs.nix-ld.libraries`, switch, retry. Do not export `LD_LIBRARY_PATH=/nix/store/.../lib`.

Case 3: buildFHSEnv for an installer that shells out

Save as `fhs.nix`:

```
# fhs.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.buildFHSEnv {
  name = "desk-fhs";
  targetPkgs = pkgs: with pkgs; [
    stdenv.cc.cc
    zlib
    openssl
    curl
    bash
    coreutils
    gnused
    gawk
  ];
  runScript = "bash";
}
```

```
nix-build fhs.nix
./result/bin/desk-fhs
```

You are now in a shell where `/usr/bin/env` and `libc.so.6` exist in FHS locations. Run the vendor installer **inside** this shell. Exit, and the host is unchanged.

`steam-run` is a pre-built FHS env in `nixpkgs` for the same trick:

```
nix run nixpkgs#steam-run -- ./desk-toolbox
```

Case 4: Package the vendor blob only when you must

If the team will run this for years, write a derivation with `autoPatchelfHook` instead of asking every laptop to enable `nix-ld` for it. That is the packaging part. `nix-ld` is the **escape hatch** for binaries you will not package this week.

Case 5: Foreign Linux

`programs.nix-ld` is a **NixOS** module. On Ubuntu, the equivalent is the `nix-ld` package plus a loader in `/lib64`, which fights the distro. Prefer `buildFHSEnv` / `steam-run` / `nix develop` on foreign distros, or run the vendor app in a container. Do not overwrite Ubuntu's `ld-linux`.

The trap

The trap is `export LD_LIBRARY_PATH=$HOME/libs` so one AppImage starts. Every Nix binary in that terminal now prefers `$HOME/libs/libssl.so`. The next day, `curl` speaks a surprise TLS stack.

The other trap is adding `pkgs.linuxPackages.kernel` to `nix-ld.libraries` “to have everything.” Keep the list short. Add libraries when `ldd` names them.

The boring rule

- Nix packages stay Nix packages. Do not wrap them in FHS.
- Vendor ELF on NixOS: `programs.nix-ld` + a **short** library list from `ldd`.
- Vendor installers that need `/usr`: `buildFHSEnv` or `steam-run`, not a fake `/usr` on the host.
- Never a global `LD_LIBRARY_PATH`.
- If you run it daily, package it with `autoPatchelfHook` later.

Try this

1. On a NixOS VM, enable Case 1, download a random upstream `rg` static/dynamic release (not from `nixpkgs`), `file` it, and run it. If it is static, `nix-ld` is unused (fine). If dynamic, `ldd` it.
2. `ldd $(readlink -f $(which curl))` on a **Nix** `curl` — every line should already be a store path. `nix-ld` must not be involved.
3. Enter the FHS env from Case 3, run `ls /usr/bin | head` and `ldd /usr/bin/bash`. Exit and confirm `/usr/bin/bash` on the host is the NixOS one (or absent), not the bubble.
4. Add one library to `nix-ld.libraries`, switch, and `nix why-depends /run/current-system pkgs.openssl` to see that the system closure grew. That is the cost of a long list.

Packages and Overlays

The Nix Package Manager in Practice

The Nix Package Manager in Practice

Nix the language is useless without Nix the tool. The boring default on Nix 2.35 is: **nix shell** for five minutes, **nix profile** only for personal CLIs that are not a project, and **nix search** against a 26.05 pin — not **nix-env -iA**.

Project compilers still belong in **devShell** / **direnv** (part 3). This chapter is the **user** profile and one-shot **PATH**.

Mental model

`~/.nix-profile` is a generation of symlinks into `/nix/store`. It is not `/usr/bin`. Adding `jq` does not delete the previous `jq`; rollback is another generation.

Command	Lifetime
<code>nix shell nixpkgs#jq</code>	This process
<code>nix run nixpkgs#jq -- --version</code>	One command
<code>nix profile install nixpkgs#jq</code>	Until rollback/remove
<code>nix search nixpkgs jq</code>	Query

Unqualified `nixpkgs#` follows your **registry** (often a moving flake). For desk work prefer a pinned flake:

```
nix shell github:NixOS/nixpkgs/nixos-26.05#jq
```

or `nix shell .#jq` inside a repo.

Worked examples

Case 1: One-shot

```
nix shell github:NixOS/nixpkgs/nixos-26.05#cowsay --command cowsay "Desk system ready"
```

Output:

```

-----
< Desk system ready >
-----
      \  ^--^
        \ (oo)\_____
          (__) \         )\/\
                ||----w |
                 ||     ||

```

Nothing new in `nix profile list`.

Case 2: `nix run`

```
nix run github:NixOS/nixpkgs/nixos-26.05#hello
```

Output:

Hello, world!

`run` builds (or substitutes) and executes the default app. Good for `nix run .#desk-api`.

Case 3: Profile install + list

```
nix profile install github:NixOS/nixpkgs/nixos-26.05#jq
nix profile list
which jq
```

Output (shape):

```
Name:                jq
Flake attribute:     packages.x86_64-linux.jq
Store path:         /nix/store/...-jq-...
```

`readlink -f $(which jq)` is a store path.

Case 4: Rollback

```
nix profile history
nix profile rollback
which jq || echo 'jq gone'
```

If `jq` was the only package in that generation, it disappears from `PATH`. The store path remains until GC.

Case 5: Search

```
nix search github:NixOS/nixpkgs/nixos-26.05 jq
```

Output (shape):

```
* legacyPackages.x86_64-linux.jq
  jq: Command-line JSON processor
```

First search can be slow (eval). After that, 26.05 is cached.

The trap

The trap is `nix-env -iA nixpkgs.jq`. Unpinned, fights flakes, surprising rollbacks. `nix-env` is legacy. `nix profile` or Home Manager.

The other trap is `nix profile install` of `go`, `nodejs`, `rustc`. The next repo wants another version. Part 3: `devShell`.

A third trap: `nix shell nixpkgs#jq` with an **unlocked** registry while CI uses 26.05. Pin the URL or the repo flake.

The boring rule

- `nix shell / nix run` for trials. Pin `nixos-26.05`.
- `nix profile` for personal CLIs (`jq`, `ripgrep`). Not for compilers.
- `nix profile rollback` when an install was a mistake.
- No `nix-env`.
- Project tools in the flake, not in the user profile.

Try this

1. `nix shell github:NixOS/nixpkgs/nixos-26.05#neofetch --command neofetch` (or `fastfetch` if you prefer).
2. `nix search github:NixOS/nixpkgs/nixos-26.05 postgresql | head`.
3. Install `hello` into the profile, run it, rollback, confirm which `hello`.
4. `nix profile list` and delete anything that is a compiler.

Understanding the Flakes Input System

Understanding the Flakes Input System

Two humans, two nixpkgs, two glibcs. The boring default is: `inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05"`, follows so nested flakes share it, and `flake.lock` committed.

Mental model

```
inputs = {
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  home-manager = {
    url = "github:nix-community/home-manager/release-26.05";
    inputs.nixpkgs.follows = "nixpkgs";
  };
};
```

Piece	Job
<code>url</code>	Where to fetch (github, git+ssh, path, tarball)
<code>follows</code>	Reuse this flake's input instead of nested default
<code>flake.lock</code>	<code>rev</code> + <code>narHash</code> for every input

`nix flake update` moves the lock. `nix flake update nixpkgs` moves one input. Do not update everything on Friday without `flake check`.

`master` / `nixpkgs-unstable` is not a pin you can name in an incident. 26.05 is.

Worked examples

Case 1: follows in the tree

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk infrastructure flake with shared nixpkgs";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
```

```
flake-utils.url = "github:numtide/flake-utils";
home-manager = {
  url = "github:nix-community/home-manager/release-26.05";
  inputs.nixpkgs.follows = "nixpkgs";
};

outputs = { self, nixpkgs, flake-utils, home-manager }: {
  formatter.x86_64-linux = nixpkgs.legacyPackages.x86_64-linux.nixfmt-rfc-style;
};
}
```

```
nix flake lock
nix flake metadata
```

Output (shape):

Inputs:

```
flake-utils: github:numtide/flake-utils/...
home-manager: github:nix-community/home-manager/...
  nixpkgs follows input 'nixpkgs'
nixpkgs: github:NixOS/nixpkgs/...
```

follows input 'nixpkgs' is the line you want. Without it, home-manager may lock a second nixpkgs.

Case 2: One-input update

```
nix flake update flake-utils
```

Output (shape):

- Updated input 'flake-utils'

nixpkgs did not move. That is the point.

Case 3: Audit the lock

```
nix flake metadata --json | jq -r '.locks.nodes.nixpkgs.locked.rev'
```

Put that rev in the PR description when you bump 26.05.

Case 4: path: for a sibling repo

```
inputs.desk-tooling.url = "path:../desk-tooling";
inputs.desk-tooling.inputs.nixpkgs.follows = "nixpkgs";
```

path: is not copied to a cache the same way as git+ssh. Fine on a laptop. CI should use a git URL + lock.

Case 5: Check after a bump

```
nix flake update nixpkgs
nix flake check
```

If check fails, `git checkout -- flake.lock` and try a smaller step. The lock is git; treat it as code.

The trap

The trap is **three copies of nixpkgs** because nobody wrote `follows`. Eval RAM and download time explode. `nix flake metadata` every time you add an input.

The other trap is not committing `flake.lock`. Then there is no team.

The boring rule

- `nixos-26.05` (and matching `release-26.05` for HM). Not `master`.
- `inputs.<dep>.inputs.nixpkgs.follows = "nixpkgs";`
- Commit the lock. Update one input per PR when you can.
- `metadata` to see `follows`.
- `path:` locally; git URLs in CI.

Try this

1. `nix flake metadata` on this flake; count `nixpkgs` nodes. Should be one.
2. Remove `follows`, `nix flake lock --update-input home-manager`, `metadata` again, put `follows` back.
3. `nix flake update flake-utils` and `git diff flake.lock`.
4. `jq` the locked `narHash` for `nixpkgs`.

Creating and Composing Overlays

Creating and Composing Overlays

You need a patch, not a nixpkgs fork. The boring default is: **final: prev: { ... }**, **prev.pkg.override / overrideAttrs**, **final** for siblings, and a **short** overlay — not a private nixpkgs.

Mental model

```
final: prev: {
  deskGreet = prev.writeShellScriptBin "desk-greet" "echo desk";
  hello = prev.hello.overrideAttrs (old: { pname = old.pname; });
}
```

Arg	Meaning
prev	Package set before this overlay. Override this package from prev.
final	Package set after all overlays. Use for <i>other</i> packages this overlay also defines.

`final.hello.overrideAttrs` when you meant `prev.hello` is infinite recursion.

Apply:

```
import nixpkgs { overlays = [ overlayA overlayB ]; }
```

or NixOS `nixpkgs.overlays = [ ... ]`;

Worked examples

Case 1: Add a script to pkgs

Save as `overlay_demo.nix`:

```
# overlay_demo.nix
let
  deskOverlay = final: prev: {
    deskGreet = prev.writeShellScriptBin "desk-greet" ''
      echo "Welcome to the desk infrastructure"
    ''
```

```

    '';
  };
  pkgs = import <nixpkgs> { overlays = [ deskOverlay ]; };
in
{
  name = pkgs.deskGreet.name;
}

```

```
nix eval --impure --file overlay_demo.nix --json
```

(`--impure` because `<nixpkgs>`. In a flake, pass `nixpkgs` from inputs — no `impure`.)

Output:

```
{"name":"desk-greet"}
```

Case 2: Compose two overlays

Save as `compose_overlays.nix`:

```

# compose_overlays.nix
let
  overlayA = final: prev: { servicePort = 8080; };
  overlayB = final: prev: {
    serviceUrl = "http://127.0.0.1:${toString final.servicePort}";
  };
  pkgs = import <nixpkgs> { overlays = [ overlayA overlayB ]; };
in
{ url = pkgs.serviceUrl; }

```

```
nix eval --impure --file compose_overlays.nix --apply 'x: x.url'
```

Output:

```
"http://127.0.0.1:8080"
```

`overlayB` reads `final.servicePort` so it sees `overlay A`'s binding.

Case 3: `overrideAttrs` from `prev`

```

# hello-overlay.nix
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    pname = "desk-hello";
  });
}

```

```
}
```

The output path name changes. Downstream `pkgs.hello` is your patched one **if** they take `hello` from the same `pkgs`.

Case 4: Infinite recursion (the foot-gun)

```
# bad-overlay.nix
final: prev: {
  hello = final.hello.overrideAttrs (old: { });
}
```

```
nix eval --impure --expr 'import <nixpkgs> { overlays = [ (import ./bad-overlay.nix) ]; }' -
```

You will get infinite recursion (or a hang/error). Use `prev.hello`.

Case 5: Flake overlay export

```
# flake.nix fragment
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }: {
    overlays.default = final: prev: {
      deskGreet = prev.writeShellScriptBin "desk-greet" "echo desk";
    };
  };
}
```

Consumers: `overlays = [ desk-tooling.overlays.default ];` with follows on `nixpkgs`.

The trap

The trap is `final.pkg.override...` Recursion.

The other trap is a 2000-line overlay that copies half of `nixpkgs`. Pin a package with `fetchFromGitHub` + `callPackage` (next chapter) or an overlay of **one** attr. Fork `nixpkgs` only when you are actually contributing upstream.

The boring rule

- Signature `final: prev:.`
- Override **this** package from `prev`.
- Refer to **other** overlay packages via `final`.

- Keep overlays small. Files under `overlays/`.
- 26.05 follows so the overlay applies to one pkgs.

Try this

1. Overlay `desk-status` with `writeShellScriptBin`; `nix-build -E 'with import <nixpkgs> { overlays = [ ... ]; }; desk-status'` (impure lab) or a flake package.
2. Two overlays: A sets `deskPort = 9`; B builds a string from `final.deskPort`.
3. Write the bad `final.hello` overlay and trigger recursion; switch to `prev`.
4. `nix flake metadata` after adding an overlay-only flake input.

Custom Package Definitions

Custom Package Definitions

Internal CLIs need a recipe, not a gist in `$HOME`. The boring default is: `{ lib, stdenv, ... }`: `+ pkgs.callPackage ./desk-tool.nix { }`, store-path wrappers, and no hardcoded `pkgs.curl` inside the recipe.

Mental model

<code>desk-tool.nix</code>	function of dependencies
<code>callPackage</code>	fills args from <code>pkgs</code>
<code>\$out/bin/...</code>	the artifact

`callPackage ./f.nix { curl = otherCurl; }` overrides one arg. That is how overlays and cross work.

Scripts: `writeShellApplication` (fundamentals) is enough for many desk tools. `stdenv.mkDerivation` when you compile. `makeWrapper` / `wrapProgram` when a script must see `curl` on `PATH` at **run-time**.

Worked examples

Case 1: `writeShellApplication` recipe

Save as `desk-tool.nix`:

```
# desk-tool.nix
{ writeShellApplication, curl, jq }:

writeShellApplication {
  name = "desk-tool";
  runtimeInputs = [ curl jq ];
  text = ''
    echo "Querying desk API with curl and jq..."
  '';
}
```

Save as `default.nix`:

```
# default.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.callPackage ./desk-tool.nix { }

nix-build default.nix
./result/bin/desk-tool
```

Output:

Querying desk API with curl and jq...

runtimeInputs puts curl and jq on PATH inside the wrapper. The host need not have them.

Case 2: stdenv + wrapProgram

Save as desk-tool-stdenv.nix:

```
# desk-tool-stdenv.nix
{ lib, stdenv, makeWrapper, curl, jq }:

stdenv.mkDerivation {
  pname = "desk-tool";
  version = "1.0.0";
  src = ./.;
  dontUnpack = true;
  nativeBuildInputs = [ makeWrapper ];
  installPhase = ''
    mkdir -p $out/bin
    echo '#!/bin/sh' > $out/bin/desk-tool
    echo 'echo desk-tool' >> $out/bin/desk-tool
    chmod +x $out/bin/desk-tool
    wrapProgram $out/bin/desk-tool \
      --prefix PATH : ${lib.makeBinPath [ curl jq ]}
  '';
}
```

Prefer Case 1 unless you already have an installPhase.

Case 3: Override an input

```
# default.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.callPackage ./desk-tool.nix {
```



```
jq = pkgs.jq;
}
```

Passing `jq` explicitly is how you inject a patched `jq` from an overlay.

Case 4: Flake package

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      pkgs = nixpkgs.legacyPackages.x86_64-linux;
    in
      {
        packages.x86_64-linux.desk-tool = pkgs.callPackage ./desk-tool.nix { };
        packages.x86_64-linux.default = self.packages.x86_64-linux.desk-tool;
      };
}

nix build
./result/bin/desk-tool
```

Case 5: meta

```
meta = {
  description = "Internal desk operations utility";
  license = lib.licenses.mit;
  platforms = lib.platforms.unix;
};
```

`nix search / nix flake show` readers see this. Internal tools still get `meta`.

The trap

The trap is `pkgs.curl` inside `desk-tool.nix`. Then `callPackage` cannot swap `curl`. Cross-compile and overlays lose. Arguments on the function, always.

The other trap is `src = ./.`; including `.git` and `README` (caching chapter). `lib.fileset / cleanSource` when the recipe grows.

The boring rule

- Recipe is a function. `callPackage` injects.
- `writeShellApplication` for scripts; `stdenv` for builds.
- Runtime tools via `runtimeInputs` / `wrapProgram`, not the user's `PATH`.
- Flake `packages.<system>.desk-tool` on 26.05.
- Override through `callPackage ./f.nix { curl = ...; }`.

Try this

1. Add `hello` to `runtimeInputs` and call `hello` from the script.
2. `callPackage ./desk-tool.nix { jq = pkgs.jq; }` explicitly.
3. `nix path-info -Shr ./result` and find `curl` in the closure.
4. Break the shebang by using a raw `echo > $out/bin` without `wrap`; run on a `PATH` without `curl`; then restore the wrapper.

Managing Dependencies Across Projects

Managing Dependencies Across Projects

Five repos, five copies of `desk-checker`, five versions. The boring default is: **one tooling flake on nixpkgs 26.05 exporting packages and overlays.default; consumers follows that nixpkgs and pin the tooling input.**

This is the same idea as the platform-engineering IDP chapter, at **package** scale: a linter, not a whole OS.

Mental model

```
desk-tooling (flake)
  packages.*.desk-checker
  overlays.default
```

```
billing-service  inputs.desk-tooling
                  inputs.nixpkgs.follows = "desk-tooling/nixpkgs"
                  # or both follow a platform flake
```

`nix flake update desk-tooling` is the upgrade. Not `curl | sh`.

Worked examples

Case 1: Tooling flake

Save as `tooling/flake.nix`:

```
# flake.nix
{
  description = "Shared desk tooling";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
      desk-checker = pkgs.writeShellApplication {
```

```

    name = "desk-checker";
    text = ''
        echo "Validating desk organizational standards: OK"
    '';
};
in
{
    packages.${system}.desk-checker = desk-checker;
    packages.${system}.default = desk-checker;
    overlays.default = final: prev: {
        desk-checker = self.packages.${prev.stdenv.hostPlatform.system}.desk-checker;
    };
};
}

```

(If `hostPlatform.system` is awkward in an overlay, export only `packages` and reference `desk-tooling.packages.${system}.desk-checker` from the consumer — often clearer.)

Case 2: Consumer with follows

Save as `project/flake.nix`:

```

# flake.nix
{
    description = "Billing service";

    inputs = {
        nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
        desk-tooling.url = "path:../tooling";
        desk-tooling.inputs.nixpkgs.follows = "nixpkgs";
    };

    outputs = { self, nixpkgs, desk-tooling }:
        let
            system = "x86_64-linux";
            pkgs = import nixpkgs {
                inherit system;
                overlays = [ desk-tooling.overlays.default ];
            };
        in
        {
            devShells.${system}.default = pkgs.mkShell {
                packages = [ pkgs.desk-checker ];
            };
        };
}

```

```
cd project
nix develop --command desk-checker
```

Output:

Validating desk organizational standards: OK

Case 3: Prefer packages over overlay

```
devShells.${system}.default = pkgs.mkShell {
  packages = [ desk-tooling.packages.${system}.desk-checker ];
};
```

No overlay, no `pkgs.desk-checker` magic. Overlay when many packages must look like `nixpkgs`.

Case 4: Update the pin

```
nix flake update desk-tooling
nix flake check
```

CI on the consumer fails if the tooling flake broke `desk-checker`. That is the contract.

Case 5: CI uses git, not path:

```
desk-tooling.url = "git+ssh://git@git.desk.internal/desk-tooling";
```

`path: ../tooling` is for this chapter's laptop. Lock the git rev in `flake.lock` for humans.

The trap

The trap is **copying `desk-checker.sh` into every repo**. The fifth copy diverges. One flake.

The other trap is **not follows**. Tooling on 26.05, app on another `nixpkgs`, two glibcs in one shell.

The boring rule

- One tooling flake. 26.05. Lockfile.
- Export `packages`; overlay only if it earns its keep.
- Consumers **follows** `nixpkgs`.
- `nix flake update desk-tooling` is the bump.
- git URL in CI; `path:` only locally.

Try this

1. Two directories as in Cases 1–2; `nix develop --command desk-checker`.
2. Change the echo text in tooling, `update` the consumer, run again.
3. `nix flake metadata` in the consumer; one `nixpkgs`.
4. Replace the overlay with a direct `packages` reference (Case 3).

Fetchers and Fixed-Output Derivations

Fetchers and Fixed-Output Derivations

Every package starts as bytes from somewhere. The boring default is: `fetchurl` / `fetchFromGitHub` with an SRI hash = "sha256-...", and a hash mismatch is a stop, not a prompt to disable the check.

Mental model

A **fixed-output derivation** (FOD) may use the network. It must produce bytes whose hash you declared. The store path is a function of that hash. Mirrors can change; the hash must not.

Fetcher	When
<code>fetchurl</code>	A tarball or file at a URL
<code>fetchFromGitHub</code>	<code>owner + repo + rev</code> (tag or commit)
<code>fetchgit</code>	Arbitrary git remote
<code>fetchzip</code>	A zip you want unpacked
<code>fetchpatch</code>	One patch file

Language vendor hashes (`vendorHash`, `cargoHash`) are the same idea: FODs for Go modules / crates.io.

`url + rev` `hash you wrote`

```
download  cmp  sha256  ok  /nix/store/<fod-hash>-src
```

```
fail → error: hash mismatch (got ...)
```

Empty hash `sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=` is a **probe**. You run the build once, paste got, commit. Leaving the probe in git is how supply chain theatre happens.

Worked examples

Case 1: `fetchurl` with a probe hash

Save as `fod.nix`:

```
# fod.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.fetchurl {
  url = "https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz";
  hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
}
```

```
nix-build fod.nix
```

Output (shape):

```
error: hash mismatch in fixed-output derivation '/nix/store/...-hello-2.12.1.tar.gz':
  specified: sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=
    got:     sha256-0P4s2aQwHzpwC3w3ys8ls6A6/JbuiJvTYdbBr9pQMnQ=
```

Paste got into hash, rebuild:

```
nix-build fod.nix
tar -tzf result | head
```

The path is now stable. Changing only the URL to a mirror of **the same bytes** keeps the path.

Case 2: fetchFromGitHub

Save as `gh.nix`:

```
# gh.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.fetchFromGitHub {
  owner = "NixOS";
  repo = "nix";
  rev = "2.35.2";
  hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
}
```

Use a **tag or full commit**. `rev = "master"` is a moving label; the hash will mismatch the moment master moves, or worse, you use `lib.fakeHash` and skip the mismatch.

Probe, paste, pin. Prefer `rev = "abc1234..."` (full git sha) for anything the desk ships.

Case 3: nix-prefetch so you do not fake-hash in a loop

```
nix-prefetch-url https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz
nix hash convert --hash-algo sha256 --to sri "$(nix-prefetch-url https://ftp.gnu.org/gnu/hel
```

or:

```
nix-prefetch-url --unpack https://github.com/NixOS/nix/archive/2.35.2.tar.gz
```

`fetchFromGitHub` unpacks; the hash is over the **unpacked** tree (`hash / outputHashMode = "recursive"`), not the `.tar.gz` bytes. That is why a `fetchurl` hash and a `fetchFromGitHub` hash for “the same release” differ.

Case 4: A package src

Save as `pkg.nix`:

```
# pkg.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.stdenv.mkDerivation rec {
  pname = "hello";
  version = "2.12.1";
  src = pkgs.fetchurl {
    url = "https://ftp.gnu.org/gnu/hello/hello-${version}.tar.gz";
    hash = "sha256-0P4s2aQwHzpwC3w3ys8ls6A6/JbuiJvTYdbBr9pQMnQ=";
  };
}
```

The `hash` in this listing is illustrative. Always use the `got` from **your** fetch. If GNU republishes the tarball, the mismatch is the news.

Case 5: `lib.fakeHash` is a development aid

```
hash = pkgs.lib.fakeHash;
```

It expands to a dummy SRI. Fine in a dirty worktree. `git grep fakeHash` must be empty on `main`.

The trap

The trap is `sha256 = ""`; or `hash = lib.fakeSha256` in production so CI “stays green” when upstream retags. You are no longer pinning bytes. You are pinning a URL, which is not a pin.

The other trap is hashing the wrong thing: `fetchurl` hash on a GitHub **archive URL** while the package uses `fetchFromGitHub` (unpacked). Always mismatch. Prefetch with the same fetcher you will use.

The boring rule

- Every `src` is a FOD with a committed SRI hash.
- Probe once (`fakeHash / AAAA...`), paste `got`, commit.
- Pin `rev` to a tag or git sha, never `master`.
- `git grep fakeHash` is empty on the default branch.
- Prefetch with the same fetcher (packed vs unpacked) you ship.

Try this

1. Run Case 1, paste the real hash, rebuild, then flip one character of the hash and read the mismatch.
2. `nix hash convert --help` and convert a hex sha256 to SRI.
3. Fetch the same hello tarball with `fetchurl` and, unpacked, with `fetchzip` / `fetchFromGitHub` if applicable. Confirm the hashes differ.
4. `git grep -n fakeHash` on a flake repo you maintain. Delete any hit on the main branch.

NIX_PATH, Channels, and Pinning Inputs

NIX_PATH, Channels, and Pinning Inputs

<nixpkgs> is whichever tree \$NIX_PATH names today. The boring default is: **do not import from Nix PATH in anything you ship; pin with a flake lock on nixos-26.05, or npins if a repo cannot use flakes yet.**

This is the “input determinism” lesson: same expression, same bytes, six months later.

Mental model

Mechanism	What it pins	Use
NIX_PATH / <nixpkgs>	Whatever the machine has	Repl, throwaway <code>nix eval --impure</code>
Channels (nix-channel)	A moving URL per user	Legacy; do not start new work here
npins / niv flake.lock	JSON of rev + hash in git rev + narHash	Non-flake trees Desk default

```
<nixpkgs>    → lookup NIX_PATH → laptop A  laptop B
flake.lock   → git-committed hash → laptop A = laptop B = CI
```

niv still works. npins is the smaller, currently maintained pin tool. Flakes made both optional.

Worked examples

Case 1: See what <nixpkgs> actually is

```
echo "$NIX_PATH"
nix eval --impure --expr '<nixpkgs>'
```

Output (shape):

```
nixpkgs=/nix/var/nix/profiles/per-user/root/channels/nixpkgs:...
/nix/store/...-nixpkgs-unpacked
```

Two engineers with different channels evaluate different **hello**.

Case 2: Flake pin (the default)

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }: {
    packages.x86_64-linux.hello = nixpkgs.legacyPackages.x86_64-linux.hello;
  };
}
```

```
nix flake lock
nix flake metadata --json | jq -r '.locks.nodes.nixpkgs.locked.rev'
```

Commit `flake.lock`. That rev is the pin.

Case 3: npins without flakes

```
nix run github:andir/npins -- init
nix run github:andir/npins -- add github NixOS nixpkgs --branch nixos-26.05
```

Save as `default.nix`:

```
# default.nix
let
  sources = import ./npins;
  pkgs = import sources.nixpkgs { };
in
pkgs.hello
```

`npins/sources.json` is the lockfile. Commit it. `niv` is the same idea with `nix/sources.nix` — do not mix `niv` and `npins` in one repo.

Case 4: Override `NIX_PATH` for one command (lab only)

```
NIX_PATH=nixpkgs=flake:nixpkgs nix eval --impure --expr 'import <nixpkgs> {}' --apply 'p: p.'
```

Useful to compare. Not a team pin. The next shell forgets it.

Case 5: Fetch a tarball by hash (no flake, no npins)

```
# pin.nix
let
  nixpkgs = builtins.fetchTarball {
    url = "https://github.com/NixOS/nixpkgs/archive/nixos-26.05.tar.gz";
    sha256 = "0000000000000000000000000000000000000000000000000000000000000000";
  };
in
import nixpkgs { }
```

Probe the hash once (`lib.fakeHash`). The tarball URL `nixos-26.05` **moves**; the `sha256` does not. When you want a newer 26.05 commit, bump both.

The trap

The trap is `import <nixpkgs> {}` in `configuration.nix` on a server. Rebuilds follow the last `nix-channel --update`. CI, the laptop, and prod disagree.

The other trap is pinning `master`. There is no “26.05” in the incident report. Branch `nixos-26.05` plus a lock hash.

The boring rule

- Ship flakes + `flake.lock` on 26.05.
- No `<nixpkgs>` in production files.
- `npins` only when flakes are forbidden.
- One pin tool per repo.
- `fetchTarball` without a hash is not a pin.

Try this

1. `echo $NIX_PATH` on two machines (or a container vs the host). Compare `<nixpkgs>` paths.
2. `nix flake metadata` and write down the `nixpkgs rev`.
3. `npins init` in a throwaway dir; open `sources.json`.
4. `git grep -n '<nixpkgs>'` on the desk repo — production files should be empty.

Building From Source

The Derivation Build Process

The Derivation Build Process

`stdenv.mkDerivation` is how Unix software becomes a store path. The boring default is: **keep the standard phases, override only the ones you must, and let `stdenv` set `$CC`, `$out`, `RPATH`, and the sandbox.**

Raw derivation `{ }` (fundamentals) is the primitive. This chapter is what you write for real C.

Mental model

Phase	Default
<code>unpackPhase</code>	Unpack <code>\$src</code>
<code>patchPhase</code>	<code>patch -p1</code> each of <code>patches</code>
<code>configurePhase</code>	<code>./configure --prefix=\$out</code> if that script exists
<code>buildPhase</code>	<code>make</code>
<code>checkPhase</code>	<code>make check</code> if <code>doCheck = true</code>
<code>installPhase</code>	<code>make install</code>
<code>fixupPhase</code>	<code>patchelf RPATH</code> , <code>strip</code> , <code>gzip</code> man pages

Skip a phase with `dontConfigure = true`; (or `dontBuild`, `dontInstall` — rarely). Override a phase with `buildPhase = '...'`; Always `mkdir -p $out/bin` before `cp` if you write `installPhase` by hand.

`$CC` is the `stdenv` compiler (`gcc` or `clang`), already wrapped with hardening flags. Do not call `gcc` by name.

`$src` → `unpack` → `patch` → `configure` → `build` → `check` → `install` → `fixup` → `$out`

Worked examples

Case 1: One C file, no autotools

Save as `c_tool.nix`:

```
# c_tool.nix
{ lib, stdenv }:

stdenv.mkDerivation {
```

```

pname = "desk-status-c";
version = "1.0";
src = ./.;
dontConfigure = true;
buildPhase = ''
    cat > main.c << 'EOF'
    #include <stdio.h>
    int main(void) {
        puts("Desk C worker active");
        return 0;
    }
    EOF
    $CC -O2 -o desk-status-c main.c
'';
installPhase = ''
    mkdir -p $out/bin
    cp desk-status-c $out/bin/
'';
meta = {
    description = "Compiled C desk status check";
    license = lib.licenses.mit;
};
}

```

Save as default.nix:

```

# default.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.callPackage ./c_tool.nix { }

nix-build default.nix
./result/bin/desk-status-c

```

Output:

Desk C worker active

Case 2: doCheck

```

# in c_tool.nix
doCheck = true;
checkPhase = ''
    ./desk-status-c
'';

```

checkPhase runs **before** installPhase, in the build directory. The binary is still `./desk-status-c`, not `$out/bin`.

```
nix-build default.nix
```

A failing check fails the derivation. Good.

Case 3: \$src as a real tree

Put `main.c` in `git` and drop the `cat` from `buildPhase`:

```
stdenv.mkDerivation {
  pname = "desk-status-c";
  version = "1.0";
  src = ./src;
  dontConfigure = true;
  buildPhase = ''
    $CC -O2 -o desk-status-c main.c
  '';
  installPhase = ''
    mkdir -p $out/bin
    cp desk-status-c $out/bin/
  '';
}
```

Filter `src` (source-filtering chapter) so `.git` is not an input.

Case 4: Inspect the wrapped compiler

```
nix-build default.nix
nix-store -q --references result
```

You should see `glibc` (or `musl`), not `gcc`. `gcc` is a **build-time** input; `fixupPhase` did not copy it into the runtime closure.

```
readelf -d result/bin/desk-status-c | grep PATH
```

`RUNPATH` points at `store/libc`. That is `stdenv`, not you.

Case 5: outputs (preview)

```
outputs = [ "out" "dev" ];
```

Multiple outputs split runtime (`out`) from headers (`dev`). Skip until a closure-size measurement says headers are the problem. One `$out` is the desk default.

The trap

The trap is `installPhase without mkdir -p $out/bin. cp: cannot create ... No such file.` The build log is long; the last line is the one that matters.

The other trap is `gcc main.c` instead of `$CC`. Cross and clang stdenvs then break. `$CC` is the contract.

The boring rule

- Standard phases. `dontConfigure` when there is no `./configure`.
- `$CC`, not `gcc`. `$out`, not `/usr`.
- `mkdir -p` before `cp` in a custom `installPhase`.
- `doCheck = true` when tests exist and are hermetic.
- Language builders (`buildGoModule`, `buildRustPackage`) wrap this. Use them when they exist.

Try this

1. Add Case 2; `nix-build`; then `checkPhase = "false"`; and confirm failure.
2. Replace `$CC` with `gcc`; cross later will hurt — put `$CC` back.
3. `nix path-info -Shr result` vs `nix-store -q --deriver result` references — runtime vs build.
4. Forget `mkdir`; save the error; restore.

Building C and C++ Projects with CMake

Building C and C++ Projects with CMake

CMake projects should not grow a hand-written `cmake -B build` in `buildPhase`. The boring default is: `cmake` in `nativeBuildInputs`, optional `ninja`, flags in `cmakeFlags`, and `stdenv`'s `configure/build/install` hooks.

Mental model

Putting `cmake` on `nativeBuildInputs` enables the CMake **setup hook**:

```
cmake -B build -DCMAKE_INSTALL_PREFIX=$out [+ cmakeFlags]
cmake --build build
cmake --install build
```

`nativeBuildInputs` = tools that **run on the build machine** (`cmake`, `ninja`, `pkg-config`).
`buildInputs` = libraries **linked into** the binary (`zlib`, `openssl`). Mixing them up is how cross breaks.

```
stdenv.mkDerivation {
  nativeBuildInputs = [ cmake ninja ];
  buildInputs = [ zlib ];
  cmakeFlags = [ "-DENABLE_DESK=ON" ];
}
```

Worked examples

Case 1: Tiny CMake project

Save as `cmake_project.nix`:

```
# cmake_project.nix
{ lib, stdenv, cmake }:

stdenv.mkDerivation {
  pname = "desk-math-engine";
  version = "1.0.0";
  src = ./.;
  nativeBuildInputs = [ cmake ];
```



```

preConfigure = ''
  cat > CMakeLists.txt << 'EOF'
  cmake_minimum_required(VERSION 3.20)
  project(DeskMath C)
  add_executable(desk-calc main.c)
  install(TARGETS desk-calc DESTINATION bin)
  EOF
  cat > main.c << 'EOF'
  #include <stdio.h>
  int main(void) {
    puts("Desk Math Engine: 42");
    return 0;
  }
  EOF
'';
meta = {
  description = "Desk calculation engine";
  license = lib.licenses.mit;
};
}

```

```

# default.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.callPackage ./cmake_project.nix { }

nix-build default.nix
./result/bin/desk-calc

```

Output:

Desk Math Engine: 42

Move CMakeLists.txt / main.c into git and delete preConfigure for a real tree.

Case 2: cmakeFlags

```

cmakeFlags = [
  "-DCMAKE_BUILD_TYPE=Release"
  "-DBUILD_TESTING=OFF"
];

```

Do not append these onto a custom buildPhase. The hook reads cmakeFlags.

Case 3: Ninja

```
nativeBuildInputs = [ cmake ninja ];
```

The hook prefers Ninja when `ninja` is on `nativeBuildInputs`. Faster incremental compiles in the sandbox still start from zero each derivation — Ninja mainly helps huge trees and nicer logs.

Case 4: Link zlib

```
# cmake_zlib.nix
{ stdenv, cmake, zlib }:

stdenv.mkDerivation {
  pname = "desk-z";
  version = "1.0";
  src = ./.;
  nativeBuildInputs = [ cmake ];
  buildInputs = [ zlib ];
}
```

CMake's `find_package(ZLIB)` sees `pkg-config` / `cmake` files from the `zlib` derivation. You did not set `ZLIB_ROOT` by hand.

Case 5: Tests

```
doCheck = true;
# CMake: enable tests in cmakeFlags if needed
# cmakeFlags = [ "-DBUILD_TESTING=ON" ];
```

Hermetic tests only. A test that `curls` the network fails the sandbox. `doCheck = false;` with a comment when upstream tests are broken on Nix — then a `passthru.tests` later.

The trap

The trap is `buildPhase = "cmake -B build && cmake --build build";`. You skip `CMAKE_INSTALL_PREFIX=$out` and the `stdenv` flags. Use `cmakeFlags` and the hook.

The other trap is `buildInputs = [ cmake ];`. `cmake` then looks like a **runtime** library. `nativeBuildInputs`.

The boring rule

- `cmake` (and `ninja`) in `nativeBuildInputs`.
- Linked libs in `buildInputs`.
- Options via `cmakeFlags`, not a custom `cmake` command.
- Real `CMakeLists.txt` in git; `preConfigure` only for the listing in this book.
- `doCheck` when tests do not need the network.

Try this

1. Case 2: add `-DCMAKE_BUILD_TYPE=Release`; `nix log` and `grep CMAKE_BUILD_TYPE`.
2. Move `cmake` to `buildInputs`; `nix-build`; note warnings or cross pain; put it back.
3. Add `zlib` and `find_package(ZLIB)` in a lab `CMakeLists.txt`.
4. `nix path-info -Shr result` — `cmake` should not appear in the **runtime** closure.

Rust Packages with Cargo

Rust Packages with Cargo

The sandbox cannot talk to crates.io. The boring default is: `rustPlatform.buildRustPackage` with `cargoHash` (or `cargoLock.lockFile`) and a committed `Cargo.lock` — Crane is optional when that FOD is too coarse.

This is the Rust peer of `buildGoModule`.

Mental model

`Cargo.lock` → `cargo vendor FOD (cargoHash)` → `cargo build --offline --release` → `$out/bin`

Style	When
<code>cargoHash = "sha256-..."</code>	26.05 default. One hash of the vendored crates.
<code>cargoLock.lockFile = ./Cargo.lock</code> Crane	Git deps / patches in lockfile; a bit more setup. Incremental crate FODs (like <code>gomod2nix</code>). Extra flake input.
<code>fenix / rust-overlay</code>	Pin <code>rustc</code> newer than <code>nixpkgs</code> . Only when 26.05's <code>rustc</code> is too old.

Commit `Cargo.lock` even for bins (`cargo generate-lockfile`). Without it you are not reproducible.

Worked examples

Case 1: `buildRustPackage` + `cargoHash`

Save as `rust_app.nix`:

```
# rust_app.nix
{ lib, rustPlatform }:

rustPlatform.buildRustPackage {
  pname = "desk-rust-agent";
  version = "0.1.0";
  src = ./.;
  cargoHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
```

```

    meta = {
      description = "Desk Rust agent";
      license = lib.licenses.mit;
    };
  }

```

With a real `Cargo.toml` / `Cargo.lock` / `src/main.rs` in `..`:

```

# default.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.callPackage ./rust_app.nix { }

nix-build default.nix
./result/bin/desk-rust-agent

```

Probe `cargoHash` with `lib.fakeHash` / `cargoHash = ""`; once; paste got. `edition = "2021"` or `"2024"` as the crate requires.

Case 2: `cargoLock.lockFile`

```

rustPlatform.buildRustPackage {
  pname = "desk-rust-agent";
  version = "0.1.0";
  src = ./.;
  cargoLock = {
    lockFile = ./Cargo.lock;
  };
}

```

Prefer this when the lockfile has `git` sources. You may need `outputHashes` for those `git` crates — `nixpkgs`'s `buildRustPackage` docs list the attr. Hash mismatches on `git` deps are FODs too.

Case 3: Native libs (`openssl`, `pkg-config`)

```

{ rustPlatform, pkg-config, openssl }:

rustPlatform.buildRustPackage {
  pname = "desk-agent";
  src = ./.;
  cargoHash = "sha256-...";
  nativeBuildInputs = [ pkg-config ];
  buildInputs = [ openssl ];
}

```

`pkg-config` is native. `openssl` is linked. Same split as `CMake`.

Case 4: Tests

`buildRustPackage` runs `cargo test` by default. Disable only with a reason:

```
doCheck = false; # upstream tests need network - see passthru.tests
```

Hermetic tests stay on. A test that hits `crates.io` or `GitHub` fails the sandbox — good.

Case 5: Crane (optional)

Crane builds deps as their own derivations so a one-line `src` change does not rebuild the world of crates. Use it when CI of a large workspace is the pain. It is a flake input (`github:ipetkov/crane`) with `follows` on `nixpkgs 26.05`.

Do not start a 200-line `hello` crate on Crane. Measure `nix-build` first.

`fenix / oxalica rust-overlay`: pin a nightly or a `rustc` newer than 26.05. The desk default is `pkgs.rustPlatform` from the channel.

The trap

The trap is `cargoSha256`. Deprecated. `cargoHash` (SRI) or `cargoLock`.

The other trap is no `Cargo.lock` in git because “it’s a bin.” Then every CI run resolves `crates.io` differently. Commit the lock.

`src = ./.`; including `target/` will bust hashes. `cleanSource / fileset`; add `/target` to `.gitignore`.

The boring rule

- `buildRustPackage` + `cargoHash` + committed `Cargo.lock`.
- Probe the hash; never leave `fakeHash` on main.
- `pkg-config` native; C libs in `buildInputs`.
- Crane / rust-overlay only when measured.
- Filter `target/` out of `src`.

Try this

1. Tiny crate; `fakeHash`; paste `got`; `nix-build`; run the bin.
2. Add a crates.io dep; bump `cargoHash`.
3. `doCheck = true` (default); add a `#[test]` that `assert!(true)`.
4. `nix path-info -Shr result` — rustc should not be in the **runtime** closure.

Python Packages with Pip and Setuptools

Python Packages with Pip and Setuptools

`pip install` on the server is a moving PyPI. The boring default is: `python3.pkgs.buildPythonApplication` (CLI) or `buildPythonPackage` (library), deps from `python3.pkgs.*` on `nixpkgs 26.05` — not `pip` in the sandbox.

`poetry2nix` / `uv2nix` exist. Use them when a lockfile is the team contract **and** you have measured `buildPythonApplication` as too coarse. They are not the first recipe.

Mental model

Builder	For
<code>buildPythonApplication</code>	A CLI with <code>project.scripts</code> — wraps <code>PYTHONPATH</code>
<code>buildPythonPackage</code>	A library other Python code imports
<code>python3.withPackages</code> (ps: [<code>ps.requests</code>])	A devShell interpreter, not a shipped package

```
pyproject.toml
  build-system = setuptools / hatchling / ...
  dependencies = python3.pkgs.requests (from nixpkgs, not PyPI at build time)
```

```
$out/bin/desk-reporter (wrapped)
```

```
pyproject = true; on modern trees. format = "setuptools" on old setup.py only.
```

Worked examples

Case 1: Application

Save as `desk_reporter.nix`:

```
# desk_reporter.nix
{ lib, python3 }:

python3.pkgs.buildPythonApplication {
  pname = "desk-reporter";
```

```

version = "1.0.0";
pyproject = true;
src = ./.;
build-system = [ python3.pkgs.setuptools ];
dependencies = [ python3.pkgs.requests ];
meta = {
    description = "Desk metrics reporting CLI";
    license = lib.licenses.mit;
};
}

```

With `pyproject.toml` declaring `desk-reporter = "reporter:main"` and `reporter.py` in git:

```

# default.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.callPackage ./desk_reporter.nix { }

nix-build default.nix
./result/bin/desk-reporter

```

Output (shape):

Desk reporter ready (requests version: 2.x.x)

The wrapper's `PYTHONPATH` includes `requests`. The host python does not matter.

Case 2: Library vs application

```

python3.pkgs.buildPythonPackage {
    pname = "desklib";
    version = "1.0.0";
    pyproject = true;
    src = ./.;
    build-system = [ python3.pkgs.setuptools ];
    dependencies = [ python3.pkgs.pydantic ];
}

```

Libraries do not always install a `bin/`. Applications do. Mixing them up gives a store path with no `bin/desk-reporter`.

Case 3: Tests

```
nativeCheckInputs = [ python3.pkgs.pytest ];  
doCheck = true;
```

pytest is a **check** input, not **dependencies**, unless the app imports pytest at runtime (it should not). Tests that download from PyPI fail — they must use `python3.pkgs`.

Case 4: Dev shell

```
pkgs.mkShell {  
  packages = [  
    (pkgs.python3.withPackages (ps: [ ps.requests ps.pytest ]))  
  ];  
}
```

Humans run `pytest` here. CI builds Case 1. Do not `pip install --user` inside the shell “just this once.”

Case 5: Missing from nixpkgs

If `desk-internal-sdk` is not on 26.05:

1. `buildPythonPackage` it in *this* flake (`callPackage ./sdk.nix`).
2. Put it in `dependencies` of the app.

`uv2nix` / `poetry2nix` generate many package derivations from a lockfile. Worth it for a 80-dep web app. Not for a 40-line reporter. Overlaying a single `buildPythonPackage` is the boring missing-dep path.

The trap

The trap is `pip install -r requirements.txt` in `buildPhase`. Sandbox: no network. Even with `--offline`, those wheels are not Nix inputs unless you FOD them.

The other trap is `propagatedBuildInputs` copied from a 2019 gist. On current `buildPythonPackage`, use `dependencies` / `build-system` / `nativeCheckInputs` as in 26.05 nixpkgs python docs.

The boring rule

- `buildPythonApplication` for CLIs; `buildPythonPackage` for libs.
- `pyproject = true`; + `build-system` + `dependencies` from `python3.pkgs`.
- No `pip` in the derivation. No unpinned `requirements.txt` as the only lock.
- Tests: `nativeCheckInputs`, `hermetic`.
- `uv2nix`/`poetry2nix` optional; one in-tree `buildPythonPackage` for a missing lib.

Try this

1. Case 1 with a real `pyproject.toml`; `nix-build`; `head -5 result/bin/desk-reporter` (wrapper).
2. Drop `requests` from `dependencies`; run; read `ModuleNotFoundError`; put it back.
3. `nix path-info -Shr result` and find `requests`.
4. `python3.withPackages` in a shell; `python -c 'import requests'`; confirm which python is a store path.

Go Modules with buildGoModule

Go Modules with buildGoModule

Go is how the desk API ships. The boring default is: `pkgs.buildGoModule` with a real `vendorHash`, committed `go.mod / go.sum`, `CGO_ENABLED=0` unless you need C, and the Go from nixpkgs 26.05 — not `go get` in the sandbox.

This is the Go analogue of `buildRustPackage`: one FOD for modules, one offline compile.

Mental model

```
go.mod + go.sum
    vendorHash (FOD)

/nix/store/...-desk-api-go-modules
    go build -mod=vendor (offline)

$out/bin/desk-api
```

Field	Role
<code>vendorHash</code>	Hash of the vendored module tree. Probe with <code>lib.fakeHash</code> .
<code>vendorHash = null</code>	Only when there are zero module deps (stdlib only).
<code>proxyVendor</code>	Let the FOD use the module proxy (usual).
<code>modRoot</code>	If <code>go.mod</code> is not at <code>src</code> root.
<code>subPackages</code>	<code>["cmd/desk-api"]</code> so you do not build every <code>./... main</code> .
<code>env.CGO_ENABLED</code>	<code>"0"</code> for static-ish binaries; <code>"1"</code> needs gcc in <code>nativeBuildInputs</code> .

`go test` runs in `checkPhase` when `doCheck = true` (the default for `buildGoModule`). Tests must be hermetic.

Worked examples

Case 1: Stdlib-only binary

Save as `desk_service_go.nix`:

```
# desk_service_go.nix
{ lib, buildGoModule }:

buildGoModule {
  pname = "desk-service";
  version = "1.0.0";
  src = ./.;
  vendorHash = null;
  meta = {
    description = "Desk Go backend";
    license = lib.licenses.mit;
  };
}
```

With `go.mod + main.go` in `.` (no require lines):

```
nix-build -E 'with import <nixpkgs> {}; callPackage ./desk_service_go.nix {}'
./result/bin/desk-service
```

Output:

Desk Go backend operational

(Your `fmt.Println` text.)

Case 2: Dependencies — probe `vendorHash`

```
# desk_api.nix
{ buildGoModule, lib }:

buildGoModule {
  pname = "desk-api";
  version = "1.0.0";
  src = lib.fileset.toSource {
    root = ./.;
    fileset = lib.fileset.unions [ ./go.mod ./go.sum ./main.go ./internal ];
  };
  vendorHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
  subPackages = [ "." ];
}

nix-build
```

Mismatch prints `got: sha256-....`. Paste it. Changing `go.mod` without bumping the hash is a **stop**, not a network call in `buildPhase`.

Case 3: Pin the toolchain

```
# go-pin.nix
{ pkgs ? import <nixpkgs> { } } :

(pkgs.buildGoModule.override { go = pkgs.go_1_24; }) {
  pname = "desk-api";
  version = "1.0.0";
  src = ./.;
  vendorHash = null;
}
```

pkgs.go on 26.05 is whatever that channel ships. When the desk must match Boring-Go's 1.27 (or a security pin), override **the builder**, not PATH in a wrapper. Confirm with:

```
nix-build
./result/bin/desk-api
# or: strings result/bin/desk-api | grep '^go1\.'
```

Case 4: CGO_ENABLED=0

```
buildGoModule {
  pname = "desk-api";
  version = "1.0.0";
  src = ./.;
  vendorHash = null;
  env.CGO_ENABLED = "0";
  ldflags = [ "-s" "-w" ];
}
```

No gcc. Cross to GOOS=linux is then just the Go toolchain. If you import **net** with cgo DNS, you want this anyway for boring deploys. SQLite with mattn/go-sqlite3 needs CGO **on** and `nativeBuildInputs = [ pkgs.pkg-config ]`; plus the C library — then you left the boring path.

Case 5: Tests in the sandbox

```
buildGoModule {
  pname = "desk-api";
  version = "1.0.0";
  src = ./.;
  vendorHash = "sha256-...";
  doCheck = true;
  # default checkPhase is go test
```

```
}
```

A test that dials `https://api.github.com` fails. Fixtures in `testdata/`. `vendorHash` must include test-only modules too (`go.sum` lines).

The trap

The trap is `vendorHash = null with a require in go.mod`. Eval may succeed; the build tries the network and dies. `null` is not “figure it out.”

The other trap is `go get / GOPROXY=https://proxy.golang.org` in `buildPhase`. The sandbox has no network. The FOD already fetched.

The boring rule

- `buildGoModule + vendorHash` (or `null` only for `stdlib-only`).
- Commit `go.mod` and `go.sum`. Filter `src` to them + packages.
- `subPackages` for the mains you ship.
- `CGO_ENABLED=0` unless a C library is the point.
- Pin `go` via `buildGoModule.override { go = ...; }` when the channel default is not the desk compiler.

Try this

1. `Stdlib main.go; vendorHash = null; nix-build`; run the binary.
2. `go get github.com/google/uuid` in a throwaway module; set `fakeHash`; paste `got`.
3. Flip one character of `vendorHash`; read the mismatch.
4. `env.CGO_ENABLED = "0"`; file `result/bin/desk-api` — dynamically linked to `musl/glibc` still possible via Go’s net; the point is no `gcc`.

Cross-Compilation for Multiple Platforms

Cross-Compilation for Multiple Platforms

Hand-rolled `aarch64-linux-gnu-gcc` prefixes are a wiki. The boring default is: `pkgs.pkgsCross.<target>` on `nixpkgs 26.05`, or a `native**` remote builder when you need speed and KVM**.

Mental model

Platform	Meaning
<code>build</code>	Machine compiling (laptop)
<code>host</code>	Machine running the output
<code>target</code>	What a compiler emits (only when building gcc)

`pkgsCross.aarch64-multiplatform.hello` is hello for ARM, built on `x86_64` with a cross toolchain.

Remote builder (`ssh-ng` to an ARM box) is **native**. Prefer it for Go, kernels, and NixOS tests. Cross is fine for small C.

Worked examples

Case 1: Cross hello

```
nix build -f '<nixpkgs>' pkgsCross.aarch64-multiplatform.hello
file result/bin/hello
```

Output (shape):

```
result/bin/hello: ELF 64-bit LSB executable, ARM aarch64, ...
```

You cannot run it on `x86_64` without `qemu-user`.

Case 2: Your C program

Save as `cross_demo.nix`:

```

# cross_demo.nix
let
  pkgs = import <nixpkgs> { };
  armPkgs = pkgs.pkgsCross.aarch64-multiplatform;
in
armPkgs.stdenv.mkDerivation {
  pname = "desk-arm";
  version = "1.0";
  src = pkgs.writeTextDir "main.c" ''
    #include <stdio.h>
    int main(void) {
      puts("desk-arm");
      return 0;
    }
  '';
  buildPhase = ''
    $CC $src/main.c -o desk-arm
  '';
  installPhase = ''
    mkdir -p $out/bin
    cp desk-arm $out/bin/
  '';
}

```

```

nix-build cross_demo.nix
file result/bin/desk-arm

```

`$CC` is the cross compiler. Do not hardcode `gcc`.

Case 3: nativeBuildInputs vs buildInputs

```

{
  nativeBuildInputs = [ pkg-config ]; # runs on the laptop
  buildInputs = [ zlib ]; # links into the ARM binary
  doCheck = stdenv.buildPlatform == stdenv.hostPlatform;
}

```

Tests that execute the binary must be off when crossing (`doCheck false`).

Case 4: Flake overlay

```

# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
}

```

```

outputs = { self, nixpkgs }:
  let
    pkgs = nixpkgs.legacyPackages.x86_64-linux;
  in
  {
    packages.x86_64-linux.desk-arm =
      pkgs.pkgsCross.aarch64-multiplatform.callPackage ./desk.nix { };
  };
}

```

Case 5: Prefer a builder for NixOS

`nixos-rebuild / nixosTest` for `aarch64`: remote `aarch64-linux` builder with `kvm`. Cross-compiling a full NixOS closure is possible and painful. The `remote-builders` chapter is the boring path for OS images.

The trap

The trap is **running `./result/bin/desk-arm` on the laptop** and calling the compiler broken. `file` first. `qemu-user` if you must execute; a builder if you must test for real.

The boring rule

- `pkgsCross.*` for small C. Native builder for OS and Go.
- `$CC`, not `gcc`. `nativeBuildInputs` vs `buildInputs`.
- `doCheck` only when `build == host`.
- 26.05 pin so the cross toolchain matches prod.
- file the output.

Try this

1. `nix eval --raw -f '<nixpkgs>' 'pkgsCross.aarch64-multiplatform.stdenv.cc' | xargs basename`
2. Case 1 file.
3. Enable `doCheck = true` on a cross drv that runs the binary; watch it fail; add the platform guard.
4. `nix eval --expr 'builtins.attrNames (import <nixpkgs> {}).pkgsCross' | tr ' ' '\n' | head`

Trivial Builders

Trivial Builders

Not every package needs `stdenv.mkDerivation` and a compiler. The boring default is: `writeShellApplication` / `writeText` / `runCommand` / `symlinkJoin` for scripts, files, and combining closures — reach for `mkDerivation` when you compile.

Mental model

`nixpkgs` “trivial builders” are functions that produce derivations without a full `stdenv` phase dance.

Builder	Produces
<code>writeText</code> / <code>writeTextFile</code>	One file at <code>\$out</code>
<code>writeTextDir</code>	<code>\$out/name</code>
<code>writeScript</code> / <code>writeScriptBin</code>	Executable (weak shebang)
<code>writeShellApplication</code>	Script + <code>runtimeInputs</code> + <code>set -euo pipefail</code>
<code>runCommand</code>	Tiny custom builder
<code>symlinkJoin</code>	One dir of symlinks to many packages
<code>copyPathToStore</code>	Copy a path as a store object

`writeShellApplication` is the desk default for shell. `writeScriptBin` is the older, sloppier cousin.

Worked examples

Case 1: `writeText`

Save as `banner.nix`:

```
# banner.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.writeText "desk-banner.txt" ''
  desk operations - window A
''
```



```
nix-build banner.nix
cat result
```

Output:

desk operations - window A

\$out *is* the file. No bin/.

Case 2: writeShellApplication

Save as tool.nix:

```
# tool.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.writeShellApplication {
  name = "desk-status";
  runtimeInputs = [ pkgs.coreutils pkgs.jq ];
  text = ''
    echo '{"window":"A","ok":true}' | jq -c .
  '';
}

nix-build tool.nix
./result/bin/desk-status
```

Output:

{"ok":true,"window":"A"}

jq is on PATH inside the wrapper only.

Case 3: runCommand

Save as run.nix:

```
# run.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.runCommand "desk-tree" { } ''
  mkdir -p $out/share/desk
  echo 1.0 > $out/share/desk/version
  ln -s ${pkgs.hello}/bin/hello $out/hello-link
''
```

```
nix-build run.nix
cat result/share/desk/version
```

Use `runCommand` when you need a few `mkdir/cp` lines. If it grows a `configurePhase`, you wanted `mkDerivation`.

Case 4: `symlinkJoin`

Save as `join.nix`:

```
# join.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.symlinkJoin {
  name = "desk-cli";
  paths = [ pkgs.jq pkgs.ripgrep pkgs.hello ];
}

nix-build join.nix
ls result/bin
```

One GC root, several tools. Home Manager `home.packages` is this idea at user scale. A “kitchen sink” join of `gcc+python+node` is how closures explode — join what operators actually run.

Case 5: `writeTextDir` for a default config

```
# conf.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.writeTextDir "desk/config.json" (builtins.toJSON {
  window = "A";
  replicas = 2;
})

nix-build conf.nix
cat result/desk/config.json
```

Good for embedding JSON next to a service. Not for secrets.

The trap

The trap is `writeScriptBin` with `curl` in the script and no `runtimeInputs`. It works on your `PATH` and dies in `systemd`. `writeShellApplication` + `runtimeInputs`.

The other trap is `runCommand` that compiles C. That belongs in `stdenv`. Trivial builders are trivial.

The boring rule

- Scripts: `writeShellApplication`.
- One file: `writeText` / `writeTextDir`.
- Glue: `runCommand`. Union of bins: `symlinkJoin`.
- Compilers: `mkDerivation` / language builders.
- No secrets in `writeText`.

Try this

1. Case 2: add `pkgs.hello` to `runtimeInputs` and call `hello`.
2. `symlinkJoin jq+hello; nix path-info -Shr result | tail -1`.
3. Replace `writeShellApplication` with `writeScriptBin` and drop `jq` from the host `PATH` in a clean `nix shell --pure`; watch it fail; restore.
4. `writeText` a unit snippet and `cat` it; confirm `$out` is a file, not a directory.

Source Filtering

Source Filtering

`src = ./.` hashes the README, `.git`, and `result`. The boring default is: `lib.fileset` (or `cleanSourceWith`) so only the files the build reads are in the FOD, especially in a monorepo.

Mental model

The source tarball is an **input**. Extra files → extra hash → extra rebuilds → extra cache misses.

Tool	When
<code>lib.cleanSource</code>	Drop <code>.git</code> , <code>result</code> , editor junk. Fast default.
<code>lib.cleanSourceWith</code>	Plus a custom filter
<code>lib.fileset.toSource</code>	Union of paths; preferred on 26.05
<code>nix-gitignore.gitignoreSource</code>	Follow <code>.gitignore</code>

```
repo/  
  apps/desk-api/  ← fileset  
  docs/           ← not in src  
  README.md       ← not in src
```

CI chapter already used `cleanSource`. This chapter is the packaging form.

Worked examples

Case 1: `cleanSource`

Save as `filter.nix`:

```
# filter.nix  
{ pkgs ? import <nixpkgs> { } }:  
  
pkgs.stdenv.mkDerivation {  
  pname = "desk-src-demo";  
  version = "1.0";  
  src = pkgs.lib.cleanSource ./.;  
  dontUnpack = false;  
  buildPhase = "echo ok > $out";
```

```

    installPhase = "true";
}

nix-build filter.nix
echo "# noise" >> README.md
nix-build filter.nix

```

If README is filtered, the second build is a cache hit. If it rebuilds, README is still in `src`.

Case 2: fileset union

```

# fileset.nix
{ pkgs ? import <nixpkgs> { }, lib ? pkgs.lib }:

pkgs.stdenv.mkDerivation {
  pname = "desk-api-src";
  version = "1.0";
  src = lib.fileset.toSource {
    root = ./.;
    fileset = lib.fileset.unions [
      ./main.go
      ./go.mod
      ./go.sum
    ];
  };
  # ...
}

```

A CSS change under `./web` does not rebuild this derivation.

Case 3: Exclude by name

```

src = pkgs.lib.cleanSourceWith {
  src = ./.;
  filter = path: type:
    let base = baseNameOf path; in
    !(builtins.elem base [ "README.md" "docs" ".envrc" ]);
};

```

Case 4: gitignore

```

src = pkgs.nix-gitignore.gitignoreSource [ ] ./.;

```

Only as good as `.gitignore`. If you commit `data.bin` that the build does not need, it still busts. `fileset` is stricter.

Case 5: Prove it with `--dry-run`

```
nix-build filter.nix
echo x >> README.md
nix-build filter.nix --dry-run
```

Empty “will be built” list → filter worked. `gcc` in the list → `src` still too wide (or you used `src = ./.` ;).

The trap

The trap is **filtering out `go.sum` / `Cargo.lock`**. The FOD hash of vendors depends on the lock. Include lockfiles. Exclude docs and `.git`.

The other trap is `filterSource` that depends on `builtins.currentTime` or `readDir` of a changing cache dir. The filter itself must be deterministic.

The boring rule

- Never raw `src = ./.` on a repo root for a leaf package.
- `fileset` on 26.05 when you can name the files.
- Lockfiles in; READMEs out.
- `--dry-run` after a docs-only edit.
- One filter style per repo (`fileset` *or* `gitignore`, not both fighting).

Try this

1. Case 1 with and without `cleanSource`; time the second build after a README edit.
2. `fileset` of a single `main.go`; add a sibling file; `--dry-run`.
3. Forget `go.sum` in the `fileset`; watch `go` complain; add it.
4. `nix path-info` the `src` store path; `tar -tf` if it is a NAR; confirm `.git` is absent.

Patching Packages

Patching Packages

Upstream is almost right. The boring default is: a `.patch` file in `git`, `patches = [ ./desk-fix.patch ]`; or `prev.pkg.overrideAttrs`, not a fork of `nixpkgs`.

Mental model

`stdenv patchPhase` runs `patch -p1` for each entry in `patches`. Fetch a patch with `fetchpatch` / `fetchpatch2` when it lives on GitHub; vendor it in-tree when it is *your* fix.

`src tarball` → `patchPhase` → `configure/build`

`./desk-fix.patch` (in `git`)

Overlays compose: `prev.hello.overrideAttrs (old: { patches = (old.patches or []) ++ [ ./x.patch ]; })`.

`fetchpatch2` is the 24.11+ spelling that hashes the **applied** result more reliably. On 26.05, prefer `fetchpatch2` for remote patches.

Worked examples

Case 1: In-tree patch on your package

Save as `desk-fix.patch` (illustrative):

```
# desk-fix.patch - unified diff against src root
--- a/main.c
+++ b/main.c
@@ -1,3 +1,3 @@
   int main(void) {
-   return 1;
+   return 0;
  }
```

```
# patched.nix
{ stdenv }:
```

```
stdenv.mkDerivation {
  pname = "desk-c";
  version = "1.0";
  src = ./src;
  patches = [ ./desk-fix.patch ];
}
```

The patch is an input. Change the diff → new hash → rebuild. That is what you want.

Case 2: Overlay on nixpkgs hello

Save as `overlay.nix`:

```
# overlay.nix
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    patches = (old.patches or [ ]) ++ [ ./hello-desk.patch ];
  });
}
```

You do not copy `hello`'s expression. You append.

Case 3: `fetchpatch2` from a PR

```
# fetch-patch.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.hello.overrideAttrs (old: {
  patches = (old.patches or [ ]) ++ [
    (pkgs.fetchpatch2 {
      url = "https://github.com/example/hello/commit/abc123.patch";
      hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
    })
  ];
})
```

Probe the hash. When GitHub force-pushes the PR, the hash mismatches — good.

Case 4: `sed` in `postPatch` (last resort)

```
postPatch = ''
  substituteInPlace Makefile --replace-fail /usr/local "$out"
'';
```

`--replace-fail` (not `--replace`) so a silent no-op cannot ship. Prefer a real patch when the change is more than one substitution.

Case 5: Drop an upstream patch

```
hello = prev.hello.overrideAttrs (old: {  
    patches = builtins.filter (p: !(pkgs.lib.hasSuffix "bad.patch" (baseNameOf p))) (old.patches  
});
```

Rare. Document *why* in a comment. Next nixpkgs bump may make the filter a no-op — check the diff.

The trap

The trap is **editing files in `/nix/store/...-hello` “to test.”** Read-only. Copy the expression, patch, overlay.

The other trap is a patch with Windows line endings or the wrong `-p` strip level. `patchPhase` fails. Generate with `git diff` from the unpacked `src` root.

The boring rule

- Your fix: `./foo.patch` in git + `patches = [ ]`.
- Upstream PR: `fetchpatch2` + hash.
- Overlay `overrideAttrs`; do not fork nixpkgs for one hunk.
- `substituteInPlace --replace-fail` for path rewrites.
- Patch is an input. It must apply cleanly on 26.05’s `src`.

Try this

1. `git diff` a one-line change; save as `.patch`; add to `patches`; `nix-build`.
2. Break the hunk context; read `patchPhase` failure in `nix log`.
3. Overlay `hello` with an empty extra patch file and `nix-build -A hello` under that overlay.
4. `fetchpatch2` with `fakeHash`, paste `got`.

Node.js Packages

Node.js Packages

Node is the desk frontend. The boring default is: `buildNpmPackage` + `npmDepsHash` from `package-lock.json`, or `pnpm.fetchDeps` when the lockfile is `pnpm` — not `npm install` in `buildPhase`.

Go moved to its own chapters. This page is Node only.

Mental model

Same two-step FOD as Go:

`package-lock.json` → `npmDepsHash` FOD → `npm ci --offline` → `$out`

Lock	Builder
<code>package-lock.json</code>	<code>buildNpmPackage</code>
<code>pnpm-lock.yaml</code>	<code>pnpm.fetchDeps</code> + <code>pnpm hooks</code> (26.05)
Yarn	yarn-specific fetchers; do not mix

Commit the lockfile. `npm install` without a lock is not a pin.

Worked examples

Case 1: `buildNpmPackage`

Save as `node_app.nix`:

```
# node_app.nix
{ buildNpmPackage, lib }:

buildNpmPackage {
  pname = "desk-web";
  version = "1.0.0";
  src = lib.cleanSource ./.;
  npmDepsHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
  installPhase = ''
    runHook preInstall
```

```

    mkdir -p $out/dist
    cp -r dist/* $out/dist/
    runHook postInstall
  '';
}

```

Probe `npmDepsHash`. `npm ci` uses the lock, not the latest registry.

Case 2: A CLI with a bin

If `package.json` has `"bin": { "desk-web": "bin/cli.js" }`, `buildNpmPackage` often wraps it. Check `./result/bin`.

Case 3: pnpm

```

# pnpm.nix
{ lib, stdenv, pnpm, nodejs }:

stdenv.mkDerivation (finalAttrs: {
  pname = "desk-web";
  version = "1.0.0";
  src = lib.cleanSource ./.;
  pnpmDeps = pnpm.fetchDeps {
    inherit (finalAttrs) pname version src;
    hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
    fetcherVersion = 2;
  };
  nativeBuildInputs = [ nodejs pnpm.configHook ];
  buildPhase = ''
    pnpm run build
  '';
  installPhase = ''
    mkdir -p $out
    cp -r dist $out/
  '';
})

```

`fetcherVersion` and option names move; `nixos-option` / the 26.05 `nixpkgs` doc for `pnpm.fetchDeps` wins if this snippet drifts.

Case 4: Node in a devShell, not in the output

```
pkgs.mkShell {  
  packages = [ pkgs.nodejs pkgs.pnpm ];  
}
```

CI builds with Case 1. Humans `pnpm test` inside `nix develop`. Do not `npm install -g` on the laptop.

Case 5: Native addons

`node-gyp` needs `python3`, `gcc`, `headers`. Add `nativeBuildInputs = [ pkgs.python3 pkgs.pkg-config ]`; and the C library. If the addon can be skipped (`optionalDependencies`), prefer a pure-JS path for the desk image.

The trap

The trap is `npm install` in `buildPhase`. Sandbox: `EAI_AGAIN` / network disabled. Hash the lockfile FOD.

The other trap is no lockfile (`package-lock.json` gitignored). Then `npmDepsHash` is a lie next week.

The boring rule

- Lockfile in git. `npmDepsHash` / `pnpm.fetchDeps` hash.
- `buildNpmPackage` for `npm`. `pnpm` fetcher for `pnpm`. Do not mix.
- Probe hashes; never leave `fakeHash` on main.
- Toolchain in the `devShell`; artifacts from the derivation.
- Native addons are a CGO-shaped exception — document `gcc`.

Try this

1. Tiny `package.json` + lock; `fakeHash`; paste got.
2. Change a dep in `package.json` without `npm install`; watch the FOD mismatch or `npm ci` fail.
3. `nix path-info -Shr` result on a built `dist`; confirm `nodejs` is or is not in the runtime closure (static `dist/` should not need `node` to *serve* if `nginx` serves files).
4. `git check-ignore -v package-lock.json` — it must **not** be ignored.

Go Toolchains and gomod2nix

Go Toolchains and gomod2nix

`buildGoModule` is Cargo's `buildRustPackage`. `gomod2nix` is closer to **Crane**: it generates Nix from `go.mod` so module FODs are per-crate, not one giant `vendorHash`. The boring default is still `buildGoModule` until a repo's vendor FOD is painfully slow; then `gomod2nix`.

There is no Fenio-for-Go. Pin the compiler with `buildGoModule.override { go = pkgs.go_1_24; }` or an overlay. `go` in `nixpkgs 26.05` *is* the toolchain overlay.

`dream2nix` / `cargo2nix` analogues exist and are **not** the desk default — too much generated Nix for a service that `buildGoModule` already builds.

Mental model

Tool	Like (Rust course)	When
<code>buildGoModule + vendorHash</code>	<code>buildRustPackage + cargoHash</code>	Default
<code>gomod2nix</code>	Crane	Many modules, incremental FODs
<code>go overlay / .override { go = ... }</code>	<code>fenix / rust-overlay</code>	Pin compiler version
<code>vendor/</code> in git	<code>cargo vendor</code> in-tree	Air-gap; last resort

```
go.mod    gomod2nix    gomod2nix.toml + buildGoApplication
          buildGoModule    one vendorHash FOD
```

Worked examples

Case 1: Stay on `buildGoModule`

If `nix-build` of the API is seconds after a cache hit, stop. `gomod2nix` is extra files to review. The previous chapter is enough.

Case 2: Pin Go like `fenix` pins `rustc`

Save as `overlay.nix`:

```
# overlay.nix
final: prev: {
  deskGo = prev.go_1_24;
  buildGoModule = prev.buildGoModule.override { go = final.deskGo; };
}
```

Then `pkgs.buildGoModule { ... }` in that `pkgs` uses 1.24. Document the version in the flake description. When 26.05's default `go` is already what you want, **do not** overlay.

Case 3: gomod2nix generate

```
nix run github:nix-community/gomod2nix -- --dir .
```

That writes `gomod2nix.toml` (and maybe a stub). Commit it.

Save as `flake.nix` fragment:

```
# flake.nix
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    gomod2nix = {
      url = "github:nix-community/gomod2nix";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  outputs = { self, nixpkgs, gomod2nix }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in
    {
      packages.${system}.desk-api =
        (gomod2nix.legacyPackages.${system}.buildGoApplication {
          pname = "desk-api";
          version = "1.0.0";
          src = ./.;
          modules = ./gomod2nix.toml;
        });
    };
}
```

Exact attr names (`buildGoApplication` vs `mk`) follow the `gomod2nix` README for the lock you pin. `follows` so you do not eval two `nixpkgs`.

Case 4: vendor/ in git (air-gap)

```
go mod vendor

buildGoModule {
  src = ./.;
  vendorHash = null;
  # with a vendor/ directory, some versions use:
  # vendorHash = null; and copy vendor
}
```

On current `buildGoModule`, an in-tree `vendor/` plus the right flags skips the FOD. It bloats git. Use when the module proxy is forbidden. Prefer `vendorHash` FODs on the desk.

Case 5: Private modules

```
# in the FOD, not in buildPhase:
# GOPRIVATE=git.desk.internal
# access-tokens in nix.conf (CI secrets chapter)
```

`buildGoModule`'s vendor FOD **may** use the network. Give it `access-tokens` / `NIX_PREFETCH_GIT` SSH. Do not `git config` secrets into the sandbox via Nix strings.

The trap

The trap is **switching to gomod2nix because a blog compared it to Crane**, then checking in 400 generated files for a 3-module API. Measure `nix-build` time first.

The other trap is three Go versions: `home.packages = [ go_1_22 ]`, a devShell `go_1_24`, and `buildGoModule` default. One overlay / one override. CI and laptop match.

The boring rule

- `buildGoModule` first. `gomod2nix` when vendor FODs hurt.
- Pin go with `.override { go = pkgs.go_1_24; }` (or whatever 26.05 offers), not a random tarball.
- follows on `gomod2nix`'s `nixpkgs`.
- No `dream2nix` as the desk default.
- One compiler version per flake.

Try this

1. `nix eval -f '<nixpkgs>' go.version` on 26.05. Write it down.
2. Overlay `buildGoModule` to `go_1_24` if that attr exists; `strings` the binary for `go1..`
3. `nix run github:nix-community/gomod2nix -- --help`.
4. `git grep GOPATH` in the flake — should not set a host `GOPATH` for the build.

Go Development Environments

Go Development Environments

A Go service that builds in CI and not in `$HOME/sdk` is the point of Nix. The boring default is: `mkShell` with `go`, `gopls`, `gotools`, and `golangci-lint` from the same **26.05** pin as `buildGoModule` — `direnv` loads it; Home Manager does not install Go.

Mental model

```
flake.nix
packages.desk-api      = buildGoModule { ... }
devShells.default      = mkShell { packages = [ go gopls ... ]; }
```

Same `go` attr in both. `direnv` + use `flake` (part 3). `CGO_ENABLED=0` in `shellHook` if the package sets it.

Private modules: `GOPRIVATE=git.desk.internal` in the shell. Tokens stay in the environment / `netrc`, not in the `flake`.

Worked examples

Case 1: Shell that matches the package

Save as `flake.nix`:

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
      go = pkgs.go;
    in
    {
      packages.${system}.desk-api = pkgs.buildGoModule {
        pname = "desk-api";
        version = "1.0.0";
        src = ./.;
      }
```

```

        vendorHash = null;
    };

    devShells.${system}.default = pkgs.mkShell {
        packages = [
            go
            pkgs.gopls
            pkgs.gotools
            pkgs.golangci-lint
            pkgs.delve
        ];
        env.CGO_ENABLED = "0";
        env.GOTOOLCHAIN = "local";
    };
};
}

```

GOTOOLCHAIN=local stops Go 1.21+ from downloading a different toolchain from go.mod's go line. The Nix go is the compiler.

```

nix develop --command go version
nix develop --command which gopls

```

Case 2: direnv

```

# .envrc
use flake

direnv allow
go version

```

Commit .envrc. Gitignore .direnv/.

Case 3: Tests as you type

```

nix develop --command go test ./...
nix develop --command golangci-lint run

```

CI runs the same via `nix develop --command` or `buildGoModule doCheck`. Do not `go test` with a Homebrew Go.

Case 4: CGO shell when you mean it

```
pkgs.mkShell {
  packages = [ pkgs.go pkgs.pkg-config pkgs.sqlite ];
  env.CGO_ENABLED = "1";
}
```

Now `mattn/go-sqlite3` can compile. The **package** derivation needs the same `nativeBuildInputs` and `CGO_ENABLED`, or CI will not match.

Case 5: Private GOPRIVATE

```
pkgs.mkShell {
  packages = [ pkgs.go pkgs.git pkgs.openssh ];
  env.GOPRIVATE = "git.desk.internal";
  env.GOPROXY = "https://proxy.golang.org,direct";
}
```

SSH agent / `~/.netrc` is user state (sops / HM). The flake only names the host pattern.

The trap

The trap is `go install golang.org/x/tools/gopls@latest inside the shell`. That ignores the pin and writes to `GOPATH`. `pkgs.gopls` from 26.05.

The other trap is `GOTOOLCHAIN=auto` (Go default) downloading `go1.23.x` on first `go build`. `GOTOOLCHAIN=local`.

The boring rule

- Same `go` in `mkShell` and `buildGoModule`.
- `gopls` / `golangci-lint` / `delve` from `nixpkgs`, not `@latest`.
- `GOTOOLCHAIN=local`. `CGO_ENABLED` matches the package.
- `direnv` use `flake`. No Home Manager `pkgs.go`.
- `GOPRIVATE` in the shell; tokens not in Nix.

Try this

1. `nix develop --command go env GOTOOLCHAIN GOVERSION CGO_ENABLED`.
2. `which go` inside vs outside `direnv`.
3. Add `golangci-lint` run as a `checks.` derivation.
4. Set `GOTOOLCHAIN=local` and a `go.mod` that asks for a newer `go` — read the error; bump `nixpkgs go` instead of letting Go download.

Cross-Compiling Go

Cross-Compiling Go

Go cross-compile without a sysroot **when CGO is off**. The boring default is: **CGO_ENABLED=0** and **GOOS/GOARCH** via **buildGoModule** env, or a **native remote builder** — **not qemu-user** for every **PR**.

pkgsCross still matters if CGO is on (cgo + glibc for the target). For the desk API, turn CGO off first.

Mental model

Need	Tool
linux/amd64 binary from a Mac	GOOS=linux GOARCH=amd64, CGO=0
linux/arm64 on x86_64, CGO=0	GOARCH=arm64
CGO + glibc for aarch64	pkgsCross.aarch64-multiplatform.buildGoModule or an aarch64 builder
NixOS test VM	Native aarch64-linux builder + KVM

```
buildGoModule {  
  env.CGO_ENABLED = "0";  
  env.GOOS = "linux";  
  env.GOARCH = "arm64";  
}
```

The compiler is still the **build go**. It emits target objects. **file** the result.

Worked examples

Case 1: linux/arm64 from x86_64, no CGO

```
# go-arm.nix  
{ pkgs ? import <nixpkgs> { } }:  
  
pkgs.buildGoModule {  
  pname = "desk-api";  
  version = "1.0.0";  
  src = ./.;  
}
```

```

    vendorHash = null;
    env.CGO_ENABLED = "0";
    env.GOOS = "linux";
    env.GOARCH = "arm64";
}

```

```

nix-build go-arm.nix
file result/bin/desk-api

```

Output (shape):

result/bin/desk-api: ELF 64-bit LSB executable, ARM aarch64, ...

Do not run it on the x86_64 laptop. Copy to a Pi or qemu-user.

Case 2: Flake packages per arch

```

# flake.nix fragment
packages.x86_64-linux.desk-api-arm64 = pkgs.buildGoModule {
  pname = "desk-api";
  version = "1.0.0";
  src = ./.;
  vendorHash = null;
  env.CGO_ENABLED = "0";
  env.GOARCH = "arm64";
  env.GOOS = "linux";
};

```

CI: nix build .#desk-api-arm64 on ubuntu-amd64. Fast.

Case 3: CGO — use pkgsCross or a builder

```

pkgs.pkgsCross.aarch64-multiplatform.buildGoModule {
  pname = "desk-api";
  src = ./.;
  vendorHash = null;
  env.CGO_ENABLED = "1";
}

```

Now you have a target gcc. Slow. Prefer ssh-ng to an ARM machine (remote builders chapter) and native buildGoModule.

Case 4: Windows / Darwin as GOOS

```
env.GOOS = "darwin";  
env.GOARCH = "arm64";  
env.CGO_ENABLED = "0";
```

Shipping a Mac binary from Linux CI is possible with CGO off. Notarization is not Nix's job.

Case 5: file and go version -m

```
file result/bin/desk-api  
go version -m result/bin/desk-api | head
```

Confirm GOOS/GOARCH in the build info. If linux/amd64 leaked, env.GOARCH did not apply (wrong attr, or CGO forced host).

The trap

The trap is **CGO on + GOARCH=arm64 on an x86_64 host** without a cross gcc. The build uses the **host** gcc and produces a confused binary or a compile error. CGO off, or **pkgsCross**, or a native builder.

The other trap is qemu-user in every GitHub job. Minutes. Native GOARCH with CGO=0 is seconds.

The boring rule

- CGO off for services you cross.
- env.GOOS / env.GOARCH on buildGoModule.
- file the output.
- CGO on → pkgsCross or remote native builder.
- Do not qemu the whole NixOS closure to get a Go binary.

Try this

1. Case 1; file; confirm aarch64.
2. Same drv with GOARCH=amd64; file; two different hashes.
3. Set CGO_ENABLED=1 without gcc; read the error.
4. go version -m on both binaries.

Packaging Anti-Patterns

Packaging Anti-Patterns

Most “Nix is slow / undeterministic” reports are the same five habits. The boring default is: **fix these before adding Crane, uv2nix, or another overlay file.**

Mental model

Smell	Usual cause
CI laptop	<nixpkgs> or unlocked nixpkgs#
Every commit rebuilds gcc	src = ../. includes docs / .git / target/
Eval takes minutes	import-from-derivation
go on PATH CI	nix-env / Home Manager compiler
Infinite recursion	final.pkg.overrideAttrs
Password in nix why-depends	secret in a derivation

Worked examples

Case 1: <nixpkgs> in production

Bad:

```
# configuration.nix - do not
{ pkgs ? import <nixpkgs> { }, ... }: { }
```

Fix:

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }: {
    nixosConfigurations.desk = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [ ./configuration.nix ];
    };
  };
}
```

nix flake metadata shows one nixpkgs node.

Case 2: Fat src

Bad: `src = ./.`; at the monorepo root.

Fix: `lib.fileset.toSource` / `cleanSource` (source-filtering chapter).

```
nix-build
echo x >> README.md
nix-build --dry-run
```

If `gcc` is in “will be built,” the filter failed.

Case 3: IFD

Bad:

```
let
  src = pkgs.fetchFromGitHub { owner = "desk"; repo = "mod"; rev = "abc"; hash = "sha256-...";
in
import "${src}/default.nix"
```

Eval `builds src` before it can parse `default.nix`. Flakes + IFD is a foot-gun.

Fix: copy `default.nix` into *this* repo, or add a flake input and `callPackage`.

Case 4: Compiler on the profile

Bad:

```
nix-env -iA nixpkgs.go
nix profile install nixpkgs#rustc
```

Fix: `devShells.default` with the same `go` / `rustPlatform` as `buildGoModule` / `buildRustPackage`.
`nix profile` for `jq`, not compilers.

Case 5: Overlay recursion and secrets

Bad: `hello = final.hello.overrideAttrs (_: { })` → infinite recursion.

Fix: `prev.hello.overrideAttrs`.

Bad: `stdenv.mkDerivation { ... password = "hunter2"; }` or `writeText "env" "TOKEN=..."`.

Fix: `sops-nix path` at runtime. `nix path-info -r` must not `grep` the token.

The trap

The trap is **papering over these with dream2nix / flake-parts / a mega-overlay**. The new tool then IFDs, too. Delete the anti-pattern first.

The boring rule

- Pin **nixos-26.05**. Commit `flake.lock`.
- Filter `src`. No IFD on the hot path.
- No `nix-env`. No compiler in `home.packages`.
- Override **this** package from `prev`.
- No secret bytes in `$out`.

Try this

1. `git grep -n '<nixpkgs>'` and `git grep 'src = \./\.'`.
2. `git grep nix-env`.
3. `nix flake metadata` — a single `nixpkgs` lock node.
4. `--dry-run` after a README-only edit on a filtered package.
5. `git grep -nE 'password = "|TOKEN=' -- '*.nix'`.

NixOS System

The NixOS Module System Architecture

The NixOS Module System Architecture

Editing `/etc` by hand and hoping two files agree is how machines drift. The boring default is: **the NixOS module system — typed options, merged config, and one evaluation that becomes a generation.**

You already installed a VM. This chapter is how that `configuration.nix` is not a special snowflake: it is one module among many, including the ones you will write for the desk.

Mental model

A module is a function:

```
{ config, lib, pkgs, ... }:  
{  
  options = { ... }; # what *may* be set (types, defaults, docs)  
  config = { ... }; # what *is* set (units, packages, files)  
  imports = [ ... ]; # other modules, merged in  
}
```

If you omit `options` and only set `config` keys (`services.openssh.enable = true;`), you are still a module — you are **consuming** options someone else declared (usually in `nixpkgs`).

The evaluator:

1. Loads every imported module (your files + `nixpkgs` NixOS modules).
2. Merges `options` (cannot define the same option twice).
3. Merges `config` with **priorities**.
4. Type-checks. Failures are eval errors, not “nginx failed at 3 a.m.”

Helper	Role
<code>lib.mkEnableOption "..."</code>	Boolean enable with a description
<code>lib.mkOption { type, default, ... }</code>	Any typed option
<code>lib.mkIf cond cfg</code>	Include <code>cfg</code> only when <code>cond</code> is true
<code>lib.mkDefault value</code>	Low priority (easy to override)
<code>lib.mkForce value</code>	High priority (wins over ordinary assignment)
<code>lib.mkMerge [a b]</code>	Merge two config attrsets

```
desk-module.nix (options + mkIf)  
host.nix        (enable = true; port = 9090)  
nixpkgs modules (openssh, users, ...)
```

one config → one system derivation

Worked examples

Case 1: A small desk module

Save as `desk_module.nix`:

```
# desk_module.nix
{ lib, config, pkgs, ... }:

let
  cfg = config.services.deskOperations;
in
{
  options.services.deskOperations = {
    enable = lib.mkEnableOption "Desk operations banner";

    port = lib.mkOption {
      type = lib.types.port;
      default = 8080;
      description = "TCP port printed by the banner.";
    };

    banner = lib.mkOption {
      type = lib.types.str;
      default = "Desk Operational";
      description = "Text printed by desk-banner.";
    };
  };

  config = lib.mkIf cfg.enable {
    environment.systemPackages = [
      (pkgs.writeShellScriptBin "desk-banner" ''
        echo "${cfg.banner} on port ${toString cfg.port}"
      '')
    ];
  };
}
```

`cfg.enable` is false by default. Nothing is installed until a host turns it on. That is the whole point of `mkIf`.

Case 2: Consume it from a host

Save as `host.nix`:

```
# host.nix
{
  imports = [ ./desk_module.nix ];

  services.deskOperations = {
    enable = true;
    port = 9090;
    banner = "Window A online";
  };
}
```

On a NixOS flake (or a lab `nixosSystem` eval):

```
nix eval .#nixosConfigurations.desk-vm.config.services.deskOperations.port
```

Output:

9090

The option is an integer (`lib.types.port`), not a string. Passing `"9090"` is an eval error.

Case 3: Priorities — `mkDefault` versus `mkForce`

Save as `priorities.nix`:

```
# priorities.nix
{ lib, ... }:

{
  networking.hostName = lib.mkDefault "generic-desk";
  services.openssh.enable = lib.mkForce true;
}
```

Save as `host-override.nix`:

```
# host-override.nix
{
  imports = [ ./priorities.nix ];
  networking.hostName = "desk-vm";
}
```

Ordinary assignment ("desk-vm") beats `mkDefault`. It does **not** beat `mkForce`. If a later file sets `services.openssh.enable = false;`, evaluation **fails** (conflict) unless that file also uses `mkForce`.

Use `mkDefault` in shared profiles (corp baseline). Use `mkForce` rarely: "this host *must* have sshd," not as a habit.

Case 4: Types catch mistakes at eval

Save as `bad-port.nix`:

```
# bad-port.nix
{
  imports = [ ./desk_module.nix ];
  services.deskOperations = {
    enable = true;
    port = "ninety";
  };
}

nixos-rebuild dry-build --flake .#desk-vm
```

Output (shape):

error: A definition for option `services.deskOperations.port' is not of type `16 bit unsigned i

No unit started. No half-applied /etc. That is the module system paying rent.

Case 5: Inspect options without guessing

```
nixos-option services.openssh.enable
nixos-option services.deskOperations.port
```

On a flake host:

```
nix eval .#nixosConfigurations.desk-vm.options.services.deskOperations.port.description
```

`man configuration.nix` (or `nixos-help`) is the same data as HTML. Prefer `nixos-option` / `nix eval` when you want the **live** merged value, not the upstream default.

The trap

The trap is `type = lib.types.anything` (or no type) so “it just merges.” Then `port = "8080";` ships, the script interpolates a string, and the unit fails at boot. Types are the boring check.

The other trap is defining `options.services.openssh.enable` yourself. That option already exists. You **set** it; you do not **redeclare** it. Redeclare → “option already defined.”

A third trap: putting implementation in `options` (an option whose default starts a service). Defaults should be data. Side effects belong in `config = mkIf cfg.enable { ... }`.

The boring rule

- One concern per module. Import it from the host. Do not paste the same 80 lines into three `configuration.nix` files.
- `mkIf cfg.enable` around every implementation.
- `mkDefault` in shared profiles. `mkForce` only when a conflict is the point.
- Strict types (`bool`, `port`, `str`, `listOf str`). Never **anything** for desk options.
- Inspect with `nixos-option` / `nix eval`, not by reading `/etc` after the fact.

Try this

1. Add `workers = lib.mkOption { type = lib.types.ints.positive; default = 4; };` and print it from `desk-banner`.
2. Set `port = 70000` and read the type error.
3. Put `networking.hostName = lib.mkDefault "a";` in one file and `networking.hostName = "b";` in another. Eval. Then change the second to `mkDefault "b"` and see the conflict.
4. `nixos-option services.openssh.settings.PasswordAuthentication` (or eval the flake option) and write down the default. Confirm it matches what you intend to set in the SSH chapter.

Configuring Your First NixOS Machine

Configuring Your First NixOS Machine

The installer left you with `/etc/nixos/configuration.nix` and a generation 1. The boring default from here is: **move that config into a flake in git, keep `hardware-configuration.nix` generated, and treat `nixos-rebuild switch --flake` as the only mutation path.**

The Installing NixOS chapter got a VM to boot. This chapter is how that VM becomes a **host you can clone**.

Mental model

File	Who writes it	How often it changes
<code>hardware-configuration.nix</code>	<code>nixos-generate-config</code>	When disks or initrd modules change
<code>configuration.nix</code> / host module	You	Every service, user, package
<code>flake.nix</code> + <code>flake.lock</code>	You	Pin nixpkgs; add the host output
<code>/run/current-system</code>	<code>nixos-rebuild</code>	Every successful switch

Rebuild verbs:

Command	Activates now?	Bootloader default?
<code>switch</code>	Yes	Yes
<code>test</code>	Yes	No (reboot undoes it)
<code>boot</code>	No	Yes (next reboot)
<code>dry-activate</code>	No	No (prints unit diffs)
<code>dry-build</code>	No	No (eval + build only)

```
git flake --nixos-rebuild switch--> new generation
                                     /nix/store/...-nixos-system-...
                                     bootloader entry
                                     /run/current-system
```

`system.stateVersion` stays at the **birth** release. It is not “which channel I follow.”

Worked examples

Case 1: A readable host module

Save as `configuration.nix` (or `hosts/desk-vm.nix` in a flake):

```
# configuration.nix
{ config, pkgs, ... }:

{
  imports = [ ./hardware-configuration.nix ];

  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;

  networking.hostName = "desk-workstation";
  time.timeZone = "UTC";
  i18n.defaultLocale = "en_US.UTF-8";

  environment.systemPackages = with pkgs; [
    git
    ripgrep
    htop
    curl
  ];

  services.openssh.enable = true;

  system.stateVersion = "26.05";
}
```

Leave `hardware-configuration.nix` as generated. Do not pretty-print UUIDs into `/dev/vda2`.

Case 2: Flake output for the same host

Save as `flake.nix` next to those files:

```
# flake.nix
{
  description = "Desk workstation";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    nixosConfigurations.desk-workstation = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
    };
  };
}
```

```

        modules = [ ./configuration.nix ];
    };
};
}

```

The output **name** should match **networking.hostName** (or be a short alias you document). Rebuild:

```
sudo nixos-rebuild switch --flake .#desk-workstation
```

Output:

```

building the system configuration...
updating systemd-boot...
activating the configuration...
setting up /etc...

```

`--flake .#desk-workstation` reads `flake.lock`. Two machines with the same lock build the same closure (hardware module aside).

Case 3: test before switch

Add `cowsay` to `environment.systemPackages`, then:

```

sudo nixos-rebuild test --flake .#desk-workstation
cowsay desk

```

Output:

```

----
< desk >
----

```

Reboot. `cowsay` is **gone**: `test` did not update the bootloader default. That is the point — try a service, reboot if it bricks the session, land on the last `switch`.

When you like it:

```
sudo nixos-rebuild switch --flake .#desk-workstation
```

Case 4: Dry-activate and generations

```
sudo nixos-rebuild dry-activate --flake .#desk-workstation
sudo nix-env --list-generations --profile /nix/var/nix/profiles/system
```

Output (shape):

would restart the following units: ...

would start the following units: ...

```
1  2026-09-01 10:00:00
2  2026-09-09 12:00:00  (current)
```

`dry-activate` is how you see that an `nginx` change restarts `nginx` and does **not** restart `sshd`. Read it before `switch` on a machine people are using.

Rollback:

```
sudo nixos-rebuild switch --rollback
```

Case 5: Split files without a framework

Save as `packages.nix`:

```
# packages.nix
{ pkgs, ... }:

{
  environment.systemPackages = with pkgs; [
    git
    ripgrep
    jq
  ];
}
```

In `configuration.nix`:

```
imports = [
  ./hardware-configuration.nix
  ./packages.nix
];
```

Lists of packages **concatenate**. Two modules both setting `environment.systemPackages` merge. Two modules both setting `networking.hostName` without `mkDefault` **conflict**. That is the module `system` from the previous chapter, on a real host.

The trap

The trap is **changing `system.stateVersion` when you bump `nixpkgs` to 26.11**. Databases, `stateVersion`-gated defaults, and Home Manager state dirs then migrate in surprising ways. Follow the release notes. Keep the birth value until a note tells you to bump a **specific** option.

The other trap is editing `/etc/nixos` on the live machine and never pushing git. The next `nixos-anywhere` or the next laptop rebuilds last week's host. The flake in git is the machine. `/etc/nixos` is either a checkout or a leftover from the installer — pick one.

A third trap: `nixos-rebuild switch` **without** `--flake` after you moved to a flake. You just built the old `/etc/nixos/configuration.nix` against channels. `nixos-version` and `nix flake metadata` should agree.

The boring rule

- Flake in git. `--flake .#host`. Lockfile committed.
- `hardware-configuration.nix` generated; host module written by you.
- `test` for risky changes; `switch` when you want it to survive reboot; `dry-activate` when you want to read unit diffs.
- `stateVersion` is a birth certificate.
- Split files with `imports`. Do not invent a framework for one VM.

Try this

1. `nixos-version` and `nix flake metadata` — confirm the `nixpkgs` revision in the lock matches what you think is running.
2. `sudo nixos-rebuild test` with a `hostname` change, `hostname`, `reboot`, `hostname` again. Then `switch` if you want it permanent.
3. `sudo nixos-rebuild dry-activate` after adding `services.nginx.enable = true;` (lab VM). Read which units would start. Disable `nginx` if you do not need it.
4. Split `environment.systemPackages` into `packages.nix`, `switch`, which `jq`.

Service Management with systemd

Service Management with systemd

Hand-written unit files in `/etc/systemd/system/` plus `daemon-reload` is how two hosts disagree. The boring default is: `systemd.services.<name>` as a Nix attrset, store-path binaries, and a high-level `services.<app>.enable` module when `nixpkgs` already has one.

Mental model

NixOS renders unit files into the system closure. `nixos-rebuild switch` diffs old vs new units and restarts what changed. You do not `systemctl enable` by hand; `wantedBy = [ "multi-user.target" ]`; is enablement.

Two layers:

Layer	When
<code>services.nginx.enable = true;</code>	<code>nixpkgs</code> already knows <code>nginx</code> . Use it.
<code>systemd.services.desk-worker = { ... };</code>	The <code>desk</code> binary has no module yet.

Custom units need **absolute** `ExecStart`. The service `PATH` is not your login `PATH`. `${pkgs.coreutils}/bin/echo`, not `echo`.

Hardening that costs one line:

Setting	Effect
<code>DynamicUser = true;</code>	Ephemeral uid; no <code>useradd</code>
<code>ProtectSystem = "strict";</code>	Root filesystem read-only
<code>ProtectHome = true;</code>	<code>/home</code> inaccessible
<code>PrivateTmp = true;</code>	Private <code>/tmp</code>
<code>NoNewPrivileges = true;</code>	No privilege escalation

```
systemd.services.desk-api
  wantedBy → starts at boot
  after    → ordering
  ExecStart → store path
  Restart  → on-failure
```

Worked examples

Case 1: A desk worker unit

Save as `desk_service_unit.nix`:

```
# desk_service_unit.nix
{ pkgs, ... }:

{
  systemd.services.desk-worker = {
    description = "Desk operations worker";
    wantedBy = [ "multi-user.target" ];
    after = [ "network-online.target" ];
    wants = [ "network-online.target" ];

    environment = {
      DESK_ENVIRONMENT = "production";
      PORT = "8080";
    };

    serviceConfig = {
      Type = "simple";
      ExecStart = "${pkgs.writeShellScript "desk-worker" ''
        ${pkgs.coreutils}/bin/echo "Desk worker on port $PORT"
        while true; do ${pkgs.coreutils}/bin/sleep 3600; done
      ''}";
      Restart = "on-failure";
      RestartSec = "5s";
      DynamicUser = true;
      ProtectSystem = "strict";
      ProtectHome = true;
      NoNewPrivileges = true;
      PrivateTmp = true;
      MemoryMax = "256M";
    };
  };
}
```

```
sudo nixos-rebuild switch
systemctl status desk-worker
journalctl -u desk-worker -n 20 --no-pager
```

Output (shape):

```
desk-worker.service - Desk operations worker
Loaded: loaded (/etc/systemd/system/desk-worker.service; enabled)
```


Active: active (running)

/etc/systemd/system/desk-worker.service is a symlink into /nix/store. Editing it by hand is undone on the next switch.

Case 2: Prefer a nixpkgs module when it exists

Save as nginx.nix:

```
# nginx.nix
{ pkgs, ... }:

{
  services.nginx = {
    enable = true;
    virtualHosts."desk.internal" = {
      root = pkgs.writeTextDir "index.html" "<h1>Desk</h1>";
    };
  };
}
```

This opens the right units, user, and (optionally) firewall helpers. Do not copy a 40-line systemd.services.nginx from a wiki.

```
systemctl is-active nginx
curl -sS http://127.0.0.1/ | head
```

Case 3: Timers instead of cron

Save as desk_timer.nix:

```
# desk_timer.nix
{ pkgs, ... }:

{
  systemd.services.desk-cleanup = {
    description = "Clean stale desk tickets";
    serviceConfig = {
      Type = "oneshot";
      ExecStart = "${pkgs.coreutils}/bin/echo desk-cleanup-ok";
    };
  };

  systemd.timers.desk-cleanup = {
    wantedBy = [ "timers.target" ];
    timerConfig = {
```

```

        OnCalendar = "daily";
        Persistent = true;
    };
};
}

systemctl list-timers --all | grep desk
sudo systemctl start desk-cleanup.service
journalctl -u desk-cleanup -n 5 --no-pager

```

Persistent = true catches up after the VM slept through midnight.

Case 4: Secrets as files, not unit text

```

# desk-envfile.nix
{
    systemd.services.desk-worker.serviceConfig.EnvironmentFile =
        "/run/secrets/desk-worker.env";
}

```

environment.PORT = "8080"; is fine for non-secrets. A database password in environment.PASSWORD = "..." lands in the world-readable store. EnvironmentFile points at a sops-decrypted tmpfs path (secrets part). The unit file then contains a **path**, not the secret.

Case 5: See what switch will restart

```

sudo nixos-rebuild dry-activate --flake .#desk-workstation

```

Output (shape):

```

would restart the following units: desk-worker.service
would not restart: sshd.service

```

If dry-activate says it will restart sshd because you changed a comment in a shared module, split the module. Restarts are part of the change.

The trap

The trap is ExecStart = "desk-api" or ExecStart = "/usr/bin/curl ...". The unit's PATH is minimal. desk-api: command not found at boot, while your interactive shell still has it.

Always \${pkgs.desk-api}/bin/desk-api or a writeShellScript that uses store paths inside.

The other trap is `systemctl edit desk-worker` drop-ins. They live outside the flake. The next switch may fight them, or they survive and surprise the next operator. If you needed the drop-in, it belonged in `systemd.services.desk-worker.serviceConfig`.

The boring rule

- High-level `services.<app>` when nixpkgs has it. Raw `systemd.services` for desk-specific units.
- `ExecStart` is a store path. No bare names, no `/usr/bin`.
- `DynamicUser + ProtectSystem = "strict"` on network daemons that do not need a named uid.
- Timers, not `crontab`.
- Secrets via `EnvironmentFile` / sops paths, never `environment.SECRET`.

Try this

1. Add `serviceConfig.MemoryMax = "64M";` to the worker, switch, `systemctl show desk-worker -p MemoryMax`.
2. Change `ExecStart` to `echo hello` (no store path), switch, `journalctl -u desk-worker -n 20`. Restore the store path.
3. `systemctl cat desk-worker` and confirm the unit lives under `/nix/store`.
4. Enable `services.timesyncd` (or `services.chrony`) via the **module**, not a custom unit. `systemctl status systemd-timesyncd`.

Networking, Firewall, and Security

Networking, Firewall, and Security

Ad-hoc `iptables -A` and a NetworkManager GUI click are gone after a rebuild — or worse, they **survive** in unmanaged `/etc` and fight the next activation. The boring default is: **firewall on**, **ports listed in Nix**, **NetworkManager on laptops**, **systemd-networkd on servers**, **secrets as files**.

Mental model

NixOS enables a firewall by default (`networking.firewall.enable` is `true`). Incoming is denied unless you or a module open it. `services.openssh.enable = true` opens 22 unless you tell it not to (`openFirewall = false` on modules that offer that flag).

Machine	Network backend
Desk laptop, Wi-Fi	<code>networking.networkmanager.enable = true;</code>
VM, cloud, rack server	<code>networking.useNetworkd = true;</code> (<code>systemd-networkd</code>)

Do not enable both and hope. Pick one.

`internet`

`nftables/iptables` ← `networking.firewall.allowedTCPPorts`

`sshd` / `nginx` / `desk-api`

WireGuard, Tailscale, and similar are **modules** (`networking.wireguard`, `services.tailscale`). They are not extra packages you `systemctl start` by hand.

Worked examples

Case 1: Hostname, DNS, firewall allow-list

Save as `networking_config.nix`:

```
# networking_config.nix
{
  networking = {
    hostname = "desk-gateway";
    domain = "internal.corp";

    firewall = {
      enable = true;
      allowPing = true; # ICMP; set false on a bastion if you mean it
      allowedTCPPorts = [ 22 8080 ];
      allowedTCPPortRanges = [
        { from = 9000; to = 9010; }
      ];
    };

    nameservers = [ "1.1.1.1" "9.9.9.9" ];
  };
}
```

```
sudo nixos-rebuild switch
sudo nft list ruleset | head
# or, if still on iptables backend:
sudo iptables -L INPUT -n | head
```

Port 8080 is open because you listed it. Port 3000 is not. A node app that binds :3000 is reachable on localhost only until you add 3000.

Case 2: systemd-networkd static address (server)

Save as networkd.nix:

```
# networkd.nix
{
  networking.useDHCP = false;
  networking.useNetworkd = true;
  networking.networkmanager.enable = false;

  systemd.network.networks."10-lan" = {
    matchConfig.Name = "en*";
    networkConfig = {
      Address = "10.20.0.10/24";
      Gateway = "10.20.0.1";
      DNS = "10.20.0.1";
    };
  };
}
```

Match on `en*` (or a by-id name) in the lab. On metal, prefer `matchConfig.MACAddress` or `Name = "enp1s0"` after you have seen `networkctl status`.

```
networkctl status
ip -br addr
```

Case 3: NetworkManager on the workstation

Save as `nm.nix`:

```
# nm.nix
{
  networking.networkmanager.enable = true;
  networking.useNetworkd = false;
  networking.firewall.enable = true;
}
```

Users who should click Wi-Fi:

```
users.users.deskadmin.extraGroups = [ "networkmanager" ];
```

Do not also set `networking.interfaces.eth0.ipv4.addresses` unless you know you want NixOS's scripted networking **instead**. One backend.

Case 4: WireGuard with a key file

Save as `wireguard.nix`:

```
# wireguard.nix
{
  networking.firewall.allowedUDPPorts = [ 51820 ];

  networking.wireguard.interfaces.wg0 = {
    ips = [ "10.100.0.2/24" ];
    listenPort = 51820;
    privateKeyFile = "/run/secrets/wg-desk.key";

    peers = [
      {
        publicKey = "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx";
        allowedIPs = [ "10.100.0.0/24" ];
        endpoint = "vpn.desk.internal:51820";
        persistentKeepalive = 25;
      }
    ];
  };
}
```

```
}
```

`privateKeyFile` is a **path**. The key bytes are not in the flake. Generate with `wg genkey` on the machine (or decrypt with `sops`) and persist that file (impermanence: `/persist`).

```
sudo systemctl status wg-quick-wg0
# or wg0 depending on the module's unit name
ip -br addr show wg0
```

Case 5: Trust a VPN interface without opening the world

```
# trusted.nix
{
  networking.firewall = {
    enable = true;
    trustedInterfaces = [ "lo" "wg0" ];
    allowedTCPPorts = [ 22 ];
  };
}
```

Traffic **from** `wg0` is trusted. The public NIC still only has 22. That is how the desk API can listen on 10.100.0.2:8080 without listing 8080 on the internet.

The trap

The trap is `networking.firewall.enable = false`; “to debug the app,” then committing the host. The next deploy is an open machine.

Open `allowedTCPPorts = [ 3000 ]`; instead. Leave the firewall on.

The other trap is putting a WireGuard **private** key in the attrset (`privateKey = "..."`). World-readable store. Same as a password in `configuration.nix`.

A third trap: `networking.firewall.extraCommands` with a page of `iptables`. Those rules are not typed, not easily reviewed, and break when the backend is `nftables`. Prefer `allowedTCPPorts`, `interfaces.<name>.allowedTCPPorts`, and `trustedInterfaces`.

The boring rule

- Firewall stays on. List ports. Do not disable it to debug.
- NetworkManager **or** `networkd`, not both.
- VPN keys as `privateKeyFile` (`sops` / `persist`), never as Nix strings.
- Let service modules open their own ports, or set `openFirewall` deliberately.
- `trustedInterfaces` for overlay NICs; public NIC stays tight.

Try this

1. Add 8080 to `allowedTCPPorts`, switch, `nc -l 8080` in one shell and `nc $HOST 8080` from another VM. Remove 8080 and confirm the second connect hangs/refuses.
2. `sudo nft list ruleset | grep -i 22` (or `iptables`) after `services.openssh.enable = true` with default firewall — 22 should be there.
3. Set `allowPing = false` on a lab VM, ping it from the host, then set it true again.
4. Generate a WireGuard keypair with `wg genkey | tee /tmp/k | wg pubkey` and **do not** paste the private half into a `.nix` file. Put it in a file and reference the path.

Users, Groups, and SSH Configuration

Users, Groups, and SSH Configuration

`useradd` on a server is an account `git` does not know about. The boring default is: `users.mutableUsers = false`, SSH keys in the flake, password hashes from `mkpasswd` or a file, and `sshd` with password auth off.

Mental model

With `mutableUsers = false`, every `nixos-rebuild switch` **rewrites** `/etc/passwd` and `/etc/shadow` from the config. An account that left the flake left the machine. `passwd` as a user may appear to work until the next switch (or be blocked). That is the feature.

Field	Use
<code>isNormalUser = true;</code>	Human: home, shell, extraGroups
<code>isSystemUser = true;</code>	Daemon uid you named (rare if <code>DynamicUser</code> suffices)
<code>extraGroups = ["wheel"];</code>	sudo (if sudo is enabled)
<code>openssh.authorizedKeys.keys</code>	Login without a password
<code>hashedPassword</code>	Hash in the store (still not plaintext)
<code>hashedPasswordFile</code>	Hash outside the store (better)
<code>initialPassword</code>	Lab only. Plaintext in the store. Never production.

```
flake → users.users.deskadmin
      uid/gid
      ~/.ssh/authorized_keys (generated)
      hashedPasswordFile
```

`services.openssh` is the daemon. User keys do not start `sshd`. Enable the service, then disable password authentication.

Worked examples

Case 1: Immutable users and hardened sshd

Save as `users_ssh.nix`:

```

# users_ssh.nix
{
  users.mutableUsers = false;

  users.users.deskadmin = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    openssh.authorizedKeys.keys = [
      "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDeskAdminLaptop deskadmin@laptop"
    ];
  };

  users.users.deployer = {
    isNormalUser = true;
    description = "CI deploy key";
    extraGroups = [ "wheel" ];
    openssh.authorizedKeys.keys = [
      "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleDeployerKeyCorp"
    ];
  };

  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      KbdInteractiveAuthentication = false;
      PermitRootLogin = "no";
      X11Forwarding = false;
    };
  };

  security.sudo.wheelNeedsPassword = true;
}

```

`security.sudo.wheelNeedsPassword = false` is convenient on a lab VM and a foot-gun on a laptop someone might walk away from. Default to needing a password (or a key + `sudo` timestamp) unless this is a CI image.

```

sudo nixos-rebuild switch
getent passwd deskadmin
sudo grep -E 'PasswordAuthentication|PermitRootLogin' /etc/ssh/sshd_config

```

Case 2: Hash a console password

On a trusted machine:

```
mkpasswd -m sha-512
```

Output (shape):

```
$6$rounds=656000$SALT$HASH...
```

```
# password_hash.nix
{
  users.users.operator = {
    isNormalUser = true;
    hashedPassword = "$6$rounds=656000$SALT$HASH";
  };
}
```

The hash is in the store. That is weaker than `hashedPasswordFile` but **not** the same as `initialPassword = "hunter2"`. Anyone with store access can **offline-crack** the hash; they do not see the password in `grep`. Prefer a file:

```
{
  users.users.operator.hashedPasswordFile = "/persist/secrets/operator.hash";
}
```

Generate the file once, mode 0400, persist it (impermanence). `sops-nix` can decrypt it at boot.

Case 3: Root is not a daily login

```
# root.nix
{
  users.users.root.openssh.authorizedKeys.keys = [ ];
  services.openssh.settings.PermitRootLogin = "no";
}
```

Break-glass in a lab VM can use a root password hash **on the console**, not over SSH. Production: serial console + the previous generation, not `PermitRootLogin = "yes"`.

Case 4: Groups for devices, not for folklore

```
# groups.nix
{
  users.users.deskadmin.extraGroups = [
    "wheel"
    "networkmanager" # if NM is enabled
    "video"
    "dialout"         # USB serial, lab only
  ];
}
```

```
}
```

Each group is a capability. `extraGroups = [ "wheel" "docker" "libvirt" "adbusers" "..."]` on every human is how laptops become root. Add groups when a device or module requires them.

`users.groups.desk.members = [ "deskadmin" "deployer" ];` for a shared directory.

Case 5: Prove password SSH is dead

From another machine:

```
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no deskadmin@desk-workstation
```

Output:

```
deskadmin@desk-workstation: Permission denied (publickey).
```

Then:

```
ssh -i ~/.ssh/id_ed25519 deskadmin@desk-workstation true
```

Output: silence, exit 0.

If password still works, `PasswordAuthentication` did not land (wrong `settings`. attr on this release) or you are not hitting this `sshd`.

The trap

The trap is `initialPassword = "changeme"` left in the flake after install. The string is in `/nix/store` forever (and in git history). Even after you “change” it with `passwd`, `mutableUsers = false` may reset or ignore that, and the store still has `changeme`.

Use `hashedPassword` / `hashedPasswordFile`. Delete `initialPassword` from the first commit that leaves the lab.

The other trap is `users.mutableUsers = true` on a server “so the new hire can `passwd`.” They never appear in git. Six months later nobody can say who has a shell. `false` plus a PR that adds their key.

The boring rule

- `mutableUsers = false` on anything you would call production (and on the lab VM, so you practise).
- SSH keys in the flake. Password SSH off. Root SSH off.
- Hashes from `mkpasswd -m sha-512`. Prefer `hashedPasswordFile`.
- Never `initialPassword` outside a throwaway installer snippet.
- `wheel` is `sudo`. Do not sprinkle extra groups “just in case.”

Try this

1. Run `mkpasswd -m sha-512`, paste the hash on a lab user, switch, log in on the VM console.
2. Add a second authorized key, switch, `ssh-add -l` / connect with that key. Remove the key, switch, confirm that key is refused.
3. `sudo useradd leftover` on a `mutableUsers = false` host, then `sudo nixos-rebuild switch`, then `getent passwd leftover`. The account should be gone.
4. `nix-store --query --references /run/current-system | xargs grep -l initialPassword` — should print nothing. If it prints, you still have plaintext in the closure.

Hardware Configuration and Drivers

Hardware Configuration and Drivers

The host module is identity (hostname, users, services). Hardware is silicon (disks, CPU, GPU, firmware). The boring default is: **generated hardware-configuration.nix, UUIDs not /dev/sda, CPU microcode on, and nixos-hardware** when a profile exists for this chassis.

Mental model

```
flake
modules = [
  ./configuration.nix      # you
  ./hardware-configuration.nix # generated
  nixos-hardware...       # community, optional
]
```

`nixos-generate-config --root /mnt` (installer) or `--show-hardware-config` (running system) writes filesystem UUIDs and `boot.initrd.availableKernelModules` (nvme, virtio, ahci). If `initrd` lacks the disk driver, the VM boots to an emergency shell.

Piece	Typical source
File systems, swap, <code>initrd</code> modules	<code>hardware-configuration.nix</code>
CPU microcode	<code>hardware.cpu.intel/amd.updateMicrocode</code>
Laptop quirks (audio, fingerprint)	<code>nixos-hardware</code>
GPU	<code>hardware.graphics</code> + vendor module if needed
Firmware blobs	<code>hardware.enableRedistributableFirmware = true;</code>

Do not merge hardware into `configuration.nix` “to have one file.” The next disk clone will fight you.

Worked examples

Case 1: Read the generated file; do not invent it

Save as `hardware-configuration.nix` (illustrative — **your** UUIDs come from `blkid`):

```
# hardware-configuration.nix
{ config, lib, pkgs, modulesPath, ... }:

{
  imports = [ (modulesPath + "/installer/scan/not-detected.nix") ];

  boot.initrd.availableKernelModules = [ "nvme" "xhci_pci" "ahci" "usbhid" "sd_mod" ];
  boot.kernelModules = [ "kvm-amd" ];

  fileSystems."/ = {
    device = "/dev/disk/by-uuid/a1b2c3d4-e5f6-7890-abcd-1234567890ab";
    fsType = "ext4";
  };

  fileSystems."/boot" = {
    device = "/dev/disk/by-uuid/1234-ABCD";
    fsType = "vfat";
  };

  swapDevices = [
    { device = "/dev/disk/by-uuid/f9e8d7c6-b5a4-3210-fedc-ba9876543210"; }
  ];

  nixpkgs.hostPlatform = lib.mkDefault "x86_64-linux";
  hardware.cpu.amd.updateMicrocode = lib.mkDefault config.hardware.enableRedistributableFirm
}

```

On a QEMU VM you will see `virtio_pci`, `virtio_blk`, `virtio_net` instead of `nvme`. That is correct. Do not copy an NVMe file onto a virtio VM.

```
blkid
ls /dev/disk/by-uuid

```

Case 2: Firmware and microcode

Save as `firmware.nix`:

```
# firmware.nix
{ config, ... }:

{
  hardware.enableRedistributableFirmware = true;
  hardware.cpu.intel.updateMicrocode = true;
  # hardware.cpu.amd.updateMicrocode = true; # AMD hosts
}

```

Pick **one** CPU vendor. Enabling both is harmless-ish but noisy. Without microcode, you skip CPU bug fixes the vendor shipped as firmware.

Case 3: nixos-hardware for a known laptop

Save as `flake.nix` fragment (inputs + import):

```
# flake-hardware.nix
{
  description = "Desk laptop";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    nixos-hardware.url = "github:NixOS/nixos-hardware";
  };

  outputs = { self, nixpkgs, nixos-hardware }: {
    nixosConfigurations.desk-laptop = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        ./configuration.nix
        ./hardware-configuration.nix
        nixos-hardware.nixosModules.framework-13-7040-amd
      ];
    };
  };
}
```

Replace the module name with the one that matches **your** chassis (ls the repo's module list, or the nixos-hardware README). A ThinkPad module on a Dell is worse than no module.

Case 4: Graphics — Mesa first, vendor blobs only if needed

Save as `graphics.nix`:

```
# graphics.nix
{
  hardware.graphics = {
    enable = true;
    enable32Bit = true; # Steam / wine; skip on a headless VM
  };
}
```

Intel and AMD generally need this plus firmware. NVIDIA:

```
# nvidia.nix
{
  services.xserver.videoDrivers = [ "nvidia" ];
  hardware.nvidia.open = false; # or true, per GPU generation - read the option doc
  hardware.nvidia.modesetting.enable = true;
}
```

Do not enable NVIDIA modules on a VM with virtio-gpu. The guest will load a driver for hardware that is not there.

Case 5: KVM on a workstation that runs VMs

```
# kvm.nix
{
  virtualisation.libvirtd.enable = true;
  boot.kernelModules = [ "kvm-intel" ]; # or kvm-amd
  users.users.deskadmin.extraGroups = [ "libvirtd" "kvm" ];
}
```

`boot.kernelModules` here is **host** virtualisation, not `initrd` disk drivers. Keep disk drivers in the generated hardware file.

```
ls /dev/kvm
```

The trap

The trap is `fileSystems."/".device = "/dev/sda2"`. Plug in a USB disk, reboot, the kernel enumerates `sda` as the USB stick, and the root UUID is now `sdb2`. Emergency shell. Always `by-uuid` or `by-id`.

The other trap is **hand-editing** `boot.initrd.availableKernelModules` out of the generated file because it “looked messy,” then cloning the disk to a machine whose storage driver you deleted.

Regenerate on the new chassis (`nixos-generate-config`) and keep the host module.

The boring rule

- Generated hardware file. Your services stay in `configuration.nix`.
- Filesystems by UUID (`blkid`), never `sda/vda` in production configs. Disko `by-id` is the fleet version of this rule.
- Microcode + redistributable firmware on real machines.
- `nixos-hardware` only when the module name matches the chassis.
- GPU modules only for GPUs that exist.

Try this

1. `blkid` and compare UUIDs to `hardware-configuration.nix`. They must match.
2. `lsinitrd /boot/EFI/nixos/*.efi 2>/dev/null | grep -i virtio` (or `lsinitrd $(readlink /run/current-system/initrd)`) and find the disk driver.
3. `lscpu | grep -i vendor` and enable **that** vendor's `updateMicrocode`.
4. Browse `nixos-hardware` module names (`nix flake show github:NixOS/nixos-hardware` may be huge; the GitHub tree is enough). Write down the one that matches your laptop, or “none — VM.”

Declarative Disk Partitioning with Disko

Declarative Disk Partitioning with Disko

`fdisk` notes in a wiki are a snowflake. The boring default is: **Disko describes GPT, LUKS, and filesystems in Nix; you format from that file; nixos-anywhere uses the same file on a remote disk.**

The Installing NixOS chapter partitioned a VM by hand so you would see mounts. From here on, the layout lives in git.

Mental model

Disko (`github:nix-community/disko`) is a NixOS module plus a script.

Mode	What it does
<code>format</code>	Partition, LUKS, <code>mkfs</code> . Destroys the disk.
<code>mount</code>	Mounts in dependency order under <code>/mnt</code>
NixOS module	Also emits <code>fileSystems.*</code> so you may not need a separate hardware file for mounts

`disk-config.nix`

```
nixos-rebuild → fileSystems."/\" etc.
disko script  → wipe /dev/disk/by-id/... (once)
```

Device names (`/dev/sda`, `/dev/vda`, `/dev/nvme0n1`) **move**. Prefer `/dev/disk/by-id/...` on metal. On a throwaway QEMU disk, `/dev/vda` is acceptable if the VM has one disk.

`nixos-anywhere` SSHes into rescue, `kexecs` NixOS, runs Disko, copies the closure, installs the bootloader, reboots. Same `disk-config.nix`.

Worked examples

Case 1: Lab VM — one disk, no LUKS

Save as `disk-config.nix`:

```
# disk-config.nix
{
  disko.devices.disk.main = {
```

```

type = "disk";
device = "/dev/vda";
content = {
  type = "gpt";
  partitions = {
    ESP = {
      size = "1G";
      type = "EF00";
      content = {
        type = "filesystem";
        format = "vfat";
        mountpoint = "/boot";
        mountOptions = [ "umask=0077" ];
      };
    };
    root = {
      size = "100%";
      content = {
        type = "filesystem";
        format = "ext4";
        mountpoint = "/";
      };
    };
  };
};
};
};
}

```

Import it from the host:

```

# configuration.nix fragment
{ inputs, ... }:
{
  imports = [
    inputs.disko.nixosModules.disko
    ./disk-config.nix
  ];
}

```

On a **throwaway** VM that is allowed to lose /dev/vda:

```

sudo nix --experimental-features 'nix-command flakes' run github:nix-community/disko -- \
  --mode disko ./disk-config.nix

```

`--mode disko` formats **and** mounts. There is no undo. Not your laptop's NVMe.

Case 2: Metal — by-id + LUKS + Btrfs subvolumes

Save as disk-luks-btrfs.nix:

```
# disk-luks-btrfs.nix
{
  disko.devices.disk.main = {
    type = "disk";
    device = "/dev/disk/by-id/nvme-VENDOR_MODEL_SERIAL";
    content = {
      type = "gpt";
      partitions = {
        ESP = {
          size = "1G";
          type = "EF00";
          content = {
            type = "filesystem";
            format = "vfat";
            mountpoint = "/boot";
            mountOptions = [ "umask=0077" ];
          };
        };
      };
    };
    luks = {
      size = "100%";
      content = {
        type = "luks";
        name = "crypted";
        settings.allowDiscards = true;
        passwordFile = "/tmp/luks.key";
        content = {
          type = "btrfs";
          extraArgs = [ "-f" ];
          subvolumes = {
            "@root" = {
              mountpoint = "/";
              mountOptions = [ "compress=zstd" "noatime" ];
            };
            "@nix" = {
              mountpoint = "/nix";
              mountOptions = [ "compress=zstd" "noatime" ];
            };
            "@persist" = {
              mountpoint = "/persist";
              mountOptions = [ "compress=zstd" "noatime" ];
            };
            "@swap" = {
              mountpoint = "/swap";
            };
          };
        };
      };
    };
  };
}
```

```

        swap.swapfile.size = "8G";
    };
};
};
};
};
};
};
};
}

```

`ls -l /dev/disk/by-id/` on the machine (or rescue) and paste the **id** that is not a `-partN` partition. `passwordFile` is for **provisioning**; after install, unlock is passphrase or TPM (later hardening). Do not commit `/tmp/luks.key`.

`@persist` is the impermanence contract. Create it even if you still boot a persistent root today.

Case 3: Flake input

```

# flake.nix
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    disk = {
      url = "github:nix-community/disko";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  outputs = { self, nixpkgs, disk }: {
    nixosConfigurations.desk-server = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      specialArgs = { inherit disk; };
      modules = [
        disk.nixosModules.disko
        ./disk-config.nix
        ./configuration.nix
      ];
    };
  };
}

```

```
nix eval .#nixosConfigurations.desk-server.config.fileSystems.\"/\".device
```

You should see a mapper or UUID-style device coming from Disko, not a guess.

Case 4: nixos-anywhere

```
nix run github:nix-community/nixos-anywhere -- \
  --flake .#desk-server \
  --generate-hardware-config nixos-generate-config ./hardware-configuration.nix \
  root@203.0.113.42
```

Output (shape):

```
[nixos-anywhere] Connected to rescue over SSH.
[nixos-anywhere] kexec NixOS installer...
[nixos-anywhere] Running Disko on /dev/disk/by-id/...
[nixos-anywhere] Copying closure...
[nixos-anywhere] Installing bootloader...
[nixos-anywhere] Rebooting.
```

The remote must be a rescue environment you are allowed to wipe. Hetzner rescue, a cloud “reinstall,” or a live ISO with SSH. Not `root@production-that-has-data`.

Case 5: Parameterise the device per host

Save as `disk-config.nix`:

```
# disk-config.nix
{ diskDevice ? "/dev/vda", ... }:

{
  disk.devices.disk.main.device = diskDevice;
  # ... rest of the layout using `main` ...
}
```

Or two files: `hosts/desk-vm/disk.nix` (`/dev/vda`) and `hosts/desk-metal/disk.nix` (`by-id`). Same subvolume names so impermanence and backups stay identical.

The trap

The trap is **pointing Disko at the wrong disk** because `/dev/nvme0n1` was the installer USB on this firmware. `by-id` includes the serial. Read `lsblk -o NAME,SIZE,MODEL,SERIAL` twice.

The other trap is committing `passwordFile = "/tmp/luks.key"` **contents** into the flake, or leaving a LUKS passphrase in `settings.keyFile` as a Nix string. Same store leak as any secret.

A third trap: running `--mode format` on a machine that already has `/persist` data. Disko is an installer, not a resize tool. Back up, then format.

The boring rule

- Disk layout in git. Format from that file. Do not keep a parallel `fdisk` runbook.
- `by-id` on real disks. `/dev/vda` only on single-disk lab VMs.
- LUKS passphrase / key file is provisioning state, not a Nix string.
- Same Disko file for `nixos-anywhere` and for `fileSystems` in the running config.
- `@nix` and `@persist` as separate subvolumes before you turn on impermanence.

Try this

1. On a **disposable** QEMU VM, apply Case 1. `findmnt / /boot` after Disko mount. Confirm `/dev/vda1` is `vfat` and `/dev/vda2` is `ext4` (names may vary).
2. `ls -l /dev/disk/by-id` on any Linux box and pick the id you would use. Do not run Disko there.
3. Eval `config.fileSystems` from a flake that imports Disko and list mountpoints.
4. Read `nixos-anywhere --help` and note `--generate-hardware-config`. That flag is how `initrd` modules still get generated when Disko owns mounts.

Ephemeral Roots and Impermanence

Ephemeral Roots and Impermanence

`/usr/local/bin` leftovers, an undocumented `/etc` drop-in, a rootkit in `/bin` — all of that is “state you did not declare.” The boring default is: **throw / away at boot (tmpfs or a blank Btrfs snapshot), keep /nix and /persist, and list every surviving path in the impermanence module.**

This is optional on a laptop you are still learning on. It is the right default on a desk **server** once Disko already created `@persist` and `@nix`. Pair it with the backups chapter: git rebuilds the OS; restic rebuilds `/persist`.

Mental model

Path	Survives reboot?
<code>/</code> (tmpfs or wiped subvol)	No
<code>/nix</code>	Yes (own subvolume / partition)
<code>/persist</code>	Yes
<code>/boot</code>	Yes (ESP)
Anything not listed in <code>environment.persistence</code>	No

The `impermanence` module bind-mounts or links `/persist/...` onto the live paths (`/etc/machine-id`, `/var/log`, ...) so programs still write where they always did.

```
boot
  tmpfs → /
  btrfs @nix → /nix
  btrfs @persist → /persist
  bind /persist/etc/machine-id → /etc/machine-id
```

If you forget a path, the service fails after reboot (empty DB, new SSH host key). That failure is the audit. Add the path, or decide the data should die.

Worked examples

Case 1: tmpfs root + persist list

Save as `impermanence.nix`:

```

# impermanence.nix
{ inputs, ... }:

{
  imports = [ inputs.impermanence.nixosModules.impermanence ];

  fileSystems."/ " = {
    device = "none";
    fsType = "tmpfs";
    options = [ "defaults" "size=4G" "mode=755" ];
  };

  fileSystems."/persist" = {
    neededForBoot = true;
  };

  environment.persistence."/persist" = {
    hideMounts = true;
    directories = [
      "/var/log"
      "/var/lib/nixos"
      "/var/lib/systemd/coredump"
      "/var/lib/systemd/timers"
    ];
    files = [
      "/etc/machine-id"
    ];
  };
}

```

`fileSystems."/persist"` `device/fsType` usually come from `Disko`. Do not fight `Disko` with a second definition of the same mount unless you are not using `Disko`.

`neededForBoot = true` is required so activation can see `/persist` before it tries to link `machine-id`.

Case 2: SSH host keys — the one everyone forgets

```

# persist-ssh.nix
{
  environment.persistence."/persist".files = [
    "/etc/ssh/ssh_host_ed25519_key"
    "/etc/ssh/ssh_host_ed25519_key.pub"
  ];
}

```

Without this, every reboot is a new host key. Clients print `REMOTE HOST IDENTIFICATION HAS CHANGED`. That looks like MITM. It is you.

After first boot with persistence, `ls -l /persist/etc/ssh/` should show the keys. If the files exist only under `/etc/ssh` on a still-persistent root, **copy them to /persist** before you enable tmpfs root, or the next boot generates new keys.

Case 3: Desk service state

```
# persist-desk.nix
{
  environment.persistence."/persist".directories = [
    "/var/lib/desk-api"
    "/var/lib/postgresql"
    "/var/lib/tailscale"
    "/var/lib/nixos"
  ];
}
```

If the unit uses `DynamicUser`, state may live under `/var/lib/private/...`. Persist **that** path or give the service a named user and a stable `StateDirectory`. Guessing wrong looks like “the database reset.”

Users’ homes:

```
{
  environment.persistence."/persist".users.deskadmin = {
    directories = [
      ".ssh"
      ".local/share"
    ];
    files = [
      ".bash_history"
    ];
  };
}
```

Do not persist all of `/home` “to be safe” if the point was to stop accumulating junk. Persist keys and the few tools that store tokens.

Case 4: Flake input

```
# flake.nix fragment
{
  inputs.impermanence.url = "github:nix-community/impermanence";
}
```



```

outputs = { self, nixpkgs, impermanence, ... }: {
  nixosConfigurations.desk-server = nixpkgs.lib.nixosSystem {
    system = "x86_64-linux";
    modules = [
      impermanence.nixosModules.impermanence
      ./impermanence.nix
      ./configuration.nix
    ];
  };
};
}

```

```
nix eval .#nixosConfigurations.desk-server.config.environment.persistence.\"/persist\".files
```

Output (shape):

```
[ "/etc/machine-id" "/etc/ssh/ssh_host_ed25519_key" ... ]
```

If eval does not list the SSH keys, reboot will not either.

Case 5: Prove it on a VM

On a lab VM **after** persistence is on:

```

sudo touch /etc/untracked-state.txt
sudo touch /persist/i-should-survive.txt
sudo reboot

```

After boot:

```

ls /etc/untracked-state.txt
ls /persist/i-should-survive.txt
ls /etc/machine-id

```

Output:

```

ls: cannot access '/etc/untracked-state.txt': No such file or directory
/persist/i-should-survive.txt
/etc/machine-id

```

`machine-id` is back because it was listed. The untracked file is gone. If `machine-id` is also gone, `neededForBoot` or the files list is wrong — fix before you enable `sshd` clients.

Btrfs variant (no tmpfs): rollback @root to a blank snapshot in `boot.initrd.postDeviceCommands` (or the current impermanence btrfs helper). Same persist list. Prefer one mechanism; do not tmpfs **and** rollback.

The trap

The trap is **enabling tmpfs root without copying SSH host keys onto /persist first**. You lock yourself out over SSH. Console or a second VM disk is the recovery.

The other trap is forgetting `/var/lib/nixos` (uids for `DynamicUser` / allocated ids). After reboot, users get new uids and cannot read old state you *did* persist.

A third trap: persisting `/etc` wholesale. You are back to a mutable `/etc` with extra steps. List files.

The boring rule

- `/nix` and `/persist` (and `/boot`) are durable. `/` is not.
- List SSH host keys, `machine-id`, `/var/lib/nixos`, and each service's state dir **before** the first ephemeral boot.
- `neededForBoot = true` on `/persist`.
- Copy existing keys to `/persist` before switching root to tmpfs.
- Back up `/persist`. The flake will not grow the Postgres data back.

Try this

1. Eval the `persist files` and `directories` lists from the flake. Read them aloud. Name one path that would lock you out if missing.
2. On a **lab** VM with a snapshot you can restore, enable Case 1 + Case 2, copy host keys to `/persist`, switch, reboot, `ssh` in. Confirm the host key warning does **not** appear.
3. `sudo touch /root/junk && sudo reboot` and confirm `/root/junk` is gone.
4. Add `/var/lib/desk-api` to `directories`, deploy a dummy file there, reboot, confirm it survived. Remove it from the list on a second experiment and confirm it dies.

Image Generation with nixos-generators

Image Generation with nixos-generators

Every team eventually needs a bootable artifact: a rescue ISO burned to a USB stick, a QEMU image for local integration testing, an SD card for a rack of Raspberry Pis, or an AMI pushed to AWS. The boring default for all of these is **nixos-generators** — a flake library that accepts a standard NixOS module configuration and a format string, then produces a ready-to-use image from the Nix store. No hand-crafted **genisoimage** invocations, no bespoke **mkimage.sh** scripts, no per-format wiki pages.

Mental model

nixos-generators (hosted at [github:nix-community/nixos-generators](https://github.com/nix-community/nixos-generators)) exposes one primary function: **nixosGenerate**. It is a thin wrapper around NixOS's own **nixos-rebuild** evaluation machinery.

```
nixosGenerate {
  system = "x86_64-linux";    # or "aarch64-linux"
  format = "iso";             # selects the image format module
  modules = [ ./my-config.nix ];
}
```

Under the hood, **nixosGenerate** imports `<nixpkgs/nixos>` with the caller-supplied modules **plus** a format-specific module bundled inside **nixos-generators**. That format module enables or disables NixOS options (bootloader type, filesystem layout, network seed, cloud-init, etc.) appropriate for the target image. The result is a standard Nix derivation whose output path contains the finished image file.

Supported formats (selection):

Format string	Output file	Use case
iso	.iso	Bootable USB / CD
qcow	.qcow2	QEMU / libvirt local VMs
amazon	.vhd	AWS AMI import
sd-aarch64	.img	Raspberry Pi / ARM SBCs
vagrant-virtualbox	.box	Vagrant + VirtualBox
do	.img.gz	DigitalOcean custom images
azure	.vhd	Azure managed disk import

Key properties:

1. **Reproducible.** The same flake lock produces the same image byte-for-byte.
2. **Cacheable.** Images are regular Nix derivations; binary caches work normally.
3. **Composable.** Any NixOS module — services, users, firewall rules — works without modification.
4. **No extra tools on the build host.** All format-specific utilities (`genisoimage`, `qemu-img`, `e2fsprogs`, etc.) are pulled into the build sandbox automatically.

Worked examples

Case 1: Bootable rescue ISO for the infrastructure desk

The team keeps a rescue ISO on a shelf USB drive. When a desk workstation fails to boot, the technician plugs in the USB and gets an SSH-accessible minimal shell with the usual diagnostic tools already present.

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk rescue ISO";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

    nixos-generators = {
      url = "github:nix-community/nixos-generators";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  outputs = { self, nixpkgs, nixos-generators, ... }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in
    {
      packages.${system}.rescue-iso = nixos-generators.lib.nixosGenerate {
        inherit system;
        format = "iso";
        modules = [
          ({ pkgs, ... }: {
            # ISO label shown in boot menu
            isoImage.isoName = "desk-rescue-26.05.iso";

            # Make the ISO available without authentication for emergency access.
            # Restrict by placing this on a physically separate VLAN in production.
          })
        ];
      };
    }
  }
```

```

services.openssh = {
  enable = true;
  settings.PermitRootLogin = "yes";
};

users.users.root.password = "rescue";

# Diagnostic tools every technician expects
environment.systemPackages = with pkgs; [
  vim
  curl
  htop
  tcpdump
  smartmontools
  parted
  pciutils
  usbutils
];

# Stateless - no mutable filesystem needed
system.stateVersion = "26.05";
})
];
};
}

```

Build:

```
nix build .#rescue-iso
```

```

warning: Git tree '/home/ops/desk-infra' is dirty
building '/nix/store/3j7k...-nixos-rescue-iso.drv'...
/nix/store/vq84...-iso/iso/desk-rescue-26.05.iso

```

Inspect the result symlink:

```
readlink -f result
```

```
/nix/store/vq84p3rz8f2a1n06xbkj5wsywi3hqd29-iso/iso/desk-rescue-26.05.iso
```

Flash to USB:

```
sudo dd if=$(readlink -f result) of=/dev/sdX bs=4M status=progress conv=fsync
```

```
1134264320 bytes (1.1 GB, 1.1 GiB) copied, 47.3 s, 24.0 MB/s
```

Case 2: QEMU qcow2 image for local VM testing

Before deploying a new service configuration to production hosts, engineers spin up a local QEMU VM to validate it. The `qcow` format produces a copy-on-write disk image that QEMU understands natively. No conversion step is needed.

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk service VM image";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

    nixos-generators = {
      url = "github:nix-community/nixos-generators";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  outputs = { self, nixpkgs, nixos-generators, ... }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in
    {
      packages.${system}.desk-vm = nixos-generators.lib.nixosGenerate {
        inherit system;
        format = "qcow";
        modules = [
          ({ pkgs, ... }: {
            # A minimal internal metrics collector for VM validation
            services.prometheus = {
              enable = true;
              exporters.node = {
                enable = true;
                openFirewall = true;
              };
            };
          });

          networking.firewall.allowedTCPPorts = [ 9090 9100 ];

          # QEMU guest agent for clean shutdown via virsh/libvirt
          services.qemuGuest.enable = true;

          users.users.ops = {
```

```

        isNormalUser = true;
        extraGroups   = [ "wheel" ];
        password      = "ops";
    };

    services.openssh = {
        enable = true;
        settings.PasswordAuthentication = true;
    };

    system.stateVersion = "26.05";
})
];
};
};
}

```

Build:

```
nix build .#desk-vm
```

```
building '/nix/store/a9xm...-desk-vm-qcow2.drv'...
/nix/store/flyw...-nixos.qcow2
```

Launch the VM with QEMU directly:

```

qemu-system-x86_64 \
  -m 2048 \
  -smp 2 \
  -enable-kvm \
  -drive file=$(readlink -f result),format=qcow2,if=virtio \
  -net nic,model=virtio \
  -net user,hostfwd=tcp::2222-:22,hostfwd=tcp::9090-:9090 \
  -nographic

```

```

[ 0.000000] Linux version 6.6.56 (nixbld@localhost) ...
[ 2.148391] EXT4-fs (vda): mounted filesystem with ordered data mode.
[ OK ] Started OpenSSH Daemon.
[ OK ] Started Prometheus Node Exporter.

```

```
<<< NixOS 26.05 (desk-vm) >>>
```

desk-vm login:

SSH in from a second terminal:


```
ssh -p 2222 ops@127.0.0.1
```

```
[ops@desk-vm:~]$
```

Case 3: Raspberry Pi SD card image (aarch64 cross-compile from x86_64)

The team operates a small fleet of Raspberry Pi 4 boards as out-of-band management nodes on the server rack. Images must be built on a standard x86_64 workstation and flashed to SD cards.

Save as `flake.nix`:

```
# flake.nix
{
  description = "Rack OOB node SD image";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

    nixos-generators = {
      url = "github:nix-community/nixos-generators";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  outputs = { self, nixpkgs, nixos-generators, ... }:
    let
      buildSystem = "x86_64-linux";
      # Cross-compilation: build on x86_64, target aarch64
      pkgsCross = nixpkgs.legacyPackages.${buildSystem}.pkgsCross.aarch64-multiplatform;
    in
    {
      packages.${buildSystem}.oob-sd = nixos-generators.lib.nixosGenerate {
        # The *host* (target board) architecture
        system = "aarch64-linux";
        # Supply pkgs built via the cross toolchain so no aarch64 runners are needed
        pkgs = pkgsCross;
        format = "sd-aarch64";
        modules = [
          ({ pkgs, lib, ... }: {
            # Raspberry Pi 4B hardware support
            hardware.raspberry-pi."4".apply-overlays-dtmerge.enable = true;
            hardware.deviceTree.enable = true;

            networking.hostName = "oob-node";

            services.openssh = {
```

```

        enable = true;
        settings = {
            PermitRootLogin      = "no";
            PasswordAuthentication = false;
        };
    };

    users.users.ops = {
        isNormalUser      = true;
        extraGroups       = [ "wheel" ];
        openssh.authorizedKeys.keys = [
            "ssh-ed25519 AAAA... ops@desk"
        ];
    };

    security.sudo.wheelNeedsPassword = false;

    environment.systemPackages = with pkgs; [
        vim
        curl
        iproute2
        iputils
    ];

    system.stateVersion = "26.05";
})
];
};
};
}

```

Build (cross-compilation; no QEMU emulation required):

```
nix build .#oob-sd
```

```
building '/nix/store/c2hm...-sd-image-aarch64-linux.drv'...
/nix/store/p9bw...-nixos-sd-image-26.05.aarch64-linux.img
```

Find the image path and flash it:

```
SD_IMG=$(readlink -f result)

sudo dd if="$SD_IMG" of=/dev/mmcblk0 bs=4M status=progress conv=fsync
```

```
3936280576 bytes (3.9 GB, 3.7 GiB) copied, 124 s, 31.7 MB/s
```

Eject the card, insert it into the Pi, and power on. The node appears on the rack management VLAN within 30 seconds.

Case 4: Sharing a NixOS module between a live machine and an image build

The biggest maintenance risk when maintaining images is drift: the image diverges from the live system because they evolved separately. The fix is to extract shared configuration into a standalone module file and import it in both the `nixosConfigurations` output and the `nixosGenerate` call.

Save as `modules/desk-base.nix`:

```
# modules/desk-base.nix
{ pkgs, lib, ... }:
{
  # Shared by both live desk machines and generated images

  networking.domain = "ops.example.internal";

  services.openssh = {
    enable = true;
    settings = {
      PermitRootLogin      = "no";
      PasswordAuthentication = false;
      X11Forwarding         = false;
    };
  };

  environment.systemPackages = with pkgs; [
    vim
    git
    curl
    htop
    jq
  ];

  security.sudo.wheelNeedsPassword = true;

  time.timeZone = "UTC";

  nix = {
    settings.experimental-features = [ "nix-command" "flakes" ];
    gc = {
      automatic = true;
      dates      = "weekly";
      options    = "--delete-older-than 30d";
    };
  };
};
```

```

    system.stateVersion = "26.05";
}

```

Save as `flake.nix`:

```

# flake.nix
{
  description = "Desk infrastructure - shared module demo";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

    nixos-generators = {
      url = "github:nix-community/nixos-generators";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  outputs = { self, nixpkgs, nixos-generators, ... }:
    let
      system = "x86_64-linux";
      pkgs    = nixpkgs.legacyPackages.${system};

      # The shared module - imported in both outputs below
      deskBase = ./modules/desk-base.nix;
    in
    {
      # Live system deployed to real hardware
      nixosConfigurations.desk-01 = nixpkgs.lib.nixosSystem {
        inherit system;
        modules = [
          deskBase
          ./hosts/desk-01/hardware-configuration.nix
          ({ ... }: {
            networking.hostName = "desk-01";
            users.users.alice = {
              isNormalUser = true;
              extraGroups  = [ "wheel" ];
            };
          })
        ];
      };

      # ISO image for emergency rescue of desk machines
      packages.${system}.desk-rescue-iso = nixos-generators.lib.nixosGenerate {
        inherit system;

```

```

format = "iso";
modules = [
  # Same shared module → same tools, same SSH policy, same Nix config
  deskBase
  ({ lib, ... }: {
    networking.hostName = "desk-rescue";

    # Rescue-only override: allow root login for emergency access
    services.openssh.settings.PermitRootLogin = lib.mkForce "yes";
    users.users.root.password = "rescue";

    isoImage.isoName = "desk-rescue-26.05.iso";
  })
];
};

# QEMU image for pre-deploy validation
packages.${system}.desk-vm = nixos-generators.lib.nixosGenerate {
  inherit system;
  format = "qcow";
  modules = [
    deskBase
    ({ lib, ... }: {
      networking.hostName = "desk-vm";
      services.qemuGuest.enable = true;
      users.users.root.password = "vm";
      services.openssh.settings.PermitRootLogin = lib.mkForce "yes";
    })
  ];
};
};
}

```

Build both images without touching `modules/desk-base.nix`:

```
nix build .#desk-rescue-iso
```

`/nix/store/8k3n...-iso/iso/desk-rescue-26.05.iso`

```
nix build .#desk-vm
```

`/nix/store/r7cj...-nixos.qcow2`

Any change to `modules/desk-base.nix` — adding a package, tightening an SSH setting — is reflected in **both** the live system and every image output on the next build. There is no separate image-maintenance checklist.

The trap

Building sd-aarch64 on x86_64 without cross-compilation

The sd-aarch64 format targets aarch64-linux. If you forget to supply a cross-compiled pkgs and simply pass the host pkgs, Nix will attempt to run aarch64 ELF binaries inside the build sandbox on an x86_64 kernel:

```
# flake.nix (wrong - do not use)
packages.x86_64-linux.bad-sd = nixos-generators.lib.nixosGenerate {
  system = "aarch64-linux";
  # No pkgs override - falls back to x86_64 pkgs, then tries to exec aarch64
  format = "sd-aarch64";
  modules = [ ./modules/desk-base.nix ];
};
```

Build error:

```
building '/nix/store/...-bash-5.2-aarch64-unknown-linux-gnu.drv'...
error: builder for '/nix/store/...-bash-5.2-aarch64-unknown-linux-gnu.drv' failed:
  error: executing '/nix/store/...-bash-5.2/bin/bash':
  Exec format error
```

The kernel refuses to execute an aarch64 ELF. There are two correct fixes.

Fix A — Cross-compilation (no emulation needed, recommended):

```
# flake.nix (relevant excerpt)
let
  buildSystem = "x86_64-linux";
  pkgsCross   = nixpkgs.legacyPackages.${buildSystem}.pkgsCross.aarch64-multiplatform;
in
{
  packages.${buildSystem}.correct-sd = nixos-generators.lib.nixosGenerate {
    system = "aarch64-linux";
    pkgs    = pkgsCross;          # cross-compiled pkgs supplied explicitly
    format = "sd-aarch64";
    modules = [ ./modules/desk-base.nix ];
  };
}
```

Fix B — binfmt emulation (slower, requires a NixOS build host):

Add this to your build host's configuration.nix, then nixos-rebuild switch:

```
# configuration.nix (build host only)
{ ... }:
{
```

```
boot.binfmt.emulatedSystems = [ "aarch64-linux" ];
}
```

```
nixos-rebuild switch
```

```
activating the configuration...
setting up binfmt handlers...
registering aarch64-linux as binfmt_misc handler
```

With binfmt active, aarch64 ELF binaries are transparently executed under QEMU user-mode emulation. Builds succeed but take significantly longer than cross-compilation. Use Fix A for routine CI pipelines; Fix B only when cross-compilation is not available.

The boring rule

- **Use nixos-generators instead of custom image scripts.** All format-specific tooling is inside the Nix sandbox; your repository stays clean.
- **Share modules between live systems and image builds.** Put common configuration in `modules/` files and import them in both `nixosConfigurations` and `nixosGenerate` calls. Images reflect exactly what runs in production.
- **Always pin nixos-generators in flake.lock.** Run `nix flake update nixos-generators` deliberately, not on every build. Image reproducibility depends on a stable lock.
- **Use pkgsCross.aarch64-multiplatform for cross-architecture builds.** Do not rely on binfmt emulation in CI; it is slower and silently fails on kernels without the `binfmt_misc` module loaded.
- **Name image outputs descriptively** (`rescue-iso`, `desk-vm`, `oob-sd`) so `nix build .#<tab>` is self-documenting.

Try this

1. **Extend the rescue ISO.** Add `nmap` and `wireguard-tools` to the `rescue-iso` packages list and rebuild. Verify both binaries appear inside the ISO by mounting it: `sudo mount -o loop result /mnt` followed by `ls /mnt/nix/store | grep -E "nmap|wireguard"`.
2. **Boot the QEMU image with a persistent overlay.** Create a 4 GB writable overlay with `qemu-img create -f qcow2 -b $(readlink -f result) -F qcow2 overlay.qcow2 4G`, then boot it with the overlay path. Confirm that writes survive a reboot while the base image stays unchanged.
3. **Add a DigitalOcean image format without duplicating config.** In the Case 4 flake, add a `packages.x86_64-linux.desk-do` output using `format = "do"` and the same `deskBase` module. Build it with `nix build .#desk-do` and inspect the resulting `.img.gz`.

4. **Measure image closure size.** After building any image, run `nix path-info -rS $(readlink -f result) | sort -k2 -rn | head -20` to identify the largest packages in the closure. Remove one unnecessary package from the module and compare closure sizes before and after.

Backups and Generation Hygiene

Backups and Generation Hygiene

A flake rebuilds the **operating system**. It does not recreate Postgres, `/persist`, or the Age key you generated once. The boring default is: **restic (or borg) of state, a restore drill, and GC that keeps two weeks of generations — not `rm -rf /nix`**.

Mental model

Rebuild from git + cache	Must be backed up
<code>/nix/store</code>	<code>/persist</code> (impermanence)
Kernel, units, packages	Database files / dumps
Most of <code>/etc</code>	Secrets material you cannot reissue
Home Manager managed files	Unmanaged <code>\$HOME</code> (photos, SSH private keys if not in sops)

3-2-1 still applies: live + backup + offsite. A green `restic snapshots` line without a restore is theatre.

Generation hygiene is the other disk: old NixOS generations are rollback insurance **and** gigabytes. Keep a window. Automatic GC with `--delete-older-than 14d` is the boring timer. `-d` is for incidents.

```
wipe disk
  nixos-anywhere / installer → OS from flake
  restic restore             → /persist + DB
```

Worked examples

Case 1: What to include

Save as `backup-paths.md` in the desk repo (documentation, not Nix):

```
# backup-paths.md
Include:
  /persist
  /var/lib/postgresql
  /var/lib/desk-api
```

```
Exclude:
/nix/store
/tmp
/var/cache
result
.direnv
```

If you use impermanence, **/persist** is the list. Do not also back up the ephemeral root.

Case 2: Restic on NixOS

Save as `restic.nix`:

```
# restic.nix
{ config, pkgs, ... }:

{
  services.restrict.backups.desk = {
    initialize = true;
    passwordFile = config.sops.secrets.restrict-password.path;
    repository = "sftp:backup@backup.desk.internal:/backups/desk-vm";
    paths = [
      "/persist"
      "/var/lib/desk-api"
    ];
    pruneOpts = [
      "--keep-daily 7"
      "--keep-weekly 4"
      "--keep-monthly 6"
    ];
    timerConfig = {
      OnCalendar = "daily";
      Persistent = true;
    };
  };
}
```

The repository password is a sops secret, not a string in this file. `sftp:` is one backend; `s3:s3.amazonaws.com/desk-backups` is the same module.

```
sudo nixos-rebuild switch
sudo systemctl start restrict-backups-desk.service
sudo journalctl -u restrict-backups-desk.service -n 40
```

Case 3: Restore drill (the part people skip)

```
sudo restic -r sftp:backup@backup.desk.internal:/backups/desk-vm \  
--password-file /run/secrets/restic-password \  
snapshots  
  
sudo restic -r sftp:backup@backup.desk.internal:/backups/desk-vm \  
--password-file /run/secrets/restic-password \  
restore latest --target /tmp/desk-restore-drill  
  
ls /tmp/desk-restore-drill/persist | head
```

On a lab VM, go further: restore `/var/lib/desk-api` onto a **second** VM and start the service. A file listing is not a drill. A responding API is.

Schedule this quarterly. Write the date in the desk repo.

Case 4: Generation window on NixOS

Save as `gc.nix`:

```
# gc.nix  
{  
  nix.gc = {  
    automatic = true;  
    dates = "weekly";  
    options = "--delete-older-than 14d";  
  };  
  nix.optimise.automatic = true;  
  boot.loader.systemd-boot.configurationLimit = 10;  
}
```

`configurationLimit` caps bootloader entries so `/boot` (often a small ESP) does not fill. GC and the bootloader limit work together: deleting generations without limiting entries leaves stale boot menu rows; limiting entries without GC leaves the store fat.

```
sudo nixos-rebuild switch  
sudo nix-env --list-generations --profile /nix/var/nix/profiles/system
```

Case 5: `/boot` full is a different incident

```
df -h /boot  
sudo nixos-rebuild switch
```

If /boot is 100%, switch fails while /nix still has space. `configurationLimit` plus removing old entries:

```
sudo nix-collect-garbage --delete-older-than 7d
sudo nixos-rebuild switch
```

Do not `rm /boot/EFI/nixos/*` by hand unless you know which generation you are running.

The trap

The trap is **backing up /nix/store**. It is huge, it is reconstructable, and restic will spend all night hashing NARs. Back up state. Substitute the store from the flake + cache.

The other trap is `nix-collect-garbage -d` on Friday before a Monday upgrade. Keep 14 days. Use `-d` when `df` is the outage.

A third trap: restic password in repository URL query string inside `configuration.nix`. That string is in the world-readable store.

The boring rule

- Flake + cache = OS. Restic (or borg) = state. You need both.
- Restore drill with a second VM, not only `restic snapshots`.
- Weekly GC with `--delete-older-than 14d`. `configurationLimit` on the ESP.
- Never back up /nix/store as the backup strategy.
- Backup passwords via sops, not Nix strings.

Try this

1. Inventory `/persist` (or `/var/lib`) on a lab VM and write the include/exclude list.
2. Enable Case 4, `systemctl status nix-gc.timer`, and list generations.
3. Fill a dummy `/persist/desk-note.txt`, take a restic snapshot to a local `repository = "/var/backup/restic";` (lab), delete the file, restore, confirm the text.
4. Set `configurationLimit = 2` on a VM, switch twice, reboot, and count `systemd-boot` entries. Put it back to 10.

Home Manager

Home Manager and the Module System

Home Manager and the Module System

NixOS configures the machine (kernel, systemd, users, /etc). Home Manager configures **one user** (dotfiles, user packages, user services). The boring default is: **home.packages** for binaries, **programs.<name>** for software that has a module, and **home.stateVersion** left at the birth release.

Mental model

Home Manager is the NixOS module system aimed at \$HOME:

Field	Effect
home.username / home.homeDirectory	Who this config is for
home.stateVersion	Birth release. Do not bump casually.
home.packages	On the user PATH (~/.nix-profile/bin)
programs.<name>.enable	Typed module: writes dotfiles and maybe a user systemd unit
home.file / xdg.configFile	Raw files into \$HOME / ~/.config

Two ways to apply it:

Mode	Command	When
Standalone	home-manager switch --flake .#deskadmin	Nix-on-Linux, macOS, or a user who must not touch NixOS
NixOS module	nixos-rebuild switch	The desk workstation is NixOS (next chapters in this part)

Standalone is how you start on a foreign distro. The NixOS module is how you stay once the machine is yours.

```
home.nix
  home.packages      → ~/.nix-profile
  programs.git       → ~/.config/git/config
  xdg.configFile     → ~/.config/...
```

```
home-manager switch
```


Worked examples

Case 1: A standalone home.nix

Save as home.nix:

```
# home.nix
{ pkgs, ... }:

{
  home.username = "deskadmin";
  home.homeDirectory = "/home/deskadmin";
  home.stateVersion = "26.05";

  home.packages = with pkgs; [
    bat
    fzf
    jq
    ripgrep
  ];

  programs.bash.enable = true;
  programs.home-manager.enable = true;
}
```

On a machine with the home-manager CLI:

```
home-manager switch -f home.nix
```

Output:

```
Starting Home Manager activation
Activating checkFilesChanged
Activating installPackages
...
Home Manager switch complete!
```

```
which jq
jq --version
```

jq resolves under ~/.nix-profile/bin or ~/.local/state/nix/profiles/home-manager/....

Case 2: A flake with homeConfigurations

Save as flake.nix:

```
# flake.nix
{
  description = "Desk user environment";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    home-manager = {
      url = "github:nix-community/home-manager/release-26.05";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  outputs = { self, nixpkgs, home-manager }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in
    {
      homeConfigurations.deskadmin = home-manager.lib.homeManagerConfiguration {
        inherit pkgs;
        modules = [ ./home.nix ];
      };
    };
}
```

`inputs.nixpkgs.follows` prevents Home Manager from pulling a second `nixpkgs`.

```
home-manager switch --flake .#deskadmin
```

Pin both inputs in `flake.lock`. The same lock on a second laptop is the same `jq`.

Case 3: User packages are not system services

`home.packages = [ pkgs.nginx ]`; puts `nginx` on **your** `PATH`. It does not open port 80 and it does not start a daemon. Daemons belong in NixOS `services.*` (or a user `systemd` module you wrote on purpose).

```
# wrong mental model
home.packages = [ pkgs.openssh ]; # a binary, not sshd

# NixOS, not Home Manager
services.openssh.enable = true;
```

Case 4: Generation rollback for \$HOME

```
home-manager generations
home-manager switch --rollback
```

Output:

```
2026-09-09 10:11 : id 42 -> /nix/store/...-home-manager-generation
2026-09-08 18:02 : id 41 -> /nix/store/...-home-manager-generation
Switching to generation 41
```

Broken programs.neovim? Roll back. You do not rm ~/.config/nvim.

Case 5: stateVersion is a birth certificate

```
home.stateVersion = "26.05";
```

This is **not** “track 26.05.” It tells Home Manager which defaults your state dirs were created with. When you upgrade nixpkgs to a newer release, you keep `stateVersion` until a release note says a specific option needs a bump.

The trap

The trap is **managing the same program in both NixOS and Home Manager** (`environment.systemPackages = [ git ]` and `programs.git.enable = true` with conflicting `userName`). Two `git` binaries, two configs, a week of “why is my email wrong.”

Pick a layer: `git` config for humans → Home Manager. `git` as a system tool for root scripts → NixOS. Not both with different emails.

The other trap is running standalone Home Manager against a different nixpkgs than NixOS on the same box. `follows` in the flake, or the NixOS module with `useGlobalPkgs` (later chapter).

The boring rule

- Home Manager owns `$HOME`. NixOS owns the machine.
- `home.packages` for extra CLIs. `programs.<name>` when a module exists.
- Flake + `follows` = “nixpkgs” so there is one nixpkgs.
- `home.stateVersion` stays at the birth release.
- Roll back with `home-manager switch --rollback`, not by deleting dotfiles.

Try this

1. Add `pkgs.hello` to `home.packages`, switch, run `hello`, remove it, switch again, and confirm `which hello` fails.
2. `readlink -f $(which jq)` and confirm the path is under `/nix/store`.
3. Run `home-manager generations` and identify the current id.
4. In the flake, omit `inputs.nixpkgs.follows` on purpose, `nix flake metadata`, and notice a second `nixpkgs`. Put `follows` back.

Managing Dotfiles Declaratively

Managing Dotfiles Declaratively

Git-bare + `stow` leaves stale symlinks and a README of “also copy this file.” The boring default is: `xdg.configFile` and `home.file` so every live config is a symlink into `/nix/store`, generated from this Home Manager generation.

Mental model

Option	Live path
<code>xdg.configFile."desk/settings.json"</code>	<code>~/.config/desk/settings.json</code>
<code>home.file".desk.rc"</code>	<code>~/.desk.rc</code>
<code>xdg.dataFile."desk/banner.txt"</code>	<code>~/.local/share/desk/banner.txt</code>

Each value can be `.text` (inline string), `.source` (a file in git), or `.template` (less common). Home Manager creates the parent directories and a generation-specific symlink.

The file in `$HOME` is **read-only** when it points at the store. That is the feature: if you need to change it, you change git and switch. An editor that “helpfully” writes in place will get **Permission denied** or silently copy-break the symlink depending on the editor.

```
git: desk/settings.json
    home.file / xdg.configFile

/nix/store/...-home-manager-files/.config/desk/settings.json

~/.config/desk/settings.json  (symlink)
```

Worked examples

Case 1: Inline JSON from Nix

Save as `dotfiles.nix`:

```
# dotfiles.nix
{
  xdg.configFile."desk/settings.json".text = builtins.toJSON {
    theme = "dark";
    autoSync = true;
```

```

        window = "A";
    };
}

```

Import this from `home.nix`. After `home-manager switch`:

```

readlink -f ~/.config/desk/settings.json
cat ~/.config/desk/settings.json

```

Output:

```

/nix/store/...-settings.json
{"autoSync":true,"theme":"dark","window":"A"}

```

`builtins.toJSON` keeps commas honest. Do not hand-write JSON in a Nix multiline string if you can avoid it.

Case 2: Source a file that lives in git

Save as `gitconfig-extra.txt` next to the module:

```

# gitconfig-extra.txt
[alias]
    st = status
    co = checkout

```

Save as `dotfiles-source.nix`:

```

# dotfiles-source.nix
{
    xdg.configFile."git/ignore".text = ''
        result
        .direnv/
        *~
    '';

    home.file.".desk/aliases".source = ./gitconfig-extra.txt;
}

home-manager switch
cat ~/.desk/aliases

```

The source file is copied into the store at build time. Editing `~/.desk/aliases` does not change git. Editing `./gitconfig-extra.txt` and switching does.

Case 3: On-change hook

Save as `onchange.nix`:

```
# onchange.nix
{
  xdg.configFile."desk/reload-flag".text = "reload-v1";
  home.activation.deskPing = ''
    echo "desk configs refreshed at $(date -u +%H:%M:%S)" >> "$HOME/.desk/activation.log"
  '';
}
```

Prefer dedicated modules (`programs.git`, `programs.ssh`) over `home.activation` scripts. Activation runs as your user on every switch; keep it idempotent.

Case 4: Mutable file when a program insists on writing

Some tools rewrite their config on every start (a desktop app dumping window geometry). Forcing a store symlink makes the app crash or clone the file.

```
# mutable.nix
{
  xdg.configFile."desk/geometry.json" = {
    text = builtins.toJSON { x = 0; y = 0; };
    force = true;
  };
}
```

`force = true` replaces a **user-created** file with the symlink. It does not make the store writable. If the app must write, **do not** manage that file. Manage a `config.example` and leave the live file alone.

Case 5: Inspect what this generation linked

```
home-manager files | head
ls -l ~/.config/desk
```

Every managed path should be a symlink into `/nix/store`. A regular file at a path you thought you managed means the app (or you) replaced the symlink.

The trap

The trap is `chmod u+w ~/.config/desk/settings.json` and editing in place. The next `home-manager switch` either overwrites you or refuses to clobber. Your change was never in git.

The other trap is stowing **and** Home Manager on the same path. Two tools fight. Pick Home Manager; delete the stow tree.

The boring rule

- Config that should be identical on every desk machine lives in git and is linked via `xdg.configFile / home.file`.
- Prefer `.text + toJSON` / real modules over huge copied blobs.
- Store symlinks are read-only. Edit the source, then switch.
- If an app writes the file, do not manage that path.
- Do not mix stow, chezmoi, and Home Manager on one directory.

Try this

1. Add `xdg.configFile."desk/motd".text = "window A";`, switch, `cat ~/.config/desk/motd`.
2. `ls -l ~/.config/desk/motd` — confirm a symlink. Try `echo x >> ~/.config/desk/motd` and record the error.
3. Change the motd text, switch, confirm the store path hash on `readlink -f` moved.
4. Create a real file at `~/.config/desk/motd` after `rm` of the symlink, switch without **force**, and read Home Manager's conflict message. Then restore management with **force = true once**, and stop using force as a habit.

Shell and Editor Configuration

Shell and Editor Configuration

Copying `.zshrc` snippets between laptops is how aliases drift. The boring default is: **typed** **programs.*** modules for the shell, prompt, git, and editor, with plugins fetched by Nix instead of a plugin manager that writes into `$HOME`.

Mental model

Home Manager modules know the file format. `programs.git` writes `~/.config/git/config`. `programs.starship` writes `~/.config/starship.toml`. You set Nix attributes; you do not maintain a parallel dotfile unless the module is missing.

Module	Replaces
<code>programs.bash</code> / <code>zsh</code> / <code>fish</code>	Hand-written rc files for aliases, history, completions
<code>programs.starship</code>	Prompt theme copied from a gist
<code>programs.git</code>	<code>git config --global</code>
<code>programs.neovim</code> / <code>programs.vim</code>	Plugin managers (<code>vim-plug</code> curling GitHub on first launch)

Enable the shell you actually log into. Enabling `programs.zsh` does not change `/etc/passwd`. The **login shell** is a NixOS `users.users.deskadmin.shell` option (or `chsh` on a foreign distro, once).

Worked examples

Case 1: Git as a module, not a ritual

Save as `shell_editor.nix`:

```
# shell_editor.nix
{ pkgs, ... }:

{
  programs.git = {
    enable = true;
    userName = "Desk Engineer";
    userEmail = "desk@corp.internal";
  };
}
```

```

extraConfig = {
  init.defaultBranch = "main";
  pull.rebase = true;
  rerere.enabled = true;
};
aliases = {
  st = "status";
  co = "checkout";
};
ignores = [ "result" ".direnv/" ];
};
}

```

```

home-manager switch
git config --global --list | head

```

Output:

```

user.name=Desk Engineer
user.email=desk@corp.internal
init.defaultbranch=main
alias.st=status

```

`git config --global user.email` in a terminal will write a **different** file or fight the symlink. Stop doing that.

Case 2: Starship prompt

```

# starship.nix
{
  programs.starship = {
    enable = true;
    enableBashIntegration = true;
    settings = {
      add_newline = false;
      format = "$directory$git_branch$character";
      character.success_symbol = "[>](bold green)";
    };
  };

  programs.bash.enable = true;
}

```

Starship's native format is TOML. Home Manager accepts a Nix attrset and renders TOML. You do not keep a second `starship.toml` in git unless you source it wholesale with `.source`.

Case 3: Bash aliases and history

```
# bash.nix
{
  programs.bash = {
    enable = true;
    historyControl = [ "ignoredups" "erasedups" ];
    historySize = 10000;
    shellAliases = {
      ll = "ls -l";
      gs = "git status";
      ".." = "cd ..";
    };
    bashrcExtra = ''
      # rare exceptions that have no module option
      export DESK_WINDOW=A
    '';
  };
}
```

`shellAliases` is the boring place for aliases. `bashrcExtra` is an escape hatch; if it grows past twenty lines, look for a module.

Case 4: Neovim with plugins from nixpkgs

```
# nvim.nix
{ pkgs, ... }:

{
  programs.neovim = {
    enable = true;
    viAlias = true;
    vimAlias = true;
    extraConfig = ''
      set number
      set relativenumber
    '';
    plugins = with pkgs.vimPlugins; [
      vim-nix
      vim-commentary
    ];
  };
}
```

Plugins are store paths. There is no `~/.local/share/nvim/site/pack` filled by a job on first open. CI can `nvim --headless +q` without a network.

Case 5: Login shell versus module

On NixOS:

```
# in configuration.nix
users.users.deskadmin.shell = pkgs.bashInteractive;
```

Home Manager's `programs.bash.enable = true` writes `~/.bashrc`. It does not change the `passwd` shell. If you enable `programs.zsh` but leave `shell = pkgs.bashInteractive`, you will wonder why `zsh` settings never run.

The trap

The trap is **Oh My Zsh / a plugin manager plus Home Manager**. OMZ clones repos into `$HOME` on the next login and overwrites what Home Manager just linked. Pick one. The boring pick is Home Manager + `nixpkgs` plugins.

The other trap is putting secrets in `extraConfig` (`smtp.password`, GitHub tokens). Those land in the world-readable store. Use `sops-nix` / `agenix` (secrets part) and point the module at the decrypted path.

The boring rule

- If a `programs.<name>` module exists, use it.
- Git identity and aliases live in `programs.git`, not in a one-off `git config`.
- Plugins come from `pkgs.vimPlugins` / `pkgs.zshPlugins`, not from a curl-on-login manager.
- Set the login shell in NixOS (or `chsh` once on a foreign distro).
- No secrets in module `extraConfig`.

Try this

1. Add `programs.git.aliases.lg = "log --oneline --graph -20";`, switch, run `git lg`.
2. `readlink -f ~/.config/git/config` and confirm a store path.
3. Enable `starship`, open a new `bash`, and confirm `echo $STARSHIP_SESSION_KEY` is set (or that the prompt changed). Disable and switch to compare.
4. List `pkgs.vimPlugins` in a `nix repl` (`:l <nixpkgs>` then `pkgs.vimPlugins.vim-nix`) to see that the plugin is an ordinary derivation.

Application Settings and Preferences

Application Settings and Preferences

CLIs are not the only thing in `$HOME`. Terminals, editors, and browsers all grow JSON/TOML/dconf trees. The boring default is: **use a Home Manager module when one exists; generate the native file from Nix; do not manage files the app rewrites on every launch.**

Mental model

Home Manager modules compile Nix into the format the app already understands:

Module	Writes
<code>programs.alacritty</code>	<code>~/.config/alacritty/alacritty.toml</code>
<code>programs.kitty</code>	<code>~/.config/kitty/kitty.conf</code>
<code>programs.vscode</code>	<code>settings.json</code> + extensions from <code>nixpkgs</code>
<code>programs.ssh</code>	<code>~/.ssh/config</code>
<code>programs.tmux</code>	<code>~/.config/tmux/tmux.conf</code>

If there is no module, `xdg.configFile."app/config".text` is still better than a README. If the app treats the config as a database (window geometry, crash dumps, “recent files”), leave it unmanaged.

Worked examples

Case 1: Alacritty

Save as `apps.nix`:

```
# apps.nix
{ pkgs, ... }:

{
  programs.alacritty = {
    enable = true;
    settings = {
      window.opacity = 0.95;
      font.size = 12.0;
      font.normal.family = "JetBrains Mono";
      colors.primary.background = "#1e1e1e";
    };
  };
}
```



```
};
};
}
```

```
home-manager switch
test -f ~/.config/alacritty/alacritty.toml && echo ok
```

Output:

ok

Open Alacritty. Opacity and font come from this generation. A distro-packaged Alacritty that ignores XDG is a different binary — install Alacritty via the same Home Manager module (`programs.alacritty.enable` already installs the package).

Case 2: SSH client config

```
# ssh.nix
{
  programs.ssh = {
    enable = true;
    extraConfig = ''
      Host *
        IdentityFile ~/.ssh/id_ed25519
        IdentitiesOnly yes
    '';
    matchBlocks = {
      "desk-bastion" = {
        hostname = "bastion.desk.internal";
        user = "deskadmin";
        port = 22;
      };
      "desk-*" = {
        proxyJump = "desk-bastion";
        user = "deskadmin";
      };
    };
  };
}
```

Host keys and **private** keys are not in this file. `IdentityFile` **points** at `~/.ssh/id_ed25519`, which you create with `ssh-keygen` or decrypt with `sops`. Putting a private key in `.text` copies it into `/nix/store`.

Case 3: tmux

```
# tmux.nix
{ pkgs, ... }:

{
  programs.tmux = {
    enable = true;
    clock24 = true;
    keyMode = "vi";
    terminal = "tmux-256color";
    extraConfig = ''
      bind r source-file ~/.config/tmux/tmux.conf
    '';
    plugins = with pkgs.tmuxPlugins; [ sensible yank ];
  };
}
```

Plugins are store paths, same story as neovim.

Case 4: VS Code extensions from nixpkgs

```
# vscode.nix
{ pkgs, ... }:

{
  programs.vscode = {
    enable = true;
    extensions = with pkgs.vscode-extensions; [
      jnoortheen.nix-ide
      tamasfe.even-better-toml
    ];
    userSettings = {
      "editor.tabSize" = 2;
      "files.insertFinalNewline" = true;
    };
  };
}
```

Marketplace extensions that are **not** in nixpkgs will not appear here. That is inconvenient and correct: an unsigned VSIX from the internet is not a pinned input. Use `pkgs.vscode-utils.extensionFromVscodeMarketplace` only when you pin sha256.

Case 5: Fonts the terminal named

```
# fonts.nix
{ pkgs, ... }:

{
  fonts.fontconfig.enable = true;
  home.packages = with pkgs; [
    jetbrains-mono
    nerd-fonts.jetbrains-mono
  ];
}
```

`programs.alacritty.settings.font.normal.family = "JetBrains Mono";` does nothing if the font is not installed. Put the font in `home.packages` (or NixOS `fonts.packages`).

The trap

The trap is **managing** `~/.config/Code/User/globalStorage` or a browser profile. Those directories are mutable state. Home Manager will either clobber your session or fail the switch.

The other trap is committing `programs.ssh.extraConfig` with `IdentityFile /nix/store/...-id_ed25519`. That store path is world-readable. Keys live outside the store, or in a sops-decrypted tmpfs.

The boring rule

- Modules first, `xdg.configFile` second, unmanaged mutable state last.
- Terminal + font + editor settings are code. Window geometry is not.
- SSH **config** is code. SSH **keys** are not.
- VS Code extensions come from `nixpkgs` (or a pinned marketplace fetch), not from “Install” clicks you cannot replay.
- If `home-manager switch` fights the app, stop managing that path.

Try this

1. Set Alacritty `font.size` to 14.0, switch, open a new terminal, then set it back.
2. Add an SSH `matchBlocks` entry for a lab host and `ssh -G desk-bastion | grep hostname`.
3. `readlink -f ~/.config/alacritty/alacritty.toml` — store path.
4. Intentionally add a private key as `home.file.".ssh/id_ed25519".text = "-----BEGIN ..."`; in a throwaway VM, `nix-store -q --hash` that path, then delete it and GC. That hash is why we never do this.

Combining NixOS and Home Manager

Combining NixOS and Home Manager

Two rebuild commands (`nixos-rebuild` and `home-manager switch`) means two nixpkgs, two lock files, and a rollback that restores the kernel but not `~/.bashrc`. The boring default on a NixOS workstation is: **Home Manager as a NixOS module, one flake, one `nixos-rebuild switch`**.

Mental model

```
flake.nix
nixosConfigurations.desk-vm
  NixOS modules (boot, users, services)
  home-manager.nixosModules.home-manager
    home-manager.users.deskadmin = { ... };
```

`nixos-rebuild switch --flake .#desk-vm` then:

1. Builds the system closure.
2. Builds the user's Home Manager generation.
3. Activates both.
4. Records **one** system generation that includes the home generation.

`--rollback` restores both.

Option	Why
<code>home-manager.useGlobalPkgs = true;</code>	One nixpkgs: the system's. No second eval.
<code>home-manager.useUserPackages = true;</code>	User packages land in the NixOS user profile, not a disconnected HM profile
<code>home-manager.users.<name></code>	Must match <code>users.users.<name></code>

Standalone Home Manager remains correct on a foreign distro (next chapter). On NixOS, the module wins.

Worked examples

Case 1: Minimal unified module

Save as `unified.nix`:

```
# unified.nix
{ config, pkgs, ... }:

{
  home-manager.useGlobalPkgs = true;
  home-manager.useUserPackages = true;

  home-manager.users.deskadmin = { pkgs, ... }: {
    home.stateVersion = "26.05";
    programs.git = {
      enable = true;
      userName = "Desk Engineer";
      userEmail = "desk@corp.internal";
    };
    home.packages = [ pkgs.jq ];
  };
}
```

The user `deskadmin` must already exist in NixOS (`users.users.deskadmin.isNormalUser = true;`). Home Manager will not create a system user.

Case 2: Flake wiring

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk workstation";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    home-manager = {
      url = "github:nix-community/home-manager/release-26.05";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  outputs = { self, nixpkgs, home-manager }: {
    nixosConfigurations.desk-vm = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        ./configuration.nix
        home-manager.nixosModules.home-manager
        ./unified.nix
      ];
    };
  };
}
```

```
};  
}
```

```
sudo nixos-rebuild switch --flake .#desk-vm
```

Output:

```
building the system configuration...  
activating the configuration...  
Starting Home Manager activation  
...
```

One command. `jq` is on the user's PATH after login.

Case 3: Split files per user

Save as `home/deskadmin.nix`:

```
# home/deskadmin.nix  
{ pkgs, ... }:  
  
{  
  home.stateVersion = "26.05";  
  programs.bash.enable = true;  
  home.packages = with pkgs; [ ripgrep jq ];  
}
```

In `unified.nix`:

```
# unified.nix  
{  
  home-manager.useGlobalPkgs = true;  
  home-manager.useUserPackages = true;  
  home-manager.users.deskadmin = import ./home/deskadmin.nix;  
}
```

Do not invent a second flake output `homeConfigurations.deskadmin` on the same NixOS host unless you have a reason to apply it separately (you probably do not).

Case 4: Backup files on clobber

```
{  
  home-manager.backupFileExtension = "bak";  
}
```

If a real file is sitting where Home Manager wants a symlink, the switch copies it to `*.bak` instead of aborting. Useful once during migration. If you see new `.bak` files every week, something is still writing in place.

Case 5: Rollback includes \$HOME

```
sudo nixos-rebuild switch --rollback
```

The previous system generation's Home Manager generation comes back. `~/.config/git/config` points at the old store path. You do not run `home-manager switch --rollback` in addition.

The trap

The trap is **keeping a standalone home-manager switch --flake .#deskadmin cron on a NixOS box** that also imports the module. Two tools write `~/.bashrc`. The last one to run wins. Disable the standalone CLI in muscle memory once the module is on.

The other trap is `useGlobalPkgs = false` “so I can pin a newer git.” You now evaluate `nixpkgs` twice and can mix library ABIs in one user session. If you need a newer git, overlay it on the **system** `nixpkgs`.

The boring rule

- On NixOS: Home Manager as a NixOS module. One flake. One rebuild.
- `useGlobalPkgs = true; useUserPackages = true;`.
- `home-manager.users.<name>` matches `users.users.<name>`.
- `follows = "nixpkgs"` on the home-manager input.
- Rollback is `nixos-rebuild switch --rollback`. Stop running two CLIs.

Try this

1. Wire Case 2 on a lab VM, add `pkgs.hello` to the user's `home.packages`, switch, run `hello` as `deskadmin`.
2. `ls /nix/var/nix/profiles/system` and find the home-manager generation referenced from the current system.
3. Change git `userEmail`, switch, then `--rollback` and confirm the old email is back.
4. Grep your flake for `homeConfigurations`. If it exists **and** `nixosConfigurations` applies the same user, delete the duplicate output.

CI/CD with Nix

Setting Up Binary Caches

Setting Up Binary Caches

Compiling llvm on every laptop is not a culture. The boring default is: **a signed substituter (cache.nixos.org plus one team cache), public keys in nix.conf, and CI that uploads what it built.**

Mental model

When Nix needs `/nix/store/<hash>-pkg` it:

1. Looks in the local store.
2. Queries each **substituter** for a `.narinfo` signed with a **trusted public key**.
3. Downloads the NAR if the signature matches.
4. Builds from the `.drv` only if nobody had the bytes.

Piece	Role
Substituter URL	<code>https://cache.nixos.org</code> , <code>https://desk.cachix.org</code> , <code>http://attic.desk.internal</code>
Trusted public key	<code>cache.nixos.org-1:6NCHdD59X431oOgWypbMrAURkbJ16ZPMQF</code>
Signing private key	Lives on CI / the cache host. Never in git.

Cachix is the hosted boring choice. Attic or Harmonia is the self-hosted one. The NixOS module is the same shape: extra substituters + extra keys.

```
laptop  --query--> desk-cache  --miss--> cache.nixos.org  --miss--> build
CI      --build--> sign      --put--> desk-cache
```

Worked examples

Case 1: Consume a team cache on NixOS

Save as `cache_config.nix`:

```
# cache_config.nix
{
  nix.settings = {
    substituters = [
```

```

    "https://cache.nixos.org"
    "https://desk-cache.cachix.org"
  ];
  trusted-public-keys = [
    "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
    "desk-cache.cachix.org-1:aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa="
  ];
};
}

```

Replace the `desk-cache` key with the real key Cachix showed when you created the cache. A truncated key in this book is **not** trusted; Nix will ignore that substituter.

```

sudo nixos-rebuild switch
nix show-config | grep substituters

```

Case 2: Push from a workstation (once)

```

nix build .#desk-api
cachix watch-exec desk-cache -- nix build .#desk-api

```

or:

```

nix build .#desk-api
cachix push desk-cache ./result

```

`CACHIX_AUTH_TOKEN` is an environment variable from the Cachix dashboard. It is a secret. It is not a flake input.

Case 3: Verify a hit versus a miss

```

nix build .#desk-api --print-build-logs 2>&1 | tee /tmp/desk-build.log
grep -E 'copying path|building' /tmp/desk-build.log | head

```

copying path '...' from 'https://desk-cache.cachix.org' is a hit. building '...drv' is a miss. A team that still builds `llvm` locally has a key mismatch or is pushing different hashes (different `nixpkgs` lock).

Case 4: Serve a tiny cache with `nix copy`

Without Cachix, an S3 bucket or a directory works:

```
nix copy --to file://$PWD/desk-nar .#desk-api
nix copy --from file://$PWD/desk-nar --to ssh://deskadmin@desk-vm .#desk-api
```

file:// is a lab. Production is HTTP + signatures (`nix-serve`, `Harmonia`, `Attic`). Unsigned file:// on a USB stick is fine for air-gap day; it is not a fleet cache.

Case 5: Generate a signing key for a self-hosted cache

```
nix key generate-secret --key-name desk-cache > /run/secrets/desk-cache.secret
nix key convert-secret-to-public < /run/secrets/desk-cache.secret
```

Put the **public** half in `trusted-public-keys`. Put the secret on the builder only (sops). `nix copy --to 's3://desk-cache?secret-key=/run/secrets/desk-cache.secret'` signs on upload.

The trap

The trap is **adding a substituter without the public key**. Nix will skip it (or refuse, depending on settings). You will compile the world and blame the network.

The other trap is putting the **private** signing key in `flake.nix` or in a GitHub Actions log (`echo $CACHIX_AUTH_TOKEN`). Rotate immediately.

A third trap: `substituters` in `~/.config/nix/nix.conf` on a multi-user machine. The daemon reads `/etc/nix/nix.conf`. Your user file is ignored for builds the daemon runs.

The boring rule

- `cache.nixos.org` plus **one** team cache. Not five random community caches you do not audit.
- Public keys next to substituter URLs, in the daemon's `nix.conf`.
- CI pushes. Laptops pull. Do not require every laptop to upload.
- Never commit signing secrets.
- Confirm a hit with “copying path from ...” before you call the cache done.

Try this

1. `nix show-config | grep -E 'substituters|trusted-public-keys'` and identify which keys match which URLs.
2. `nix build nixpkgs#hello --max-jobs 0` (no local build). It should substitute. If it errors, your substituter list is empty.
3. Create a `file://` cache of `hello` (Case 4) and copy it into a second store (`nix copy --from ...`).
4. Intentionally typo the public key, rebuild a package you do not have locally, and read the error. Restore the key.

GitHub Actions with Nix

GitHub Actions with Nix

actions/setup-go plus actions/setup-python plus a matrix of cache keys is how CI drifts from laptops. The boring default is: **install Nix on the runner, nix flake check / nix build, and push results to the team binary cache.**

Mental model

The runner starts empty. You give it Nix, the same `flake.lock` as the laptop, and a cache token. The graph that `nix develop` used locally is the graph CI builds. There is no second `Dockerfile` for CI.

Step	Job
checkout	The flake and the lock
install Nix	Daemon + flakes
cache auth	Cachix (or magic-nix-cache for GHA's own cache)
<code>nix flake check</code>	Eval + checks + tests
<code>nix build</code>	The artifacts you ship

Use **pinned action SHAs or release tags**, not `@main`, once the workflow is real. Examples below use tags for readability.

Worked examples

Case 1: Check and build, push to Cachix

Save as `.github/workflows/ci.yml`:

```
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
```

```

steps:
  - uses: actions/checkout@v4
  - uses: cachix/install-nix-action@v27
    with:
      nix_path: nixpkgs=channel:nixos-26.05
  - uses: cachix/cachix-action@v15
    with:
      name: desk-cache
      authToken: ${ secrets.CACHIX_AUTH_TOKEN }
  - name: Check and build
    run: |
      nix flake check
      nix build --print-build-logs

```

CACHIX_AUTH_TOKEN is a repository secret. Pull requests from forks should **not** receive a write token (GitHub’s default is correct: pull_request from forks cannot read org secrets).

Case 2: nix flake check is the gate

Save as a flake fragment (merge into outputs):

```

# checks.nix - conceptually part of flake outputs
{
  checks.x86_64-linux.fmt = pkgs.runCommand "fmt" { } ''
    echo ok > $out
  '';
}

```

A failing check fails the job. You do not add a second “lint” job with a different Python.

```
nix flake check
```

Output on success:

```

evaluating flake...
checking flake...

```

Case 3: Magic Nix Cache when you do not have Cachix yet

```

# .github/workflows/ci-magic.yml
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest

```



```

steps:
  - uses: actions/checkout@v4
  - uses: DeterminateSystems/nix-installer-action@v16
  - uses: DeterminateSystems/magic-nix-cache-action@v8
  - run: nix build

```

This uses GitHub’s cache API. It is per-repository and evicts. It is a start. A team cache (Cachix/Attic) is what laptops share with CI.

Case 4: Pin the installer, pin nixpkgs

The flake lock already pins nixpkgs. The installer action should not float @main:

```

- uses: cachix/install-nix-action@v27

```

When you bump, bump in a dedicated commit. Same discipline as `flake.lock`.

Case 5: Free disk on the runner

GitHub’s Ubuntu image is busy. If `nix build` dies with `No space left on device`:

```

- name: Free disk
  run: |
    sudo rm -rf /usr/share/dotnet /opt/ghc /usr/local/share/boost
    df -h

```

Then GC at the end of self-hosted runners only (`nix-collect-garbage -d`). Hosted runners die anyway.

The trap

The trap is `nix-env -iA nixpkgs.go` in CI after installing Nix, then `go test`. You just rebuilt the old world: an unpinned go, no flake, no cache sharing with the laptop.

The other trap is `cachix-action` with a write token on `pull_request_target`. A fork PR can then push malware NARs to the team cache. Write tokens: `push` to `main` and trusted workflows only.

The boring rule

- One install-Nix action, then `nix flake check` and `nix build`.
- Same `flake.lock` as developers.
- Push to the team cache from trusted branches.
- Pin action versions.
- Do not `setup-go` / `setup-python` next to Nix “just in case.”

Try this

1. Add Case 1 to a throwaway repo with a `flake.nix` that outputs `packages.x86_64-linux.default = pkgs.hello`. Confirm the Actions log shows a substitute or a build, then a second run that hits the cache.
2. Break `flake.nix` syntax, push, and confirm the job fails at eval, not at a later custom script.
3. List repository secrets and confirm `CACHIX_AUTH_TOKEN` is not printed in logs (`echo ${secrets.CACHIX_AUTH_TOKEN}` is a firing offence).
4. Time `nix flake check` locally versus in CI. If CI is 10× slower, you are compiling; fix substituters.

GitLab CI Integration

GitLab CI Integration

GitLab jobs are containers. The boring default is: a Nix image (or a runner that already has Nix), flakes enabled, `nix flake check` / `nix build`, and a cache that is not `GIT_STRATEGY: none` plus hope.

Mental model

Runner kind	How Nix appears
Shared SaaS runner	Job image <code>nixos/nix</code> (or <code>gitpod/workspace-nix</code>)
Self-hosted with Nix	<code>image:</code> optional; use the host store + a mounted <code>/nix</code>
Kubernetes executor	Persistent <code>/nix</code> volume or you download the world every job

`NIX_CONFIG` in `variables:` is the reliable way to turn on flakes inside `nixos/nix` images that ship a conservative `nix.conf`.

The same binary cache as GitHub Actions applies. Cachix works from GitLab. `CACHIX_AUTH_TOKEN` is a GitLab CI/CD variable, masked, protected if it can write.

Worked examples

Case 1: Official Nix image, flakes on

Save as `.gitlab-ci.yml`:

```
# .gitlab-ci.yml
image: nixos/nix:2.35.2

variables:
  NIX_CONFIG: "experimental-features = nix-command flakes"

stages:
  - test
  - build
```

```

test-job:
  stage: test
  script:
    - nix flake check

build-job:
  stage: build
  script:
    - nix build --print-build-logs
    - nix path-info -Shr ./result

```

Pin the image tag (2.35.2), not latest.

Case 2: Cachix from GitLab

```

# .gitlab-ci.yml
image: nixos/nix:2.35.2

variables:
  NIX_CONFIG: "experimental-features = nix-command flakes"

before_script:
  - nix-env -iA nixpkgs.cachix
  - cachix use desk-cache
  - echo "$CACHIX_AUTH_TOKEN" | cachix authtoken --stdin

build:
  script:
    - nix flake check
    - nix build
    - cachix push desk-cache ./result

```

`nix-env -iA` for **cachix the client** on a throwaway image is acceptable. Do not use it to install the application under test.

A cleaner variant: `nix run nixpkgs#cachix -- use desk-cache`.

Case 3: Cache /nix on a self-hosted runner

On a runner VM that keeps its disk:

```

# .gitlab-ci.yml
default:
  tags: [nix-runner]
  image: nixos/nix:2.35.2
  cache:

```

```

key: nix-store
paths:
  - /nix/store

```

GitLab's cache: paths on /nix/store is **slow and incomplete** (no db.sqlite). Prefer a **persistent runner** with Nix installed on the host and:

```

# .gitlab-ci.yml
default:
  tags: [nix-host]
  # no image: - job runs on the host
variables:
  NIX_CONFIG: "experimental-features = nix-command flakes"
build:
  script:
    - nix flake check
    - nix build

```

The host store is the cache. GC with `--delete-older-than 7d` in a nightly cron.

Case 4: rules so forks do not push

```

# .gitlab-ci.yml
push-cache:
  script:
    - nix build
    - cachix push desk-cache ./result
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

```

Write tokens stay on the default branch.

Case 5: Show what you shipped

```

# .gitlab-ci.yml
build:
  script:
    - nix build
    - nix path-info -Shr ./result
    - nix-store -q --tree ./result | head -40
artifacts:
  paths:
    - result
  expire_in: 1 day

```

The `result` symlink as an artifact is **not** a closure. It is a pointer that will not work on a machine without `/nix`. Use `nix copy --to file://$PWD/nar` and artifact `nar/` if humans must download.

The trap

The trap is **image: ubuntu:24.04 plus curl | sh the Nix installer on every job**. You pay minutes and get a different Nix version than last week. Use a Nix image or a Nix-equipped runner.

The other trap is caching only `result` in GitLab artifacts and calling it a binary cache. Artifacts are not signed NARs.

The boring rule

- Pin `nixos/nix:<version>` or use a self-hosted runner that already has Nix.
- `NIX_CONFIG` enables flakes inside the job.
- Same team cache as GitHub Actions and laptops.
- Write tokens only on the default branch.
- `nix path-info -Shr` in the log so a fat closure is visible in review.

Try this

1. Run Case 1 against a flake that builds `hello`. Confirm the job log contains `Hello` or a store path for `hello`.
2. Pin the image digest (`nixos/nix:2.35.2@sha256:...`) and record why that is stricter than a moving tag.
3. Add `nix path-info -Shr ./result` and paste the closure size into the merge request template once.
4. On a self-hosted runner, run `nix-collect-garbage --delete-older-than 7d` after a week and note `df -h /nix`.

Build Caching Strategies

Build Caching Strategies

CI that recompiles llvm because someone edited `README.md` is not a pipeline, it is a heater. The boring default is: **substituters first, a persistent team cache second, GitHub’s cache API only as a bump, and src filters so docs do not bust compiles.**

Mental model

Nix does not cache “step 3 of a Dockerfile.” It caches **store paths**. If the `.drv` hash is unchanged, the path is substituted. If `src` includes the whole git tree, every commit is a new `.drv`.

Layers, cheapest first:

Layer	Lifetime	Who uses it
Local <code>/nix/store</code>	Until GC	This runner / laptop
Team cache (Cachix / Attic)	Months	Every laptop + every CI job
<code>magic-nix-cache</code> (GHA)	Days, per repo	GitHub-hosted runners only

Docker layer cache busts everything after a changed line. Nix rebuilds only the node whose inputs moved, plus dependents. A docs derivation and a Go derivation are siblings, not a stack.

```
README.md change
    hello-docs.drv    rebuilt
    desk-api.drv      same hash → substitute
```

Worked examples

Case 1: Team cache is the real cache

The binary-cache chapter already added substituters. CI must **push** what it built:

```
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
  push:
    branches: [main]
```

```

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
      - uses: cachix/cachix-action@v15
        with:
          name: desk-cache
          authToken: ${ secrets.CACHIX_AUTH_TOKEN }
      - run: nix build --print-build-logs

```

A laptop with the same `flake.lock` then substitutes. `magic-nix-cache` alone does **not** help the laptop.

Case 2: magic-nix-cache as a spare tyre

```

# .github/workflows/ci-magic.yml
name: CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: DeterminateSystems/nix-installer-action@v16
      - uses: DeterminateSystems/magic-nix-cache-action@v8
      - run: nix build

```

Fine for a personal repo. Evicts. Does not sign NARs for other machines. Graduate to Cachix when a second human waits on CI.

Case 3: lib.cleanSource / gitignore

Save as `package.nix`:

```

# package.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.stdenv.mkDerivation {
  pname = "desk-banner";
  version = "1.0.0";
  src = pkgs.lib.cleanSource ./.;
  buildPhase = "echo ok > $out";
  installPhase = "true";
}

```

```
}
```

`cleanSource` drops `.git`, `result`, and editor junk. Stricter:

```
src = pkgs.lib.cleanSourceWith {
  src = ./.;
  filter = path: type:
    baseNameOf path != "README.md"
    && baseNameOf path != "docs";
};
```

Or `pkgs.nix-gitignore.gitignoreSource [ ] ./.` with a `.gitignore` you actually maintain.

```
nix-build package.nix
echo "# note" >> README.md
nix-build package.nix
```

If the second build is “checking paths” / a cache hit, the filter worked. If it rebuilds, `README.md` is still in `src`.

Case 4: Vendor once, compile often

Save as `go.nix`:

```
# go.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.buildGoModule {
  pname = "desk-api";
  version = "1.0.0";
  src = pkgs.lib.cleanSource ./.;
  vendorHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
}
```

`vendorHash` is a FOD. Changing `main.go` does not re-fetch modules. Changing `go.mod` does. Probe the hash once; do not set `vendorHash = null` in CI “to make it work.”

Case 5: See what would rebuild

```
nix build --dry-run
nix path-info -Shr ./result | tail -1
```

`--dry-run` lists derivations that are not in the store or substituters. If that list includes `gcc` after a comment-only change, `src` is too wide or an input leaked (`currentTime`, impure env).

The trap

The trap is `src = ./.` on a monorepo root for a leaf package. A frontend CSS change rebuilds the Go API. Filter to `./cmd/desk-api` or `fileset (lib.fileset.gitTracked)`.

The other trap is treating GHA cache as the team cache. A new contributor's first `nix build` still compiles the world.

The boring rule

- Push to a signed team cache from trusted CI. Laptops pull.
- Filter `src`. Docs and `.git` are not compile inputs.
- Language vendor hashes (Go, Cargo, npm) are FODs. Pin them.
- `--dry-run` after a trivial edit. If `gcc` is in the list, stop and fix inputs.
- `magic-nix-cache` is optional. Substituters are not.

Try this

1. `nix path-info -Shr ./result` before and after a README-only commit. The store path of the binary should be identical if `src` is filtered.
2. `--dry-run` twice in a row with no edits. Second list should be empty (or only substitutes).
3. Break `vendorHash` by one character, watch the FOD mismatch, restore it.
4. Time `nix build` on a clean store with the team cache configured versus `nix build --option substituters ''`. The delta is the cache earning its keep.

Testing in CI Pipelines

Testing in CI Pipelines

Tests that need the office Wi-Fi are not tests. The boring default is: **hermetic checkPhase / go test inside the derivation, nix flake check on every PR against nixpkgs 26.05, and NixOS VM tests when a unit must listen.**

Mental model

The sandbox has no network. `go test` that downloads modules fails. `vendorHash` (FOD) already fetched modules. `npm test` that hits staging fails. Mock it.

Kind	Where
Unit	<code>doCheck</code> / <code>checkPhase</code> on the package
Lint	<code>checks.<system>.lint</code>
Integration	<code>testers.runNixOSTest</code> (needs KVM)
Gate	<code>nix flake check -L</code> in CI

GitHub Actions: install Nix 2.35 (or the 26.05-pinned Nix), then `nix flake check`. No `setup-go`.

Worked examples

Case 1: Hermetic Go tests

Save as `service.nix`:

```
# service.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.buildGoModule {
  pname = "desk-service";
  version = "1.0.0";
  src = pkgs.lib.cleanSource ./.;
  vendorHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
  doCheck = true;
}
```

buildGoModule already runs `go test` when `doCheck` is true. Probe `vendorHash`. `null` is not a pin.

```
nix-build service.nix
```

Case 2: Flake checks

Save as `flake.nix`:

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
      pkg = pkgs.callPackage ./service.nix { };
    in
    {
      packages.${system}.default = pkg;
      checks.${system}.unit = pkg;
      checks.${system}.vet = pkgs.runCommand "vet" {
        nativeBuildInputs = [ pkgs.go ];
        src = ./.;
      } ''
        cd $src
        GOPROXY=off go vet ./...
        touch $out
      '';
    };
}
```

`GOPROXY=off` proves you are not downloading. If `vet` needs modules, use the same `buildGoModule` env.

```
nix flake check --print-build-logs
```

Case 3: CI workflow

```
# .github/workflows/check.yml
name: check
on: [pull_request, push]
jobs:
  check:
```

```

runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: cachix/install-nix-action@v27
  - uses: cachix/cachix-action@v15
    with:
      name: desk-cache
      authToken: ${ secrets.CACHIX_AUTH_TOKEN }
      skipPush: ${ github.event_name == 'pull_request' }
  - run: nix flake check -L

```

Push cache on main only (secrets chapter).

Case 4: VM test job (KVM)

```
nix build .#checks.x86_64-linux.nginx -L
```

GitHub hosted runners often lack nested KVM. Use a self-hosted runner with `/dev/kvm` or skip VM tests on GHA and run them on the builder.

Case 5: Fail the gate

```

# break a test, then:
nix flake check
echo $?

```

Non-zero. The PR does not merge. Logs are in `nix log / -L`, not in a screenshot.

The trap

The trap is **go test that hits `https://staging.desk.internal`**. Sandbox: connection refused. “Works on my laptop” because your laptop is not a sandbox. `doCheck` must be hermetic. Integration against staging is a **deploy** job with secrets, not `flake check`.

The boring rule

- `nix flake check -L` is the PR gate. 26.05 pin. Nix 2.35 on the runner.
- `vendorHash` / `cargoHash` so tests do not download.
- VM tests need KVM; do not pretend GHA ubuntu has it.
- No prod credentials in check.
- `-L` so failures are in the log.

Try this

1. Flip an assertion, `nix flake check`, restore.
2. Add `curl https://example.com` to `checkPhase` and confirm sandbox failure.
3. `nix flake check` twice; second should substitute.
4. Document whether VM tests run on GHA or on the builder.

Secrets Management in CI

Secrets Management in CI

The store is world-readable. CI logs are copied. Fork pull requests are hostile. The boring default is: **no secret bytes in Nix; access-tokens for private inputs; Age/sops keys only on trusted jobs; never pull_request_target plus a write token.**

Mental model

Kind of secret	Where it lives	How Nix sees it
GitHub PAT for private flakes	Runner <code>nix.conf</code> <code>access-tokens</code>	Fetch, not a derivation input
Age private key for sops	Runner env / <code>SOPS_AGE_KEY</code>	Decrypt at eval/deploy time
DB password for the app	Encrypted in git; decrypted on the host	Path <code>/run/secrets/...</code> only
Cachix write token	Actions secret	<code>cachix push</code> , not <code>flake.nix</code>

Builds should not need production passwords. If `nix flake check` requires Vault, the check is not hermetic.

```
trusted push to main
  → runner gets tokens
  → nix build    (no app secrets)
  → deploy      (decrypt on target, not in the NAR)
```

Worked examples

Case 1: Private flake inputs without putting the PAT in Nix

Save as `.github/workflows/deploy.yml`:

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]

jobs:
```

```

deploy:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: cachix/install-nix-action@v27
      with:
        extra_nix_config: |
          access-tokens = github.com=${{ secrets.GH_PAT_FOR_PRIVATE_MODULES }}
    - run: nix flake check
    - run: nix build

```

`access-tokens` is daemon config. The token is not hashed into `desk-api`. Fork PRs must **not** receive this secret (GitHub's default for `pull_request` from forks).

Case 2: Eval sops without decrypting values into the store

```

# .github/workflows/sops-eval.yml
name: Sops eval
on:
  push:
    branches: [main]
jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
      - name: Eval secret *paths*
        env:
          SOPS_AGE_KEY: ${{ secrets.CI_AGE_PRIVATE_KEY }}
        run: nix eval .#nixosConfigurations.desk-server.config.sops.secrets

```

The eval prints **paths** (`/run/secrets/db-password`), not password strings. If the log shows a password, the module leaked it into config (usually `hashedPassword = "plaintext"` or `environment.FOO = config.sops.secrets.x.path` is fine; interpolating `builtins.readFile` is not).

Case 3: The leak, on purpose, once

Save as `leak.nix`:

```

# leak.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.writeText "database.conf" ''

```

```
password = SuperSecretPassword123
''
```

```
nix-build leak.nix
cat result
nix path-info result
```

Output:

```
password = SuperSecretPassword123
/nix/store/...-database.conf
```

Anyone with `nix` copy of this closure has the password. Delete the file, GC, **rotate** the password. Then never do this.

Case 4: Cachix write only on main

```
# .github/workflows/push-cache.yml
name: Push cache
on:
  push:
    branches: [main]
jobs:
  push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
      - uses: cachix/cachix-action@v15
        with:
          name: desk-cache
          authToken: ${ secrets.CACHIX_AUTH_TOKEN }
      - run: nix build
```

Do not use `pull_request_target` with this token. A fork can run your workflow against their branch and upload NARs to the team cache.

Case 5: nix flake check without credentials

```
unset SOPS_AGE_KEY CACHIX_AUTH_TOKEN GH_TOKEN
nix flake check
```

If this fails because sops cannot decrypt, split checks: `eval` that only looks at **paths** and types, versus a `deploy` job that has keys. Checks that need production secrets will fail on contributor laptops too.

The trap

The trap is `builtins.readFile ./prod.key` in a module. Eval copies the bytes into the store. `nix why-depends /run/current-system` will show a path whose cat is the key.

The other trap is printing secrets in `set -x bash`. `echo "$SOPS_AGE_KEY"` in a workflow is a rotation event.

The boring rule

- Store is public. GitHub logs are public-ish. Treat both as hostile.
- `access-tokens` for fetches. Encrypted files in git for app secrets. Decrypt on the **host**.
- Write tokens (Cachix, deploy keys) only on **push** to **main**.
- `nix flake check` without production credentials.
- After any leak, rotate. GC does not un-copy a cache.

Try this

1. Case 3 on a throwaway password. `cat $(nix-build leak.nix --no-out-link)`. Then GC and confirm you still understand copies in the cache if you had pushed.
2. `nix-instantiate --eval -E 'builtins.readFile ./test.txt'` with a dummy file. Note that the string is in the eval result.
3. Confirm `nix flake check` on a clone with no `SOPS_AGE_KEY`.
4. Open the Actions secret list and tick “review logs for accidental echo” on the last deploy job.

Remote Builders

Remote Builders

Binary caches move **results**. Remote builders move **work**. The boring default is: **evaluate on the laptop, realise on a machine that has CPU, KVM, and the right system, then push the result to the team cache**.

A laptop should not compile Chromium. A Darwin laptop cannot natively produce `x86_64-linux` test VMs. Both problems are remote builders, not “more -j”.

Mental model

	Binary cache	Remote builder
Needs the path to already exist?	Yes	No
What moves first	NAR (result)	.drv + input paths, then NAR back
Trust	Signing keys	SSH user on the builder
Typical role	Everyone pulls	One beefy box (or a queue) builds

```
laptop --drv--> builder --push--> cache --substitute--> fleet
```

`nix.buildMachines` (NixOS) or `--builders` (CLI) lists machines:

- `hostName`, `sshUser`, `sshKey`
- `system` (`x86_64-linux`, `aarch64-linux`, ...)
- `protocol` (`ssh-ng` preferred)
- `maxJobs`, `speedFactor`
- `supportedFeatures` (`kvm`, `big-parallel`, `nixos-test`)

If a derivation requires `kvm` and the builder does not advertise it, Nix builds locally (or fails). Mysterious local compiles are often a missing feature flag.

Worked examples

Case 1: NixOS client configuration

Save as `builders.nix`:


```
# builders.nix
{
  nix.distributedBuilds = true;
  nix.settings.builders-use-substitutes = true;

  nix.buildMachines = [
    {
      hostName = "builder.desk.internal";
      system = "x86_64-linux";
      protocol = "ssh-ng";
      sshUser = "nixbuilder";
      sshKey = "/root/.ssh/id_builder";
      maxJobs = 8;
      speedFactor = 4;
      supportedFeatures = [ "nixos-test" "big-parallel" "kvm" ];
    }
  ];
}
```

`builders-use-substitutes = true` lets the **builder** download from the team cache instead of asking the laptop to upload gcc.

The SSH key is a host key for root (the daemon builds as root's SSH). Protect it with sops. `nixbuilder` on the far side must be a trusted Nix user.

```
sudo nixos-rebuild switch
```

Case 2: One-shot without a module

```
nix build .#desk-api --max-jobs 0 \
  --builders 'ssh-ng://nixbuilder@builder.desk.internal x86_64-linux - 8 1 kvm,big-parallel,
```

`--max-jobs 0` means “do not build locally.” If the builder is down, the command fails instead of melting the laptop.

Case 3: Builder host — NixOS

Save as `builder-host.nix`:

```
# builder-host.nix
{ pkgs, ... }:

{
  users.users.nixbuilder = {
    isNormalUser = true;
  }
}
```

```

    extraGroups = [ "wheel" ];
    openssh.authorizedKeys.keys = [
        "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDeskLaptopRootBuilderKey desk-laptop"
    ];
};

nix.settings.trusted-users = [ "root" "nixbuilder" ];
nix.settings.max-jobs = 8;

services.openssh.enable = true;
}

```

`trusted-users` is required for `nixbuilder` to start builds as the daemon. Do not put your whole team in `trusted-users` on a laptop; do it on the builder.

Test SSH as the key the daemon will use:

```

sudo ssh -i /root/.ssh/id_builder nixbuilder@builder.desk.internal nix-store --version

```

If this asks for a password, the daemon will hang.

Case 4: Darwin → Linux

An Apple silicon laptop producing Linux containers:

```

# darwin-builders.nix
{
    nix.distributedBuilds = true;
    nix.buildMachines = [
        {
            hostname = "linux-builder.desk.internal";
            system = "aarch64-linux";
            protocol = "ssh-ng";
            sshUser = "nixbuilder";
            sshKey = "/var/root/.ssh/id_builder";
            maxJobs = 8;
            supportedFeatures = [ "kvm" "big-parallel" "nixos-test" ];
        }
    ];
}

```

`linux-builder` can be a VM on the Mac (`nix-darwin linux-builder`) or a real ARM server. Prefer a real server for `nixosTest` (KVM).

Case 5: Features — why tests ran locally

```
nix build .#checks.x86_64-linux.desk-vm --max-jobs 0
```

If the check is a `nixosTest`, the builder **must** list `kvm` and `nixos-test`. Missing those, Nix tries the local machine; on Darwin that is a slow VM or a failure.

```
nix show-config | grep builders
```

Confirm the live config, not just the `.nix` file you meant to switch.

The trap

The trap is **pointing buildMachines at a machine that is also a substituter with a different nixpkgs**. You upload inputs, it rebuilds everything because its store is on 23.11. Same `flake.lock` everywhere; builder substitutes from the team cache (`builders-use-substitutes`).

The other trap is user SSH config (`~/.ssh/config` Host aliases) that root's daemon does not read. Use `hostName` IPs or put the alias in `/root/.ssh/config`.

The boring rule

- Cache for results, builders for first compiles and foreign **system**.
- `ssh-ng`, `builders-use-substitutes`, `--max-jobs 0` when you mean “remote only.”
- Advertise `kvm` / `nixos-test` / `big-parallel` honestly.
- Daemon SSH keys, not your interactive `ssh-agent`, unless you designed for that.
- Builder `nixpkgs` + cache = laptop `flake lock` + team cache.

Try this

1. `sudo ssh` from Case 3 until `nix-store --version` prints without a password.
2. `nix build nixpkgs#hello --max-jobs 0 --builders '...'` and watch the builder's `nix-daemon log` (`journalctl -fu nix-daemon`).
3. Remove `kvm` from `supportedFeatures`, run a `nixosTest`, and note where it builds. Put `kvm` back.
4. After a remote build, `nix path-info -Shr ./result` on the laptop — the NAR should have been copied back. Then `cachix push` so the next laptop never builds it.

Containers and Kubernetes

Creating Container Images with Nix (dockerTools)

Creating Container Images with Nix (dockerTools)

FROM debian:latest plus apt-get is a different closure every Tuesday. The boring default is: `pkgs.dockerTools.buildLayeredImage` (or `streamLayeredImage`) from `nixpkgs 26.05`, no daemon at build time, cacert if you speak TLS.

Mental model

Nix builds an OCI tarball from store paths. Layers split by derivation so glibc stays cached when `desk-api` changes. No `/bin/bash` unless you listed `pkgs.bash`.

Function	Output
<code>buildLayeredImage</code>	<code>.tar.gz</code> you <code>docker load</code>
<code>streamLayeredImage</code>	a script that streams to <code>docker load</code> (smaller eval)

Podman and Docker both load the tarball. Building does **not** need root.

`desk-api drv + cacert → layered OCI → docker load / podman load`

Worked examples

Case 1: Layered image

Save as `image.nix`:

```
# image.nix
{ pkgs ? import <nixpkgs> {} }:

let
  deskApp = pkgs.writeShellApplication {
    name = "desk-api";
    text = ''
      echo "desk-api listening (fake)"
    '';
  };
};
```

```

in
pkgs.dockerTools.buildLayeredImage {
  name = "desk-api";
  tag = "1.0.0";
  contents = [ deskApp pkgs.cacert ];
  config = {
    Cmd = [ "${deskApp}/bin/desk-api" ];
    ExposedPorts."8080/tcp" = { };
    Env = [ "SSL_CERT_FILE=${pkgs.cacert}/etc/ssl/certs/ca-bundle.crt" ];
  };
}

nix-build image.nix
podman load < result
podman run --rm desk-api:1.0.0

```

Output (shape):

desk-api listening (fake)

Prefer a version tag over latest.

Case 2: vendorHash for a real Go module

```

# image-go.nix
{ pkgs ? import <nixpkgs> {} }:

let
  deskApp = pkgs.buildGoModule {
    pname = "desk-api";
    version = "1.0.0";
    src = pkgs.lib.cleanSource ./.;
    vendorHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
  };
in
pkgs.dockerTools.buildLayeredImage {
  name = "desk-api";
  tag = "1.0.0";
  contents = [ deskApp pkgs.cacert pkgs.tzdata ];
  config.Cmd = [ "${deskApp}/bin/desk-api" ];
}

```

Probe vendorHash once. null is not a pin.

Case 3: Inspect layers

```
tar -tf result | head
nix path-info -Shr result | tail -1
```

The image closure should be the app + libc + certs, not gcc. If gcc is in `path-info -r`, you copied `buildInputs` into `contents`.

Case 4: Flake package

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in
    {
      packages.${system}.image = import ./image.nix { inherit pkgs; };
    };
}
```

```
nix build .#image
```

CI runs that, then `skopeo copy docker-archive:./result docker://registry.desk.internal/desk-api:1.0.0`

Case 5: Nobody is home in the image

```
podman run --rm --entrypoint /bin/sh desk-api:1.0.0
```

Output:

Error: no such file or directory

That is success for a production image. Debug with a **separate** image that includes `pkgs.busybox`. Do not ship busybox in the API tag.

The trap

The trap is **forgetting cacert**. HTTPS fails with `x509: certificate signed by unknown authority`. Include `pkgs.cacert` and `SSL_CERT_FILE`.

The other trap is `contents = [ pkgs.stdenv ]`; so “debug is easy.” You just shipped a compiler.

The boring rule

- `buildLayeredImage` / `streamLayeredImage`. No Dockerfile `FROM ubuntu`.
- Pin tags. Include `cacert` (and `tzdata` if you format times).
- `vendorHash` is a FOD. Probe once.
- Build in CI without `dockerd`. Load or `skopeo copy` after.
- Debug images are a different output, not extra packages in prod.

Try this

1. Build Case 1, `tar -tf result | wc -l`.
2. Drop `cacert`, run a one-liner that `curl https://example.com` (add `curl`), read the TLS error, put `cacert` back.
3. `nix path-info -Shr result` and confirm `gcc` is absent.
4. Tag `1.0.0` and `1.0.1` after a script change; confirm the base layers' hashes stayed when only the top layer moved.

Kubernetes Manifests as Nix Modules

Kubernetes Manifests as Nix Modules

Hand-written YAML that drifts from the image tag in CI is how Saturday pages start. The boring default is: **generate manifests from Nix (same pin as `.#image`), commit the generated YAML if the cluster cannot eval Nix, and skip Helm until a chart is forced on you.**

Mental model

Nix can produce JSON/YAML for `kubectl apply`. Libraries (`nixidy`, `kubenix`, or a 40-line `writeText`) all do the same job: **one evaluation, one tag, one registry.**

The cluster still runs kubelet. Nix does not replace Kubernetes. It replaces the YAML copypaste.

```
flake → image tag + digest
      → Deployment YAML with that digest
      → kubectl apply --server-side
```

If the platform already speaks Helm only, next chapter. If you own the app, raw manifests are enough.

Worked examples

Case 1: A Deployment as JSON

Save as `k8s.nix`:

```
# k8s.nix
{ pkgs ? import <nixpkgs> {} }:

let
  image = "registry.desk.internal/desk-api:1.0.0";
  doc = {
    apiVersion = "apps/v1";
    kind = "Deployment";
    metadata = { name = "desk-api"; namespace = "desk"; };
    spec = {
      replicas = 2;
      selector.matchLabels.app = "desk-api";
      template = {
```

```

        metadata.labels.app = "desk-api";
        spec.containers = [
            {
                name = "desk-api";
                inherit image;
                ports = [ { containerPort = 8080; } ];
                resources.requests = { cpu = "50m"; memory = "64Mi"; };
            }
        ];
    };
};
in
pkgs.writeText "desk-api-deploy.json" (builtins.toJSON doc)

nix-build k8s.nix
python3 -m json.tool result | head

```

Case 2: Same flake as the image

```

# flake.nix fragment
{
    inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    outputs = { self, nixpkgs }: {
        packages.x86_64-linux.manifests = import ./k8s.nix {
            pkgs = nixpkgs.legacyPackages.x86_64-linux;
        };
    };
}

```

Bump the tag in **one** place (an overlay attr `desk.imageTag`) and rebuild image + manifests together.

Case 3: Apply

```

kubectl apply --server-side -f result
kubectl -n desk rollout status deploy/desk-api

```

`--server-side` plays nicer with fields you do not own. Dry-run first on a shared cluster:

```

kubectl apply --dry-run=server -f result

```

Case 4: ConfigMap from Nix, not from kubectl create

```
# configmap.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.writeText "desk-cm.json" (builtins.toJSON {
  apiVersion = "v1";
  kind = "ConfigMap";
  metadata = { name = "desk-api"; namespace = "desk"; };
  data.WINDOW = "A";
})
```

Secrets still do not belong here. Use External Secrets / sealed-secrets / CSI — or do not put the password in git, same rule as the Nix store.

Case 5: When the cluster cannot run Nix

```
nix build .#manifests
cp result generated/desk-api-deploy.json
git add generated/
```

CI diffs `git diff --exit-code generated/` so humans cannot “just kubectl edit” without updating the flake.

The trap

The trap is **templating the image as latest**. Every node may run a different digest. Pin tag and digest (`image: repo@sha256:...`) when the registry supports it.

The boring rule

- Manifests and image from one flake pin (nixpkgs 26.05).
- JSON/YAML generated; apply server-side.
- No passwords in ConfigMaps.
- Pin digests in production.
- Generate-and-commit if GitOps tools cannot eval Nix.

Try this

1. Case 1, `jq .spec.replicas result`.
2. Change replicas to 3, rebuild, `diff` the JSON.
3. Put `image: ...:latest` in a lab, list why you will not ship it.

4. Add a Service JSON next to the Deployment in the same file (list of docs or a YAML stream).

Helm Charts Generated from Nix

Helm Charts Generated from Nix

Helm is the cluster's package manager whether you like it or not. The boring default is: **pin the chart *and* the values in Nix (nixhelm / vendored Chart.yaml), render with helm template, and never helm install from floating latest.**

Mental model

A chart is a tarball plus values. Unpinned helm repo add && helm upgrade is npm install on the control plane.

Pin	How
Chart version	<code>chart = { name, repo, version, hash }</code> FOD
Values	Nix attrset $\rightarrow$ YAML
Image inside values	Same digest as <code>image</code>

If the app is yours, prefer raw manifests (previous chapter). Helm when the vendor only ships a chart (ingress-nginx, cert-manager).

Worked examples

Case 1: Vendor a chart tarball

```
helm pull ingress-nginx --version 4.11.0 --repo https://kubernetes.github.io/ingress-nginx
nix hash path ingress-nginx-4.11.0.tgz
```

Save as `chart.nix`:

```
# chart.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.fetchurl {
  url = "https://github.com/kubernetes/ingress-nginx/releases/download/helm-chart-4.11.0/ing
  hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
}
```

Probe the hash. That tarball is now as pinned as `hello`.

Case 2: Render values from Nix

Save as values.nix:

```
# values.nix
{
  controller = {
    replicaCount = 2;
    image.tag = "v1.11.2";
    service.type = "ClusterIP";
  };
}
```

```
nix-instantiate --eval --json -E 'import ./values.nix' > /tmp/values.json
helm template desk-ing ./ingress-nginx-4.11.0.tgz -f /tmp/values.json
```

Commit the rendered YAML if Argo cannot run Helm+Nix.

Case 3: Wrapper script in a flake

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      pkgs = nixpkgs.legacyPackages.x86_64-linux;
    in
    {
      packages.x86_64-linux.render-ing = pkgs.writeShellApplication {
        name = "render-ing";
        runtimeInputs = [ pkgs.kubernetes-helm pkgs.jq ];
        text = ''
          helm template desk-ing ${./ingress-nginx-4.11.0.tgz} -f values.yaml
        '';
      };
    };
}
```

values.yaml can be builtins.toJSON written by a derivation so it is not hand-edited.

Case 4: helm upgrade with a pinned package

```
nix run .#render-ing | kubectl apply --server-side -f -
```

or helm upgrade --install from the vendored tgz, not from a live repo index.

Case 5: Diff in CI

```
nix run .#render-ing > /tmp/new.yaml  
git diff --no-index generated/ingress.yaml /tmp/new.yaml
```

If the diff is unexpected (chart author pushed a hook), you see it before the cluster does.

The trap

The trap is **helm repo update in CI** with no version. Monday's job is not Friday's chart. Version + hash, like any FOD.

The boring rule

- Helm for vendor charts. Nix manifests for your apps.
- Chart tarball in the store (fetchurl + hash) or git-vendor.
- Values from Nix. Images from the same flake.
- Render in CI; apply the bits you reviewed.
- No **latest** chart, no **latest** image.

Try this

1. **helm pull** a tiny chart, **nix hash path** the tgz, put it in **fetchurl**.
2. Change **replicaCount**, re-render, **diff**.
3. Break the hash by one character and read the FOD error.
4. Search CI for **helm repo update** and delete it.

GitOps with Argo CD and Flux

GitOps with Argo CD and Flux

SSH-from-laptop `kubectl apply` does not scale and does not audit. The boring default is: **the cluster pulls generated YAML from git (Flux or Argo), and Nix only runs in CI to refresh that YAML.**

Mental model

Tool	Pull model
Flux	controllers reconcile <code>GitRepository</code> + <code>Kustomization</code>
Argo CD	Application CR points at a git path

Both want **plain YAML** in the repo (or a Helm release they render). They do not eval your flake on the control plane. CI:

```
nix build .#manifests → commit generated/ → Flux applies
```

Pick **one**. Running Argo and Flux on the same cluster is a fight.

Worked examples

Case 1: Generated tree layout

```
generated/  
desk/  
  namespace.yaml  
  deploy.yaml  
  svc.yaml
```

```
nix build .#manifests  
rsync -a --delete result/ generated/desk/  
git add generated && git diff --cached
```

Human reviews the YAML, not the Nix, at merge time — or reviews both.

Case 2: Flux GitRepository (illustrative)

Save as `generated/flux/gitrepo.yaml`:

```
# gitrepo.yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: desk
  namespace: flux-system
spec:
  interval: 1m
  url: https://git.desk.internal/platform/desk.git
  ref:
    branch: main
```

The URL is your repo. Auth is a Secret Flux already knows, not a token in Nix.

Case 3: Kustomization pointing at generated YAML

```
# kustomization-flux.yaml
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: desk-api
  namespace: flux-system
spec:
  interval: 5m
  sourceRef:
    kind: GitRepository
    name: desk
  path: ./generated/desk
  prune: true
```

`prune: true` deletes objects you removed from git. That is the point. Test in staging first.

Case 4: CI gate

```
# .github/workflows/manifests.yml
name: manifests
on: [pull_request]
jobs:
  gen:
    runs-on: ubuntu-latest
    steps:
```

```

- uses: actions/checkout@v4
- uses: cachix/install-nix-action@v27
- run: nix build .#manifests
- run: diff -ru generated/desk result

```

If someone edited YAML by hand, the PR fails. Edit Nix, regenerate, commit both.

Case 5: Argo Application (if the org standardised on Argo)

```

# application.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: desk-api
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://git.desk.internal/platform/desk.git
    targetRevision: main
    path: generated/desk
  destination:
    server: https://kubernetes.default.svc
    namespace: desk
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

```

Same generated path. Do not point Argo at `helm/` with unpinned deps.

The trap

The trap is **the laptop as GitOps**: `flux reconcile` after `kubectl apply` after `nix build` that never got pushed. Git is the desired state. If it is not in `main`, the cluster should not have it.

The boring rule

- One GitOps controller. Generated YAML in git. Nix in CI.
- `diff` generated output on every PR.
- `prune` in staging before production.
- Secrets still not in that YAML (sops + KSOPS / External Secrets).
- Pin git `targetRevision` to a tag for prod if you need freeze.

Try this

1. `diff -ru two nix build .#manifests` trees with no edits — empty.
2. Change a replica count in Nix, regenerate, read the YAML diff as if you were on-call.
3. Write a one-page “we use Flux xor Argo” note in the desk repo.
4. Confirm `kubect1` on your laptop is **read** against prod, not write (RBAC).

Service Mesh and Observability

Service Mesh and Observability

A mesh is another control plane. The boring default is: **Prometheus + logs + a NetworkPolicy before Istio; add a mesh only when mTLS between 20+ services is a written requirement.**

Mental model

Need	Boring tool
CPU/RAM/HTTP metrics	<code>prometheus + node_exporter / app /metrics</code>
Logs	journald on NixOS; Loki if you already have it
mTLS inside the cluster	Mesh (Istio/Linkerd) or app TLS
Traffic split	Ingress / Service, then mesh

NixOS can run Prometheus on the **ops** host without Kubernetes (capstone). In-cluster, same rule: pin charts, generate YAML.

This chapter does not install Istio. It tells you what to measure first so you do not.

Worked examples

Case 1: App metrics without a mesh

Save as `metrics.nix` (NixOS ops host):

```
# metrics.nix
{
  services.prometheus = {
    enable = true;
    scrapeConfigs = [
      {
        job_name = "desk-api";
        static_configs = [ { targets = [ "10.100.0.11:8080" ] }; ];
      }
    ];
  };
}
```

The Go service exposes `/metrics`. If it does not, a mesh will not invent useful RED metrics for you.

Case 2: Probe the endpoint

```
curl -sS http://10.100.0.11:8080/metrics | head
```

Output (shape):

```
# HELP go_goroutines Number of goroutines
# TYPE go_goroutines gauge
go_goroutines 7
```

No output → fix the app, do not add Envoy.

Case 3: NetworkPolicy before sidecar

```
# netpol.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: desk-api
  namespace: desk
spec:
  podSelector:
    matchLabels:
      app: desk-api
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: desk-ing
      ports:
        - port: 8080
```

Generate this from Nix like any manifest. Most “we need a mesh” tickets are “we need NetworkPolicy.”

Case 4: If you must mesh

Pin the Linkerd or Istio **chart version + hash** (Helm chapter). Inject sidecars in **one** namespace first. Measure CPU. Roll back with GitOps prune.

Do not enable the mesh globally on day one.

Case 5: Logs on NixOS already exist

```
journalctl -u desk-api -n 50 --no-pager
```

Ship them with `services.promtail` or `systemd-journal-upload` if you have a Loki. A mesh access log is extra, not a substitute for application logs.

The trap

The trap is **installing Istio to “get observability.”** You get a second failure domain, CNI races, and a dashboard. Start with `/metrics` and `NetworkPolicy`.

The boring rule

- `/metrics` on the app. Prometheus scrape config in Nix.
- `NetworkPolicy` before sidecars.
- Mesh is optional and pinned. One namespace first.
- `journalctl` is a valid log system.
- Do not run two meshes.

Try this

1. Hit `/metrics` on a desk service or add a one-line `expvar/prometheus` handler in a lab binary.
2. Write a `NetworkPolicy` that allows only the ingress pods.
3. List three problems a mesh does **not** fix (bad schema, slow SQL, missing auth).
4. `nixos-option services.prometheus.scrapeConfigs` on 26.05.

Infrastructure as Code

Managing Terraform Providers with Nix

Managing Terraform Providers with Nix

`terraform init` downloading whatever the registry serves this morning is a supply chain. The boring default is: `pkgs.opentofu` or `pkgs.terraform.withPlugins` from `nixpkgs 26.05` so providers are store paths.

OpenTofu is the next chapter; the wrapper pattern is the same. Prefer OpenTofu on a new desk. This chapter names Terraform because that is the API people google.

Mental model

`withPlugins` builds a Terraform/OpenTofu **wrapper** whose plugin directory is in `/nix/store`. `init` does not need the network for providers. State still lives where you put it (S3, local) — Nix does not replace the backend.

```
nix develop → tofu/terraform in PATH
              plugins = /nix/store/...-terraform-providers
terraform init -plugin-dir=... → offline providers
```

Worked examples

Case 1: Shell with pinned providers

Save as `terraform-env.nix`:

```
# terraform-env.nix
{ pkgs ? import <nixpkgs> {} }:

let
  tf = pkgs.terraform.withPlugins (p: [
    p.aws
    p.random
    p.local
  ]);
in
pkgs.mkShell {
  packages = [ tf ];
}
```

```
nix-shell terraform-env.nix --run "terraform version"
```

Output (shape):

```
Terraform v1.x.x  
+ provider registry.terraform.io/hashicorp/aws v...
```

Versions are whatever 26.05 pinned. To freeze harder, overlay a specific provider derivation.

Case 2: Flake devShell

```
# flake.nix  
{  
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";  
  outputs = { self, nixpkgs }:  
    let  
      pkgs = nixpkgs.legacyPackages.x86_64-linux;  
      tf = pkgs.terraform.withPlugins (p: [ p.random p.local ]);  
    in  
    {  
      devShells.x86_64-linux.default = pkgs.mkShell { packages = [ tf ]; };  
    };  
}
```

```
nix develop --command terraform version
```

Case 3: Offline init

```
nix develop --command terraform init -get=false
```

If providers are already in the wrapper, init should not download. If it tries the registry, the wrapper is not on PATH (you ran distro terraform). `which terraform` must be `/nix/store`.

Case 4: Tiny plan

Save as `main.tf`:

```
# main.tf  
resource "random_id" "desk" {  
  byte_length = 4  
}
```

```
nix develop --command terraform init
nix develop --command terraform plan
```

No AWS credentials required. Proves plugins execute.

Case 5: CI

```
# .github/workflows/tf.yml
# after install-nix:
#   nix develop --command terraform plan -input=false
```

Same flake as laptops. No `hashicorp/setup-terraform`.

The trap

The trap is **terraform init in CI with a clean \$HOME and no withPlugins**. Registry outage or a yanked version becomes your outage.

The boring rule

- Providers from nixpkgs (26.05 pin), not the registry at apply time.
- `which terraform` is a store path.
- State backend is still your problem (lock table, encryption).
- Prefer OpenTofu for new work (next chapter).
- Overlay only when you must pin a provider the channel does not have.

Try this

1. Case 1, `which terraform, ls $(dirname $(which terraform))/../libexec` or equivalent plugin dir.
2. `terraform init` with `NIX_CURL_TIMEOUT=1` / offline to confirm no download.
3. Add `p.null` to the plugin list, enter a new shell, `terraform version`.
4. `git grep setup-terraform` and delete it from CI.

OpenTofu Integration

OpenTofu Integration

Terraform's license change is why OpenTofu exists. The boring default for a **new** desk is: **pkgs.opentofu** with **withPlugins** on **nixpkgs 26.05**, **.tofu** or **.tf** files, same **backend discipline**.

Mental model

OpenTofu is a Terraform-compatible CLI. Providers from nixpkgs attach the same way. State files from Terraform 1.x often import; test in a clone of state first.

```
pkgs.opentofu.withPlugins (p: [ p.aws p.random ])
```

Commands: `tofu init|plan|apply`. Do not install both Terraform and OpenTofu on `PATH` without wrapping.

Worked examples

Case 1: devShell

Save as `flake.nix`:

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      pkgs = nixpkgs.legacyPackages.x86_64-linux;
      tofu = pkgs.opentofu.withPlugins (p: [ p.random p.local ]);
    in
    {
      devShells.x86_64-linux.default = pkgs.mkShell { packages = [ tofu ]; };
    };
}
```

```
nix develop --command tofu version
```

Case 2: Same main.tf as Terraform

```
# main.tf
resource "random_pet" "desk" {}

nix develop --command tofu init
nix develop --command tofu plan
```

Case 3: Backend still not in Nix

```
# backend.tf
terraform {
  backend "s3" {
    bucket = "desk-tf-state"
    key    = "desk/terraform.tfstate"
    region = "eu-central-1"
  }
}
```

Credentials for the backend come from the environment / sops, not from the flake. Nix pins the **binary**, not the AWS account.

Case 4: CI

```
nix develop --command tofu fmt -check
nix develop --command tofu plan -input=false -out=plan.bin
```

Apply is a separate, protected job.

Case 5: Do not mix CLIs

```
alias terraform=tofu # in a desk-only shellHook, if humans muscle-memory `terraform`
```

Better: teach tofu. A shell that has both unwrapped is how state gets written by the wrong binary.

The trap

The trap is **migrating state on Friday in production** without `tofu plan` against a copied state. Clone the backend key, plan, then cut.

The boring rule

- New work: OpenTofu from 26.05. `withPlugins`.
- One CLI on PATH.
- State credentials are secrets (sops / env), not Nix strings.
- `fmt` + `plan` in CI. Apply on main with protection.
- Test state migration on a copy.

Try this

1. `nix develop --command tofu version` and confirm a store path.
2. `tofu plan` on `random_pet` without AWS.
3. `tofu fmt -check` on a badly indented file, fix, re-run.
4. Write down where state lives for the desk (bucket + key) in the repo README — not the credentials.

Ansible Provisioning with Nix

Ansible Provisioning with Nix

Ansible mutates hosts. NixOS **replaces** that for machines you own. The boring default is: **do not** Ansible a NixOS box; if you must drive a fleet of Ubuntu leftovers, pin `pkgs.ansible` in a 26.05 devShell and inventory in git.

Mental model

Host OS	Tool
NixOS	<code>nixos-rebuild</code> / Colmena / <code>nixos-anywhere</code>
Legacy Ubuntu/Debian you cannot reinstall this quarter	Ansible from a Nix shell

Using both on the same NixOS host (`ansible ad-hoc apt`) fights the next switch.

Worked examples

Case 1: Pinned ansible CLI

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      pkgs = nixpkgs.legacyPackages.x86_64-linux;
    in
    {
      devShells.x86_64-linux.default = pkgs.mkShell {
        packages = [ pkgs.ansible ];
      };
    };
}
```

```
nix develop --command ansible --version
```

Case 2: Inventory in git

Save as `inventory.ini`:

```
# inventory.ini
[legacy]
ubuntu-leftover.desk.internal

nix develop --command ansible all -i inventory.ini -m ping
```

Case 3: Collections pinned, not ansible-galaxy at runtime

```
# collections.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.ansible.overrideAttrs (_: {
  # prefer pkgs.ansible with extra collections from nixpkgs when they exist
})
```

If `nixpkgs` has the collection as a package, use it. If not, vendor the collection tarball as a FOD into `ANSIBLE_COLLECTIONS_PATH`. Do not `ansible-galaxy install` on the runner every job.

Case 4: Playbook that only talks to legacy

Save as `ping.yml`:

```
# ping.yml
- hosts: legacy
  gather_facts: false
  tasks:
    - ping:

nix develop --command ansible-playbook -i inventory.ini ping.yml
```

Case 5: Refuse NixOS in the inventory

```
# assert.yml
- hosts: all
  tasks:
    - name: skip nixos
      fail:
        msg: "Use nixos-rebuild, not Ansible"
        when: ansible_facts['os_family'] | default('') == 'NixOS'
```

The trap

The trap is **Ansible copy of a configuration.nix then shell: nixos-rebuild**. You now have two sources of truth. SSH to the host and rebuild from the flake, or Colmena.

The boring rule

- NixOS hosts: no Ansible.
- Legacy hosts: Ansible from a Nix-pinned CLI + vendored collections.
- Inventory in git. No ad-hoc **ansible all**.
- Collections are FODs, not galaxy-on-Monday.
- Plan the reinstall to NixOS; Ansible is a bridge.

Try this

1. **which ansible** inside **nix develop**.
2. Add a fake inventory host, **ansible-inventory --list**.
3. **git grep ansible** on NixOS modules — should be empty.
4. Write the retirement date for the last Ubuntu leftover in the desk README.

Cloud Provider Configuration

Cloud Provider Configuration

Cloud consoles are mutable. The boring default is: **NixOS images from nixos-generators (or official AMIs), cloud-init only for SSH keys at first boot, then the flake owns the instance.**

Mental model

Piece	Owner
AMI / qcow / gce image	nixos-generators on 26.05, or NixOS AMIs
Instance size, disk, VPC	OpenTofu withPlugins
OS config	nixosConfigurations in the same repo
First-boot SSH	cloud-init / metadata once

Do not bake AWS keys into the image.

Worked examples

Case 1: Official AMI lookup (OpenTofu)

```
# ami.tf
data "aws_ami" "nixos" {
  owners      = ["427812963091"]
  most_recent = true
  filter {
    name     = "name"
    values   = ["nixos/26.05*"]
  }
  filter {
    name     = "architecture"
    values   = ["x86_64"]
  }
}
```

Owner ID is the NixOS AMI account as documented on nixos.org/download. Confirm it when you copy.

Case 2: Image you built

```
nix build .#amazon
# upload the VHD / AMI import per AWS docs
```

The generator chapter produces the artifact. This chapter just consumes the AMI id in tofu.

Case 3: user-data is not configuration.nix

```
# userdata.tf
user_data = <<-EOT
# cloud-config
users:
  - name: deskadmin
    ssh-authorized-keys:
      - ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI...
EOT
```

After first boot, Colmena / `nixos-rebuild --flake` takes over. Do not keep growing user-data with packages.

Case 4: Security group matches the firewall chapter

Open 22 from the office VPN, 443 on the gateway. Do not 0.0.0.0/0 on 22 “for debug.” The NixOS firewall and the cloud SG should tell the same story.

Case 5: Instance tags

```
tags = {
  Name      = "desk-app-01"
  Flake     = "git.desk.internal/platform/desk"
  HostKey   = "app-01"
}
```

Tags are how you find the box when DNS is wrong. They are not secrets.

The trap

The trap is **an AMI from 24.11 still in tofu** after the flake moved to 26.05. The instance boots old software, then the first rebuild is a surprise kernel. Pin AMI name `nixos/26.05*` or a **specific** AMI id you tested.

The boring rule

- Images: 26.05 generator or official 26.05 AMI.
- tofu owns cloud objects. Flake owns the OS.
- user-data: SSH keys only.
- SG + NixOS firewall agree.
- No cloud keys in the image.

Try this

1. `aws ec2 describe-images --owners 427812963091 --filters Name=name,Values=nixos/26.05*` (if you have AWS) and record an AMI id.
2. Compare that id to what tofu would pick with `most_recent`.
3. Read the generator chapter's amazon format name on 26.05.
4. List ports in SG vs `networking.firewall.allowedTCPPorts` for one host — they should match.

Multi-Cloud Deployment Patterns

Multi-Cloud Deployment Patterns

Two clouds is two IAM models, two images, two outages. The boring default is: **one cloud until you have a written reason; if you must span, one flake with two nixosConfigurations and two tofu roots** — not a unified abstraction layer.

Mental model

NixOS **closures** are cloud-agnostic. **Images and metadata** are not. An AWS AMI is not a GCP image. nixos-generators emits per-format artifacts from the **same** modules.

```
modules/desk-api.nix
    nixosConfigurations.desk-aws
    nixosConfigurations.desk-gcp
generators: amazon | gce | azure
```

Do not share state files across clouds.

Worked examples

Case 1: Shared module, two hosts

```
# hosts.nix
{
  desk-aws = { modules = [ ./common.nix ./aws.nix ]; };
  desk-gcp = { modules = [ ./common.nix ./gcp.nix ]; };
}
```

common.nix has users, sshd, desk-api. Cloud files have disks and networking the provider needs.

Case 2: stateVersion

```
# common.nix
{
  system.stateVersion = "26.05";
}
```

Both clouds, same birth if they were born on 26.05.

Case 3: Images

```
nixos-amazon-image-26.05.vhd  
nixos-gce-image-26.05.tar.gz
```

Build from the generator chapter. Do not convert an AMI to GCE with a random script if the generator already has `gce`.

Case 4: Separate tofu directories

```
tofu/aws/  
tofu/gcp/
```

Different backends. Different credentials. A `for_each` over `{aws, gcp}` in one state file is how you lock the wrong cloud.

Case 5: WireGuard between clouds

If the only reason for two clouds is “HA,” a WireGuard mesh (networking chapter) plus Colmena is enough. Multi-cloud Kubernetes is not a first project.

The trap

The trap is a “cloud-agnostic” Nix wrapper that hides AMI vs GCE differences until apply time. Keep the differences in `aws.nix` / `gcp.nix` where you can read them.

The boring rule

- One cloud first.
- Same NixOS modules; different image formats and tofu roots.
- `stateVersion` 26.05 on new hosts.
- No shared terraform state.
- Mesh if you need routing; not a service mesh across clouds on day one.

Try this

1. List what actually differs between two clouds for the desk (image, IP, disk).
2. Draw two tofu dirs on paper. Do not merge them.
3. Build `.#amazon` and `.#gce` from one module set (generator chapter).
4. Write the one-sentence reason you need the second cloud — or delete the work.

Secrets and Security

Why Secrets Are Hard with Declarative Config

Why Secrets Are Hard with Declarative Config

Nix wants every input in the expression. Attackers want every input too. The boring default is: **treat /nix/store as public, never put secret bytes in Nix, and decrypt only onto /run at boot.**

Mental model

/nix/store items are world-readable. Binary caches replicate them. CI logs echo eval results. A private git repo does **not** make a string in `configuration.nix` private — the next `nix copy` to a cache, a coworker, or a backup disk copies the secret.

Safe in the flake	Never in the flake
Path <code>/run/secrets/db</code>	Password string
Age public keys	Age private keys
sops <i>file names</i>	Decrypted YAML
<code>hashedPasswordField = "/persist/..."</code>	<code>initialPassword = "hunter2"</code>

Builds are sandboxed: they cannot prompt you. If a derivation “needs” the production DB password, the design is wrong. Fetch private **source** with `access-tokens`. Fetch private **runtime** secrets on the machine that runs the unit.

```
git → encrypted blob → eval knows the path
boot → decrypt to tmpfs /run/secrets → unit EnvironmentFile
```

Worked examples

Case 1: Prove the leak

Save as `leak.nix`:

```
# leak.nix
{ pkgs ? import <nixpkgs> {} } :

pkgs.writeText "database.conf" ''
  password = SuperSecretPassword123
''
```

```
nix-build leak.nix
cat result
ls -l result
```

Output:

```
password = SuperSecretPassword123
-r--r--r-- 1 root root ... /nix/store/...-database.conf
```

Mode `r--r--r--`. Any local uid can cat it. `nix copy --to ssh://untrusted` sends it off-box.

Case 2: `builtins.readFile` is the same leak

Save as `read.nix`:

```
# read.nix
builtins.readFile ./prod.password

echo 'hunter2' > prod.password
nix-instantiate --eval read.nix
```

Output:

```
"hunter2\n"
```

The string is now in the eval. If a module interpolates it into a unit, it is in the store. Use this only with **public** files (a banner, a JSON schema).

Case 3: Reference a runtime path

Save as `safe.nix`:

```
# safe.nix
{ config, ... }:

{
  systemd.services.desk-api.serviceConfig = {
    EnvironmentFile = "/run/secrets/desk-api.env";
    DynamicUser = true;
  };
}
```

Eval contains the path. The file is created at boot by `sops-nix` / `agenix` / Vault agent. `nix path-info -r /run/current-system` should **not** contain the password.

Case 4: Hashes versus plaintext

hashedPassword = "\$6\$..." is still in the store. It is an **offline crack** target, not a live password. Prefer hashedPasswordFile on a 0400 persist path. Never initialPassword.

```
# hash-ok-ish.nix
{
  users.users.deskadmin.hashedPasswordFile = "/persist/secrets/deskadmin.hash";
}
```

Case 5: Hunt the closure

```
# after a switch, on a lab VM - dummy strings only
sudo grep -R "SuperSecret" /nix/store 2>/dev/null | head
nix why-depends /run/current-system /nix/store/...-database.conf
```

If grep hits, that generation is tainted. Roll back, rotate, GC, and if you pushed a cache, consider the secret public.

The trap

The trap is **private GitHub = safe**. Clones, Actions artifacts, nix-copy to the team cache, and a laptop left at security all see the store. Encrypt in git (sops/age) so the repo is safe to copy. Decrypt on the host.

The boring rule

- Store is public. Write configs as if a stranger can **cat** every NAR.
- Paths in Nix. Bytes on /run (tmpfs) or a 0400 persist file.
- No readFile of secrets. No secret strings in **environment**.
- After a leak: rotate, then delete from git history if you must, but assume copies exist.
- **nix flake check** must not need production secret **values**.

Try this

1. Case 1, then `stat -c '%a %n' $(readlink -f result)` — expect 444 or 555 dir.
2. Put a dummy token in a .nix file, nixos-rebuild dry-build, grep the built unit in the store. Remove it.
3. `git grep -nE 'initialPassword|privateKey |=|password =' -- '*.nix'` on the desk repo.
4. Confirm /run/secrets is tmpfs (`findmnt /run/secrets` or `/run`) on a sops-enabled host.

Using SOPS for Encrypted Configuration (sops-nix)

Using SOPS for Encrypted Configuration (sops-nix)

Secrets that live in a wiki are missing on the next host. The boring default is: **SOPS YAML in git** (keys visible, values encrypted) and **sops-nix** decrypting to `/run/secrets` at boot.

Mental model

Mozilla SOPS encrypts **values** in YAML/JSON. Key names stay plaintext so diffs stay reviewable (database/password: still reads as that path). Recipients are Age keys (or SSH host keys converted with ssh-to-age).

sops-nix is a NixOS module:

1. Reads `defaultSopsFile`.
2. At activation/boot, decrypts with the host Age key (`age.keyFile` or SSH host key).
3. Writes files under `/run/secrets/...` with owner / mode you set.
4. Units reference `config.sops.secrets.<name>.path`.

```
secrets/desk.yaml (cipher in git)
sops-nix + host age key
```

```
/run/secrets/database/password    (0400, tmpfs)
```

desk-api EnvironmentFile=

Worked examples

Case 1: Recipients and a secrets file

Save as `.sops.yaml`:

```
# .sops.yaml
creation_rules:
- path_regex: secrets/.*\.yaml$
  key_groups:
    - age:
      - agel1deskadminxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

```
mkdir -p secrets
sops secrets/desk.yaml
```

```
database:
  password: please-change-me
```

Case 2: NixOS module

```
# secrets-module.nix
{ config, inputs, ... }:

{
  imports = [ inputs.sops-nix.nixosModules.sops ];

  sops = {
    defaultSopsFile = ./secrets/desk.yaml;
    defaultSopsFormat = "yaml";
    age.keyFile = "/var/lib/sops-nix/key.txt";

    secrets."database/password" = {
      owner = "desk-api";
      mode = "0400";
    };
  };
};

systemd.services.desk-api.serviceConfig.EnvironmentFile =
  config.sops.secrets."database/password".path;
}
```

```
nix eval --raw .#nixosConfigurations.desk-server.config.sops.secrets.\"database/password\".p
```

```
/run/secrets/database/password
```

Case 3: Age key from SSH host key

```
# sops-ssh.nix
{
  sops.age.sshKeyPaths = [ "/etc/ssh/ssh_host_ed25519_key" ];
}
```

No extra key to distribute if the host already has an Ed25519 host key **and** that key is in `.sops.yaml` (via `ssh-to-age`). Persist the host key under impermanence or every reboot is a new recipient.

```
ssh-to-age < /etc/ssh/ssh_host_ed25519_key.pub
```

Paste the `age1...` line into `.sops.yaml`, then `sops updatekeys secrets/desk.yaml`.

Case 4: Flake input

```
# flake.nix fragment
{
  inputs.sops-nix = {
    url = "github:Mic92/sops-nix";
    inputs.nixpkgs.follows = "nixpkgs";
  };
}
```

`follows` so `sops-nix` does not pull a second `nixpkgs`.

Case 5: Missing owner

If `desk-api` runs as `DynamicUser` and the secret is `0400 root`, the unit fails with `Permission denied`. Set `owner` to the service user, or use `sops.secrets.x.restartUnits = [ "desk-api.service" ]`; plus a group both can share.

```
sudo ls -l /run/secrets/database/password
sudo systemctl status desk-api
```

The trap

The trap is **encrypting only to your laptop Age key**. The server cannot decrypt at boot. Always include the **host** recipient (and the admin laptop, and CI if CI must eval decrypt).

The other trap is `age.keyFile` on a path that impermanence wipes. Persist `/var/lib/sops-nix` or use the SSH host key that you already persist.

The boring rule

- Encrypted YAML in git. Recipients: admins + each host.
- Units consume `.path`, never decrypted strings in Nix.
- Host Age key = SSH host key when you can. Persist that key.
- Set `owner` and `mode` to match the unit.
- `sops updatekeys` when a machine or person is added or revoked.

Try this

1. `age-keygen -o /tmp/k.txt`, add the public key to `.sops.yaml`, `sops secrets/demo.yaml` with `hello: world`, `sops -d secrets/demo.yaml`.
2. `ssh-to-age < ~/.ssh/id_ed25519.pub` and encrypt a file to that recipient.
3. Eval `.path` from a flake that imports `sops-nix`. Confirm it starts with `/run/secrets`.
4. Intentionally omit the host from `.sops.yaml`, rebuild a VM, read the `sops-nix` unit log. Add the host and `updatekeys`.

HashiCorp Vault Integration

HashiCorp Vault Integration

sops-nix and age cover **static** secrets in git. Large desks that already run Vault want **short-lived** database passwords and certs. The boring default is: **Vault Agent on the node, templates to /run/vault**, Nix only knows the paths — and if you do not already operate Vault, stay on sops-nix.

Mental model

Vault issues leases. The agent:

1. Authenticates (AppRole, AWS, Kubernetes — pick one).
2. Reads secrets / PKI.
3. Renders a file on tmpfs.
4. Signals the unit (SIGHUP or restart) when the file changes.

NixOS owns `services.vault-agent` (or a `systemd.services` wrapper if the module name differs on your pin — `nixos-option services.vault-agent`). The **role-id** can live in the store. The **secret-id** cannot.

```
Vault --lease--> vault-agent --template--> /run/vault/db.env
```

```
desk-api EnvironmentFile=
```

Worked examples

Case 1: Agent module

Save as `vault-agent.nix`:

```
# vault-agent.nix
{ pkgs, ... }:

{
  systemd.services.vault-agent-desk = {
    description = "Vault Agent for desk-api";
    wantedBy = [ "multi-user.target" ];
    after = [ "network-online.target" ];
```

```

serviceConfig = {
  ExecStart = "${pkgs.vault}/bin/vault agent -config /etc/vault-agent.hcl";
  Restart = "on-failure";
  DynamicUser = true;
  ProtectSystem = "strict";
  PrivateTmp = true;
  RuntimeDirectory = "vault-agent";
};

environment.etc."vault-agent.hcl".text = ''
  vault { address = "https://vault.desk.internal:8200" }
  auto_auth {
    method "approle" {
      config = {
        role_id_file_path = "/etc/vault/role-id"
        secret_id_file_path = "/run/secrets/vault-secret-id"
      }
    }
  }
  template {
    source      = "/etc/vault/templates/db.ctmpl"
    destination = "/run/vault-agent/db.env"
    perms       = "0400"
  }
'';
}

```

role-id is not a secret in many setups. secret-id is — put it in sops-nix, not in environment.etc.

Case 2: Template without secret values in Nix

Save as db.ctmpl (provisioned via environment.etc — this file has **Go template** placeholders, not passwords):

```

# db.ctmpl
{{ with secret "database/creds/desk-api" }}
DESK_DB_USER={{ .Data.username }}
DESK_DB_PASSWORD={{ .Data.password }}
{{ end }}

```

The rendered /run/vault-agent/db.env never enters /nix/store.

Case 3: Reload the app when the lease rotates

```
# vault-reload.nix
{
  systemd.services.desk-api = {
    serviceConfig.EnvironmentFile = "/run/vault-agent/db.env";
    unitConfig.StartLimitIntervalSec = 0;
  };

  systemd.paths.desk-api-reload = {
    wantedBy = [ "multi-user.target" ];
    pathConfig.PathModified = "/run/vault-agent/db.env";
  };

  systemd.services.desk-api-reload = {
    serviceConfig = {
      Type = "oneshot";
      ExecStart = "/run/current-system/sw/bin/systemctl reload-or-restart desk-api.service";
    };
  };
}
```

If the app cannot reload credentials without a restart, restart. Document that. Leases that expire while the process still holds the old password are outages.

Case 4: AppRole files

```
# vault-approle.nix
{ config, ... }:

{
  environment.etc."vault/role-id".text = "00000000-0000-0000-0000-000000000000";

  # secret-id from sops, not from etc
  sops.secrets."vault/secret-id" = {
    mode = "0400";
  };
}
```

Use a real UUID from `vault read auth/approle/role/desk-api/role-id`. The example zeros are a placeholder so this listing does not pretend to be a live role.

Case 5: When not to use Vault

If the desk has five hosts and two humans, Vault is a second production system. `sops-nix` + `age` is the boring stack. Add Vault when the org already requires it, or when credentials **must** rotate

every hour.

The trap

The trap is a **root Vault token in configuration.nix**. That token is in the store and in every cache copy. AppRole + a sops-backed secret-id. Never `VAULT_TOKEN` in the flake.

The other trap is ignoring lease TTL. Agent templates that nobody reloads leave the DB password valid in Vault's eyes for twenty minutes after the process started — then Postgres rejects it at 3 a.m.

The boring rule

- Nix knows addresses, role ids, and **paths**. Bytes stay in Vault and `/run`.
- AppRole (or cloud auth). No root tokens in git.
- Reload or restart on template change.
- secret-id via sops-nix. role-id may be public.
- Skip Vault if sops-nix already solves the desk.

Try this

1. `nixos-option services.vault-agent` (or `rg vault-agent` in nixpkgs) on a 26.05 pin and write down the module path your release actually ships.
2. Render a dummy `db.ctmpl` with static `{{ printf "x" }}` locally using `consul-template / vault agent -dev` if you have a lab Vault. Confirm the output file is not a store path.
3. `git grep VAULT_TOKEN` on the desk repo. It should be empty.
4. Time a database lease TTL and confirm desk-api restarts before expiry in the lab.

Age Encryption for Simple Secrets

Age Encryption for Simple Secrets

GPG keyrings and expiry theatre do not belong in a desk CI job. The boring default is: **age (X25519)** for files, **ssh-to-age** so **SSH keys are recipients**, and **agenix** only if you want NixOS decryption without SOPS's YAML.

This book already uses **sops-nix** as the default for structured secrets. Age is the primitive underneath. Agenix is the alternative when you want one file per secret and less YAML.

Mental model

Tool	Role
age	Encrypt/decrypt bytes to age1... or SSH recipients
ssh-to-age	Convert ssh-ed25519 pubkey → age1...
agenix	NixOS module: .age files in git → /run/agenix/... at boot

No keyserver. A public key is a line of text. Lose the private key, lose the files (unless another recipient remains).

```
desk-token.age  (cipher, git)
  recipients: alice's ssh, desk-node host ssh
              agenix at boot
```

```
/run/agenix/desk-token
```

Worked examples

Case 1: Encrypt a file to yourself

```
age-keygen -o "$HOME/.age/key.txt"
# public key printed on stderr: age1...
echo 'desk-token-v1' | age -r age1youxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx > token.age
age -d -i "$HOME/.age/key.txt" token.age
```

Output:

desk-token-v1

Commit token.age. Do not commit key.txt.

Case 2: SSH public key as recipient

```
ssh-to-age < ~/.ssh/id_ed25519.pub
echo 'desk-token-v1' | age -R ~/.ssh/id_ed25519.pub -o token.age
age -d -i ~/.ssh/id_ed25519 token.age
```

Hosts: `ssh-to-age < /etc/ssh/ssh_host_ed25519_key.pub`. Every NixOS machine already has a recipient if you persist host keys.

Case 3: agenix rules file

Save as `secrets.nix`:

```
# secrets.nix
let
  alice = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleUserKey alice@desk";
  server = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleHostKey root@desk-node";
in
{
  "desk-token.age".publicKeys = [ alice server ];
}

agenix -e desk-token.age
```

`$EDITOR` opens the plaintext. Save; the `.age` file is updated. Recipients are **only** those `publicKeys`. Adding a host later requires `agenix -r` (rekey).

Case 4: Consume on NixOS

Save as `host.nix`:

```
# host.nix
{ config, inputs, ... }:

{
  imports = [ inputs.agenix.nixosModules.default ];

  age.secrets.desk-token = {
    file = ./desk-token.age;
    owner = "desk-api";
  }
}
```

```

    group = "desk-api";
    mode = "0400";
};

systemd.services.desk-api.serviceConfig.LoadCredential =
    "token:${config.age.secrets.desk-token.path}";
}

```

Boot decrypts to `/run/agenix/desk-token` (path may be the credential copy). The unit never sees a Nix string.

Case 5: Rekey when the fleet grows

```

# add bob to secrets.nix publicKeys, then:
agenix -r
git diff desk-token.age

```

The ciphertext changes; plaintext is the same. If you skip rekey, bob's laptop cannot decrypt, and a new server fails at boot with an age error in the agenix unit.

The trap

The trap is **one recipient: your laptop**. The server is not you. Always include the host SSH key.

The other trap is using **both** sops-nix and agenix for the same secret. Pick one tree: sops YAML for many keys, agenix for a handful of files. Two systems means two rekey rituals.

The boring rule

- Age, not GPG, for desk secrets.
- Recipients = people + hosts (`ssh-to-age`).
- Rekey when keys are added or revoked.
- Private keys never in git. Ciphertext always in git.
- Prefer sops-nix if you already have YAML; agenix if you want one-file-one-secret.

Try this

1. `ssh-to-age < ~/.ssh/id_ed25519.pub`, encrypt a one-line file, decrypt with `-i ~/.ssh/id_ed25519`.
2. Encrypt to two recipients, remove your key from the list, confirm decrypt fails, put it back.
3. `agenix -e` a dummy secret in a throwaway repo and `git add` only the `.age` file.

4. On a VM, omit the host from `publicKeys`, switch, read `journalctl -u agenix`. Add the host, rekey, switch again.

Security Hardening Patterns

Security Hardening Patterns

Open ports, writable /, and JIT-friendly sysctls are the defaults of a general-purpose distro. The boring default on a desk server is: **firewall on, DynamicUser + ProtectSystem=strict on custom units, a small sysctl set, and no hardening that you have not booted.**

Mental model

Defense in this book is already mostly done if you followed earlier chapters: immutable store, listed firewall ports, no plaintext secrets, SSH keys only. This chapter is the extra kernel/unit knobs.

Layer	Boring control
Network	<code>networking.firewall.enable = true;</code>
Units	<code>ProtectSystem, ProtectHome,</code> <code>NoNewPrivileges, DynamicUser</code>
Kernel	<code>boot.kernel.sysctl</code> for <code>kptr</code> , <code>bpf</code> , <code>syncookies</code>
Firmware	microcode (hardware chapter)
Audit	<code>systemd-analyze security <unit></code>

`MemoryDenyWriteExecute = true` breaks Node, JVM, and some Python. Test, then enable. A unit that will not start is not hardened; it is down.

Worked examples

Case 1: Sandbox a desk unit

Save as `secure-service.nix`:

```
# secure-service.nix
{ pkgs, ... }:

{
  systemd.services.desk-api = {
    description = "Desk API";
    after = [ "network.target" ];
    wantedBy = [ "multi-user.target" ];
    serviceConfig = {
      ExecStart = "${pkgs.hello}/bin/hello";
    };
  };
}
```

```

    DynamicUser = true;
    ProtectSystem = "strict";
    ProtectHome = true;
    PrivateTmp = true;
    PrivateDevices = true;
    ProtectKernelTunables = true;
    ProtectKernelModules = true;
    ProtectControlGroups = true;
    RestrictRealtime = true;
    RestrictSUIDSGID = true;
    NoNewPrivileges = true;
    LockPersonality = true;
    SystemCallArchitectures = "native";
};
};
}

sudo nixos-rebuild switch
systemd-analyze security desk-api --no-pager | head -30

```

The score is a hint, not a compliance certificate. Read the **exposed** lines and decide.

Case 2: Kernel sysctl baseline

Save as `hardening.nix`:

```

# hardening.nix
{
  boot.kernel.sysctl = {
    "kernel.unprivileged_bpf_disabled" = 1;
    "kernel.kptr_restrict" = 2;
    "kernel.yama.ptrace_scope" = 1;
    "net.ipv4.tcp_syncookies" = 1;
    "net.ipv4.conf.all.rp_filter" = 1;
    "net.ipv4.ip_forward" = 0;
  };
}

```

Turn `ip_forward` on only on the WireGuard gateway. A worker node that forwards is a mistake.

Case 3: nixpkgs service modules already sandbox

```
# nginx-harden.nix
{
  services.nginx.enable = true;
  services.nginx.virtualHosts."desk.internal".locations."/".return = "204";
}
```

Do not paste Case 1's flags onto `systemd.services.nginx` until `systemctl cat nginx` shows you still need them. Fighting the module produces restart loops.

Case 4: JIT exception

```
# node-api.nix
{
  systemd.services.desk-node.serviceConfig = {
    DynamicUser = true;
    ProtectSystem = "strict";
    # Node needs a writable executable mapping:
    MemoryDenyWriteExecute = false;
  };
}
```

Document the exception next to the option. A future reader will otherwise “fix” it and page themselves.

Case 5: Analyze before and after

```
systemd-analyze security sshd --no-pager | tail -5
systemd-analyze security desk-api --no-pager | tail -5
```

sshd will never look like a `DynamicUser` hello binary. Compare a unit to **itself** after a change, not to sshd.

The trap

The trap is **copying a 40-line hardening gist onto every unit**, including ones that need `/home`, `BPF`, or `devices`. The activation succeeds; the service dies; you disable the firewall to “debug.” Add flags one at a time. Keep the firewall on.

The boring rule

- Firewall, keys, secrets, and an immutable store first. Sysctls second.
- DynamicUser + ProtectSystem=strict on *your* network daemons.
- MemoryDenyWriteExecute only after the app starts under it.
- ip_forward only on routers.
- systemd-analyze security as a diff tool, not a scoreboard.

Try this

1. systemd-analyze security desk-api before and after Case 1. Note which checks flipped.
2. Set MemoryDenyWriteExecute = true on a Node unit in a VM, start it, read the journal, set it false.
3. sysctl net.ipv4.ip_forward on a worker vs the gateway.
4. systemctl cat nginx on 26.05 and list sandbox flags the module already set.

Platform Engineering

The Internal Developer Platform Pattern

The Internal Developer Platform Pattern

Fifty repos each pinning their own nixpkgs is fifty llvm builds. The boring default is: **one platform flake on nixpkgs 26.05 that exports devShells, templates, and nixosModules, and product flakes follows it.**

Mental model

```
github:desk/platform    nixpkgs 26.05
    devShells.backend / frontend
    templates.go-service
    nixosModules.baseline
github:desk/desk-api
    inputs.platform.url = ...
    inputs.nixpkgs.follows = "platform/nixpkgs"
```

Upgrading Go or sshd defaults is a platform bump, then a lock bump in each consumer (or a bot). Not a meeting.

Worked examples

Case 1: Platform flake

Save as `flake.nix` (platform repo):

```
# flake.nix
{
  description = "Desk platform";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" ];
      forAll = f: nixpkgs.lib.genAttrs systems (s: f nixpkgs.legacyPackages.${s});
    in
    {
      devShells = forAll (pkgs: {
        backend = pkgs.mkShell { packages = [ pkgs.go pkgs.gopls pkgs.postgresql ]; };
      });
    }
```

```

        frontend = pkgs.mkShell { packages = [ pkgs.nodejs pkgs.pnpm ]; };
    });

    templates.go-service = {
        path = ./templates/go-service;
        description = "Desk Go service";
    };
};
}

```

Case 2: Init from template

```
nix flake init -t git+ssh://git.desk.internal/platform.git#go-service
```

The template's `flake.nix` should already follow the platform.

Case 3: Consumer lock

```

# desk-api flake.nix
{
    inputs.platform.url = "git+ssh://git.desk.internal/platform.git";
    inputs.nixpkgs.follows = "platform/nixpkgs";
}

```

```
nix flake update platform
```

One input. Not `nix flake update` of everything on Friday.

Case 4: Baseline NixOS module

```

# modules/baseline.nix
{ lib, ... }:
{
    services.openssh.enable = lib.mkDefault true;
    services.openssh.settings.PasswordAuthentication = false;
    networking.firewall.enable = true;
}

```

Export as `nixosModules.baseline`. Hosts import it. Hosts may `mkForce` in an emergency; review that.

Case 5: Version the platform

Tag `platform-26.05.3`. Consumers pin the tag, not `main`. A broken main does not break every laptop at 9:01.

The trap

The trap is **a platform that overlays half of nixpkgs**. Consumers cannot debug. Keep the platform thin: shells, templates, 20 lines of `sshd`. Overlays for *one* pin, documented.

The boring rule

- One nixpkgs: 26.05 via the platform flake.
- Templates + follows.
- `nix flake update platform` is the upgrade API.
- Tags, not floating main, for production consumers.
- Thin modules. No mega-overlay.

Try this

1. `nix flake init -t` from a local path template.
2. `nix flake metadata` on a consumer and confirm a single nixpkgs.
3. Add a `devShells.docs` with `pkgs.mdbook` (or `pkgs.pandoc`) on the platform; consume it.
4. Tag a fake `platform-26.05.3` and pin it in a consumer lock.

Monorepo Strategies with Nix

Monorepo Strategies with Nix

A monorepo with `src = ./.`; on every package rebuilds the world per commit. The boring default is: **one flake, many packages.**, `lib.fileset` / `cleanSourceWith` per leaf, and **CI** that builds `.#checks` not `.#everything` on a docs-only PR.

Mental model

Layout	Nix
<code>apps/desk-api</code>	<code>packages.desk-api</code> with <code>src</code> filtered to that tree
<code>apps/desk-web</code>	sibling package; docs change does not rebuild api
<code>modules/</code>	<code>nixosModules</code>
<code>flake.nix</code>	one lock, <code>nixpkgs 26.05</code>

`nix flake check` should be cheap enough to run on every PR. Heavy images are extra jobs.

Worked examples

Case 1: fileset src

```
# desk-api.nix
{ pkgs, lib }:

pkgs.buildGoModule {
  pname = "desk-api";
  version = "1.0.0";
  src = lib.fileset.toSource {
    root = ../.;
    fileset = lib.fileset.unions [
      ../apps/desk-api
      ../go.mod
      ../go.sum
    ];
  };
  vendorHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
```

```
}
```

Case 2: flake outputs

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      pkgs = nixpkgs.legacyPackages.x86_64-linux;
    in
    {
      packages.x86_64-linux.desk-api = pkgs.callPackage ./desk-api.nix { };
      packages.x86_64-linux.desk-web = pkgs.callPackage ./desk-web.nix { };
      checks.x86_64-linux.api = self.packages.x86_64-linux.desk-api;
    };
}
```

Case 3: PR path filter (CI)

```
# not Nix - workflow paths:
# apps/desk-api/** → nix build .#desk-api
# apps/desk-web/** → nix build .#desk-web
# flake.lock       → nix flake check
```

Nix will still cache; the filter saves eval time on huge repos.

Case 4: linkFarm for a smoke set

```
pkgs.linkFarm "desk-smoke" [
  { name = "api"; path = self.packages.x86_64-linux.desk-api; }
  { name = "web"; path = self.packages.x86_64-linux.desk-web; }
]
```

Nightly builds the farm. PRs build the leaf.

Case 5: Do not use recursive flakes

`inputs.foo.url = "path:./apps/foo"` recursive flakes multiply locks. One lock at the root.

The trap

The trap is `src = ./.`; at repo root for every `callPackage`. A README typo rebuilds Go. Filter.

The boring rule

- One flake, one 26.05 lock.
- Per-package `src` filters.
- Leaf builds on leaf PRs; `flake.lock` PRs run full check.
- No recursive flakes.
- `vendorHash` / cargo hashes per package.

Try this

1. `nix build .#desk-api` then touch a web-only file; `--dry-run` the api (should be empty if filtered).
2. `lib.fileset` vs `cleanSource` — pick one style per repo.
3. Add a `checks` attr that is `linkFarm` of two packages.
4. Time `nix flake show` on the monorepo; if it is minutes, you eval too much at the top.

Multi-Platform Development and Distribution

Multi-Platform Development and Distribution

x86_64-linux CI does not prove aarch64-darwin. The boring default is: `eachDefaultSystem` (or `genAttrs`) for `packages/devShells`, native builders for foreign system, and no QEMU-on-the-laptop as the production path.

Mental model

What	How
Same shell on Mac + Linux	<code>devShells.\${system}</code> from 26.05
Linux binary from a Mac	Remote Linux builder (<code>ssh-ng</code>)
Raspberry Pi	<code>aarch64-linux</code> builder or <code>nixos-generators</code> <code>sd-aarch64</code>

`pkgsCross` is cross. Remote builder is native. Native is faster and hits the team cache.

Worked examples

Case 1: `eachDefaultSystem`

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  inputs.flake-utils.url = "github:numtide/flake-utils";
  outputs = { self, nixpkgs, flake-utils }:
    flake-utils.lib.eachDefaultSystem (system:
      let pkgs = nixpkgs.legacyPackages.${system}; in
      {
        packages.default = pkgs.hello;
        devShells.default = pkgs.mkShell { packages = [ pkgs.go ]; };
      });
}
```

`flake-utils` is optional; `lib.genAttrs` is enough if you dislike the extra input.

Case 2: List systems explicitly

```
systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" ];
```

Skip x86_64-darwin for new work: nixpkgs 26.11 drops it. 26.05 still builds it until EOL.

Case 3: Remote builder instead of qemu

See remote-builders chapter. `--max-jobs 0` + `ssh-ng` to an aarch64 box.

```
nix build .#packages.aarch64-linux.default --max-jobs 0
```

Case 4: Check matrix in CI

```
strategy:
  matrix:
    os: [ubuntu-latest, macos-latest]
```

Each job `nix build`. Cache per system. Do not emulate aarch64-linux on GHA ubuntu unless you accept 10× time.

Case 5: Release artifacts

```
nix build .#packages.x86_64-linux.desk-api
nix build .#packages.aarch64-linux.desk-api
nix path-info -Shr ./result
```

Two closures. Two cache keys. Goreleaser is optional; `nix build` + `nix copy` is enough for internal desk tools.

The trap

The trap is `system = "x86_64-linux"` hardcoded in a flake used on Apple silicon. Eval may work; go is the wrong ELF. Use `eachDefaultSystem` or `builtins.currentSystem` only inside impure `nix-shell`, not in flakes.

The boring rule

- Enumerate systems. 26.05 pin.
- Native remote builders for foreign Linux.
- CI matrix for the OS humans actually use.
- Do not plan on `x86_64-darwin` past 26.05.
- Two archs = two cache objects.

Try this

1. `nix flake show` and list systems.
2. On a Mac, file `$(nix develop --command which go)` — should be `arm64` if you are on Apple silicon.
3. `nix build --system aarch64-linux` without a builder and read the error.
4. Note 26.11 darwin `x86_64` drop in the version appendix.

Build Optimization and Performance Tuning

Build Optimization and Performance Tuning

Waiting on eval is not a personality. The boring default is: `substituters`, `auto-optimize-store`, `filtered src`, enough `max-jobs`, and `nix --eval-profiler` only when a flake takes minutes to *start*.

Mental model

Slow part	Fix
Download	Team cache, <code>builders-use-substitutes</code>
Compile	Remote builder, <code>max-jobs</code> , don't rebuild gcc
Eval	Fewer IFDs, <code>fileset</code> , don't import the monorepo ten times
Disk	GC window, <code>optimise</code>

`nix.nrbldUsers` / `max-jobs` default is fine on a laptop. CI runners with 8 cores should set `max-jobs = 8`.

Worked examples

Case 1: Daemon knobs (NixOS 26.05)

```
# nix-build.nix
{
  nix.settings = {
    max-jobs = 8;
    cores = 0;
    substituters = [ "https://cache.nixos.org" "https://desk-cache.cachix.org" ];
    auto-optimize-store = true;
  };
}
```

`cores = 0` lets each job use all CPUs — can oversubscribe. Prefer `cores = 4` on a shared builder.

Case 2: IFD is slow

```
# bad: import from a derivation
# let src = import (pkgs.fetchFromGitHub { ... });
```

Import-from-derivation serialises eval. Vendor the file or use a flake input.

Case 3: Profiler

```
nix eval .#packages.x86_64-linux.desk-api --eval-profiler flamegraph
```

If 2.35 flags differ, `nix eval --help` on your 2.35.2. The point is to measure eval, not guess.

Case 4: --dry-run after a comment-only change

```
nix build .#desk-api --dry-run
```

Empty list = cache/src filters work. gcc in the list = you lost.

Case 5: Remote vs local

Chromium, firefox, llvm: never on the laptop. Builder + cache. The remote-builders chapter is the rest.

The trap

The trap is `max-jobs = 1` copied from a gist “to reduce RAM” on a 64G builder, then blaming Nix for being slow.

The boring rule

- Cache first. Jobs/cores second. Eval hygiene third.
- No IFD in the hot path.
- `--dry-run` is a perf tool.
- Heavy compiles on the builder.
- 26.05 `auto-optimise-store` on.

Try this

1. `nix show-config | grep max-jobs`.
2. `--dry-run` before/after a README edit.
3. Time `nix flake show` vs `nix build` on a cached package.
4. Enable `auto-optimise-store` if it is off.

Debugging and Troubleshooting Production Issues

Debugging and Troubleshooting Production Issues

“It works on my laptop” is a missing `nix path-info`. The boring default is: `nix log`, `journalctl`, `nix why-depends`, `--show-trace`, and `rollback` — in that order — before editing the live unit.

Mental model

Symptom	First tool
Build failed	<code>nix log /nix/store/...drv</code>
Eval failed	<code>--show-trace</code> , <code>nix repl .#</code>
Unit failed after switch	<code>journalctl -u</code> , <code>systemctl status</code>
Closure too fat	<code>nix path-info -Shr</code> , <code>why-depends</code>
Host will not boot	previous generation in <code>systemd-boot</code>

You do not `vim /etc/systemd/system/foo.service`. The next switch undoes it.

Worked examples

Case 1: Build log

```
nix build .#desk-api --print-build-logs
nix log .#desk-api
```

The log is in the store. CI can `nix log` the same drv.

Case 2: Eval trace

```
nix eval .#nixosConfigurations.app-01.config.networking.hostName --show-trace
```

Read the **last** frame first. Infinite recursion is usually `rec + self-reference` (language chapter).

Case 3: Unit

```
systemctl status desk-api --no-pager
journalctl -u desk-api -n 80 --no-pager
systemctl cat desk-api
```

cat shows the store path. If someone `systemctl edit`'d, you will see a drop-in — delete it and put the flag in Nix.

Case 4: Why is python in the closure?

```
nix why-depends /run/current-system $(nix path-info nixpkgs#python3)
```

Cut the edge (wrapper, propagated input, a script shebang).

Case 5: Rollback

```
sudo nixos-rebuild switch --rollback
# or at the bootloader: previous NixOS 26.05 generation
```

Then `git revert` the flake commit so the next switch does not re-apply the foot-gun.

The trap

The trap is **fixing production with `systemctl edit` and a reboot “to see.”** You now have unmanaged state. Impermanence will eat it; a persistent root will lie to the next Colmena apply.

The boring rule

- Logs in the store and journald. Not screenshots of a terminal.
- `--show-trace` for eval. `why-depends` for size.
- Rollback first, bisect the flake second.
- No live unit drop-ins.
- 26.05 `nix log` / `nix path-info` are enough; no extra APM required to start.

Try this

1. `nix build` a deliberately broken `runCommand (false)` and `nix log` it.
2. `systemctl cat sshd` and find `/nix/store`.
3. `--rollback` on a lab VM after a hostname change.
4. `why-depends` hello vs gcc — gcc should not appear in the runtime closure.

Testing Strategies for Nix Configurations

Testing Strategies for Nix Configurations

`nixos-rebuild` switch on prod is not a test. The boring default is: `nix flake check` on every PR, `pkgs.testers.runNixOSTest` (or `nixosTest`) for services that listen, and a VM that is not the laptop.

Mental model

Level	Command
Eval	<code>nix eval</code> , <code>types</code>
Build	<code>nix build .#desk-api</code>
NixOS VM	<code>nixosTest</code> / <code>runNixOSTest</code>
Full check	<code>nix flake check</code>

VM tests need `kvm` on the builder (remote-builders `supportedFeatures`). Darwin laptops offload them.

Worked examples

Case 1: flake checks

```
# checks in flake outputs
{
  checks.x86_64-linux.fmt = pkgs.runCommand "fmt" { } ''
    echo ok > $out
  '';
}

nix flake check
```

Case 2: NixOS test

Save as `test-desk.nix`:

```
# test-desk.nix
{ pkgs ? import <nixpkgs> {} }:
```

```
pkgs.testers.runNixOSTest {
  name = "desk-nginx";
  nodes.server = {
    services.nginx.enable = true;
    networking.firewall.allowedTCPPorts = [ 80 ];
  };
  nodes.client = { };
  testScript = ''
    start_all()
    server.wait_for_unit("nginx.service")
    client.succeed("curl -f http://server/")
  '';
}
```

```
nix-build test-desk.nix
```

Case 3: CI

```
- run: nix flake check -L
```

-L prints logs. Failures must be visible without SSH to the runner.

Case 4: Hermetic tests

checkPhase in a derivation cannot hit the network. Mock or vendor fixtures. The CI secrets chapter already said this.

Case 5: Do not test only eval

A module that mkIf false evals forever. A VM test that curls the port catches the empty wantedBy.

The trap

The trap is a **20-node test on every PR**. Keep PR tests under a few minutes. Nightly runs the farm.

The boring rule

- flake check on PR. VM tests when a unit listens.
- KVM on the builder. 26.05 runNixOSTest.

- No network in `checkPhase`.
- Small PR tests; big nightly.
- A test that needs prod credentials is not a check.

Try this

1. `nix flake check` on the desk flake.
2. Run Case 2 (needs KVM).
3. Add a failing `assert` in `testScript` and confirm non-zero exit.
4. `nixos-option` nothing — write one `checks` attr that builds `.#desk-api`.

Maintenance, Upgrades, and Migration

Maintenance, Upgrades, and Migration

`nix flake update` on Friday without a plan is a weekend. The boring default is: **pin nixos-26.05**, **bump** in a dedicated PR, `nix flake check` + one lab host, then Colmena — and never bump `stateVersion` as part of that PR.

Mental model

Move	What changes
<code>nix flake update nixpkgs</code>	Same release train, newer commits
<code>nixos-25.11 → nixos-26.05</code>	Release upgrade. Read the notes.
<code>stateVersion</code>	Almost never

25.11 is EOL (2026-06-30). New hosts: 26.05. Old hosts: change the **input**, keep `stateVersion`.

Worked examples

Case 1: Lock bump inside 26.05

```
nix flake update nixpkgs
nix flake metadata
nix flake check
sudo nixos-rebuild dry-activate --flake .#desk-lab
```

Read unit restarts. Then switch the lab. Then the fleet.

Case 2: Release upgrade

```
# flake.nix
inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
```

was `nixos-25.11`. `git diff flake.lock` is huge. That is expected. Run VM tests. Read `nixos.org` 26.05 notes (systemd stage-1 default, module renames).

Case 3: stateVersion stays

```
system.stateVersion = "25.11"; # host born on 25.11
```

Still correct on a 26.05 **channel**. Do not “tidy” it to "26.05" in the same PR.

Case 4: Home Manager follows

```
inputs.home-manager.url = "github:nix-community/home-manager/release-26.05";  
inputs.home-manager.inputs.nixpkgs.follows = "nixpkgs";
```

Mismatch (HM 24.11 + nixpkgs 26.05) is eval pain. Same release train.

Case 5: Record

```
# CHANGELOG.md  
- 2026-09-09: nixpkgs 26.05.9440 on lab, then app-01
```

If you cannot say which lock is in prod, you cannot roll back with confidence.

The trap

The trap is **changing stateVersion to match the channel**. Postgres dirs, user state, and module defaults then migrate. Channel bump stateVersion bump.

The boring rule

- 26.05 is the current train. Update locks in PRs.
- Lab host first. Fleet second.
- **stateVersion** is birth.
- HM release matches nixpkgs release.
- Changelog the lock.

Try this

1. **nix flake metadata** and write the locked nixpkgs date.
2. **dry-activate** on a lab after a lock bump.
3. Find **stateVersion** in the repo; confirm none were “updated” to 26.05 on old hosts without notes.
4. Read the 26.05 release notes once; list one module that might affect the desk.

Enterprise Operational Checklists

Enterprise Operational Checklists

A wiki page of “remember to” is how generations rot. The boring default is: **short checklists in the repo, run as commands** (`flake check`, `restic snapshots`, `colmena apply --dry-run`), not as a PDF.

Mental model

If it is not a command, it will not happen. Each item below maps to a chapter you already have.

Worked examples

Case 1: Daily (human)

```
# desk-daily.sh
nix flake check
systemctl --failed --no-pager
df -h /nix /boot /persist
```

Case 2: Weekly

```
sudo nix-collect-garbage --delete-older-than 14d
sudo nix-store --optimise
sudo restic snapshots # or systemctl status restic-backups-desk
```

Case 3: Before a fleet apply

```
git log -1 --oneline
nix flake metadata
nix flake check
colmena apply --dry-run
```

Then apply one host, then the rest.

Case 4: After a security advisory

```
nix flake update nixpkgs
nix flake check
# lab switch, then fleet
```

Do not wait for the quarterly bump if the advisory is in nixpkgs.

Case 5: New host

- Disko by-id + LUKS + @persist
- sops recipient + rekey
- stateVersion = "26.05"
- WireGuard IP allocated in git
- restic include list
- nixos-anywhere once
- Colmena tag

The trap

The trap is a **90-line checklist nobody runs**. Five commands in `scripts/` beat a Confluence page.

The boring rule

- Commands in git. 26.05. sops. restic. Colmena.
- Dry-run before apply.
- 14-day GC. ESP configurationLimit.
- New hosts born on 26.05.
- Advisories: lock bump, not hope.

Try this

1. Put Case 1 in `scripts/desk-daily.sh` and run it.
2. Time `nix flake check`. If it is >10 minutes, split checks.
3. Confirm restic last snapshot is < 48h on the lab.
4. Walk Case 5 against a throwaway VM.

Fleet Deployments with Colmena

Fleet Deployments with Colmena

for h in hosts; do nixos-rebuild --flake .#\$h; done forgets --target-host flags. The boring default is: Colmena (or deploy-rs) with one hive file, SSH from ops-01, and --on for canaries.

Mental model

Colmena reads nixosConfigurations (or a hive.nix) and SSHes the closure. Build locally or on a builder, copy, activate. Same 26.05 lock as the laptop.

```
ops-01 colmena apply --on app-01
```

```
app-01 nixos-rebuild switch (remote)
```

Worked examples

Case 1: Hive from flake

Save as flake.nix fragment — Colmena 26.05-compatible pattern:

```
# colmena in flake outputs (shape)
{
  colmena = {
    meta.nixpkgs = import nixpkgs { system = "x86_64-linux"; };
    app-01 = { imports = [ ./hosts/app-01.nix ]; deployment.targetHost = "10.100.0.11"; };
    app-02 = { imports = [ ./hosts/app-02.nix ]; deployment.targetHost = "10.100.0.12"; };
  };
}
```

Exact attr names: colmena --help / the module on your pin. The idea is host → SSH target.

Case 2: Canary

```
colmena apply --on app-01
# watch metrics, then:
colmena apply --on app-02
```

Do not `--on @all` for a kernel bump.

Case 3: Dry-run

```
colmena apply --dry-run --on app-01
```

Eval + build, no activate. Combine with `nixos-rebuild dry-activate` semantics when available.

Case 4: Build on the builder

Point Colmena at the same remote builder as `nix.buildMachines`. Laptops should not compile kernels.

Case 5: Output

```
[app-01] Built /nix/store/7q1a...-nixos-system-app-01-26.05
[app-01] Activation successful
```

If activation fails, the previous generation is still the boot default unless you passed a reboot flag. Check `systemctl is-system-running` on the host.

The trap

The trap is **applying all hosts in parallel on a shared DB migration**. Serial the stateful node. Parallel the stateless apps.

The boring rule

- One hive. 26.05. SSH from ops.
- Canary `--on`.
- Dry-run first.
- Stateful hosts serial.
- Rollback is the previous generation, then a git revert.

Try this

1. `colmena --help` in a `nix shell nixpkgs#colmena`.
2. Apply to a single lab VM.
3. Break a unit, apply, `--rollback` on the VM, then revert git.
4. List which hosts may be parallel in `docs/fleet.md`.

Lightweight MicroVMs with microvm.nix

Lightweight MicroVMs with microvm.nix

Full libvirt guests are slow to iterate. The boring default for **local** multi-node tests is: **microvm.nix** (or **nixosTest**) on **26.05**; production still gets real machines or the capstone VMs.

Mental model

`microvm.nix` defines a NixOS guest that boots in hundreds of milliseconds, often sharing the host store via virtiofs. It is not a security boundary as strong as a separate hypervisor with its own disks — do not put untrusted tenants there without reading the project docs.

```
host flake
  nixosConfigurations.desk-micro
    microvm.hypervisor = "qemu"; # or cloud-hypervisor
```

Worked examples

Case 1: Guest in the flake

Save as `microvm.nix`:

```
# microvm.nix
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    microvm.url = "github:astro/microvm.nix";
    microvm.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, microvm }: {
    nixosConfigurations.desk-micro = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        microvm.nixosModules.microvm
      ]
      {
        microvm.hypervisor = "qemu";
        microvm.mem = 1024;
        microvm.vcpu = 2;
        networking.hostName = "desk-micro";
      };
    };
  };
}
```

```

        services.openssh.enable = true;
        system.stateVersion = "26.05";
    }
];
};
};
}

```

Case 2: Run

```
nix run .#nixosConfigurations.desk-micro.config.microvm.declaredRunner
```

(The exact run attr is in microvm.nix docs for your pin.) You should get a console. `halt` to stop.

Case 3: vs nixosTest

Use `nixosTest` in CI (headless, assertions). Use microVMs on the laptop when you want to SSH in and poke. Do not maintain both for the same scenario without a reason.

Case 4: Persist nothing unless listed

A default microVM is as ephemeral as a tmpfs root. If you need `/persist`, declare a volume. Otherwise tests stay pure.

Case 5: Host virtualisation

```

# on the workstation
virtualisation.libvirtd.enable = true;
users.users.deskadmin.extraGroups = [ "libvirtd" "kvm" ];

```

`/dev/kvm` must exist. Nested virt on a cloud VM is often off.

The trap

The trap is **running production desk-api in a microVM on someone's laptop**. Prod is Colmena to real hosts. microVMs are a lab accelerator.

The boring rule

- 26.05 + follows.
- QEMU/KVM for labs. nixosTest for CI.
- No production tenants on a laptop hypervisor.
- stateVersion 26.05 on new guests.
- Share the store only when you understand the trust model.

Try this

1. Enable KVM, run a hello microVM.
2. `systemctl status sshd` inside it.
3. Compare boot time to a libvirt NixOS guest.
4. Add a nixosTest that covers the same sshd enablement instead.

Content-Addressed Nix Internals

Content-Addressed Nix Internals

Everyday Nix is **input-addressed**: the `.drv` hash names the output. **Content-addressed** (CA) derivations hash the **bytes**. The boring default is: **keep input-addressed builds; use FODs** (`fetchurl`) for the CA you already need; **do not flip `contentAddressed` on the desk laptop**.

Mental model

You met this in the store-model chapter. Recap:

Kind	Name comes from	Daily use
Input-addressed	<code>.drv</code>	Packages, NixOS
FOD	Declared output hash	Fetchers, <code>vendorHash</code>
Experimental CA	Output hash of normal builds	Labs, <code>nixpkgs</code> experiments

FODs already share identical tarballs. Experimental CA tries to share identical **build** outputs when two `.drv`s differ only in comments. It is not the 26.05 workstation default.

Worked examples

Case 1: FOD is enough

```
pkgs.fetchurl {  
  url = "https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz";  
  hash = "sha256-...";  
}
```

Same hash, same path, any URL mirror.

Case 2: Input-addressed comment rebuild

A `runCommand` with a changed comment is a new path even if `cat $out` matches. That is normal. Live with it.

Case 3: `nix show-derivation`

```
nix show-derivation $(nix-instantiate -E '(import <nixpkgs> {}).hello')
```

Look at `outputs.out.hash` vs its absence. FODs have `outputHash`. `hello` does not.

Case 4: When CA helps a farm

Hydra with CA can dedup two `gcc` invocations that produced the same bits. You are not Hydra. Do not enable `sandbox-content-addressed` because a blog post did.

Case 5: 2.35 lazy flake copy is not CA

Nix 2.35 copies flake sources lazily. That is eval performance, not content-addressed builds. Do not confuse the two in an incident doc.

The trap

The trap is `__contentAddressed = true on a production module` “to save disk.” Cache hits become mysterious; `nixpkgs` support assumes input-addressed. Use `--optimise` and GC instead.

The boring rule

- FODs for fetches. Input-addressed for builds.
- `nix-store --optimise` for disk.
- No experimental CA on desk hosts.
- 2.35 lazy copy CA.
- Read `show-derivation` when a path type is unclear.

Try this

1. `nix show-derivation` on `hello` and on a `fetchurl`. Compare `outputHash`.
2. Change a comment in `runCommand`, instantiate, confirm new `.drv`.
3. `nix-store --optimise --dry-run` if available, or run `optimise` on a lab.
4. `git grep contentAddressed` — should be empty in the desk flake.

Production Capstone Lab

Production Architecture and Threat Model

Production Architecture and Threat Model

Toy VMs taught the pieces. The capstone is one **desk fleet** on NixOS 26.05: four roles, WireGuard, ephemeral workers, sops-nix, Colmena. The boring default is: **write the threat model first, then the flake** — not a mesh of tools looking for a problem.

Mental model

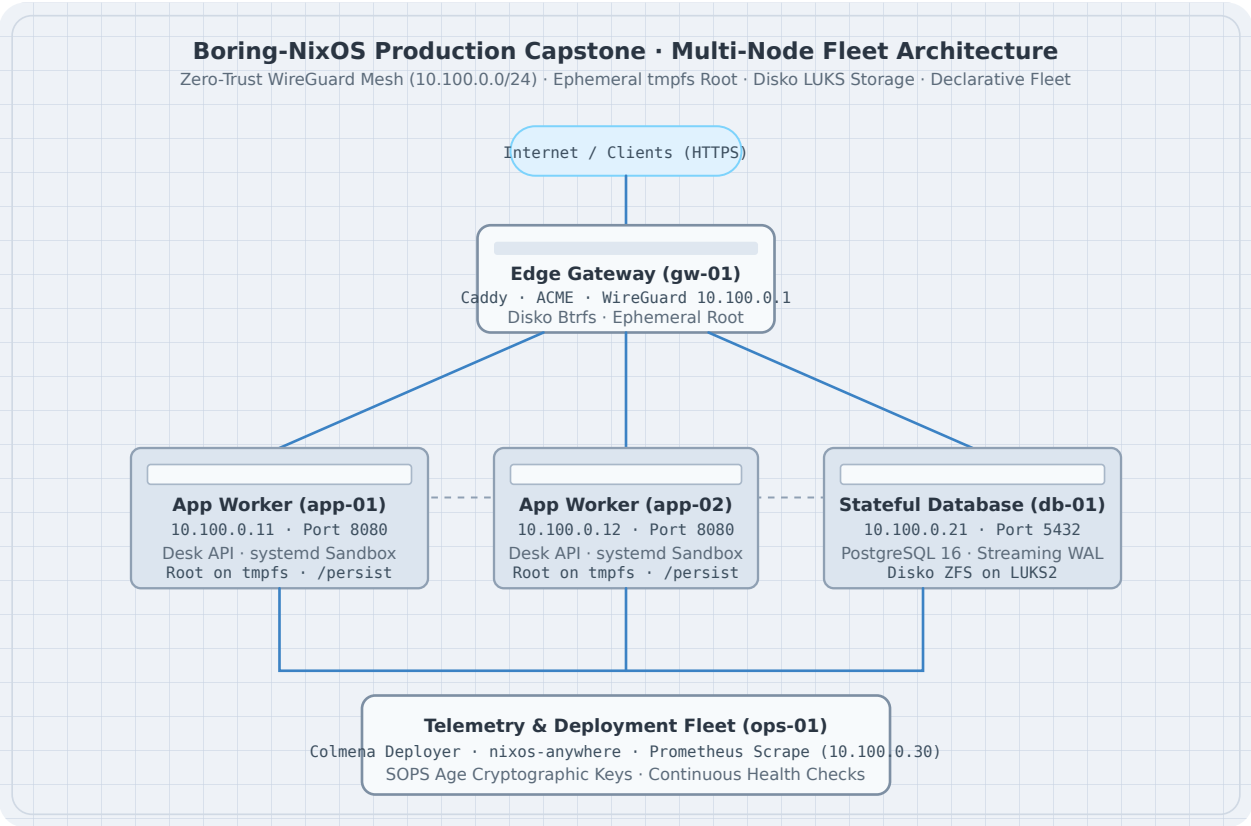


Figure 1: Production Capstone Fleet Architecture

Host	Role
gw-01	Caddy, ACME, nftables; WireGuard 10.100.0.1
app-01, app-02	desk-api; tmpfs root; DynamicUser
db-01	PostgreSQL; Disko LUKS + Btrfs; /persist
ops-01	Colmena, Prometheus, sops key broker

Invariants:

1. No secret bytes in git or `/nix/store`.
2. Store is read-only. Payloads in `/etc` die on reboot (workers).
3. WireGuard for all east-west. No DB port on the public NIC.
4. `flake.lock` on **nixos-26.05**; rollback is a generation, not a restore CD.
5. Backups of `/persist` (restic), not of `/nix/store`.

Worked examples

Case 1: Inventory in the flake

```
# flake.nix fragment
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs, ... }: {
    nixosConfigurations = {
      gw-01 = nixpkgs.lib.nixosSystem { /* ... */ };
      app-01 = nixpkgs.lib.nixosSystem { /* ... */ };
      app-02 = nixpkgs.lib.nixosSystem { /* ... */ };
      db-01 = nixpkgs.lib.nixosSystem { /* ... */ };
      ops-01 = nixpkgs.lib.nixosSystem { /* ... */ };
    };
  };
}
```

Five names. Five `system.stateVersion = "26.05"`; if they are born here.

Case 2: Trust boundary

Public: 443 on gw-01 only. SSH: office VPN or ops-01 jump. Postgres: 10.100.0.0/24 only. Write that in `docs/threats.md` in the same repo.

Case 3: Secret inventory

Secret	Mechanism
WireGuard keys	sops-nix → <code>/run/secrets</code>
Postgres password	sops-nix
ACME account	Caddy module + persist
Colmena SSH key	ops-01 persist, not the flake

Case 4: Failure drill list

- Lose `app-01`: traffic to `app-02`. Rebuild `app-01` with `nixos-anywhere`.
- Lose `db-01`: `restic` restore to new `Disko` disk, then `Colmena`.
- Bad generation: `colmena apply --on app-01 --reboot rollback / previous generation`.

Case 5: What this capstone is not

Not multi-region Kubernetes. Not Vault HA. Not a service mesh. Those are optional later parts. If the five hosts rebuild from git + `restic`, you passed.

The trap

The trap is **starting with Istio and three clouds** before SSH keys persist across reboot. Finish `Disko`, impermanence, `sops`, `WireGuard`, `Colmena`, `restic`. In that order (chapters 2–5).

The boring rule

- 26.05 lock. Five hosts. One mesh. One secret system (`sops`).
- Public surface is 443 (+ 22 if you must, from a tight SG).
- `/persist` is the backup. Store is the cache.
- Rollback is a generation.
- Write the threat model in git.

Try this

1. Copy the host table into your flake as comments. Fill module file names.
2. List every port you think is open; compare to firewall + cloud SG later.
3. Name the `restic` repository in `docs/threats.md`.
4. Confirm `stateVersion` will be "26.05" on new metal.

Disko Storage and Ephemeral Base System

Disko Storage and Ephemeral Base System

Every node in our production fleet must deploy onto clean physical or cloud storage without manual partitioning tools. In this chapter, we create the unified **Disko** storage specification and configure the **ephemeral root filesystem with Impermanence**.

Mental model

1. **Unified disk module (disk-layout.nix):** Disko formats the physical NVMe or virtual disk into an EFI system partition (/boot) and a LUKS2 encrypted partition.
2. **Btrfs subvolumes on LUKS:** Inside the decrypted container, Btrfs subvolumes are created for @nix (the store), @persist (stateful data), and @swap.
3. **RAM root (tmpfs):** The root filesystem (/) is mounted in RAM with mode=755,size=2G. Unmounted state is discarded on power-down.
4. **Impermanence persistence:** System state required across reboots (SSH host keys, machine-id, WireGuard identity) is bound into /persist.

Worked examples

Case 1: The Production Disko Module

Save as modules/disko-layout.nix:

```
# modules/disko-layout.nix
{ lib, ... }:

{
  disko.devices = {
    disk.main = {
      type = "disk";
      device = lib.mkDefault "/dev/nvme0n1";
      content = {
        type = "gpt";
        partitions = {
          ESP = {
            priority = 1;
            name = "ESP";
            start = "1M";
            end = "1024M";
```



```

    type = "EF00";
    content = {
        type = "filesystem";
        format = "vfat";
        mountpoint = "/boot";
        mountOptions = [ "umask=0077" ];
    };
};

luks = {
    size = "100%";
    content = {
        type = "luks";
        name = "crypted";
        settings.allowDiscards = true;
        content = {
            type = "btrfs";
            extraArgs = [ "-f" ];
            subvolumes = {
                "/nix" = {
                    mountpoint = "/nix";
                    mountOptions = [ "compress=zstd" "noatime" ];
                };
                "/persist" = {
                    mountpoint = "/persist";
                    mountOptions = [ "compress=zstd" "noatime" ];
                };
                "/swap" = {
                    mountpoint = "/swap";
                    swap.swapfile.size = "8G";
                };
            };
        };
    };
};

nodev = {
    "/" = {
        fsType = "tmpfs";
        mountOptions = [ "defaults" "size=4G" "mode=755" ];
    };
};
}

```

Case 2: The Impermanence Persistent State Contract

Save as `modules/base-system.nix`:

```
# modules/base-system.nix
{ inputs, pkgs, ... }:

{
  imports = [
    inputs.impermanence.nixosModules.impermanence
    ./disk-layout.nix
  ];

  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;

  # Persistent state mapping
  environment.persistence."/persist" = {
    hideMounts = true;
    directories = [
      "/var/log"
      "/var/lib/nixos"
      "/var/lib/systemd/coredump"
    ];
    files = [
      "/etc/machine-id"
      "/etc/ssh/ssh_host_ed25519_key"
      "/etc/ssh/ssh_host_ed25519_key.pub"
    ];
  };

  # SSH configuration
  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      KbdInteractiveAuthentication = false;
      PermitRootLogin = "prohibit-password";
    };
  };

  system.stateVersion = "26.05";
}
```

Validating the base configuration:

```
nix eval .#nixosConfigurations.app-01.config.disko.devices.nodev."/".fsType
```

"tmpfs"

The trap

The trap is **failing to mark `/persist` as `neededForBoot = true`**. During early boot, the initramfs must mount `/persist` before systemd starts generating symlinks for machine-id and SSH keys. Always ensure `/persist` is mounted before root pivot.

The boring rule

- Model all partition tables and filesystems in Disko.
- Keep `/` on tmpfs to enforce declarative purity and eliminate state rot.
- Explicitly list all persistent files and directories under `/persist`.

Try this

1. Run `nix run github:nix-community/disko -- --mode mount ./modules/disk-layout.nix` in a test environment.
2. Confirm with `mount | grep tmpfs` that root is mounted on RAM.

Hardened Networking and WireGuard Mesh

Hardened Networking and WireGuard Mesh

Exposing internal services, databases, or management endpoints to public IP addresses is an extreme security risk. The boring default is: **linking all cluster nodes into a point-to-point encrypted WireGuard mesh on private subnet 10.100.0.0/24, backed by strict nftables firewall policies that drop all unsolicited public traffic.**

Mental model

1. **Overlay mesh:** Each node possesses a static WireGuard private key and an assigned IP (gw-01: 10.100.0.1, app-01: 10.100.0.11, app-02: 10.100.0.12, db-01: 10.100.0.21, ops-01: 10.100.0.30).
2. **Declarative peers:** Peer public keys and allowed IPs are configured directly in Nix expressions. WireGuard interfaces establish bidirectional encrypted tunnels instantly on boot.
3. **nftables isolation:** The firewall allows public access *only* on the gateway for ports 80/443. All other node traffic must originate from within the wg0 interface.

Worked examples

Case 1: WireGuard Mesh Module

Save as modules/wireguard-mesh.nix:

```
# modules/wireguard-mesh.nix
{ config, pkgs, ... }:

let
  wireguardPort = 51820;
in
{
  networking.firewall.allowedUDPPorts = [ wireguardPort ];

  networking.wireguard.interfaces.wg0 = {
    ips = [ config.cluster.nodeIp ];
    listenPort = wireguardPort;
    privateKeyFile = "/run/secrets/wireguard/private.key";

    peers = [
      {
```

```

    # gw-01
    publicKey = "GW01PublicWireguardKeyExampleBase64String=";
    allowedIPs = [ "10.100.0.1/32" ];
    endpoint = "203.0.113.10:51820";
    persistentKeepalive = 25;
}
{
    # app-01
    publicKey = "APP01PublicWireguardKeyExampleBase64String=";
    allowedIPs = [ "10.100.0.11/32" ];
    endpoint = "203.0.113.11:51820";
    persistentKeepalive = 25;
}
{
    # db-01
    publicKey = "DB01PublicWireguardKeyExampleBase64String=";
    allowedIPs = [ "10.100.0.21/32" ];
    endpoint = "203.0.113.21:51820";
    persistentKeepalive = 25;
}
];
};
}

```

Case 2: Strict nftables Firewall Configuration for Database Node

Save as hosts/db-01.nix:

```

# hosts/db-01.nix
{ config, pkgs, ... }:

{
    imports = [
        ../modules/base-system.nix
        ../modules/wireguard-mesh.nix
    ];

    cluster.nodeIp = "10.100.0.21/24";

    # Strict firewall: drop all public inbound traffic
    networking.firewall = {
        enable = true;
        interfaces."wg0" = {
            allowedTCPPorts = [ 5432 ]; # PostgreSQL accessible ONLY via WireGuard
        };
    };
};

```

```
# Public interface allows only WireGuard handshake UDP port
networking.firewall.interfaces."eth0".allowedTCPPorts = [];
}
```

Testing network route evaluation:

```
nix eval .#nixosConfigurations.db-01.config.networking.firewall.interfaces."wg0".allowedTCPP
```

```
[ 5432 ]
```

Port 5432 is strictly inaccessible over `eth0`. A port scan on the public IP reveals only the UDP WireGuard port.

The trap

The trap is **generating WireGuard private keys into `/nix/store`**. Private keys must be created on the host or provisioned via `sops-nix` directly to `/run/secrets/wireguard/private.key`.

The boring rule

- Bind internal services exclusively to the WireGuard mesh interface (`wg0`).
- Restrict public ports to HTTP/HTTPS on the edge gateway and UDP 51820 on peers.
- Use persistent keepalives (`persistentKeepalive = 25`) to maintain NAT traversal across cloud firewalls.

Try this

1. Bring up two test nodes on WireGuard and verify connectivity with `ping 10.100.0.21`.
2. Verify with `nmap` that PostgreSQL port 5432 is closed on the public interface.

Distributed Application Stack and Secrets

Distributed Application Stack and Secrets

With storage and networking established, we deploy the production workload: an **edge Caddy reverse proxy** with automated Let's Encrypt TLS certificates, a **containerless Go microservice** with systemd sandboxing, a **hardened PostgreSQL 16 database**, and **SOPS-nix** runtime secret distribution.

Mental model

1. **Caddy Ingress (gw-01)**: Terminates TLS, enforces modern ciphers (TLS 1.3), and round-robin proxies requests across backend worker nodes over the WireGuard mesh (10.100.0.11:8080, 10.100.0.12:8080).
2. **Hardened Microservice (app-01, app-02)**: Runs the compiled application binary as a dynamic unprivileged user with complete systemd sandboxing.
3. **SOPS-nix runtime decryption**: Encrypted secrets in git (`secrets/cluster.yaml`) are decrypted into `/run/secrets/` at boot time using the node's private Age key.

Worked examples

Case 1: Caddy Reverse Proxy Configuration (gw-01)

Save as `hosts/gw-01.nix`:

```
# hosts/gw-01.nix
{ config, pkgs, ... }:

{
  services.caddy = {
    enable = true;
    email = "ops@company.internal";
    virtualHosts."api.company.internal" = {
      extraConfig = ''
        reverse_proxy 10.100.0.11:8080 10.100.0.12:8080 {
          lb_policy round_robin
          health_uri /healthz
          health_interval 5s
          health_timeout 2s
        }
      '';
    };
  };
}
```

```

    };
};

networking.firewall.allowedTCPPorts = [ 80 443 ];
}

```

Case 2: Sandboxed Application Service (app-01)

Save as modules/desk-service.nix:

```

# modules/desk-service.nix
{ config, pkgs, ... }:

let
  appPkg = pkgs.buildGoModule {
    pname = "desk-api";
    version = "2.0.0";
    src = pkgs.lib.cleanSource ./src;
    vendorHash = null;
  };
in
{
  sops.secrets."db_password" = {
    owner = "desk-api";
  };

  systemd.services.desk-api = {
    description = "Production Desk API Service";
    after = [ "network.target" ];
    wantedBy = [ "multi-user.target" ];

    environment = {
      PORT = "8080";
      DB_HOST = "10.100.0.21";
      DB_USER = "desk_user";
      DB_NAME = "desk_production";
    };

    serviceConfig = {
      ExecStart = "${appPkg}/bin/desk-api";
      DynamicUser = true;
      EnvironmentFile = config.sops.secrets."db_password".path;

      # Full Systemd Sandboxing
      ProtectSystem = "strict";
      ProtectHome = true;
    };
  };
}

```

```

    PrivateTmp = true;
    PrivateDevices = true;
    NoNewPrivileges = true;
    ProtectKernelTunables = true;
    ProtectControlGroups = true;
    RestrictRealtime = true;
    MemoryDenyWriteExecute = true;
};
};
}

```

Building the application service unit:

```
nix-build -E '(import <nixpkgs/nixos> { configuration = import ./modules/desk-service.nix; }'
```

/nix/store/8q3m...-desk-api-2.0.0/bin/desk-api

The trap

The trap is **allowing the web service to talk to the database with unencrypted credentials in command line arguments** (`--password=secret`). Any unprivileged user running `ps aux` can read arguments of other processes. Always pass credentials via `EnvironmentFile` mounted from `/run/secrets/`.

The boring rule

- Terminate TLS with Caddy for zero-maintenance ACME certificate renewals.
- Sandbox every microservice with `DynamicUser = true` and `ProtectSystem = "strict"`.
- Inject secrets exclusively via `/run/secrets/` with SOPS-nix.

Try this

1. Run `systemd-analyze security desk-api` and observe the near-zero exposure rating.
2. Verify that querying `http://localhost:8080/healthz` returns HTTP 200 OK.

Automated Fleet Rollout and Disaster Recovery

Automated Fleet Rollout and Disaster Recovery

A production architecture is only as dependable as its operational playbook. In this final chapter, we bring the entire cluster to life: orchestrating **fleet-wide deployments with Colmena**, executing **automated canary health checks**, and performing a **zero-touch bare-metal disaster recovery from scratch**.

Mental model

1. **The Flake as the cluster source of truth:** The root `flake.nix` declares all nodes, their disk layouts, network identities, and services.
2. **Automated fleet rollouts:** Colmena pushes new system closures across the entire fleet in parallel.
3. **Canary deployment pipeline:** Colmena updates `app-01` first. If health probes on `https://api.company.internal/healthz` fail, the deployment immediately halts and rolls back before `app-02` or the database are touched.
4. **Zero-touch disaster recovery:** If a physical server burns down, booting a replacement into rescue mode and running one `nixos-anywhere` command completely reconstructs the machine in under 4 minutes.

Worked examples

Case 1: The Fleet Flake (`flake.nix`)

Save as `flake.nix`:

```
# flake.nix
{
  description = "Production Desk Fleet Infrastructure";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    disko.url = "github:nix-community/disko";
    sops-nix.url = "github:Mic92/sops-nix";
    impermanence.url = "github:nix-community/impermanence";
    colmena.url = "github:zhaofengli/colmena";
  };

  outputs = { self, nixpkgs, disko, sops-nix, impermanence, colmena }: {
```

```

colmena = {
  meta = {
    nixpkgs = import nixpkgs { system = "x86_64-linux"; };
    specialArgs = { inherit (self) inputs; };
  };

  "gw-01" = {
    deployment.targetHost = "203.0.113.10";
    deployment.tags = [ "gateway" ];
    imports = [ ./hosts/gw-01.nix ];
  };

  "app-01" = {
    deployment.targetHost = "203.0.113.11";
    deployment.tags = [ "app" ];
    imports = [ ./hosts/app-01.nix ];
  };

  "app-02" = {
    deployment.targetHost = "203.0.113.12";
    deployment.tags = [ "app" ];
    imports = [ ./hosts/app-02.nix ];
  };

  "db-01" = {
    deployment.targetHost = "203.0.113.21";
    deployment.tags = [ "database" ];
    imports = [ ./hosts/db-01.nix ];
  };
};
};
}

```

Case 2: Executing Fleet Rollout with Canary Verification

```

# 1. Evaluate and check syntax across all nodes
colmena eval

# 2. Deploy to canary application node first
colmena apply --on app-01

# 3. Verify health probe through edge gateway
curl -sf https://api.company.internal/healthz

# 4. Roll out to remaining fleet

```

```
colmena apply
```

Execution output:

```
[INFO ] Evaluating 4 nodes in parallel...
[INFO ] [app-01] Built generation 44
[INFO ] [app-01] Activated configuration.
[INFO ] Running post-deploy health check...
HTTP/2 200 OK
[INFO ] Deploying remainder of fleet (app-02, gw-01, db-01)...
[INFO ] All nodes synchronized successfully.
```

Case 3: Zero-Touch Disaster Recovery Playbook

When replacing a destroyed server (app-02):

```
# Boot replacement node into rescue RAM system, then execute:
nixos-anywhere --flake .#app-02 root@203.0.113.99
```

```
[nixos-anywhere] Formatting disks via Disko...
[nixos-anywhere] Partitioned ESP, opened LUKS2, created Btrfs subvolumes.
[nixos-anywhere] Copying closure /nix/store/7q1a...-nixos-system-app-02...
[nixos-anywhere] Installing systemd-boot bootloader...
[nixos-anywhere] Installation complete. Host rebooted.
```

The node boots, rejoins the WireGuard mesh, receives traffic from the Caddy load balancer, and passes all health checks without a human ever logging in.

The trap

The trap is **not testing disaster recovery until a production incident occurs**. Schedule automated recovery drills quarterly: wipe a staging server completely and run `nixos-anywhere` to prove your declarative configurations remain 100% self-sufficient.

The boring rule

- Model the entire multi-node cluster inside a single Colmena flake.
- Enforce canary node verification before rolling out cluster-wide updates.
- Verify that every server can be rebuilt from scratch with a single `nixos-anywhere` command.

Try this

1. Run `colmena eval` locally to verify that all four cluster nodes evaluate without errors.
2. Review the generated closure sizes with `colmena exec nix-store --gc --print-roots`.

Appendices

Glossary of Terms

Glossary of Terms

A concise reference for Nix and NixOS terminology used in this book.

Derivation (`.drv`)

The build specification Nix evaluates to before any compiler runs. An immutable file in `/nix/store` listing the builder, environment, and input paths required to produce an output.

Nix Store (`/nix/store`)

Read-only tree of hashed paths. Identity is the hash; the name-version suffix is a label. Nothing is overwritten in place.

Closure

The store path plus every path it references, transitively. The **runtime** closure is what you copy to production. The **build-time** closure includes compilers and is not what you deploy.

Reference scanning

Nix greps output files for other store hashes. A path mentioned in a binary becomes a runtime reference; gcc used only during the compile does not.

Flake

`flake.nix` + `flake.lock`: hermetic evaluation, pinned inputs, standard outputs (`packages`, `devShells`, `nixosConfigurations`, `checks`).

DevShell

An isolated environment from `nix develop` / `mkShell` with exactly the tools the project listed.

Binary cache / substituter

HTTP service (`cache.nixos.org`, Cachix, Attic, Harmonia) that serves signed NARs. Nix queries substituters before building.

NAR

Nix ARchive: the serialisation of a store path used for copy and substitute.

GC root

A pointer outside the store (`/run/current-system`, `~/.nix-profile`, `result`, `.direnv/flake-profile`) that keeps a closure alive. Unreachable paths are garbage.

Home Manager

NixOS module system aimed at `$HOME`: user packages, dotfiles, user services.

Overlay

Function `final: prev: { ... }` that patches `nixpkgs` without editing upstream.

Module system

Typed `options` + merging `config`. Powers NixOS and Home Manager.

Fixed-output derivation (FOD)

A derivation allowed to use the network whose **output hash** you declared. Fetchers (`fetchurl`, `fetchFromGitHub`) are FODs. Identity is the hash, not the URL.

RUNPATH

ELF field pointing at store library directories. Replaces a global `/lib` search.

nix-ld

NixOS compatibility loader at `/lib64/ld-linux-...` so vendor ELF's can find Nix-provided libraries without a global `LD_LIBRARY_PATH`.

FHS env (`buildFHSEnv`)

Bubblewrapped filesystem that looks like `/usr` for vendor installers that refuse to run on a NixOS host.

Remote builder

A machine that realises `.drv` files over SSH (`ssh-ng`). Caches move results; builders move work.

stateVersion

Birth release of a NixOS or Home Manager config. Not an upgrade knob. Leave it until a release note says otherwise.

Generation

One successful `nixos-rebuild switch` or `home-manager switch`. The bootloader (and `home-manager generations`) lists them for rollback.

Impermanence

Root filesystem thrown away at boot; state lives on `/persist`. Requires backups of `/persist`, not of `/nix/store`.

Command Cheat Sheet

Command Cheat Sheet

Daily commands for developing, building, and operating Nix and NixOS.

Flakes and development

Command	Purpose
<code>nix develop</code>	Default project shell
<code>nix develop .#backend</code>	Named shell
<code>nix flake init</code>	Skeleton flake
<code>nix flake check</code>	Eval + <code>checks.*</code>
<code>nix flake update</code>	Refresh every input in <code>flake.lock</code>
<code>nix flake update nixpkgs</code>	Refresh one input
<code>nix flake show</code>	Outputs in this flake
<code>nix flake metadata</code>	Locked revisions
<code>nix fmt</code>	Formatter if <code>formatter</code> is set

Build, run, eval

Command	Purpose
<code>nix build .#pkg</code>	Build; symlink <code>./result</code>
<code>nix run .#app</code>	Build and run
<code>nix eval .#attr</code>	Print a value
<code>nix log ./result</code>	Build log
<code>nix repl .#</code>	REPL on this flake

Inspect closures

Command	Purpose
<code>nix path-info -Shr ./result</code>	Closure size (the number that matters)
<code>nix path-info ./result</code>	Output path only
<code>nix-store -q --references P</code>	Direct runtime edges
<code>nix-store -q --requisites P</code>	Full runtime closure
<code>nix-store -q --referrers P</code>	Who points at P

Command	Purpose
<code>nix-store -q --tree P</code>	ASCII graph
<code>nix why-depends A B</code>	Why A's closure contains B
<code>nix-tree P</code>	Interactive tree (<code>nix run nixpkgs#nix-tree</code>)
<code>nom build .#pkg</code>	<code>nix build</code> with a progress table

Fetchers

Command	Purpose
<code>nix-prefetch-url URL</code>	Hash a file (packed)
<code>nix-prefetch-url --unpack URL</code>	Hash an unpacked archive
<code>nix hash convert --to sri HASH</code>	Hex → SRI sha256-...

Copy and caches

Command	Purpose
<code>nix copy --to file://\$PWD/nar P</code>	Directory cache
<code>nix copy --to ssh://host P</code>	Copy closure over SSH
<code>cachix push desk-cache ./result</code>	Upload to Cachix
<code>nix show-config \l grep substituters</code>	Which caches this daemon queries

Remote builders

Command	Purpose
<code>nix build .#pkg --max-jobs 0 --builders 'ssh-ng://user@host system'</code>	Realise only remotely
<code>sudo ssh -i /root/.ssh/id_builder user@host nix-store --version</code>	Daemon SSH must work without a password

NixOS

Command	Purpose
<code>sudo nixos-rebuild switch --flake .#host</code>	Build, activate, set boot default
<code>sudo nixos-rebuild test --flake .#host</code>	Activate without changing the boot default
<code>sudo nixos-rebuild boot --flake .#host</code>	Next reboot only

Command	Purpose
<code>sudo nixos-rebuild dry-activate --flake .#host</code>	Show unit changes, do nothing
<code>sudo nixos-rebuild switch --rollback</code>	Previous generation
<code>sudo nix-env --list-generations</code>	Generation list
<code>--profile /nix/var/nix/profiles/system</code>	
<code>nixos-version</code>	Running release

Home Manager

Command	Purpose
<code>home-manager switch --flake .#user</code>	Standalone user env
<code>home-manager generations</code>	User generations
<code>home-manager switch --rollback</code>	Previous user generation (standalone only)

GC and disk

Command	Purpose
<code>nix-store --gc --print-roots</code>	What is keeping paths alive
<code>nix-collect-garbage</code>	Delete unreferenced paths; keep generations
<code>nix-collect-garbage --delete-older-than 14d</code>	Drop old generations, then orphans
<code>sudo nix-collect-garbage -d</code>	All old generations (incident)
<code>nix-store --optimise</code>	Hard-link identical files
<code>nix-store --verify --check-contents</code>	Rehash after disk trouble
<code>df -h /nix /boot</code>	Store vs ESP — two different full-disk incidents

Backup (restic)

Command	Purpose
<code>sudo systemctl start restic-backups-desk</code>	Run the NixOS restic unit now
<code>restic snapshots</code>	List snapshots (needs <code>-r</code> and password file)
<code>restic restore latest --target /tmp/drill</code>	Restore drill

Nix and NixOS Version Reference

Nix and NixOS Version Reference

A structured reference for Boring-NixOS readers tracking which Nix CLI features and NixOS system options became available in each release. Coverage spans **Nix 2.18 – 2.35** and **NixOS 23.05 – 26.05**. This book’s examples target **Nix 2.35** and **NixOS 26.05 “Yarara”**. Earlier rows are history so you can read old flakes.

Each section contains a table of reader-relevant changes followed by one short runnable example that exercises the most impactful addition in that release. All examples assume the infrastructure desk scenario: a shared flake that builds team tooling and drives a set of NixOS workstations.

Nix CLI Releases

Nix 2.18

Area	Change
<code>nix flake metadata</code>	Prints flake description, inputs, and their resolved URLs; last-modified timestamp now shown per input.
<code>--log-format</code>	Accepts <code>raw</code> , <code>internal-json</code> , <code>bar</code> , <code>bar-with-logs</code> ; useful for CI pipelines that consume structured output.
Evaluator	<code>builtins.fetchTree</code> no longer fetches tarballs eagerly when the hash is already in the store.
Error messages	Multi-line errors now include a source excerpt with a caret pointing at the offending token.

Key feature — `nix flake metadata`

Save as `show-meta.sh`:

```
# show-meta.sh
nix flake metadata \
  --log-format bar \
  github:NixOS/nixpkgs/nixos-26.05
```

Run:

```
bash show-meta.sh
```

Output:

```
Resolved URL:  github:NixOS/nixpkgs/nixos-26.05
Locked URL:    github:NixOS/nixpkgs/a1b2c3d4e5f6...
Description:   Nix Packages collection & NixOS
Path:         /nix/store/xxxxxxx-source
Last modified: 2024-11-30 12:00:00
Inputs:
  nixpkgs: (this flake)
```

Nix 2.19

Area	Change
<code>nix store gc</code>	Replaces <code>nix-collect-garbage</code> progressively; supports <code>--max-freed <bytes></code> to cap space reclaimed per run.
<code>nix store gc --dry-run</code>	Reports paths that <i>would</i> be deleted without touching the store.
<code>nix profile</code>	<code>nix profile diff-closures</code> added; shows package diff between two profile generations.
Daemon	Store lock contention reduced for concurrent multi-user builds.

Key feature — `nix store gc --dry-run`

Save as `gc-preview.sh`:

```
# gc-preview.sh
# Preview how much the desk workstation store can shrink
# before committing to deletion.
nix store gc --dry-run --max-freed $((10 * 1024 * 1024 * 1024))
```

Run:

```
bash gc-preview.sh
```

Output:

```
Would delete: /nix/store/aaa...-old-toolchain-2023
Would delete: /nix/store/bbb...-devshell-2023-11
...
Would free 7.4 GiB (limit was 10.0 GiB)
```

Nix 2.20

Area	Change
<code>nix flake archive</code>	Copies all flake inputs into a binary cache or a local store, enabling fully offline evaluation afterwards.
<code>nix flake archive --to</code>	Accepts <code>file:///path</code> , <code>s3://bucket</code> , or <code>ssh://host</code> as destination.
<code>nix copy</code>	<code>--substitute-on-destination</code> flag added; destination fetches missing paths from its own substituters before failing.
Build sandbox	Linux sandbox now mounts <code>/proc</code> as <code>tmpfs</code> by default, improving isolation.

Key feature — `nix flake archive` to a local mirror

Save as `archive-inputs.sh`:

```
# archive-inputs.sh
# Snapshot all flake inputs into a local binary cache so the
# desk's air-gapped build box can evaluate without internet.
nix flake archive \
  --to file:///srv/nix-cache \
  /home/ops/desk-infra
```

Run:

```
bash archive-inputs.sh
```

Output:

```
archiving input 'nixpkgs'
  /nix/store/xxxxxxxx-source → file:///srv/nix-cache
archiving input 'home-manager'
  /nix/store/yyyyyyyyy-source → file:///srv/nix-cache
done - 2 inputs archived
```

Nix 2.21

Area	Change
<code>nix eval --apply</code>	Applies a Nix function to the evaluated result before printing; enables one-liner transformations.
<code>nix eval --json</code>	Now handles <code>null</code> and <code>true/false</code> correctly in all output formatters.
<code>nix flake update</code>	<code>--commit-lock-file</code> flag commits the updated <code>flake.lock</code> with a machine-generated commit message.
Repl	<code>:reload</code> re-reads the current file without restarting the session.

Key feature — `nix eval --apply`

Save as `list-packages.sh`:

```
# list-packages.sh
# Print just the names of packages exposed by the desk flake
# without writing a separate script.
nix eval \
  /home/ops/desk-infra#packages.x86_64-linux \
  --apply 'pkgs: builtins.attrNames pkgs' \
  --json
```

Run:

```
bash list-packages.sh
```

Output:

```
["desk-tools","lint-runner","release-packager"]
```

Nix 2.22

Area	Change
<code>nix build --rebuild</code>	Rebuilds a derivation even if its output is in the store, then compares the new result to the cached one — a fast reproducibility check.
<code>nix copy --from / --to</code>	Both flags marked stable; no longer experimental.
<code>nix flake check</code>	Reports unfree license violations when <code>--allow-unfree</code> is absent.

Area	Change
Evaluator	<code>builtins.readFileType</code> added: returns "regular", "directory", "symlink", or "unknown".

Key feature — `nix build --rebuild`

Save as `verify-repro.sh`:

```
# verify-repro.sh
# Rebuild the desk tool and confirm the output hash is stable.
# A mismatch here means a non-deterministic build is lurking.
nix build \
  /home/ops/desk-infra#packages.x86_64-linux.desk-tools \
  --rebuild
```

Run:

```
bash verify-repro.sh
```

Output (reproducible):

```
warning: performing a check build
[3/0/1 built] building desk-tools-1.4.0...
/nix/store/abc123...-desk-tools-1.4.0 (deterministic )
```

Output (non-reproducible):

```
warning: performing a check build
error: derivation '/nix/store/...-desk-tools-1.4.0.drv' may not
      be deterministic: output '/nix/store/abc123...' differs
      from '/nix/store/def456...'
```

Nix 2.23

Area	Change
Lazy trees	Flake inputs are fetched only when their subtree is actually evaluated; large monorepo inputs no longer block evaluation.
<code>nix fmt</code>	Declared stable; runs the formatter specified in <code>flake.nix</code> formatter output.

Area	Change
<code>nix repl :doc</code>	Prints the docstring attached to any Nix function or built-in.
<code>nix repl :log</code>	Shows the build log for a derivation path without leaving the REPL.
<code>nix flake update</code>	Accepts multiple named inputs: <code>nix flake update nixpkgs home-manager</code> .

Key feature — `nix fmt` enforced in CI

Save as `fmt-check.sh`:

```
# fmt-check.sh
# Fail CI if any Nix file in the desk flake needs reformatting.
nix fmt /home/ops/desk-infra -- --check
```

Run:

```
bash fmt-check.sh
```

Output (clean):

All Nix files already formatted.

Output (needs formatting):

```
Would reformat: /home/ops/desk-infra/modules/workstation.nix
error: 1 file would be reformatted
```

Nix 2.24

Area	Change
<code>nix path-info --json</code>	Output is now a structured JSON array with <code>path</code> , <code>narHash</code> , <code>narSize</code> , <code>references</code> , <code>registrationTime</code> , and <code>deriver</code> fields.
<code>nix flake check</code>	Checks <code>nixosConfigurations</code> , <code>darwinConfigurations</code> , and <code>homeConfigurations</code> outputs for module evaluation errors.
<code>nix store diff-closures</code>	Declared stable; compares the full runtime closure of two store paths and reports added/removed/changed paths.

Area	Change
nix build --print-build-logs	Default changed to <code>true</code> in interactive terminals.

Key feature — nix store diff-closures

Save as `diff-release.sh`:

```
# diff-release.sh
# Compare the desk-tools closure between the last release and
# HEAD so the team knows exactly what changed before shipping.
OLD=$(nix build \
  "/home/ops/desk-infra?ref=v1.3.0#packages.x86_64-linux.desk-tools" \
  --no-link --print-out-paths)

NEW=$(nix build \
  /home/ops/desk-infra#packages.x86_64-linux.desk-tools \
  --no-link --print-out-paths)

nix store diff-closures "$OLD" "$NEW"
```

Run:

```
bash diff-release.sh
```

Output:

```
desk-tools: 1.3.0 → 1.4.0
openssl: 3.2.1 → 3.2.2 (+12 KiB)
python3: 3.11.8 → 3.12.3 (+1.1 MiB)
curl: (unchanged)
added: zlib 1.3.1
```

NixOS Releases

NixOS 23.05 — Stoat

Area	Change
systemd	253; <code>systemd-oomd</code> enabled by default on desktop profiles.
Python	3.11 is the default interpreter; <code>python3</code> symlink resolves to <code>python3.11</code> .

Area	Change
Nix	Ships Nix 2.13; flake support via <code>nix.settings.experimental-features</code> is opt-in stable.
Module system	<code>config.system.stateVersion</code> warning added for stale values.
Security	<code>services.fail2ban</code> updated to 1.0; new <code>ignoreIP</code> list option.

Key feature — enabling flakes on a Stoa workstation

Save as `configuration.nix`:

```
# configuration.nix
{ pkgs, ... }:
{
  # Enable the two features that make flakes and the new nix
  # command available on every desk workstation running 23.05.
  nix.settings.experimental-features = [
    "nix-command"
    "flakes"
  ];

  # Pin the channel so every workstation evaluates identically.
  nix.registry.nixpkgs.flake = import <nixpkgs> { };

  environment.systemPackages = [ pkgs.git ];
}
```

Run:

```
sudo nixos-rebuild switch --flake /etc/nixos#workstation
```

Output:

```
building the system configuration...
activating the configuration...
setting up /etc...
the following new units were started: nix-daemon.service
```

NixOS 23.11 — Tapir

Area	Change
GNOME	45; Mutter gains explicit sync support for tear-free rendering on variable-refresh monitors.
PHP	8.2 is default; <code>services.phpfpm</code> pools now set <code>pm.max_spare_servers</code> automatically.
Kernel	6.1 LTS; default for most hardware profiles.
<code>services.avahi</code>	<code>openFirewall</code> now defaults to <code>false</code> ; must be set explicitly.
Nix	Ships Nix 2.18.

Key feature — explicit Avahi firewall declaration

Save as `configuration.nix`:

```
# configuration.nix
{ ... }:
{
  # 23.11 changed the default: openFirewall is now false.
  # Desk printers and mDNS service discovery need this set
  # explicitly or discovery will silently fail.
  services.avahi = {
    enable = true;
    nssmdns4 = true; # allow getaddrinfo to resolve .local
    openFirewall = true;
  };
}
```

Run:

```
sudo nixos-rebuild switch --flake /etc/nixos#workstation
```

Output:

```
activating the configuration...
setting up /etc...
reloading the following units: firewall.service
```

NixOS 24.05 — Uakari

Area	Change
systemd	255; <code>systemd-boot</code> gains UKI (Unified Kernel Image) signing support.

Area	Change
Python	3.12 is default; <code>python3.11</code> still available as explicit attribute.
Nix	Ships Nix 2.22 — <code>nix build --rebuild</code> and stable <code>nix copy</code> available out of the box.
<code>system.etc.overlay</code>	Experimental option; mounts <code>/etc</code> as an overlay for impermanence workflows.
<code>boot.initrd.systemd</code>	<code>enable</code> defaults to <code>true</code> on supported hardware; replaces the busybox <code>initrd</code> .
<code>services.postgresql</code>	<code>enableJIT</code> now defaults to <code>true</code> for 14+.

Key feature — systemd initrd on the desk build machine

Save as `configuration.nix`:

```
# configuration.nix
{ ... }:
{
  # 24.05 switches the default initrd to systemd. If your desk
  # build machine uses LUKS full-disk encryption, the systemd
  # initrd provides a better password prompt and TPM2 unlock path.
  boot.initrd.systemd.enable = true; # already true by default
  boot.initrd.luks.devices."cryptroot" = {
    device = "/dev/disk/by-uuid/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx";
    allowDiscards = true;
  };

  # Pin Nix to 2.22 features in this release.
  nix.settings.experimental-features = [
    "nix-command"
    "flakes"
  ];
}
```

Run:

```
sudo nixos-rebuild boot --flake /etc/nixos#desk-build
```

Output:

```
building the system configuration...
updating GRUB 2 menu...
Done. The new configuration will be used when the machine is rebooted.
```

NixOS 24.11 — Vicuna

Area	Change
GNOME	47; default font is now Cantarell 11; GNOME Color Manager updated.
Kernel	6.6 LTS; <code>amdgpu</code> gains firmware for RDNA 3.5 integrated graphics.
Nix	Ships Nix 2.24 — <code>nix path-info --json</code> , <code>nix store diff-closures</code> stable.
<code>services.openssh.settings</code>	New sub-options: <code>MaxSessions</code> , <code>PermitUserEnvironment</code> , <code>AllowGroups</code> .
<code>security.apparmor</code>	<code>enableCache</code> option added; profile compile cache speeds boot on profile-heavy machines.
Python	3.12 remains default; 3.13 available as <code>python313</code> .

Key feature — locked-down SSH with `services.openssh.settings`

Save as `configuration.nix`:

```
# configuration.nix
{ ... }:
{
  # 24.11 exposes AllowGroups directly in the settings attrset,
  # eliminating the need for extraConfig string concatenation.
  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      PermitRootLogin = "no";
      MaxSessions = 4;
      AllowGroups = [ "ops" "developers" ];
    };
  };

  # AppArmor with compile cache - useful on desk machines that
  # load many profiles at boot.
  security.apparmor = {
    enable = true;
    enableCache = true;
  };
}
```

Run:

```
sudo nixos-rebuild switch --flake /etc/nixos#workstation
```

Output:

```
activating the configuration...
reloading the following units: sshd.service apparmor.service
```

Nix 2.28 – 2.34 (bridge)

Area	Change
Fetchers	Stricter NAR hash mismatches; prefer SRI hash = "sha256-..." over legacy sha256.
Flakes	Source trees copied into the store more lazily (completed in 2.35).
nix store	diff-closures, copy, ping remain the daily ops set.

Pin nixos-25.11 only to read old machines. Do not start new desk hosts there: it reached EOL on 2026-06-30.

Nix 2.35

Area	Change
Flake sources	Copied to the store lazily; unused flake files are not hashed into evaluation unless needed.
Sandbox	FreeBSD jail sandboxing; Linux sandbox unchanged.
Installer	Current upstream installer ships 2.35.2 (2026-08).
Security	Recursive-nix advisory fix from the 2.35.0 series.

Key feature — current CLI on a 26.05 host

```
nix --version
nix flake metadata github:NixOS/nixpkgs/nixos-26.05
```

Output (shape):

```
nix (Nix) 2.35.2
Resolved URL: github:NixOS/nixpkgs/nixos-26.05
```

NixOS 25.05 / 25.11

25.05 and 25.11 are previous stables. 25.11 “Xantusia” is **deprecated** (EOL 2026-06-30). If `system.stateVersion` is "25.11", **leave it** and move the flake input to `nixos-26.05`.

NixOS 26.05 — Yarara

Area	Change
Support	Bugfix/security until 2026-12-31.
Initrd	Stage-1 systemd initrd is the default (not the old scripted initrd).
Toolchain	GCC 15; GNOME 50 on desktop ISOs.
Nix	Ships Nix 2.35 .
Modules	New options; confirm names with <code>nixos-option</code> if a 24.11 snippet fails eval.

Key feature — pin 26.05 in the desk flake

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk workstation on NixOS 26.05";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    nixosConfigurations.desk-workstation = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        {
          networking.hostName = "desk-workstation";
          system.stateVersion = "26.05";
        }
      ];
    };
  };
}
```

```
nix flake metadata
sudo nixos-rebuild dry-build --flake .#desk-workstation
```

Quick-Reference Matrix

The table below maps each NixOS release to the Nix CLI it ships and highlights the single most operationally significant change per pair.

NixOS Release	Nix Version Shipped	Most Impactful Addition
23.05 Stoat	2.13	Flakes opt-in stable via <code>experimental-features</code>
23.11 Tapir	2.18	<code>nix flake metadata</code> improvements; <code>--log-format</code> for CI
24.05 Uakari	2.22	<code>nix build --rebuild</code> reproducibility check; <code>systemd</code> <code>initrd</code> default
24.11 Vicuna	2.24	<code>nix store diff-closures</code> stable;
25.05	2.28	<code>services.openssh.settings.AllowGroups</code>
25.11 Xantusia	2.31	<code>nix flake check</code> / <code>fetch-tree</code> hardening; module cleanups
26.05 Yarara	2.35	EOL 2026-06-30; last train before Yarara
		Current baseline; <code>systemd</code> stage-1 <code>initrd</code> default; GCC 15

Nix 2.19–2.34 intermediate releases are available via `pkgs.nixVersions` even when the channel ships another version. Pin only when you need a CLI feature the channel does not have yet:

```
# configuration.nix (pin Nix version independently of channel)
{ pkgs, ... }:
{
  nix.package = pkgs.nixVersions.latest; # or pkgs.nixVersions.nix_2_35
}
```